

Foundations of Machine Learning
Department of Computer Science, NYU
Homework assignment 1
Due: February 8, 2005

Problem 2: Bernstein's Inequality

The objective of this problem is to prove Bennett's and Bernstein's inequalities.

- (1) Show that for any $t > 0$, and any random variable X with $E[X] = 0$, $E[X^2] = \sigma^2$, and $X \leq c$,

$$E[e^{tX}] \leq e^{f(\sigma^2/c^2)},$$

where

$$f(x) = \log\left(\frac{1}{1+x}e^{-ctx} + \frac{x}{1+x}e^{ct}\right).$$

- (2) Show that $f''(x) \leq 0$ for $x \geq 0$.
(3) Using Chernoff's bounding technique, show that

$$\Pr\left[\frac{1}{m} \sum_{i=1}^m X_i \geq \epsilon\right] \leq e^{-tm\epsilon + \sum_{i=1}^m f(\sigma_{X_i}^2/c^2)},$$

where $(\sigma_{X_i}^2)$ is the variance of X_i .

- (4) Show that $f(x) \leq f(0) + xf'(0) = (e^{ct} - 1 - ct)x$.
(5) Using the bound derived in (4), find the optimal value of t .
(6) *Bennett's inequality.* Let $X_1, \dots, X_m$ be independent real-valued random variables with zero mean such that for $i = 1, \dots, m$, $X_i \leq c$. Let $\sigma^2 = \frac{1}{m} \sum_{i=1}^m \sigma_{X_i}^2$. Show that

$$\Pr\left[\frac{1}{m} \sum_{i=1}^m X_i > \epsilon\right] \leq \exp\left(-\frac{m\sigma^2}{c^2} \theta\left(\frac{\epsilon c}{\sigma^2}\right)\right),$$

where $\theta(x) = (1+x) \log(1+x) - x$.

- (7) *Berstein's inequality.* Show that under the same conditions as Bennett's inequality

$$\Pr\left[\frac{1}{m} \sum_{i=1}^m X_i > \epsilon\right] \leq \exp\left(-\frac{m\epsilon^2}{2\sigma^2 + 2c\epsilon/3}\right).$$

Hint: show that for all $x \geq 0$, $\theta(x) \geq h(x) = \frac{3}{2} \frac{x^2}{x+3}$.

- (8) Write Hoeffding's inequality assuming the same conditions. For what values of σ is Bernstein's inequality better than Hoeffding's inequality?

Problem 3: Two-Oracle Variant of PAC model

Assume that positive and negative examples are now drawn from two separate distributions D_+ and D_- . For an accuracy $(1 - \epsilon)$, the learning algorithm must find a hypothesis h such that:

$$\Pr_{x \sim D_+} [h(x) = 0] \leq \epsilon \text{ and } \Pr_{x \sim D_-} [h(x) = 1] \leq \epsilon$$

Thus, the hypothesis must have a small error on both distributions.

Let C be any concept class and H be any hypothesis space. Let h_0 and h_1 represent the identically 0 and identically 1 functions, respectively. Prove that C is efficiently PAC-learnable using H in the standard (one-oracle) PAC model if and only if it is efficiently PAC-learnable using $H \cup \{h_0, h_1\}$ in this two-oracle PAC model.

Foundations of Machine Learning
 Department of Computer Science, NYU
 Homework assignment 1 – Solution

1. Bernstein's Inequality [40 points]

(1) [20 bonus points]

(2) [10 points] Just a series of calculations of the derivatives starting from:

$$\forall x \geq 0, f'(x) = \frac{(-cte^{-ctx} + e^{ct})(1+x) - e^{-ctx} - xe^{ct}}{(1+x)^2} \frac{1+x}{e^{-ctx} + xe^{ct}}.$$

This can be simplified into:

$$\forall x \geq 0, f'(x) = \frac{e^{ct(x+1)} - (ctx + ct + 1)}{xe^{ct(x+1)} + 1}.$$

The calculation of the second derivative leads to:

$$\forall x \geq 0, f''(x) = -\frac{e^{2ct(x+1)} + c^2t^2x^2 + (c^2t^2 + 3ct)x + ct}{(xe^{ct(x+1)} + 1)^2} \leq 0.$$

(3) [5 points] As already done in class in other instances, using Markov's inequality, for any $t > 0$,

$$\Pr[X \geq m\epsilon] = \Pr[e^{tX} \geq e^{tm\epsilon}] \leq e^{-tm\epsilon} \mathbb{E}[e^{tX}].$$

Using the inequality of (1) with $X = \sum_{i=1}^m X_i$ leads directly the desired inequality.

(4) [10 points] By the Taylor series expansion with remainder, there exists $\theta \in [0, x]$ such that:

$$f(x) = f(0) + xf'(0) + \frac{x^2}{2}f''(\theta)$$

By (2), $f''(\theta) \leq 0$, thus $f(x) \leq f(0) + xf'(0)$.

(5) [5 points] Plugging in the expression obtained in (3) in the inequality of (4) gives:

$$\Pr\left[\frac{1}{m} \sum_{i=1}^m X_i \geq \epsilon\right] = \exp[-m\Phi(t)]$$

with $\Phi(t) = t\epsilon - (e^{ct} - 1 - ct)\frac{\sigma^2}{c^2}$. It is easy to see that:

$$\Phi'(t) \geq 0 \Leftrightarrow t \leq t_0 = \frac{1}{c} \log(1 + \frac{\epsilon c}{\sigma^2}).$$

Thus, t_0 is the optimal value.

- (6) Replacing t by t_0 leads directly to Bennett's inequality.
- (7) [5 points] It is sufficient to observe that: $\theta(0) = h(0) = 0$, $\theta'(0) = h'(0) = 0$, and $\forall x, \theta''(x) \geq h''(x)$.

$$\theta''(x) = \frac{1}{1+x} \text{ and } h''(x) = \frac{27}{(x+3)^3}$$

- (8) [5 points] When $E[X_i] = 0$ and $|X| \leq c$, Hoeffding's inequality (see also lemma proved in class) gives:

$$\Pr[\frac{1}{m} \sum_{i=1}^m X_i > \epsilon] \leq e^{-\frac{m\epsilon^2}{2c^2}}.$$

For smaller values of the variance, $\sigma^2 \ll c^2$, Bernstein's inequality is tighter.

2. Two-Oracle Variant of PAC model [60 points]

- [20 points] Assume that C is efficiently PAC-learnable using H in the standard PAC model using algorithm L . Consider the distribution $D = \frac{1}{2}(D_- + D_+)$. Let $h \in H$ be the hypothesis output by L . Choose δ such that:

$$\Pr[\text{error}_D(h) \leq \epsilon/2] \geq 1 - \delta.$$

From

$$\begin{aligned} \text{error}_D(h) &= \Pr_{x \sim D} [h(x) \neq c(x)] \\ &= \frac{1}{2} (\Pr_{x \sim D_-} [h(x) \neq c(x)] + \Pr_{x \sim D_+} [h(x) \neq c(x)]) \\ &= \frac{1}{2} (\text{error}_{D_-}(h) + \text{error}_{D_+}(h)), \end{aligned}$$

it follows that:

$$\Pr[\text{error}_{D_-}(h) \leq \epsilon] \geq 1 - \delta \quad \text{and} \quad \Pr[\text{error}_{D_+}(h) \leq \epsilon] \geq 1 - \delta.$$

This implies two-oracle PAC-learning with the same computational complexity.

- [40 points] Assume now that C is efficiently PAC-learnable in the two-oracle PAC model. Thus, there exists a learning algorithm L such that for $c \in C$, $\epsilon > 0$, and $\delta > 0$, there exist m_- and m_+ polynomial in $1/\epsilon$, $1/\delta$, and $\text{size}(c)$, such that if we draw m_- negative examples or more and m_+ positive examples or more, with confidence $1 - \delta$, the hypothesis h output by L verifies:

$$\Pr[\text{error}_{D_-}(h)] \leq \epsilon \quad \text{and} \quad \Pr[\text{error}_{D_+}(h)] \leq \epsilon.$$

Now, let D be a probability distribution over negative and positive examples. If we could draw m examples according to D such that $m \geq \max\{m_-, m_+\}$, m polynomial in $1/\epsilon$, $1/\delta$, and $\text{size}(c)$, then two-oracle PAC-learning would imply standard PAC-learning:

$$\begin{aligned} \Pr[\text{error}_D(h)] &\leq \Pr[\text{error}_D(h)|c(x) = 0] \Pr[c(x) = 0] + \\ &\Pr[\text{error}_D(h)|c(x) = 1] \Pr[c(x) = 1] \leq \\ &\epsilon(\Pr[c(x) = 0] + \Pr[c(x) = 1]) = \epsilon. \end{aligned}$$

If D is not too biased, that is if the probability of drawing a positive example, or that of drawing a negative example is more than ϵ , it is not hard to show, using Chernoff bounds or just Chebyshev's inequality, that drawing a polynomial number of examples in $1/\epsilon$ and $1/\delta$ suffices to guarantee that $m \geq \max\{m_-, m_+\}$ with high confidence.

Otherwise, D is biased towards negative (or positive examples), in which case returning $h = h_0$ (respectively $h = h_1$) guarantees that $\Pr[\text{error}_D(h)] \leq \epsilon$.

To show the claim about the not-too-biased case, let S_m denote the number of positive examples obtained when drawing m examples when the probability of a positive example is ϵ . By Chernoff bounds,

$$\Pr[S_m \leq (1 - \alpha)m\epsilon] \leq e^{-m\epsilon\alpha^2/2}.$$

We want to ensure that at least m_+ examples are found. With $\alpha = \frac{1}{2}$ and $m = \frac{2m_+}{\epsilon}$,

$$\Pr[S_m > m_+] \leq e^{-m_+/4}.$$

Setting the bound to be less than or equal to $\delta/2$, leads to the following condition on m :

$$m \geq \min\left\{\frac{2m_+}{\epsilon}, \frac{8}{\epsilon} \log \frac{2}{\delta}\right\}$$

A similar analysis can be done in the case of negative examples. Thus, when D is not too biased, with confidence $1 - \delta$, we will find at least m_- negative and m_+ positive examples if we draw m examples, with

$$m \geq \min\left\{\frac{2m_+}{\epsilon}, \frac{2m_-}{\epsilon}, \frac{8}{\epsilon} \log \frac{2}{\delta}\right\}.$$

Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 1
Due: January 31, 2006

Problem 1: Probability Review

Professor Mamoru teaches at a university whose computer science and math building has $F = 30$ floors.

- (1) Assume that the floors are independent and that they have the same probability to be selected by someone taking the elevator. How many people should take the elevator in order to make it likely (probability more than half) that two of them go to the same floor? [*Hint*: use the Taylor series expansion of $e^{-x} = 1 - x + \dots$ and give an approximate general expression of the solution.]
- (2) Professor Mamoru is popular and his floor is in fact more likely to be selected than others. Assuming that all other floors are equiprobable, derive the general expression of the probability that two persons go to the same floor, using the same approximation as before. How many people should take the elevator in order to make it likely that two of them go to the same floor when the probability of Professor Mamoru's floor is .25, .35, or .5? When $q = .5$, would the answer change if the number of floors were instead $F = 1000$?
- (3) The probability models assumed in (1) and (2) are both naive. If you had access to the data collected by the elevator guard, how would you define a more faithful model.

Problem 2: PAC Learning

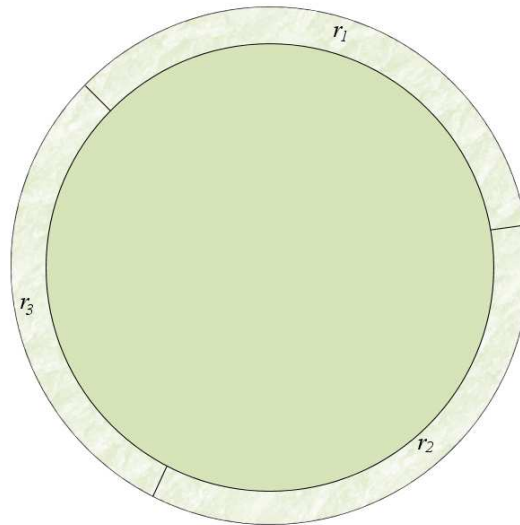
- (1) Concentric Circles

Let $X = \mathbb{R}^2$ and consider the set of concepts of the form $c = \{(x, y) : x^2 + y^2 \leq r^2\}$ for some real number r . Show that this class can be (ϵ, δ) -PAC-learned from training data of size $m \geq (1/\epsilon) \log(1/\delta)$.

- (2) Non-Concentric Circles

Let $X = \mathbb{R}^2$ and consider the set of concepts of the form $c = \{x \in \mathbb{R}^2 : \|x - x_0\| \leq r\}$ for some point $x_0 \in \mathbb{R}^2$ and real number r .

Gertrude, an aspiring machine learning researcher, attempts to show that this class of concepts may be (ϵ, δ) PAC-learned with sample complexity $m \geq (3/\epsilon) \log(3/\delta)$, but is having trouble with her proof. Her idea is that the learning algorithm would select the smallest circle consistent with the training data. She has drawn 3 regions r_1, r_2, r_3 around the edge of concept c , each of size $\epsilon/3$ in probability measure P (see the sketch below). She wants to argue that if the generalization error is $\geq \epsilon$, then one of these regions must have been missed by the training data, and hence this event will occur with probability at most δ . Can you tell Gertrude if her approach works?



Problem 1: Probability Review [50 points]

(1) [25 points]

The main result of this section is known as the standard *birthday paradox*. All floors have the same probability $\frac{1}{F}$ of being selected. Let $k \leq F$ be the number of people in the elevator and let $\bar{p}$ denote the probability that they all select a different floor. Fix the floor for the first person, then a distinct floor can be chosen for the second person with probability $(F - 1)/F$. Fixing the first two distinct floors, the floor for the third person can be chosen with probability $(F - 2)/F$ and so on. Thus, $\bar{p}$ is given by:

$$\bar{p} = 1 \cdot \frac{F-1}{F} \cdot \frac{F-2}{F} \cdots \frac{F-(k-1)}{F} = \prod_{i=0}^{k-1} \left(1 - \frac{i}{F}\right). \quad (1)$$

Using the approximation $e^{-x} \approx 1 - x$ which here holds for $k \ll F$,

$$\bar{p} \approx \prod_{i=0}^{k-1} e^{-\frac{i}{F}} = e^{-\sum_{i=0}^{k-1} \frac{i}{F}} = e^{-\frac{(k-1)k}{2F}}. \quad (2)$$

The right-hand side is smaller than $1/2$ when:

$$k^2 - k - 2 \log(2)F > 0. \quad (3)$$

that is for $k \geq \left\lceil \frac{1 + \sqrt{1 + 8 \log(2)F}}{2} \right\rceil$. For $F = 30$, it suffices that $k = 7$ for the probability that two people go the same floor to be more than half.

(2) [25 points]

Let q denote the probability of Professor Mamoru's floor. The other floors have the same probability r , thus $q + r(F - 1) = 1$. Let $\bar{p}$ be defined as before. $\bar{p} = \bar{p}_1 + \bar{p}_2$ where $\bar{p}_1$ is the probability that all k floors be distinct and different from that of Professor Mamoru, and $\bar{p}_2$ the probability that they all be distinct with one of them Professor

Mamoru's floor. $\bar{p}_1$ can be computed using the same reasoning as in the previous section:

$$\bar{p}_1 = r(F-1) \cdot r(F-2) \cdots r(F-(k-1)) = (1-q)^k \prod_{i=0}^{k-1} \left(1 - \frac{i}{F-1}\right). \quad (4)$$

To compute $\bar{p}_2$, observe that Professor Mamoru's floor can be chosen in k different ways among the k floors. The other $k-1$ floors must be chosen as before among the equiprobable floors, thus:

$$\bar{p}_2 = kq \cdot r(F-1) \cdots r(F-(k-2)) = kq(1-q)^{k-1} \prod_{i=0}^{k-2} \left(1 - \frac{i}{F-1}\right). \quad (5)$$

Thus, the general expression of the probability that two persons go to the same floor is $1 - \bar{p}$ with:

$$\begin{aligned} \bar{p} &= (1-q)^{k-1} \prod_{i=0}^{k-1} \left(1 - \frac{i}{F-1}\right) \left(k\left(q - \frac{1-q}{F-1}\right) + \frac{F}{F-1}(1-q)\right) \\ &\approx (1-q)^{k-1} e^{-\frac{(k-2)(k-1)}{2F}} \left(k\left(q - \frac{1-q}{F-1}\right) + \frac{F}{F-1}(1-q)\right) \end{aligned} \quad (6)$$

For $q = .25$, it suffices that $k = 5$ persons take the elevator, for $q = .35$, $k = 4$ persons, and for $q = .5$ only $k = 3$. For $q = .5$, the answer is the same for $F = 1000$.

Problem 2: PAC Learning [50 points]

[Thanks to Tyler Neylon for writing both the problem and the solution.]

In both solutions, our training data is the set T and our learned concept $L(T)$ is the tightest circle (with minimal radius) which is consistent with the data.

(1) [25 points] Concentric Circles

Suppose our target concept c is the circle around the origin with radius r . We will choose a slightly smaller radius s by

$$s := \inf\{s' : P(s' \leq \|x\| \leq r) < \epsilon\}.$$

Let A denote the annulus between radii s and r ; that is, $A := \{x : s \leq \|x\| \leq r\}$. By definition of s ,

$$P(A) \geq \epsilon. \quad (7)$$

In addition, our generalization error, $P(c \Delta L(T))$ must be small if T intersects A . We can state this as

$$P(c \Delta L(T)) > \epsilon \Rightarrow T \cap A = \emptyset. \quad (8)$$

Using (7), we know that any point in T chosen according to P will “miss” region A with probability at most $1 - \epsilon$. Defining $error := P(c \Delta L(T))$, we can combine this with (8) to see that

$$P(error > \epsilon) \leq P(T \cap A = \emptyset) \leq (1 - \epsilon)^m \leq e^{-m\epsilon}.$$

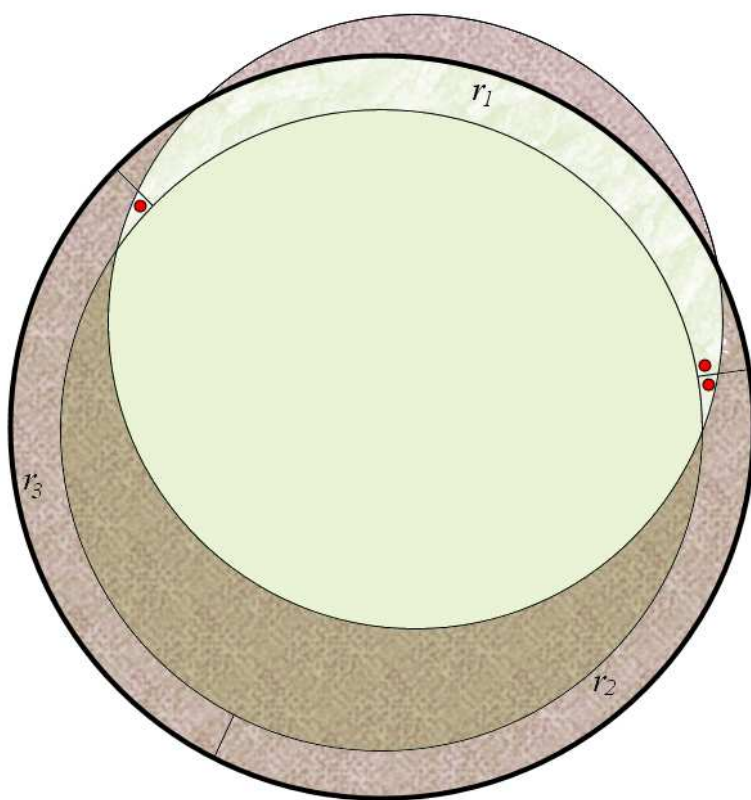
Setting δ to be greater than or equal to the right-hand side leads to $m \geq \frac{1}{\epsilon} \log(\frac{1}{\delta})$.

(2) [25 points] Non-Concentric Circles

As in the previous example, it is natural to assume the learning algorithm operates by returning the smallest circle which is consistent with the data. Gertrude is relying on the logical implication

$$error > \epsilon \Rightarrow T \cap r_i = \emptyset \text{ for some } i, \quad (9)$$

which is not necessarily true here. The figure illustrates a counterexample. In the figure, we have one training point in each region r_i . The points in r_1 and r_2 are very close together, and the point in r_3 is very close to region r_1 . On this training data (some other points may be included outside the three regions r_i), our learned circle is the “tightest” circle including these points, and hence one diameter approximately traverses the corners of r_1 . In the figure, the darkened areas are the error of this learned hypotheses versus the target circle which has a thick border. Clearly, the error may be greater than ϵ even while $T \cap r_i \neq \emptyset$ for any i ; this contradicts (9) and invalidates poor Gertrude’s proof.



Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 1
Due: February 6, 2007

A. Senate Laws

For important questions, President Mouth relies on expert advice. He selects an appropriate advisor from a collection of $H = 2800$ experts.

1. Assume that laws are proposed in a random fashion independently and identically according to some distribution D determined by an unknown group of senators. Assume that President Mouth can find and select an expert senator out of H who has consistently voted with the majority for the last $m = 200$ laws. Give a bound on the chances that such a senator incorrectly predicts the global vote for a future law. What is the value of the bound with 95% confidence?
2. Assume now that President Mouth can find and select an expert senator out of H who has consistently voted with the majority for all but $m' = 20$ of the last $m = 200$ laws. What is the value of the new bound?

B. PAC Learning of Hyper-rectangles

1. An axis-aligned hyper-rectangle in $\mathbb{R}^n$ is a set of the form $[a_1, b_1] \times \dots \times [a_n, b_n]$. Show that axis-aligned hyper-rectangles are PAC-learnable by extending the proof given in class for the case $n = 2$.

C. Bound Comparison

Let $X_1, \dots, X_m$ be a sequence of random variables taking values in $[0, 1]$ with the same mean μ and variance $\sigma^2 < \infty$ and let $\bar{X} = \frac{1}{m} \sum_{i=1}^m X_i$.

1. For any $\epsilon > 0$, give a bound on $\Pr[|\bar{X} - \mu| > \epsilon]$ using Chebyshev's inequality, then Hoeffding's inequality. For what values of σ is Chebyshev's inequality tighter?
2. Assume that the random variables X_i take values in $\{0, 1\}$. Show that $\sigma^2 \leq \frac{1}{4}$. Use this to simplify Chebyshev's inequality. Choose $\epsilon =$

.05 and plot Chebyshev's inequality thereby modified and Hoeffding's inequality as a function of m [you can use your favorite program for plotting this, e.g., matlab, R].

A. Senate Laws

[30 points]

1. [15 points] The true error in the consistent case is bounded as follows:

$$error_D(h) \leq \frac{1}{m}(\log |H| + \log \frac{1}{\delta}). \quad (1)$$

For $\delta = .05$, $m = 200$ and $H = 2800$, $error_D(h) \leq 5.5\%$.

2. [15 points] The true error in the inconsistent case is bounded as:

$$error_D(h) \leq \widehat{error}_D(h) + \sqrt{\frac{1}{2m}(\log 2|H| + \log \frac{1}{\delta})}. \quad (2)$$

For $\delta = .05$, $\widehat{error}_D(h) = m'/m = .1$, $m = 200$ and $H = 2800$, $error_D(h) \leq 27.05\%$.

B. PAC Learning of Hyper-rectangles

[30 points]

1. The proof in the case of hyper-rectangles is similar to the one given in class. The algorithm selects the tightest axis-aligned hyper-rectangle containing all the sample points. For $i \in [1, 2n]$, select a region r_i such that $\Pr_D[r_i] = \epsilon/(2n)$ for each edge of the hyper-rectangle R . Assuming that $\Pr_D[R - R'] > \epsilon$, argue that R' cannot meet all r_i s, so it must miss at least one. The probability that none of the m sample points falls into region r_i is $(1 - \epsilon/2n)^m$. By the union bound, this shows that

$$\Pr[error(R') > \epsilon] \leq 2n(1 - \epsilon/2n)^m \leq 2ne^{-\frac{\epsilon m}{2n}}. \quad (3)$$

Setting δ to the right-hand side shows that for

$$m \geq \frac{2n}{\epsilon} \log \frac{2n}{\delta}, \quad (4)$$

with probability at least $1 - \delta$, $error_D(R') \leq \epsilon$.

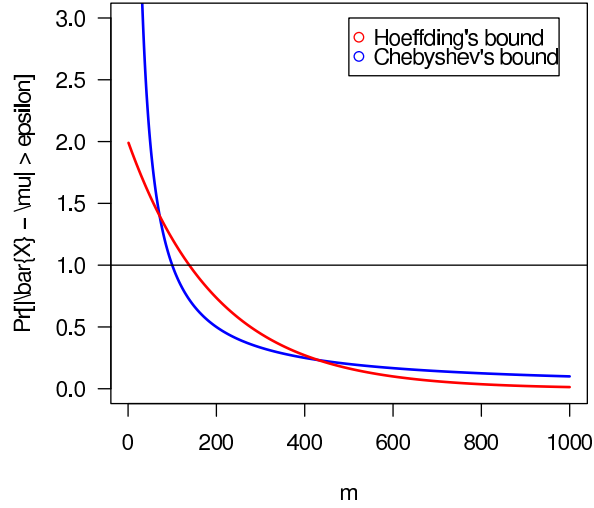


Figure 1: Comparison of Chebyshev's and Hoeffding's bound.

C. Bound Comparison

[40 points]

Let $X_1, \dots, X_m$ be a sequence of random variables taking values in $[0, 1]$ with the same mean μ and variance $\sigma^2 < \infty$ and let $\bar{X} = \frac{1}{m} \sum_{i=1}^m X_i$.

1. [20 points] Since the random variables X_i are independent, the variance of $\bar{X}$ is the sum of the variances: $\text{Var}[\bar{X}] = m(\sigma^2/m^2) = \sigma^2/m$. For any $\epsilon > 0$, using Chebyshev's inequality (see lecture 1),

$$\Pr[|\bar{X} - \mu| > \epsilon] \leq \frac{\sigma^2}{m\epsilon^2}. \quad (5)$$

Using Hoeffding's inequality,

$$\Pr[|\bar{X} - \mu| > \epsilon] \leq 2e^{-2m\epsilon^2}. \quad (6)$$

Thus, Chebyshev's inequality is tighter for $\frac{\sigma^2}{m\epsilon^2} < 2e^{-2m\epsilon^2}$, that is for $\sigma \leq \sqrt{(2m\epsilon^2) e^{-2m\epsilon^2}}$.

2. [20 points] When X_i takes values in $\{0, 1\}$, the variance of X_i is given by

$$\sigma^2 = E[X_i^2] - E[X_i]^2 = E[X_i] - E[X_i]^2, \quad (7)$$

since $X_i^2 = X_i$. For $\mu \in [0, 1]$, the function $\mu \mapsto \mu(1 - \mu)$ reaches its maximum for $\mu = \frac{1}{2}$. Thus, $\sigma^2 \leq \frac{1}{4}$. Chebyshev's inequality can then be simplified into:

$$\Pr[|\bar{X} - \mu| > \epsilon] \leq \frac{1}{4m\epsilon^2}, \quad (8)$$

The two bounds are approximately equal for $m\epsilon^2 \approx 1.075$.

Figure 1 plots these inequalities for $\epsilon = .05$. Both bounds are vacuous for values of m less than 100. Chebyshev's inequality is tighter for $m < m_0 \approx 1.075/\epsilon^2 = 430$, Hoeffding's inequality tighter for larger values of m .

Mehryar Mohri
 Foundations of Machine Learning
 Courant Institute of Mathematical Sciences
 Homework assignment 1
 Due: Feb 15th, 2008
 Credit: Ashish Rastogi, Afshin Rostamizadeh
 Ameet Talwalkar, and Eugene Weinstein.

1. Probability Review:

- (a) Imagine you are given one fair die, and you need to decide which task is harder: (i) guessing the value of one die toss or (ii) tossing the die twice and getting the same value twice. Given that the die is fair (every side has weight $1/6$), does event (i) have a greater chance of success or event (ii), or do they have the same probability of success? Make sure to give justification.
- (b) We will now generalize this result to n -sided dice with any (possibly non-uniform) distribution. First prove the following useful fact, for any $\alpha_1, \alpha_2, \dots, \alpha_n$ such that $\sum_i \alpha_i = 1$, the following holds,

$$0 \leq \sum_{i=1}^n (\alpha_i - 1/n)^2 = \sum_{i=1}^n \alpha_i^2 - 1/n$$

- (c) Let X_1 be the value of the first toss, and X_2 be the value of the second toss. Show that $\Pr(X_1 = X_2) \geq 1/n$ (hint: use part b). For what distribution is the inequality tight?

2. Concentration Bounds:

- (a) Given a sample of m bounded points $X = (x_1, x_2, \dots, x_m)$, $\forall i, |x_i| \leq M$, define the function

$$f(X) = \frac{1}{m} \sum_i x_i.$$

Can you give a bound on the probability $\Pr[|f(X) - \mathbb{E}[f(X)]| \geq \epsilon]$?

- (b) Let X and X' be two sets of size m that differ in exactly one point. That is, $|X \cap X'| = m - 1$. We say a function h is *stable* if for all such X, X' , $|h(X) - h(X')| \leq g(m)$ for some decreasing function g . How quickly does g need to decrease as a function of

m in order for McDiarmid's inequality to provide a bound on the event $\Pr[|h(X) - \mathbb{E}[h(X)]| \geq \epsilon]$ that converges to zero as $m \rightarrow \infty$?

- (c) Is the function f from part (a) stable (still assuming the bound $|x_i| \leq M, \forall i$)? Will McDiarmid's inequality provide a convergent bound? If so give the bound. Now define the function $f'(X) = \max(X)$, is f' stable? Can you give a bound with McDiarmid's inequality?

3. **PAC Learning:** Here we will consider an alternative PAC learning scenario, called the two-oracle model. Imagine you are given the ability to explicitly ask for a positive or negative sample, which are drawn from different distributions D_+ and D_- respectively. A concept is efficiently PAC-learnable if there exists an algorithm L that can generate a hypothesis h , such that $\Pr_{x \sim D_+}[h(x) = 0] \leq \epsilon$ and $\Pr_{x \sim D_-}[h(x) = 1] \leq \epsilon$ with confidence $(1 - \delta)$, after sampling $m = \text{poly}(1/\epsilon, 1/\delta)$ points.

- (a) Show that if a problem is efficiently PAC-learnable in the classic sense, it is also always efficiently PAC-learnable in the two-oracle model.
- (b) (Bonus) Show that the reverse direction is also true.

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 1 - solution
Credit: Ashish Rastogi, Afshin Rostamizadeh
Ameet Talwalkar, and Eugene Weinstein.

1. **Probability Review** [30 points]:

- (a) [10 points] Imagine you are given one fair die, and you need to decide which task is harder: (i) guessing the value of one die toss or (ii) tossing the die twice and getting the same value twice. Given that the die is fair (every side has weight $1/6$), does event (i) have a greater chance of success or event (ii), or do they have the same probability of success? Make sure to give justification.

Solution: Successfully guessing the outcome of either event is equally likely. Clearly, guessing the value of a single fair toss is $1/6$. To see that the probability of rolling the same value twice, let X_1 denote the outcome of the first toss and X_2 denote the value of the second toss. Then, we are interested in the value of $\Pr(X_1 = X_2) = \sum_{i=1}^6 \Pr(X_1 = i \wedge X_2 = i)$. Notice each toss is independent and identical, so we can write

$$\begin{aligned} \sum_{i=1}^6 \Pr(X_1 = i \wedge X_2 = i) &= \sum_{i=1}^6 \Pr(X_1 = i) \Pr(X_2 = i) \\ &= \sum_{i=1}^6 1/6 \cdot 1/6 \\ &= 1/6 \end{aligned}$$

- (b) [5 points] We will now generalize this result to n -sided dice with any (possibly non-uniform) distribution. First prove the following useful fact, for any $\alpha_1, \alpha_2, \dots, \alpha_n$ such that $\sum_i \alpha_i = 1$, the following holds,

$$0 \leq \sum_{i=1}^n (\alpha_i - 1/n)^2 = \sum_{i=1}^n \alpha_i^2 - 1/n$$

Solution: The inequality is true, because a sum of squares is always positive. To show the equality, we simply expand the terms and use the fact $\sum_i \alpha_i = 1$.

$$\begin{aligned}
 \sum_{i=1}^n (\alpha_i - 1/n)^2 &= \sum_{i=1}^n (\alpha_i^2 - \frac{2}{n} \alpha_i + 1/n^2) \\
 &= \sum_{i=1}^n \alpha_i^2 - \frac{2}{n} \sum_{i=1}^n \alpha_i + \sum_{i=1}^n 1/n^2 \\
 &= \sum_{i=1}^n \alpha_i^2 - 2/n + 1/n \\
 &= \sum_{i=1}^n \alpha_i^2 - 1/n
 \end{aligned}$$

- (c) [15 points] Let X_1 be the value of the first toss, and X_2 be the value of the second toss. Show that $\Pr(X_1 = X_2) \geq 1/n$ (hint: use part b). For what distribution is the inequality tight?

Solution: Generalizing part (a) in a straight-forward manner, we get

$$\begin{aligned}
 \Pr(X_1 = X_2) &= \sum_{i=1}^n \Pr(X_1 = i \wedge X_2 = i) \\
 &= \sum_{i=1}^n \Pr(X_1 = i) \Pr(X_2 = i) \quad (\text{independent}) \\
 &= \sum_{i=1}^n \Pr(X_1 = i)^2 \quad (\text{and identical})
 \end{aligned}$$

Notice that, $\sum_{i=1}^n \Pr(X_1 = i) = 1$ by definition, so we can think of $\Pr(X_1 = i) = \alpha_i$. From part (b), we know that $\sum_{i=1}^n \alpha_i^2 \geq 1/n$. As seen in part (a), the uniform distribution ($\alpha_i = 1/n$) achieves equality.

2. Concentration Bounds [30 points]:

- (a) [10 points] Given a sample of m bounded points $X = (x_1, x_2, \dots, x_m)$, $\forall i, |x_i| \leq M$, define the function

$$f(X) = \frac{1}{m} \sum_i x_i.$$

Can you give a bound on the probability $\Pr[|f(X) - \mathbb{E}[f(X)]| \geq \epsilon]$?

Solution: This is simply a straight-forward application of Hoeffding's inequality. We can think of our function f as the sum of random variables x_i/m , and we know the value of each variable is bounded by $2M/m$. Hoeffding's inequality give the following bound,

$$\begin{aligned} \Pr[|f(X) - \mathbb{E}[f(X)]| > \epsilon] &\leq 2 \exp\left(\frac{-2\epsilon^2}{\sum_{i=1}^m (2M/m)^2}\right) \\ &= 2 \exp\left(\frac{-2\epsilon^2 m}{4M^2}\right) \end{aligned}$$

- (b) [10 points] Let X and X' be two sets of size m that differ in exactly one point. That is, $|X \cap X'| = m - 1$. We say a function h is *stable* if for all such X, X' , $|h(X) - h(X')| \leq g(m)$ for some decreasing function g . How quickly does g need to decrease as a function of m in order for McDiarmid's inequality to provide a bound on the event $\Pr[|h(X) - \mathbb{E}[h(X)]| \geq \epsilon]$ that converges to zero as $m \rightarrow \infty$?

Solution: In order for McDiarmid's inequality to converge, we need $g(m) \in o(1/\sqrt{m})$. In the case $g(m) = 1/m^{1/2+\delta}$, we can apply McDiarmid's inequality with each $c_i = 1/m^{1/2+\delta}$,

$$\begin{aligned} \Pr[|h(X) - \mathbb{E}_X[h(X)]| \geq \epsilon] &\leq 2 \exp\left(\frac{-2\epsilon^2}{\sum_{i=1}^m (m^{-1/2}m^{-\delta})^2}\right) \\ &= 2 \exp\left(-2\epsilon^2 m^{2\delta}\right) \end{aligned}$$

Clearly we need $\delta > 0$ in order for the bound to converge to zero as m tends to infinity.

- (c) [10 points] Is the function f from part (a) stable (still assuming the bound $|x_i| \leq M, \forall i$)? Will McDiarmid's inequality provide a convergent bound? If so give the bound. Now define the function $f'(X) = \max(X)$, is f' stable? Can you give a bound with McDiarmid's inequality?

Solution: The function from part (a) is stable, with $g(m) = 2M/m$ (changing a single bounded point, will change the average by at most $2M/m$). Indeed, from part (b), we can reason that McDiarmid's inequality, with $c_i = 2M/m$ will give a convergent bound. In fact, in this case, the bound is exactly the same as the one given by Hoeffding's inequality. Here we see that McDiarmid's inequality generalized Hoeffding's.

The function f' is not stable, the only bound that we can give is $g(m) \leq M$ (without assuming anything else about the distribution). We cannot get a useful bound with McDiarmid's inequality.

3. **PAC Learning** [40 points + 20 points]: Here we will consider an alternative PAC learning scenario, called the two-oracle model. Imagine you are given the ability to explicitly ask for a positive or negative sample, which are drawn from different distributions D_+ and D_- respectively. A concept is efficiently PAC-learnable if there exists an algorithm L that can generate a hypothesis h , such that $\Pr_{x \sim D_+}[h(x) = 0] \leq \epsilon$ and $\Pr_{x \sim D_-}[h(x) = 1] \leq \epsilon$ with confidence $(1 - \delta)$, after sampling $m = \text{poly}(1/\epsilon, 1/\delta)$ points.

- (a) [40 points] Show that if a problem is efficiently PAC-learnable in the classic sense, it is also always efficiently PAC-learnable in the two-oracle model.

Solution: Let c be the true concept, then notice that

$$\begin{aligned} \text{error}(h) &= \Pr_x[h(x) \neq c(x)] \\ &= \Pr_x[h(x) = 1 \wedge c(x) = 0] + \Pr_x[h(x) = 0 \wedge c(x) = 1] \\ &= \Pr_x[h(x) = 1 | c(x) = 0] \Pr_x[c(x) = 0] + \\ &\quad \Pr_x[h(x) = 0 | c(x) = 1] \Pr_x[c(x) = 1]. \end{aligned}$$

Let $0 < \epsilon < 1/2$, then by assumption we know there exists an algorithm L that will efficiently produce a hypothesis h , such that $\text{error}(h) \leq \epsilon/2$ with confidence $1 - \delta$. From the above series of equalities this implies,

$$\begin{aligned} \Pr_x[h(x) = 1 | c(x) = 0] \Pr_x[c(x) = 0] + \\ \Pr_x[h(x) = 0 | c(x) = 1] \Pr_x[c(x) = 1] \leq \epsilon/2. \quad (1) \end{aligned}$$

Thus, in the two-oracle model, we can simulate the classic scenario by simply treating points from either distribution D_+ or D_- as coming from the same underlying distribution. If we draw points uniformly from the negative and positive oracle we will have $\Pr_x[c(x) = 0] = \Pr_x[c(x) = 1] = 1/2$, which would imply $\Pr_x[h(x) = 1|c(x) = 0] = \Pr_{x \sim D_-}[h(x) = 1] \leq \epsilon$, and similarly $\Pr_x[h(x) = 0|c(x) = 1] = \Pr_{x \sim D_+}[h(x) = 0] \leq \epsilon$.

- (b) [20 points] (Bonus) Show that the reverse direction is also true.

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 1
Due: February 17, 2009

A. Concentration Bounds

1. Let X be a non-negative random variable verifying $\Pr[X > t] \leq ce^{-2mt^2}$ for all $t > 0$ and some $c > 0$. Show that $\mathbb{E}[X^2] \leq \frac{\log(ce)}{2m}$ [*Hint*: to do that, use the identity $\mathbb{E}[X^2] = \int_0^\infty \Pr[X^2 > t]dt$, write $\int_0^\infty = \int_0^u + \int_u^\infty$, bound the first term by u and find the best u to minimize the upper bound].
2. Let $S = (X_1, \dots, X_m)$ be a sample of size m . Consider the function Φ defined by $\Phi(X_1, \dots, X_m) = \sup_{h \in H} |\text{error}(h) - \widehat{\text{error}}(h)|$. Apply McDiarmid's inequality to Φ . Does the result depend on the VC dimension of H ?

B. PAC Learning

1. Show that equilateral triangles with a base parallel to the X-axis are PAC-learnable. Give an algorithm and careful justifications using the proof given in class. What is the VC dimension of this concept class?
2. Give a PAC-learning algorithm for the subsets of the real line formed by the union of k intervals. What is the VC dimension of this concept class?

C. VC dimension

1. Show that the VC dimension of a finite hypothesis set H is at most $\log_2 |H|$.
2. What is the VC-dimension of the set of subsets I_α of the real line parameterized by a single parameter α : $I_\alpha = [\alpha, \alpha + 1] \cup [\alpha + 2, +\infty[$.
3. What is the VC dimension of the set of all ellipsoids in $\mathbb{R}^n$?

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 1
Due: February 22, 2010

A. PAC Learning

- Let $X = \mathbb{R}^2$ with orthonormal basis (e_1, e_2) and consider the set of concepts defined by the area inside a right triangle ABC with two sides parallel to the axes, with $\overrightarrow{AB}/AB = e_1$ and $\overrightarrow{AC}/AC = e_2$, and $AB/AC = \alpha$ for some positive real $\alpha \in \mathbb{R}_+$. Show, using similar methods to those used in the lecture slides for the axis-aligned rectangles, that this class can be (ϵ, δ) -PAC-learned from training data of size $m \geq (3/\epsilon) \log(3/\delta)$.

B. Biased coins

- Professor Moent has two coins in his pocket, coin x_A and coin x_B , both slightly biased as follows:

$$\Pr[x_A = 0] = 1/2 - \epsilon/2 \quad \Pr[x_B = 0] = 1/2 + \epsilon/2,$$

where $0 < \epsilon < 1$ is a small positive number and where 0 denotes head and 1 tail. He likes to play the following game with his students. He randomly picks a coin $x \in \{x_A, x_B\}$ from his pocket, tosses it m times, then reveals the sequence of 0s and 1s he obtained and asks which coin was tossed.

The goal of this exercise is to determine how large m needs to be for a student's coin prediction error to be at most $\delta > 0$.

1. Let S be a sample of size m . Professor Moent's best student, Oskar, plays according to the decision rule $f_o: \{0, 1\}^m \rightarrow \{x_A, x_B\}$ defined by $f_o(S) = x_A$ iff $N(S) < m/2$, where $N(S)$ is the number of 0's in sample S .

Suppose m is even, then show that

$$\text{error}(f_o) \geq \frac{1}{2} \Pr \left[N(S) \geq \frac{m}{2} \mid x = x_A \right]. \quad (1)$$

2. Assuming m even, use the inequalities given in the appendix to show that

$$\text{error}(f_o) > \frac{1}{4} \left[1 - \left[1 - e^{-\frac{m\epsilon^2}{1-\epsilon^2}} \right]^{\frac{1}{2}} \right]. \quad (2)$$

3. Argue that if m is odd, the probability can be lower bounded by the one for $m+1$ and conclude that for both odd and even m ,

$$\text{error}(f_o) > \frac{1}{4} \left[1 - \left[1 - e^{-\frac{2\lceil m/2 \rceil \epsilon^2}{1-\epsilon^2}} \right]^{\frac{1}{2}} \right]. \quad (3)$$

4. Using this bound, how large should m be for Oskar's error to be at most δ , where $0 < \delta < 1/4$. What is the asymptotic behavior of this lower bound as a function of ϵ ?
5. Show that no decision rule $f: \{0, 1\}^m \rightarrow \{x_a, x_B\}$ can do better than Oskar's rule f_o . Conclude that the lower bound of the previous question applies to all rules.

Appendix

0.1 Binomial inequality

Let B be a binomial (m, p) random variable with $p \leq 1/2$. Then, for $mp \leq k \leq m(1-p)$,

$$\Pr[B \geq k] \geq \Pr \left[N \geq \frac{k - mp}{\sqrt{mp(1-p)}} \right], \quad (4)$$

where N is in standard normal form.

0.2 Tail bound

If N is a random variable following the standard normal distribution, then for $u > 0$,

$$\Pr[N \geq u] \geq \frac{1}{2} \left(1 - \sqrt{1 - e^{-u^2}} \right). \quad (5)$$

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Solution assignment 1
Due: February 22, 2010

A. PAC Learning

- Let $X = \mathbb{R}^2$ with orthonormal basis (e_1, e_2) and consider the set of concepts defined by the area inside a right triangle ABC with two sides parallel to the axes, with $\overrightarrow{AB}/AB = e_1$ and $\overrightarrow{AC}/AC = e_2$, and $AB/AC = \alpha$ for some positive real $\alpha \in \mathbb{R}_+$. Show, using similar methods to those used in the lecture slides for the axis-aligned rectangles, that this class can be (ϵ, δ) -PAC-learned from training data of size $m \geq (3/\epsilon) \log(3/\delta)$.

As in the case of axis-aligned rectangles, consider three regions r_1, r_2, r_3 , along the sides of the target concept as indicated in Figure 1. Note that the triangle formed by the points A'', B'', C'' is similar to ABC (same angles) since $A''B''$ must be parallel to AB , and similarly for the other sides.

Assume that $\Pr[ABC] > \epsilon$, otherwise the statement would be trivial. Consider a triangle $A'B'C'$ similar to ABC and consistent with the training sample and such that it meets all three regions r_1, r_2, r_3 .

Since it meets r_1 , the line $A'B'$ must be below $A''B''$. Since it meets r_2 and r_3 , A' must be in r_2 and B' in r_3 (see Figure 1). Now, since the angle $\widehat{A'B'C'}$ is equal to $\widehat{A''B''C''}$, C' must be necessarily above C'' . This implies that triangle $A'B'C'$ contains $A''B''C''$ and thus $\text{error}(A'B'C') \leq \epsilon$.

$$\text{error}(A'B'C') > \epsilon \implies \exists i \in \{1, 2, 3\} : A'B'C' \cap r_i = \emptyset.$$

Thus, by the union bound,

$$\Pr[\text{error}(A'B'C') > \epsilon] \leq \sum_{i=1}^3 \Pr[A'B'C' \cap r_i = \emptyset] \leq 3(1 - \epsilon/3)^m \leq 3e^{-3m\epsilon}.$$

Setting δ to match the right-hand side gives the sample complexity $m \geq \frac{3}{\epsilon} \log \frac{3}{\delta}$.

B. Biased coins [based on theorem of (Anthony and Bartlett, 1999)]

- Professor Moent has two coins in his pocket, coin x_A and coin x_B , both slightly biased as follows:

$$\Pr[x_A = 0] = 1/2 - \epsilon/2 \quad \Pr[x_B = 0] = 1/2 + \epsilon/2,$$

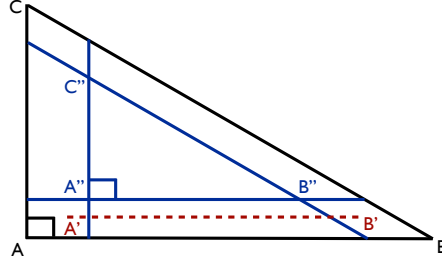


Figure 1: Rectangle triangles.

where $0 < \epsilon < 1$ is a small positive number and where 0 denotes head and 1 tail. He likes to play the following game with his students. He randomly picks a coin $x \in \{x_A, x_B\}$ from his pocket (uniformly), tosses it m times, then reveals the sequence of 0s and 1s he obtained and asks which coin was tossed.

The goal of this exercise is to determine how large m needs to be for a student's coin prediction error to be at most $\delta > 0$.

1. Let S be a sample of size m . Professor Moent's best student, Oskar, plays according to the decision rule $f_o: \{0, 1\}^m \rightarrow \{x_A, x_B\}$ defined by $f_o(S) = x_A$ iff $N(S) < m/2$, where $N(S)$ is the number of 0's in sample S .

Suppose m is even, then show that

$$\text{error}(f_o) \geq \frac{1}{2} \Pr \left[N(S) \geq \frac{m}{2} \mid x = x_A \right]. \quad (1)$$

By definition of the error of Oskar's prediction rule,

$$\begin{aligned} \text{error}(f_o) &= \Pr[f_o(S) \neq x] \\ &= \Pr[f_o(S) = x_A \wedge x = x_B] + \Pr[f_o(S) = x_B \wedge x = x_A] \\ &= \Pr \left[N(S) < \frac{m}{2} \mid x = x_B \right] \Pr[x = x_B] + \\ &\quad \Pr \left[N(S) \geq \frac{m}{2} \mid x = x_A \right] \Pr[x = x_A] \\ &= \frac{1}{2} \Pr \left[N(S) < \frac{m}{2} \mid x = x_B \right] + \frac{1}{2} \Pr \left[N(S) \geq \frac{m}{2} \mid x = x_A \right] \\ &\geq \frac{1}{2} \Pr \left[N(S) \geq \frac{m}{2} \mid x = x_A \right]. \end{aligned}$$

2. Assuming m even, use the inequalities given in the appendix to show that

$$\text{error}(f_o) > \frac{1}{4} \left[1 - \left[1 - e^{-\frac{m\epsilon^2}{1-\epsilon^2}} \right]^{\frac{1}{2}} \right]. \quad (2)$$

Note that $\Pr[N(S) \geq \frac{m}{2} | x = x_A] = \Pr[B(m, p) \geq k]$, with $p = 1/2 - \epsilon/2$, $k = \frac{m}{2}$, and $mp \leq k \leq m(1-p)$. Thus, by the binomial inequality of the Appendix,

$$\text{error}(f_o) \geq \frac{1}{2} \Pr \left[N \geq \frac{m\epsilon/2}{\sqrt{1/4(1-\epsilon^2)m}} \right] = \frac{1}{2} \Pr \left[N \geq \frac{\sqrt{m}\epsilon}{\sqrt{1-\epsilon^2}} \right].$$

Using the second inequality of the Appendix, we now obtain

$$\text{error}(f_o) \geq \frac{1}{4} \left(1 - \sqrt{1 - e^{-u^2}} \right),$$

with $u = \frac{\sqrt{m}\epsilon}{\sqrt{1-\epsilon^2}}$, which coincides with (2).

3. Argue that if m is odd, the probability can be lower bounded by the one for $m+1$ and conclude that for both odd and even m ,

$$\text{error}(f_o) > \frac{1}{4} \left[1 - \left[1 - e^{-\frac{2\lceil m/2 \rceil \epsilon^2}{1-\epsilon^2}} \right]^{\frac{1}{2}} \right]. \quad (3)$$

If m is odd, since $\Pr \left[N(S) \geq \frac{m}{2} | x = x_A \right] \geq \Pr \left[N(S) \geq \frac{m+1}{2} | x = x_A \right]$, we can use the lower bound

$$\text{error}(f_o) \geq \frac{1}{2} \Pr \left[N(S) \geq \frac{m+1}{2} | x = x_A \right].$$

Thus, in both cases we can use the lower bound expression with $\lceil m/2 \rceil$ instead of $m/2$.

4. Using this bound, how large must m be if Oskar's error is at most δ , where $0 < \delta < 1/4$. What is the asymptotic behavior of this lower bound as a function of ϵ ?

If $\text{error}(f_o)$ is at most δ , then $\frac{1}{4} \left[1 - \left[1 - e^{-\frac{2\lceil m/2 \rceil \epsilon^2}{1-\epsilon^2}} \right]^{\frac{1}{2}} \right] < \delta$, which gives

$$e^{-\frac{2\lceil m/2 \rceil \epsilon^2}{1-\epsilon^2}} < 1 - (1 - 4\delta)^2 = 4\delta(2 - 4\delta) = 8\delta(1 - 2\delta),$$

and

$$m > 2 \left\lceil \frac{1-\epsilon^2}{2\epsilon^2} \log \frac{1}{8\delta(1-2\delta)} \right\rceil.$$

The lower bound varies as $\frac{1}{\epsilon^2}$.

5. Show that no decision rule $f: \{0, 1\}^m \rightarrow \{x_a, x_B\}$ can do better than Oskar's rule f_o . Conclude that the lower bound of the previous question applies to all rules.

Let f be an arbitrary rule and denote by F_A the set of samples for which $f(S) = x_A$ and by F_B the complement. Then, by definition of the error,

$$\begin{aligned} \text{error}(f) &= \sum_{S \in F_A} \Pr[S \wedge x_B] + \sum_{S \in F_B} \Pr[S \wedge x_A] \\ &= \frac{1}{2} \sum_{S \in F_A} \Pr[S|x_B] + \frac{1}{2} \sum_{S \in F_B} \Pr[S|x_A] \\ &= \frac{1}{2} \sum_{\substack{S \in F_A \\ N(S) < m/2}} \Pr[S|x_B] + \frac{1}{2} \sum_{\substack{S \in F_A \\ N(S) \geq m/2}} \Pr[S|x_B] + \\ &\quad \frac{1}{2} \sum_{\substack{S \in F_B \\ N(S) < m/2}} \Pr[S|x_A] + \frac{1}{2} \sum_{\substack{S \in F_B \\ N(S) \geq m/2}} \Pr[S|x_A]. \end{aligned}$$

Now, if $N(S) \geq m/2$, clearly $\Pr[S|x_B] \geq \Pr[S|x_A]$. Similarly, if $N(S) < m/2$, clearly $\Pr[S|x_A] \geq \Pr[S|x_B]$. In view of these inequalities, $\text{error}(f)$ can be lower bounded as follows

$$\begin{aligned} \text{error}(f) &\geq \frac{1}{2} \sum_{\substack{S \in F_A \\ N(S) < m/2}} \Pr[S|x_B] + \frac{1}{2} \sum_{\substack{S \in F_A \\ N(S) \geq m/2}} \Pr[S|x_A] + \\ &\quad \frac{1}{2} \sum_{\substack{S \in F_B \\ N(S) < m/2}} \Pr[S|x_B] + \frac{1}{2} \sum_{\substack{S \in F_B \\ N(S) \geq m/2}} \Pr[S|x_A] \\ &= \frac{1}{2} \sum_{S: N(S) < m/2} \Pr[S|x_B] + \frac{1}{2} \sum_{S: N(S) \geq m/2} \Pr[S|x_A] \\ &= \text{error}(f_o). \end{aligned}$$

Oskar's rule is known as the maximum likelihood solution.

Appendix

0.1 Binomial inequality

Let B be a binomial (m, p) random variable with $p \leq 1/2$. Then, for $mp \leq k \leq m(1-p)$,

$$\Pr[B \geq k] \geq \Pr \left[N \geq \frac{k - mp}{\sqrt{mp(1-p)}} \right], \quad (4)$$

where N is in standard normal form.

0.2 Tail bound

If N is a random variable following the standard normal distribution, then for $u > 0$,

$$\Pr[N \geq u] \geq \frac{1}{2} \left(1 - \sqrt{1 - e^{-u^2}} \right). \quad (5)$$

Mehryar Mohri
 Foundations of Machine Learning 2014
 Courant Institute of Mathematical Sciences
 Homework assignment 1
 September 17, 2014
 Due: September 30, 2014

A. PAC learning of n -dimensional rectangles

Give a PAC-learning algorithm for C , the set of axis-aligned n -dimensional rectangles in $\mathbb{R}^n$, that is $C = \{[a_1, b_1] \times \cdots \times [a_n, b_n] : a_i, b_i \in \mathbb{R}\}$. You should give a careful proof similar to what was given in class for axis-aligned rectangles (case $n = 2$). How does the sample complexity vary as a function of n ?

B. Rademacher complexity of regularized neural networks

Let the input space be $X = \mathbb{R}^{n_1}$. In this problem, we consider the family of regularized neural networks defined by the following set of functions mapping X to $\mathbb{R}$:

$$\mathcal{H} = \left\{ \mathbf{x} \mapsto \sum_{j=1}^{n_2} w_j \sigma(\mathbf{u}_j \cdot \mathbf{x}) : \|\mathbf{w}\|_1 \leq \Lambda', \|\mathbf{u}_j\|_2 \leq \Lambda, \forall j \in [1, n_2] \right\},$$

where σ is an L -Lipschitz function. As an example, σ could be the sigmoid function which is 1-Lipschitz.

1. Show that $\widehat{\mathfrak{R}}_S(\mathcal{H}) = \frac{\Lambda'}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{\|\mathbf{u}\|_2 \leq \Lambda} \left| \sum_{i=1}^m \sigma_i \sigma(\mathbf{u} \cdot \mathbf{x}_i) \right| \right]$.
2. Use the following form of Talagrand's lemma valid for all hypothesis sets H and L -Lipschitz function Φ :

$$\frac{1}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h \in H} \left| \sum_{i=1}^m \sigma_i (\Phi \circ h)(x_i) \right| \right] \leq \frac{L}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h \in H} \left| \sum_{i=1}^m \sigma_i h(x_i) \right| \right],$$

to upper bound $\widehat{\mathfrak{R}}_S(\mathcal{H})$ in terms of the empirical Rademacher complexity of $\mathcal{H}'$, where $\mathcal{H}'$ is defined by

$$\mathcal{H}' = \{ \mathbf{x} \mapsto s(\mathbf{u} \cdot \mathbf{x}) : \|\mathbf{u}\|_2 \leq \Lambda, s \in \{-1, +1\} \}.$$

3. Use the Cauchy-Schwarz inequality to show that

$$\hat{\mathfrak{R}}_S(\mathcal{H}') = \frac{\Lambda}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\left\| \sum_{i=1}^m \sigma_i \mathbf{x}_i \right\|_2 \right].$$

4. Use the inequality $\mathbb{E}[\|\mathbf{X}\|_2] \leq \sqrt{\mathbb{E}[\|\mathbf{X}\|_2^2]}$, which holds by Jensen's inequality to upper bound $\hat{\mathfrak{R}}_S(\mathcal{H}')$.
5. Assume that for all $\mathbf{x} \in S$, $\|\mathbf{x}\|_2 \leq r$ for some $r > 0$. Use the previous questions to derive an upper bound on the Rademacher complexity of $\mathcal{H}$ in terms of r .

Mehryar Mohri
 Foundations of Machine Learning 2014
 Courant Institute of Mathematical Sciences
 Homework assignment 1
 September 17, 2014
 Due: September 30, 2014

A. PAC learning of n -dimensional rectangles

Give a PAC-learning algorithm for C , the set of axis-aligned n -dimensional rectangles in $\mathbb{R}^n$, that is $C = \{[a_1, b_1] \times \cdots \times [a_n, b_n] : a_i, b_i \in \mathbb{R}\}$. You should give a careful proof similar to what was given in class for axis-aligned rectangles (case $n = 2$). How does the sample complexity vary as a function of n ?

Solution: We let R' be the smallest n -dimensional rectangle consistent with the given sample. The proof is similar to the one given in class for rectangles in the plane except that here we need to consider $2n$ regions r_i , $i \in [1, 2n]$, along each face of the target n -dimensional rectangle with $\Pr[r_i] \geq \frac{\epsilon}{2n}$ and with $\Pr[r_i - f_i] < \frac{\epsilon}{2n}$ where f_i is the internal face of r_i . Arguing as in the proof given in class, assuming that $\Pr[R] > \epsilon$, if $\Pr[R(R') > \epsilon]$ then R' must miss at least one region r_i . The probability that none of the m sample points falls into region r_i is $(1 - \epsilon/2n)^m$. By the union bound, this shows that

$$\Pr[R(R') > \epsilon] \leq 2n(1 - \epsilon/2n)^m \leq 2ne^{-\frac{\epsilon m}{2n}}. \quad (1)$$

Setting δ to the right-hand side shows that for

$$m \geq \frac{2n}{\epsilon} \log \frac{2n}{\delta}, \quad (2)$$

with probability at least $1 - \delta$, $R(R') \leq \epsilon$.

B. Rademacher complexity of regularized neural networks

Let the input space be $X = \mathbb{R}^{n_1}$. In this problem, we consider the family of regularized neural networks defined by the following set of functions mapping X to $\mathbb{R}$:

$$\mathcal{H} = \left\{ \mathbf{x} \mapsto \sum_{j=1}^{n_2} w_j \sigma(\mathbf{u}_j \cdot \mathbf{x}) : \|\mathbf{w}\|_1 \leq \Lambda', \|\mathbf{u}_j\|_2 \leq \Lambda, \forall j \in [1, n_2] \right\},$$

where σ is an L -Lipschitz function. As an example, σ could be the sigmoid function which is 1-Lipschitz.

1. Show that $\widehat{\mathfrak{R}}_S(\mathcal{H}) = \frac{\Lambda'}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{\|\mathbf{u}\|_2 \leq \Lambda} \left| \sum_{i=1}^m \sigma_i \sigma(\mathbf{u} \cdot \mathbf{x}_i) \right| \right]$.

Solution:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(\mathcal{H}) &= \frac{1}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{\|\mathbf{w}\|_1 \leq \Lambda', \|\mathbf{u}_j\|_2 \leq \Lambda} \sum_{i=1}^m \sigma_i \sum_{j=1}^{n_2} w_j \sigma(\mathbf{u}_j \cdot \mathbf{x}_i) \right] \\
&= \frac{1}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{\|\mathbf{w}\|_1 \leq \Lambda', \|\mathbf{u}_j\|_2 \leq \Lambda} \sum_{j=1}^{n_2} w_j \sum_{i=1}^m \sigma_i \sigma(\mathbf{u}_j \cdot \mathbf{x}_i) \right] \\
&= \frac{\Lambda'}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{\|\mathbf{u}_j\|_2 \leq \Lambda} \max_{j \in [1, n_2]} \left| \sum_{i=1}^m \sigma_i \sigma(\mathbf{u}_j \cdot \mathbf{x}_i) \right| \right] \quad (\text{all the weight put on largest term}) \\
&= \frac{\Lambda'}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{\|\mathbf{u}_j\|_2 \leq \Lambda, j \in [1, n_2]} \left| \sum_{i=1}^m \sigma_i \sigma(\mathbf{u}_j \cdot \mathbf{x}_i) \right| \right] \\
&= \frac{\Lambda'}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{\|\mathbf{u}\|_2 \leq \Lambda} \left| \sum_{i=1}^m \sigma_i \sigma(\mathbf{u} \cdot \mathbf{x}_i) \right| \right].
\end{aligned}$$

2. Use the following form of Talagrand's lemma valid for all hypothesis sets H and L -Lipschitz function Φ :

$$\frac{1}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h \in H} \left| \sum_{i=1}^m \sigma_i (\Phi \circ h)(x_i) \right| \right] \leq \frac{L}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h \in H} \left| \sum_{i=1}^m \sigma_i h(x_i) \right| \right],$$

to upper bound $\widehat{\mathfrak{R}}_S(\mathcal{H})$ in terms of the empirical Rademacher complexity of $\mathcal{H}'$, where $\mathcal{H}'$ is defined by

$$\mathcal{H}' = \{ \mathbf{x} \mapsto s(\mathbf{u} \cdot \mathbf{x}) : \|\mathbf{u}\|_2 \leq \Lambda, s \in \{-1, +1\} \}.$$

Solution: By Talagrand's lemma, since σ is L -Lipschitz, the following

inequality holds:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(\mathcal{H}) &\leq \frac{\Lambda' L}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h \in H} \left\| \sum_{i=1}^m \sigma_i \mathbf{u} \cdot \mathbf{x}_i \right\| \right] \\
&= \frac{\Lambda' L}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h \in H} \sup_{s \in \{-1, +1\}} s \sum_{i=1}^m \sigma_i \mathbf{u} \cdot \mathbf{x}_i \right] \quad (\text{def. of abs. value}) \\
&= \Lambda' L \widehat{\mathfrak{R}}_S(\mathcal{H}').
\end{aligned}$$

3. Use the Cauchy-Schwarz inequality to show that

$$\widehat{\mathfrak{R}}_S(\mathcal{H}') = \frac{\Lambda}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\left\| \sum_{i=1}^m \sigma_i \mathbf{x}_i \right\|_2 \right].$$

Solution:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(\mathcal{H}') &= \frac{1}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{\|\mathbf{u}\|_2 \leq \Lambda, s \in \{-1, +1\}} \sum_{i=1}^m \sigma_i s \mathbf{u} \cdot \mathbf{x}_i \right] \\
&= \frac{1}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{\|\mathbf{u}\|_2 \leq \Lambda} \left\| \sum_{i=1}^m \sigma_i \mathbf{u} \cdot \mathbf{x}_i \right\| \right] \quad (\text{def. of abs. val.}) \\
&= \frac{1}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{\|\mathbf{u}\|_2 \leq \Lambda} \left\| \mathbf{u} \cdot \sum_{i=1}^m \sigma_i \mathbf{x}_i \right\| \right] \\
&= \frac{\Lambda}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\left\| \sum_{i=1}^m \sigma_i \mathbf{x}_i \right\|_2 \right] \quad (\text{Cauchy-Schwarz eq. case}).
\end{aligned}$$

The last equality holds by setting $\mathbf{u} = \frac{\Lambda \sum_{i=1}^m \sigma_i \mathbf{x}_i}{\|\sum_{i=1}^m \sigma_i \mathbf{x}_i\|}$.

4. Use the inequality $\mathbb{E}[\|\mathbf{X}\|_2] \leq \sqrt{\mathbb{E}[\|\mathbf{X}\|_2^2]}$, which holds by Jensen's inequality to upper bound $\widehat{\mathfrak{R}}_S(\mathcal{H}')$.

Solution:

$$\begin{aligned}
\widehat{\mathfrak{R}}_S(\mathcal{H}') &= \frac{\Lambda}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\left\| \sum_{i=1}^m \sigma_i \mathbf{x}_i \right\|_2 \right] \\
&\leq \frac{\Lambda}{m} \sqrt{\mathbb{E}_{\boldsymbol{\sigma}} \left[\left\| \sum_{i=1}^m \sigma_i \mathbf{x}_i \right\|_2^2 \right]} && \text{(Jensen's ineq.)} \\
&= \frac{1}{m} \sqrt{\sum_{i,j=1}^m \mathbb{E}_{\boldsymbol{\sigma}} [\sigma_i \sigma_j] (\mathbf{x}_i \cdot \mathbf{x}_j)} \\
&= \frac{\Lambda}{m} \sqrt{\sum_{i,j=1}^m 1_{i=j} (\mathbf{x}_i \cdot \mathbf{x}_j)} && \text{(independence of } \sigma_i \text{)} \\
&= \frac{\Lambda}{m} \sqrt{\sum_{i=1}^m \|\mathbf{x}_i\|_2^2}.
\end{aligned}$$

5. Assume that for all $\mathbf{x} \in S$, $\|\mathbf{x}\|_2 \leq r$ for some $r > 0$. Use the previous questions to derive an upper bound on the Rademacher complexity of $\mathcal{H}$ in terms of r .

Solution: In view of the previous questions,

$$\widehat{\mathfrak{R}}_S(\mathcal{H}) \leq \Lambda' L \widehat{\mathfrak{R}}_S(\mathcal{H}') \leq \frac{\Lambda' \Lambda L}{m} \sqrt{\sum_{i=1}^m \|\mathbf{x}_i\|_2^2} \leq \frac{\Lambda' \Lambda L}{m} \sqrt{mr^2} = \frac{\Lambda' \Lambda L r}{\sqrt{m}}.$$

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 1
September 16, 2016
Due: October 04, 2016

A. Probability tools

1. Let $f: (0, +\infty) \rightarrow \mathbb{R}$ be a function admitting an inverse f^{-1} and let X be a random variable. Show that if for any $t > 0$, $\Pr[X > t] \leq f(t)$, then, for any $\delta > 0$, with probability at least $1 - \delta$, $X \leq f^{-1}(\delta)$.
2. Let X be a discrete random variable taking non-negative integer values. Show that $\mathbb{E}[X] = \sum_{n \geq 1} \Pr[X \geq n]$ (*hint*: rewrite $\Pr[X = n]$ as $\Pr[X \geq n] - \Pr[X \geq n + 1]$).

B. Label bias

1. Let D be a distribution over $\mathcal{X}$ and let $f: \mathcal{X} \rightarrow \{-1, +1\}$ be a labeling function. Suppose we wish to find a good approximation of the label bias of the distribution D , that is of p_+ defined by:

$$p_+ = \Pr_{x \sim D}[f(x) = +1]. \quad (1)$$

Let S be a finite labeled sample of size m drawn i.i.d. according to D . Use S to derive an estimate $\hat{p}_+$ of p_+ . Show that for any $\delta > 0$, with probability at least $1 - \delta$, $|p_+ - \hat{p}_+| \leq \sqrt{\frac{\log(2/\delta)}{2m}}$ (carefully justify all steps).

C. Learning in the presence of noise

1. In Lecture 2, we showed that the concept class of axis-aligned rectangles is PAC-learnable. Consider now the case where the training points received by the learner are subject to the following noise: points negatively labeled are unaffected by noise but the label of a positive training point is randomly flipped to negative with probability $\eta \in (0, \frac{1}{2})$. The exact value of the noise rate η is not known to the learner but an upper bound η' is supplied to him with $\eta \leq \eta' < 1/2$. Show that the algorithm described in class returning the tightest rectangle containing positive points can still PAC-learn axis-aligned

rectangles in the presence of this noise. To do so, you can proceed using the following steps:

- (a) Using the notation of the lecture slides, assume that $\Pr[R] > \epsilon$. Suppose that $\text{error}(R') > \epsilon$. Give an upper bound on the probability that R' misses a region r_j , $j \in [1, 4]$ in terms of ϵ and η' ?
 - (b) Use that to give an upper bound on $\Pr[\text{error}(R') > \epsilon]$ in terms of ϵ and η' and conclude by giving a sample complexity bound.
2. [Bonus question] In this section, we will seek a more general result. We consider a finite hypothesis set H , assume that the target concept is in H , and adopt the following noise model: the label of a training point received by the learner is randomly changed with probability $\eta \in (0, \frac{1}{2})$. The exact value of the noise rate η is not known to the learner but an upper bound η' is supplied to him with $\eta \leq \eta' < 1/2$.
- (a) For any $h \in H$, let $d(h)$ denote the probability that the label of a training point received by the learner disagrees with the one given by h . Let h^* be the target hypothesis, show that $d(h^*) = \eta$.
 - (b) More generally, show that for any $h \in H$, $d(h) = \eta + (1 - 2\eta) \text{error}(h)$, where $\text{error}(h)$ denotes the generalization error of h .
 - (c) Fix $\epsilon > 0$ for this and all the following questions. Use the previous questions to show that if $\text{error}(h) > \epsilon$, then $d(h) - d(h^*) \geq \epsilon'$, where $\epsilon' = \epsilon(1 - 2\eta')$.
 - (d) For any hypothesis $h \in H$ and sample S of size m , let $\hat{d}(h)$ denote the fraction of the points in S whose labels disagree with those given by h . We will consider the algorithm L which, after receiving S , returns the hypothesis h_S with the smallest number of disagreements (thus $\hat{d}(h_S)$ is minimal). To show PAC-learning for L , we will show that for any h , if $\text{error}(h) > \epsilon$, then with high probability $\hat{d}(h) \geq \hat{d}(h^*)$. First, show that for any $\delta > 0$, with probability at least $1 - \delta/2$, for $m \geq \frac{2}{\epsilon'^2} \log \frac{2}{\delta}$, the following holds:

$$\hat{d}(h^*) - d(h^*) \leq \epsilon'/2$$

- (e) Second, show that for any $\delta > 0$, with probability at least $1 - \delta/2$, for $m \geq \frac{2}{\epsilon'^2} (\log |H| + \log \frac{2}{\delta})$, the following holds for all $h \in H$:

$$d(h) - \hat{d}(h) \leq \epsilon'/2$$

- (f) Finally, show that for any $\delta > 0$, with probability at least $1 - \delta$, for $m \geq \frac{2}{\epsilon^2(1-2\eta)^2}(\log |H| + \log \frac{2}{\delta})$, the following holds for all $h \in H$ with $\text{error}(h) > \epsilon$:

$$\hat{d}(h) - \hat{d}(h^*) \geq 0.$$

(*hint*: use $\hat{d}(h) - \hat{d}(h^*) = [\hat{d}(h) - d(h)] + [d(h) - d(h^*)] + [d(h^*) - \hat{d}(h^*)]$ and use previous questions to lower bound each of these three terms).

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 1
September 17, 2016
Due: October 04, 2016

A. Probability tools

1. Let $f: (0, +\infty) \rightarrow \mathbb{R}_+$ be a function admitting an inverse f^{-1} and let X be a random variable. Show that if for any $t > 0$, $\Pr[X > t] \leq f(t)$, then, for any $\delta > 0$, with probability at least $1 - \delta$, $X \leq f^{-1}(\delta)$.

Solution: For any $\delta > 0$, let $t = f^{-1}(\delta)$. Plugging this in $\Pr[X > t] \leq f(t)$ yields $\Pr[X > f^{-1}(\delta)] \leq \delta$, that is $\Pr[X \leq f^{-1}(\delta)] \geq 1 - \delta$. $\square$

2. Let X be a discrete random variable taking non-negative integer values. Show that $\mathbb{E}[X] = \sum_{n \geq 1} \Pr[X \geq n]$ (*hint: rewrite $\Pr[X = n]$ as $\Pr[X \geq n] - \Pr[X \geq n + 1]$*).

Solution: We assume that X is a bounded random variable to avoid any convergence issues (although the statement is still true in the general case).

By definition of expectation and using the hint, we can write

$$\mathbb{E}[X] = \sum_{n \geq 0} n \Pr[X = n] = \sum_{n \geq 1} n(\Pr[X \geq n] - \Pr[X \geq n + 1]).$$

Note that in this sum, for $n \geq 1$, $\Pr[X \geq n]$ is added n times and subtracted $n - 1$ times, thus $\mathbb{E}[X] = \sum_{n \geq 1} \Pr[X \geq n]$.

More generally, by definition of the Lebesgue integral, for any non-negative random variable X , the following identity holds:

$$\mathbb{E}[X] = \int_0^{+\infty} \Pr[X \geq t] dt.$$

$\square$

B. Label bias

1. Let D be a distribution over $\mathcal{X}$ and let $f: \mathcal{X} \rightarrow \{-1, +1\}$ be a labeling function. Suppose we wish to find a good approximation of the label bias of

the distribution D , that is of p_+ defined by:

$$p_+ = \Pr_{x \sim D}[f(x) = +1]. \quad (1)$$

Let S be a finite labeled sample of size m drawn i.i.d. according to D . Use S to derive an estimate $\hat{p}_+$ of p_+ . Show that for any $\delta > 0$, with probability at least $1 - \delta$, $|p_+ - \hat{p}_+| \leq \sqrt{\frac{\log(2/\delta)}{2m}}$ (carefully justify all steps).

Solution: Let $\hat{p}_+$ be the fraction of positively labeled points in $S = (x_1, \dots, x_m)$:

$$\hat{p}_+ = \frac{1}{m} \sum_{i=1}^m 1_{f(x_i)=+1}$$

Since the points are drawn i.i.d.,

$$\mathbb{E}[\hat{p}_+] = \frac{1}{m} \sum_{i=1}^m \mathbb{E}_{S \sim D^m}[1_{f(x_i)=+1}] = \mathbb{E}_{S \sim D^m}[1_{f(x_1)=+1}] = \mathbb{E}_{x \sim D}[1_{f(x)=+1}] = p_+.$$

Thus, by Hoeffding's inequality, for any $\epsilon > 0$,

$$\Pr[|p_+ - \hat{p}_+| > \epsilon] \leq 2e^{-2m\epsilon^2}.$$

Setting δ to match the right-hand side yields the result. $\square$

C. Learning in the presence of noise

1. In Lecture 2, we showed that the concept class of axis-aligned rectangles is PAC-learnable. Consider now the case where the training points received by the learner are subject to the following noise: points negatively labeled are unaffected by noise but the label of a positive training point is randomly flipped to negative with probability $\eta \in (0, \frac{1}{2})$. The exact value of the noise rate η is not known to the learner but an upper bound η' is supplied to him with $\eta \leq \eta' < 1/2$. Show that the algorithm described in class returning the tightest rectangle containing positive points can still PAC-learn axis-aligned rectangles in the presence of this noise. To do so, you can proceed using the following steps:

- (a) Using the notation of the lecture slides, assume that $\Pr[R] > \epsilon$. Suppose that $\text{error}(R') > \epsilon$. Give an upper bound on the probability that R' misses a region r_j , $j \in [1, 4]$ in terms of ϵ and η' ?

Solution: The probability that R' misses region r_j is the product of the probability p for each point x_i of the training sample to either not fall in r_j or be positive and fall in r_j with the label flipped to negative due to noise.

$$\begin{aligned}
p &= \Pr[x \notin r_j \vee (x \in r_j \wedge x \text{ positive} \wedge \text{label of } x \text{ flipped})] \\
&= \Pr[x \notin r_j \vee (x \in r_j \wedge \text{label of } x \text{ flipped})] \\
&= \Pr[x \notin r_j] + \Pr[(x \in r_j \wedge \text{label of } x \text{ flipped})] \\
&= (1 - \Pr[x \in r_j]) + \eta \Pr[x \in r_j] \\
&= (1 - \eta)(1 - \Pr[x \notin r_j]) + \eta \\
&\leq (1 - \eta)(1 - \epsilon/4) + \eta \\
&= (1 - \epsilon/4) + \eta\epsilon/4 \leq 1 - \epsilon(1 - \eta')/4.
\end{aligned}$$

□

- (b) Use that to give an upper bound on $\Pr[\text{error}(R') > \epsilon]$ in terms of ϵ and η' and conclude by giving a sample complexity bound.

Solution: The probability that $\Pr[\text{error}(R') > \epsilon]$ is upper bounded by the probability that R' misses at least one region r_j . Thus, by the union bound,

$$\Pr[\text{error}(R') > \epsilon] \leq 4 \left(1 - \epsilon(1 - \eta')/4\right)^m \leq 4e^{-m\epsilon(1 - \eta')/4}.$$

Setting δ to match the upper bound leads to the following: with probability at least $1 - \delta$, for $m \geq \frac{4}{(1 - \eta')\epsilon} \log \frac{4}{\delta}$, $\text{error}(R') \leq \epsilon$. □

2. [Bonus question] In this section, we will seek a more general result. We consider a finite hypothesis set H , assume that the target concept is in H , and adopt the following noise model: the label of a training point received by the learner is randomly changed with probability $\eta \in (0, \frac{1}{2})$. The exact value of the noise rate η is not known to the learner but an upper bound η' is supplied to him with $\eta \leq \eta' < 1/2$.

- (a) For any $h \in H$, let $d(h)$ denote the probability that the label of a training point received by the learner disagrees with the one given by h . Let h^* be the target hypothesis, show that $d(h^*) = \eta$.

Solution: The probability that the label of a point be incorrect is η . A label is incorrect iff it differs from the label given by the target h^* . □

- (b) More generally, show that for any $h \in H$, $d(h) = \eta + (1 - 2\eta) \text{error}(h)$, where $\text{error}(h)$ denotes the generalization error of h .

Solution: The label of a point disagrees with the one given by h either because its label is correct (probability $1 - \eta$) and h misclassifies that point (probability $\text{error}(h)$), or because its label is incorrect (probability η) and h classifies it correctly (probability $1 - \text{error}(h)$). Since these two events are disjoint, the probability of their union is the sum of the probability and

$$\begin{aligned} d(h) &= (1 - \eta)\text{error}(h) + \eta(1 - \text{error}(h)) \\ &= \eta + (1 - 2\eta) \text{error}(h). \end{aligned}$$

□

- (c) Fix $\epsilon > 0$ for this and all the following questions. Use the previous questions to show that if $\text{error}(h) > \epsilon$, then $d(h) - d(h^*) \geq \epsilon'$, where $\epsilon' = \epsilon(1 - 2\eta')$.

Solution: In view of the previous question, if $\text{error}(h) > \epsilon$,

$$\begin{aligned} d(h) &= \eta + (1 - 2\eta) \text{error}(h) \\ &\geq \eta + (1 - 2\eta)\epsilon \\ &\geq \eta + (1 - 2\eta')\epsilon \\ &= d(h^*) + (1 - 2\eta')\epsilon, \end{aligned}$$

where we used $d(h^*) = \eta$.

□

- (d) For any hypothesis $h \in H$ and sample S of size m , let $\hat{d}(h)$ denote the fraction of the points in S whose labels disagree with those given by h . We will consider the algorithm L which, after receiving S , returns the hypothesis h_S with the smallest number of disagreements (thus $\hat{d}(h_S)$ is minimal). To show PAC-learning for L , we will show that for any h , if $\text{error}(h) > \epsilon$, then with high probability $\hat{d}(h) \geq \hat{d}(h^*)$. First, show that for any $\delta > 0$, with probability at least $1 - \delta/2$, for $m \geq \frac{2}{\epsilon^2} \log \frac{2}{\delta}$, the following holds:

$$\hat{d}(h^*) - d(h^*) \leq \epsilon'/2$$

Solution: By Hoeffding's inequality $\Pr[\hat{d}(h^*) - d(h^*) > \epsilon'/2] \leq e^{-m\epsilon'^2/2}$. Setting $\delta/2$ to match the right-hand side yields the result. □

- (e) Second, show that for any $\delta > 0$, with probability at least $1 - \delta/2$, for $m \geq \frac{2}{\epsilon'^2}(\log |H| + \log \frac{2}{\delta})$, the following holds for all $h \in H$:

$$d(h) - \widehat{d}(h) \leq \epsilon'/2$$

Solution: By the union bound and Hoeffding's inequality $\Pr[\exists h: d(h) - \widehat{d}(h) > \epsilon'/2] \leq |H|e^{-m\epsilon'^2/2}$. Setting $\delta/2$ to match the right-hand side yields the result. $\square$

- (f) Finally, show that for any $\delta > 0$, with probability at least $1 - \delta$, for $m \geq \frac{2}{\epsilon^2(1-2\eta')^2}(\log |H| + \log \frac{2}{\delta})$, the following holds for all $h \in H$ with $\text{error}(h) > \epsilon$:

$$\widehat{d}(h) - \widehat{d}(h^*) \geq 0.$$

(*hint:* use $\widehat{d}(h) - \widehat{d}(h^*) = [\widehat{d}(h) - d(h)] + [d(h) - d(h^*)] + [d(h^*) - \widehat{d}(h^*)]$ and use previous questions to lower bound each of these three terms).

Solution: By the union bound, for any $\delta > 0$, with probability at least $1 - \delta$, for $m \geq \frac{2}{\epsilon'^2}(\log |H| + \log \frac{2}{\delta})$, both inequalities of the previous two questions hold, the previous one for all $h \in H$. Thus, using the equality of the hint, with probability at least $1 - \delta$, for $m \geq \frac{2}{\epsilon'^2}(\log |H| + \log \frac{2}{\delta})$, the following holds for all $h \in H$ with $\text{error}(h) > \epsilon$:

$$\begin{aligned} \widehat{d}(h) - \widehat{d}(h^*) &= [\widehat{d}(h) - d(h)] + [d(h) - d(h^*)] + [d(h^*) - \widehat{d}(h^*)] \\ &\geq -\epsilon'/2 + \epsilon' - \epsilon'/2 = 0, \end{aligned}$$

and thus such hypotheses h are not selected by L since they do not admit a minimal $\widehat{d}(h)$.

This shows that algorithm L can be used for PAC-learning in the presence of the noise described and in the consistent case where the target concept is in H . Nevertheless, the computational complexity of L is in general not polynomial. In general, the problem of finding the hypothesis with minimal $\widehat{d}(h)$ is NP-complete. $\square$

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 1
September 21, 2017
Due: October 04, 2017

A. Concentration inequality

In this problem, we will derive a concentration inequality that is finer than Hoeffding's inequality.

Let $X_1, \dots, X_m$ be independent random variables drawn according to some distribution $\mathcal{D}$ with mean p and support included in $[0, 1]$. Let $q \in [0, 1]$ and define $\hat{p}$ by $\hat{p} = \frac{1}{m} \sum_{i=1}^m X_i$.

1. Use Markov's inequality to prove that for any $t > 0$,

$$\mathbb{P}[\hat{p} \geq q] \leq e^{-tmq} \prod_{i=1}^m \mathbb{E}[e^{tX_i}].$$

2. Show that $\mathbb{P}[\hat{p} \geq q] \leq [e^{-tq}(1 - p + e^tp)]^m$ (*Hint*: you can show that $\forall x \in [0, 1], e^{tx} \leq 1 - x + e^tx$).
3. Choose $t > 0$ to minimize the right-hand side and prove the following inequality:

$$\mathbb{P}[\hat{p} \geq q] \leq e^{-mD(q||p)},$$

where $D(q||p) = q \log \frac{q}{p} + (1 - q) \log \frac{1-q}{1-p}$ is the binary relative entropy of p and q .

4. Show that this can be used to derive a bound that is finer than Hoeffding's inequality by choosing $q = p + \epsilon$, for any $\epsilon > 0$ (*Hint*: you can use Pinsker's inequality, which in this context gives $D(q||p) \geq 2|p - q|^2$).

B. PAC-Learning

1. Concentric spheres. Let $\mathcal{X} = \mathbb{R}^3$ and consider the set of concepts of the form $c = \{(x, y, z) : x^2 + y^2 + z^2 \leq r^2\}$ for some $r > 0$. Show that this class can be (ϵ, δ) -PAC-learned from training data of size $m \geq (1/\epsilon) \log(1/\delta)$. You should carefully justify all steps of your proof.

C. Rademacher complexity

1. Let $\mathcal{H}_1$ and $\mathcal{H}_2$ be two families of functions mapping $\mathcal{X}$ to $[0, 1]$ and let S be a sample of size m .

- (a) Show that for any sample S ,

$$\widehat{\mathfrak{R}}_S(\mathcal{H}_1 + \mathcal{H}_2) = \widehat{\mathfrak{R}}_S(\mathcal{H}_1) + \widehat{\mathfrak{R}}_S(\mathcal{H}_2).$$

- (b) Show that the empirical Rademacher complexity of $\mathcal{H}$ for any sample S of size m can be bounded as follows:

$$\widehat{\mathfrak{R}}_S(\mathcal{H}_1 \mathcal{H}_2) \leq \frac{3}{2} [\widehat{\mathfrak{R}}_S(\mathcal{H}_1) + \widehat{\mathfrak{R}}_S(\mathcal{H}_2)].$$

Hint: use the identity $x_1 x_2 = \frac{1}{4}[(x_1 + x_2)^2 - (x_1 - x_2)^2]$ valid for all $x_1, x_2 \in \mathbb{R}$ and Talagrand's contraction lemma.

2. Let $\mathcal{E}$ be the family of functions defined over $\mathcal{X}$ by $\mathcal{E} = \{x \mapsto 1_{x=z} : z \in \mathcal{X}\}$.

- (a) Let S be a sample of size m with distinct points. Prove the inequality $\widehat{\mathfrak{R}}_S(\mathcal{E}) \leq \frac{1}{m}$.
- (b) Let S be a sample of size m with a single point repeated m times. Prove the equality $\widehat{\mathfrak{R}}_S(\mathcal{E}) = \frac{1}{2m} \mathbb{E}_{\sigma} [|\sum_{i=1}^m \sigma_i|]$. Show that $\widehat{\mathfrak{R}}_S(\mathcal{E}) \leq \frac{1}{2\sqrt{m}}$.
- (c) [bonus question] Let n^* denote the largest number of repetitions of any point in the sample. Give an upper bound on $\widehat{\mathfrak{R}}_S(\mathcal{E})$ in terms of n^* .

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 1
September 21, 2017
Due: October 04, 2017

A. Concentration inequality

In this problem, we will derive a concentration inequality that is finer than Hoeffding's inequality.

Let $X_1, \dots, X_m$ be independent random variables drawn according to some distribution $\mathcal{D}$ with mean p and support included in $[0, 1]$. Let $q \in [0, 1]$ and define $\hat{p}$ by $\hat{p} = \frac{1}{m} \sum_{i=1}^m X_i$.

1. Use Markov's inequality to prove that for any $t > 0$,

$$\mathbb{P}[\hat{p} \geq q] \leq e^{-tmq} \prod_{i=1}^m \mathbb{E}[e^{tX_i}].$$

2. Show that $\mathbb{P}[\hat{p} \geq q] \leq [e^{-tq}(1 - p + e^tp)]^m$ (*Hint*: you can show that $\forall x \in [0, 1], e^{tx} \leq 1 - x + e^tx$).
3. Choose $t > 0$ to minimize the right-hand side and prove the following inequality:

$$\mathbb{P}[\hat{p} \geq q] \leq e^{-mD(q||p)},$$

where $D(q||p) = q \log \frac{q}{p} + (1 - q) \log \frac{1-q}{1-p}$ is the binary relative entropy of p and q .

4. Show that this can be used to derive a bound that is finer than Hoeffding's inequality by choosing $q = p + \epsilon$, for any $\epsilon > 0$ (*Hint*: you can use Pinsker's inequality, which in this context gives $D(q||p) \geq 2|p - q|^2$).

Solution: For any $t > 0$, by convexity of the function $x \mapsto e^{tx}$, the following inequality holds for all $x \in [0, 1]$: $e^{tx} = e^{t[(1-x) \cdot 0 + x \cdot 1]} \leq 1 - x + e^tx$. In

view of that, for any $t > 0$, we can write

$$\begin{aligned}
\mathbb{P}[\hat{p} \geq q] &= \mathbb{P}[e^{tm\hat{p}} \geq e^{tmq}] \\
&= \mathbb{P}[e^{tm\hat{p}} \geq e^{tmq}] \\
&\leq e^{-tmq} \mathbb{E}[e^{tm\hat{p}}] && \text{(by Markov's inequality)} \\
&= e^{-tmq} \mathbb{E}[e^{t \sum_{i=1}^m X_i}] \\
&= e^{-tmq} \prod_{i=1}^m \mathbb{E}[e^{tX_i}] \\
&\leq e^{-tmq} \prod_{i=1}^m \mathbb{E}[1 - X_i + e^t X_i] \quad (\forall x \in [0, 1], e^{tx} \leq 1 - x + e^t x) \\
&= [e^{-tq}(1 - p + e^t p)]^m.
\end{aligned}$$

Now, the function $f: t \mapsto e^{-tq}(1 - p + e^t p) = (1 - p)e^{-tq} + pe^{t(1-q)}$ reaches its minimum at $t = \log \frac{q(1-p)}{p(1-q)}$. Plugging in this value of t in the inequality above yields $\mathbb{P}[\hat{p} \geq q] \leq e^{-mD(q\|p)}$.

Note that for any $\epsilon > 0$, $\epsilon \leq 1 - p$, with the choice $q = p + \epsilon$, the theorem implies

$$\mathbb{P}[\hat{p} \geq p + \epsilon] \leq e^{-mD(p+\epsilon\|p)}. \quad (1)$$

This is a finer bound than Hoeffding's inequality since, by Pinsker's inequality, $D(p + \epsilon\|p) \geq \frac{1}{2}(2\epsilon)^2 = 2\epsilon^2$. $\square$

B. PAC-Learning

1. Concentric spheres. Let $\mathcal{X} = \mathbb{R}^3$ and consider the set of concepts of the form $c = \{(x, y, z): x^2 + y^2 + z^2 \leq r^2\}$ for some $r > 0$. Show that this class can be (ϵ, δ) -PAC-learned from training data of size $m \geq (1/\epsilon) \log(1/\delta)$. You should carefully justify all steps of your proof.

Solution: Let T denote the training sample and let $L(T)$ be the concept with the smallest radius that is consistent with T .

Suppose our target concept c is the sphere around the origin with radius r . We will choose a slightly smaller radius s by

$$s := \inf\{s': \mathbb{P}(s' \leq \|x\| \leq r) < \epsilon\}.$$

Let A denote the annulus between radii s and r ; that is, $A := \{x: s \leq \|x\| \leq r\}$. By definition of s , we have $\mathbb{P}(A) \geq \epsilon$. In addition, our generalization

error, $\mathbb{P}(c \Delta L(T))$, must be small if T intersects A . We can state this as

$$\mathbb{P}(c \Delta L(T)) > \epsilon \implies T \cap A = \emptyset. \quad (2)$$

Thus, we know that any point in T chosen will miss region A with probability at most $1 - \epsilon$. Defining $error := \mathbb{P}(c \Delta L(T))$, we can combine this with (2) to see that

$$\mathbb{P}(error > \epsilon) \leq P(T \cap A = \emptyset) \leq (1 - \epsilon)^m \leq e^{-m\epsilon}.$$

Setting δ to be greater than or equal to the right-hand side leads to $m \geq \frac{1}{\epsilon} \log(\frac{1}{\delta})$. $\square$

C. Rademacher complexity

1. Let $\mathcal{H}_1$ and $\mathcal{H}_2$ be two families of functions mapping $\mathcal{X}$ to $[0, 1]$ and let S be a sample of size m .

(a) Show that for any sample S ,

$$\widehat{\mathfrak{R}}_S(\mathcal{H}_1 + \mathcal{H}_2) = \widehat{\mathfrak{R}}_S(\mathcal{H}_1) + \widehat{\mathfrak{R}}_S(\mathcal{H}_2).$$

Solution:

$$\begin{aligned} \widehat{\mathfrak{R}}_S(\mathcal{H}_1 + \mathcal{H}_2) &= \frac{1}{m} \mathbb{E}_{\sigma} \left[\sup_{(h_1, h_2) \in \mathcal{H}_1 \times \mathcal{H}_2} \sum_{i=1}^m \sigma_i (h_1(x_i) + h_2(x_i)) \right] \\ &= \frac{1}{m} \mathbb{E}_{\sigma} \left[\sup_{(h_1, h_2) \in \mathcal{H}_1 \times \mathcal{H}_2} \sum_{i=1}^m \sigma_i h_1(x_i) + \sum_{i=1}^m \sigma_i h_2(x_i) \right] \\ &= \frac{1}{m} \mathbb{E}_{\sigma} \left[\sup_{h_1 \in \mathcal{H}_1} \sum_{i=1}^m \sigma_i h_1(x_i) + \sup_{h_2 \in \mathcal{H}_2} \sum_{i=1}^m \sigma_i h_2(x_i) \right] \\ &= \widehat{\mathfrak{R}}_S(\mathcal{H}_1) + \widehat{\mathfrak{R}}_S(\mathcal{H}_2). \end{aligned}$$

$\square$

- (b) Show that the empirical Rademacher complexity of $\mathcal{H}$ for any sample S of size m can be bounded as follows:

$$\widehat{\mathfrak{R}}_S(\mathcal{H}_1 \mathcal{H}_2) \leq \frac{3}{2} [\widehat{\mathfrak{R}}_S(\mathcal{H}_1) + \widehat{\mathfrak{R}}_S(\mathcal{H}_2)].$$

Hint: use the identity $x_1 x_2 = \frac{1}{4}[(x_1 + x_2)^2 - (x_1 - x_2)^2]$ valid for all $x_1, x_2 \in \mathbb{R}$ and Talagrand's contraction lemma.

Solution: By the hint, we can write

$$\mathcal{H}_1\mathcal{H}_2 = \left\{ \frac{1}{4}[(h_1 + h_2)^2 - (h_1 - h_2)^2] : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2 \right\}.$$

Thus, in view of the previous question,

$$\begin{aligned} \widehat{\mathfrak{R}}_S(\mathcal{H}_1\mathcal{H}_2) &= \frac{1}{4}\widehat{\mathfrak{R}}_S\left(\{(h_1 + h_2)^2 : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}\right) \\ &\quad + \frac{1}{4}\widehat{\mathfrak{R}}_S\left(\{-(h_1 - h_2)^2 : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}\right) \\ &= \frac{1}{4}\widehat{\mathfrak{R}}_S\left(\{(h_1 + h_2)^2 : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}\right) \\ &\quad + \frac{1}{4}\widehat{\mathfrak{R}}_S\left(\{(h_1 - h_2)^2 : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}\right). \end{aligned}$$

For any h_1, h_2 , $h_1 + h_2$ takes values in $[0, 2]$ and the function $x \mapsto x^2$ is 4-Lipschitz over $[0, 2]$. Thus,

$$\begin{aligned} &\widehat{\mathfrak{R}}_S\left(\{(h_1 + h_2)^2 : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}\right) \\ &\leq 4\widehat{\mathfrak{R}}_S\left(\{h_1 + h_2 : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}\right) = 4(\widehat{\mathfrak{R}}_S(H_1) + \widehat{\mathfrak{R}}_S(H_2)). \end{aligned}$$

Similarly, For any h_1, h_2 , $h_1 - h_2$ takes values in $[-1, +1]$ and the function $x \mapsto x^2$ is 2-Lipschitz over $[-1, +1]$. Thus,

$$\begin{aligned} &\widehat{\mathfrak{R}}_S\left(\{(h_1 - h_2)^2 : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}\right) \\ &\leq 2\widehat{\mathfrak{R}}_S\left(\{h_1 - h_2 : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}\right) = 2(\widehat{\mathfrak{R}}_S(H_1) + \widehat{\mathfrak{R}}_S(H_2)). \end{aligned}$$

Thus,

$$\widehat{\mathfrak{R}}_S(\mathcal{H}_1\mathcal{H}_2) \leq \frac{6}{4}(\widehat{\mathfrak{R}}_S(H_1) + \widehat{\mathfrak{R}}_S(H_2)) = \frac{3}{2}(\widehat{\mathfrak{R}}_S(H_1) + \widehat{\mathfrak{R}}_S(H_2)).$$

□

2. Let $\mathcal{E}$ be the family of functions defined over $\mathcal{X}$ by $\mathcal{E} = \{x \mapsto 1_{x=z} : z \in \mathcal{X}\}$.

- (a) Let S be a sample of size m with distinct points. Prove the inequality $\widehat{\mathfrak{R}}_S(\mathcal{E}) \leq \frac{1}{m}$.

Solution: We can write

$$\widehat{\mathfrak{R}}_S(\mathcal{E}) = \frac{1}{m} \mathbb{E}_{\sigma} \left[\sup_{z \in \mathcal{X}} \sum_{i=1}^m \sigma_i 1_{x_i=z} \right] = \frac{1}{m} \mathbb{E}_{\sigma} \left[\max_{i \in [m]} \sigma_i \right] \leq \frac{1}{m},$$

since $\max_{i \in [m]} \sigma_i \leq 1$.

□

- (b) Let S be a sample of size m with a single point repeated m times. Prove the equality $\hat{\mathfrak{R}}_S(\mathcal{E}) = \frac{1}{2m} \mathbb{E}_{\sigma} \left[\left| \sum_{i=1}^m \sigma_i \right| \right]$. Show that $\hat{\mathfrak{R}}_S(\mathcal{E}) \leq \frac{1}{2\sqrt{m}}$.

Solution: We can write

$$\begin{aligned} \hat{\mathfrak{R}}_S(\mathcal{E}) &= \frac{1}{m} \mathbb{E}_{\sigma} \left[\sup_{z \in \mathcal{X}} \sum_{i=1}^m \sigma_i 1_{x_i=z} \right] \\ &= \frac{1}{m} \mathbb{E}_{\sigma} \left[\max \left(\sum_{i=1}^m \sigma_i, 0 \right) \right] \\ &= \frac{1}{2m} \mathbb{E}_{\sigma} \left[\left| \sum_{i=1}^m \sigma_i \right| \right]. \end{aligned}$$

By Jensen's inequality, we can write

$$\begin{aligned} \frac{1}{2m} \mathbb{E}_{\sigma} \left[\left| \sum_{i=1}^m \sigma_i \right| \right] &\leq \frac{1}{2m} \sqrt{\mathbb{E}_{\sigma} \left[\left| \sum_{i=1}^m \sigma_i \right|^2 \right]} \\ &= \frac{1}{2m} \sqrt{\mathbb{E}_{\sigma} \left[\sum_{i,j=1}^m \sigma_i \sigma_j \right]} \\ &= \frac{1}{2m} \sqrt{\mathbb{E}_{\sigma} \left[\sum_{i=1}^m \sigma_i^2 \right]} = \frac{1}{2\sqrt{m}}. \end{aligned}$$

□

- (c) [bonus question] Let n^* denote the largest number of repetitions of any point in the sample. Give an upper bound on $\hat{\mathfrak{R}}_S(\mathcal{E})$ in terms of n^* .

Mehryar Mohri
Foundations of Machine Learning 2018
Courant Institute of Mathematical Sciences
Homework assignment 1
Sep 18, 2018
Due: Oct 01, 2018

A. Probability review

Let $h: X \rightarrow \{0, 1\}$ be a hypothesis and let S denote an i.i.d. sample of size m . For any $\epsilon > 0$, the following two-sided inequality holds:

$$\Pr_S(|\hat{R}_S(h) - R(h)| > \epsilon) \leq 2e^{-2m\epsilon^2}.$$

Show that the variance of $\hat{R}_S(h)$ satisfies $\text{var}[\hat{R}_S(h)] \leq \frac{\log(2e)}{2m}$. (*Hint:* use the identity $E[X^2] = \int_0^\infty \Pr[X^2 > t]dt$.)

B. PAC learning

1. Consider the concept class C formed by threshold functions on the real line, $C = \{[c, \infty) : \forall c \in \mathbb{R}\} \cup \{(-\infty, c] : \forall c \in \mathbb{R}\}$. Give a PAC-learning algorithm for C . The analysis is similar to that of the axis-aligned rectangles given in class, but you should carefully present and justify your proof.
2. Give a PAC-learning algorithm for the concept class C_2 on $\mathbb{R}^2$ that is formed by intersections of axis-aligned half-spaces: C_2 consists of concepts of the following forms:

$$\begin{aligned} &\{(x, y) : x \geq c_x, y \geq c_y\}, \\ &\{(x, y) : x \geq c_x, y \leq c_y\}, \\ &\{(x, y) : x \leq c_x, y \geq c_y\}, \\ &\{(x, y) : x \leq c_x, y \leq c_y\}, \end{aligned}$$

where $c_x, c_y \in \mathbb{R}$. You should carefully justify all steps of your proof.

Mehryar Mohri
Foundations of Machine Learning 2018
Courant Institute of Mathematical Sciences
Homework assignment 1
Sep 18, 2018
Due: Oct 01, 2018

A. Probability review

Let $h: X \rightarrow \{0, 1\}$ be a hypothesis and let S denote an i.i.d. sample of size m . For any $\epsilon > 0$, the following two-sided inequality holds:

$$\Pr_S(|\hat{R}_S(h) - R(h)| > \epsilon) \leq 2e^{-2m\epsilon^2}.$$

Show that the variance of $\hat{R}_S(h)$ satisfies $\text{var}[\hat{R}_S(h)] \leq \frac{\log(2e)}{2m}$. (*Hint:* use the identity $E[X^2] = \int_0^\infty \Pr[X^2 > t]dt$.)

Solution: For any $u > 0$, we have

$$\begin{aligned} \text{var}[\hat{R}(h)] &= E[(\hat{R}(h) - R(h))^2] \\ &= \int_0^u \Pr[(\hat{R}(h) - R(h))^2 > t]dt + \int_u^\infty \Pr[(\hat{R}(h) - R(h))^2 > t]dt \\ &\leq u + \int_u^\infty 2e^{-2mt} dt \\ &= u + \frac{e^{-2mu}}{m} := f(u). \end{aligned}$$

The function $f(u)$ is a convex function of u , with its minimum value attained when

$$f'(u_0) = 0 \iff 1 - 2e^{-2mu_0} = 0 \iff u_0 = \frac{\log 2}{2m}.$$

Plug in u_0 to get $f(u_0) = \frac{\log(2e)}{2m}$. □

B. PAC learning

1. Consider the concept class C formed by threshold functions on the real line, $C = \{[c, \infty) : \forall c \in \mathbb{R}\} \cup \{(-\infty, c] : \forall c \in \mathbb{R}\}$. Give a PAC-learning algorithm for C . The analysis is similar to that of the axis-aligned

rectangles given in class, but you should carefully present and justify your proof.

Solution: Let $S = \{(x_1, y_1), \dots, (x_m, y_m)\}$ denote the labeled sample of size m . Without loss of generality, assume that the true concept is $[c, \infty)$ for some unknown $c \in \mathbb{R}$. Define

$$\begin{aligned}\hat{l} &= \max\{x_i : (x_i, y_i) \in S, y_i = -1\}, \\ \hat{r} &= \min\{x_i : (x_i, y_i) \in S, y_i = 1\}.\end{aligned}$$

By definition, $\hat{l} \leq c \leq \hat{r}$.

The algorithm returns the concept $R_S = [\hat{c}, \infty)$ with $\hat{c} = (\hat{l} + \hat{r})/2$. The error region of R_S is the interval $[\hat{c}, c)$ when $\hat{c} < c$, and $[c, \hat{c})$ otherwise. In both cases, the error region is a subset of $(\hat{l}, \hat{r})$. Therefore,

$$\begin{aligned}\Pr[R(R_S) > \epsilon] &\leq \Pr[R((\hat{l}, \hat{r})) > \epsilon] \\ &\leq (1 - \epsilon)^m \leq e^{-m\epsilon}.\end{aligned}$$

Setting δ to be greater than or equal to the right-hand side leads to $m \geq \frac{1}{\epsilon} \log(\frac{1}{\delta})$. $\square$

2. Give a PAC-learning algorithm for the concept class C_2 on $\mathbb{R}^2$ that is formed by intersections of axis-aligned half-spaces: C_2 consists of concepts of the following forms:

$$\begin{aligned}&\{(x, y) : x \geq c_x, y \geq c_y\}, \\ &\{(x, y) : x \geq c_x, y \leq c_y\}, \\ &\{(x, y) : x \leq c_x, y \geq c_y\}, \\ &\{(x, y) : x \leq c_x, y \leq c_y\},\end{aligned}$$

where $c_x, c_y \in \mathbb{R}$. You should carefully justify all steps of your proof.

Solution: The algorithm returns the tightest concept in C_2 containing points labeled with 1. The proof is similar to the proof of learning axis-aligned rectangles.

Mehryar Mohri
Foundations of Machine Learning 2020
Courant Institute of Mathematical Sciences
Homework assignment 1
Sep 23, 2020
Due: Oct 07, 2020

A. Consistent hypotheses

In the second lecture, we showed that for a finite hypothesis set $\mathcal{H}$, a consistent learning algorithm $\mathcal{A}$ is a PAC-learning algorithm. Here, we consider a converse question. Let $\mathcal{Z}$ be a finite set of m labeled points. Suppose that you are given a PAC-learning algorithm $\mathcal{A}$. Show that you can use $\mathcal{A}$ and a finite training sample S to find in polynomial time a hypothesis $h \in \mathcal{H}$ that is consistent with $\mathcal{Z}$, with high probability. (*Hint*: you can select an appropriate distribution $\mathcal{D}$ over $\mathcal{Z}$ and give a condition on $R(h)$ for h to be consistent.)

B. Oracle PAC learning

1. Learning unions of intervals. Give a PAC-learning algorithm for the concept class $\mathcal{C}_3$ formed by unions of three closed intervals, that is $[a, b] \cup [c, d] \cup [e, f]$, with $a, b, c, d, e, f \in \mathbb{R}$. You should carefully describe and justify your algorithm. Extend your result to derive a PAC-learning algorithm for the concept class $\mathcal{C}_p$ formed by unions of $p \geq 1$ closed intervals, thus $[a_1, b_1] \cup \dots \cup [a_p, b_p]$, with $a_k, b_k \in \mathbb{R}$ for $k \in [p]$. What are the time and sample complexities of your algorithm as a function of p ?
2. Hypothesis testing. In the previous problem, the learning algorithm was given p as input.
 - (a) Is PAC-learning possible even when p is not provided?

Now, consider, more generally, a family of concept classes $\{\mathcal{C}_s\}_s$ where $\mathcal{C}_s$ is the set of concepts in $\mathcal{C}$ with size at most some integer s . Suppose we have a PAC-learning algorithm $\mathcal{A}$ that can be used for learning any concept class $\mathcal{C}_s$ when s is given. Can we convert

$\mathcal{A}$ into a PAC-learning algorithm $\mathcal{B}$ that does not require the knowledge of s ? This is the main objective of the rest of this problem.

To do so, we first introduce a method for testing a hypothesis h , with high probability. Fix $\epsilon > 0$, $\delta > 0$, and $i \geq 1$ and define the sample size n by $n = \frac{32}{\epsilon} [i \log 2 + \log \frac{2}{\delta}]$. Suppose we draw an i.i.d. sample S of size n according to some unknown distribution $\mathcal{D}$. We will say that a hypothesis h is *accepted* if it makes at most $3/4\epsilon$ errors on S and that it is *rejected* otherwise. Thus, h is accepted iff $\widehat{R}(h) \leq 3/4\epsilon$.

- (b) Assume that $R(h) \geq \epsilon$. Use the (multiplicative) Chernoff bound to show that in that case $\mathbb{P}_{S \sim \mathcal{D}^n}[h \text{ is accepted}] \leq \frac{\delta}{2^{i+1}}$.
- (c) Assume that $R(h) \leq \epsilon/2$. Use the (multiplicative) Chernoff bounds to show that in that case $\mathbb{P}_{S \sim \mathcal{D}^n}[h \text{ is rejected}] \leq \frac{\delta}{2^{i+1}}$.
- (d) Algorithm $\mathcal{B}$ is defined as follows: we start with $i = 1$ and, at each round $i \geq 1$, we guess the parameter size s to be $\tilde{s} = \lfloor 2^{(i-1)/\log \frac{2}{\delta}} \rfloor$. We draw a sample S of size n (which depends on i) to test the hypothesis h_i returned by $\mathcal{A}$ when it is trained with a sample of size $S_{\mathcal{A}}(\epsilon/2, 1/2, \tilde{s})$, that is the sample complexity of $\mathcal{A}$ for a required precision $\epsilon/2$, confidence $1/2$, and size $\tilde{s}$ (we ignore the size of the representation of each example here). If h_i is accepted, the algorithm stops and returns h_i , otherwise it proceeds to the next iteration. Show that if at iteration i , the estimate $\tilde{s}$ is larger than or equal to s , then $\mathbb{P}[h_i \text{ is accepted}] \geq 3/8$.
- (e) Show that the probability that $\mathcal{B}$ does not halt after $j = \lceil \log \frac{2}{\delta} / \log \frac{8}{5} \rceil$ iterations with $\tilde{s} \geq s$ is at most $\delta/2$.
- (f) Show that for $i \geq \lceil 1 + (\log_2 s) \log \frac{2}{\delta} \rceil$, the inequality $\tilde{s} \geq s$ holds.
- (g) Show that with probability at least $1 - \delta$, algorithm $\mathcal{B}$ halts after at most $j' = \lceil 1 + (\log_2 s) \log \frac{2}{\delta} \rceil + j$ iterations and returns a hypothesis with error at most ϵ .

Mehryar Mohri
Foundations of Machine Learning 2020
Courant Institute of Mathematical Sciences
Homework assignment 1
Sep 23, 2020
Due: Oct 07, 2020

A. Consistent hypotheses

In the second lecture, we showed that for a finite hypothesis set $\mathcal{H}$, a consistent learning algorithm $\mathcal{A}$ is a PAC-learning algorithm. Here, we consider a converse question. Let $\mathcal{Z}$ be a finite set of m labeled points. Suppose that you are given a PAC-learning algorithm $\mathcal{A}$. Show that you can use $\mathcal{A}$ and a finite training sample S to find in polynomial time a hypothesis $h \in \mathcal{H}$ that is consistent with $\mathcal{Z}$, with high probability. (*Hint*: you can select an appropriate distribution $\mathcal{D}$ over $\mathcal{Z}$ and give a condition on $R(h)$ for h to be consistent.)

Solution: Since PAC-learning with L is possible for any distribution, let $\mathcal{D}$ be the uniform distribution over $\mathcal{Z}$. Note that, in that case, the cost of an error of a hypothesis h on any point $z \in \mathcal{Z}$ is $\mathbb{P}_{\mathcal{D}}[z] = 1/m$. Thus, if $R_{\mathcal{D}}(h) < 1/m$, we must have $R_{\mathcal{D}}(h) = 0$ and h is consistent. Thus, choose $\epsilon = 1/(m+1)$. Then, for any $\delta > 0$, with probability at least $1 - \delta$ over samples S with $|S| \geq P((m+1), 1/\delta)$ points (where P is some fixed polynomial) the hypothesis h_S returned by L is consistent with $\mathcal{Z}$ since $R_{\mathcal{D}}(h_S) \leq 1/(m+1)$. $\square$

B. Oracle PAC learning

1. Learning unions of intervals. Give a PAC-learning algorithm for the concept class $\mathcal{C}_3$ formed by unions of three closed intervals, that is $[a, b] \cup [c, d] \cup [e, f]$, with $a, b, c, d, e, f \in \mathbb{R}$. You should carefully describe and justify your algorithm. Extend your result to derive a PAC-learning algorithm for the concept class $\mathcal{C}_p$ formed by unions of $p \geq 1$ closed intervals, thus $[a_1, b_1] \cup \dots \cup [a_p, b_p]$, with $a_k, b_k \in \mathbb{R}$ for $k \in [p]$. What are the time and sample complexities of your algorithm as a function of p ?

Solution: Given a sample S , our algorithm consists of the following

steps:

- (a) Sort S in ascending order.
- (b) Loop through sorted S , marking where intervals of consecutive positively labeled points begin and end.
- (c) Return the union of intervals found on the previous step. This union is represented by a list of tuples that indicate start and end points of the intervals.

This algorithm works both for $p = 2$ and for a general p . We will now consider the problem for $\mathcal{C}_2$. To show that this is a PAC-learning algorithm we need to distinguish between two cases.

The first case is when our target concept is a disjoint union of two closed intervals: $I = [a, b] \cup [c, d]$. Note, there are two sources of error: false negatives in $[a, b]$ and $[c, d]$ and also false positives in (b, c) . False positives may occur if no sample is drawn from (b, c) . By linearity of expectation and since these two error regions are disjoint, we have that $R(h_S) = R_{\text{FP}}(h_S) + R_{\text{FN},1}(h_S) + R_{\text{FN},2}(h_S)$, where

$$\begin{aligned} R_{\text{FP}}(h_S) &= \mathbb{P}_{x \sim \mathcal{D}}[x \in h_S, x \notin I], \\ R_{\text{FN},1}(h_S) &= \mathbb{P}_{x \sim \mathcal{D}}[x \notin h_S, x \in [a, b]], \\ R_{\text{FN},2}(h_S) &= \mathbb{P}_{x \sim \mathcal{D}}[x \notin h_S, x \in [c, d]]. \end{aligned}$$

Since we need to have that at least one of $R_{\text{FP}}(h_S)$, $R_{\text{FN},1}(h_S)$, $R_{\text{FN},2}(h_S)$ exceeds $\epsilon/3$ in order for $R(h_S) > \epsilon$, by union bound

$$\begin{aligned} \mathbb{P}(R(h_S) > \epsilon) &\leq \mathbb{P}(R_{\text{FP}}(h_S) > \epsilon/3 \text{ or } R_{\text{FN}(h_S),1} > \epsilon/3 \text{ or } R_{\text{FN}(h_S),2} > \epsilon/3) \\ &\leq \mathbb{P}(R_{\text{FP}}(h_S) > \epsilon/3) + \sum_{i=1}^2 \mathbb{P}(R_{\text{FN}(h_S),i} > \epsilon/3) \end{aligned} \quad (1)$$

We first bound $\mathbb{P}(R_{\text{FP}}(h_S) > \epsilon/3)$. Note that if $R_{\text{FP}}(h_S) > \epsilon/3$, then $\mathbb{P}((b, c)) > \epsilon/3$ and hence

$$\mathbb{P}(R_{\text{FP}}(h_S) > \epsilon/3) \leq (1 - \epsilon/3)^m \leq e^{-m\epsilon/3}.$$

Now we can bound $\mathbb{P}(R_{\text{FN}(h_S),i} > \epsilon/3)$ by $2e^{-m\epsilon/6}$ using the same argument as in the previous question. Therefore,

$$\mathbb{P}(R(h_S) > \epsilon) \leq e^{-m\epsilon/3} + 4e^{-m\epsilon/6} \leq 5e^{-m\epsilon/6}.$$

Setting, the right-hand side to δ and solving for m yields that $m \geq \frac{6}{\epsilon} \log \frac{5}{\delta}$.

The second case that we need to consider is when $I = [a, d]$, that is, $[a, b] \cap [c, d] \neq \emptyset$. In that case, our algorithm reduces to that of learning a single $[a, b]$ interval and it is easy to show that only $m \geq \frac{2}{\epsilon} \log \frac{2}{\delta}$ samples is required to learn this concept, as with the axis-aligned rectangles case discussed in class. Therefore, we conclude that our algorithm is indeed a PAC-learning algorithm.

Extension of this result to the case of $\mathcal{C}_p$ is straightforward. The only difference is that in (1), one has two summations for $p - 1$ regions of false positives and $2p$ regions of false negatives. In that case sample complexity is $m \geq \frac{2(2p-1)}{\epsilon} \log \frac{3p-1}{\delta}$.

Sorting step of our algorithm takes $O(m \log m)$ time and steps (b) and (c) are linear in m , which leads to overall time complexity $O(m \log m)$.

□

2. Hypothesis testing. In the previous problem, the learning algorithm was given p as input.

- (a) Is PAC-learning possible even when p is not provided?

Solution: Argue that it is not possible in general since the union of all $\mathcal{C}_p$ s is not PAC-learnable (infinite VC-dimension). □

Now, consider, more generally, a family of concept classes $\{\mathcal{C}_s\}_s$ where $\mathcal{C}_s$ is the set of concepts in $\mathcal{C}$ with size at most some integer s . Suppose we have a PAC-learning algorithm $\mathcal{A}$ that can be used for learning any concept class $\mathcal{C}_s$ when s is given. Can we convert $\mathcal{A}$ into a PAC-learning algorithm $\mathcal{B}$ that does not require the knowledge of s ? This is the main objective of the rest of this problem.

To do so, we first introduce a method for testing a hypothesis h , with high probability. Fix $\epsilon > 0$, $\delta > 0$, and $i \geq 1$ and define the sample size n by $n = \frac{32}{\epsilon} [i \log 2 + \log \frac{2}{\delta}]$. Suppose we draw an i.i.d. sample S of size n according to some unknown distribution $\mathcal{D}$. We will say that a hypothesis h is *accepted* if it makes at most $3/4\epsilon$ errors on S and that it is *rejected* otherwise. Thus, h is accepted iff $\widehat{R}(h) \leq 3/4\epsilon$.

- (b) Assume that $R(h) \geq \epsilon$. Use the (multiplicative) Chernoff bound to show that in that case $\mathbb{P}_{S \sim \mathcal{D}^n}[h \text{ is accepted}] \leq \frac{\delta}{2^{i+1}}$.

Solution: By definition of acceptance,

$$\begin{aligned}
\mathbb{P}[h \text{ is accepted}] &= \mathbb{P}[\widehat{R}_S(h) \leq 3/4\epsilon] \\
&\leq \mathbb{P}[\widehat{R}_S(h) \leq 3/4 R(h)] && (R(h) \geq \epsilon) \\
&\leq \exp\left(-\frac{n}{2} R(h) (1/4)^2\right) && (\text{Chernoff bound}) \\
&= \exp\left(-\frac{R(h)}{\epsilon} \log \frac{2^{i+1}}{\delta}\right) && (\text{def. of } n) \\
&= \exp\left(-\log \frac{2^{i+1}}{\delta}\right) = \frac{\delta}{2^{i+1}}. && (R(h) \geq \epsilon).
\end{aligned}$$

□

- (c) Assume that $R(h) \leq \epsilon/2$. Use the (multiplicative) Chernoff bounds to show that in that case $\mathbb{P}_{S \sim \mathcal{D}^n}[h \text{ is rejected}] \leq \frac{\delta}{2^{i+1}}$.

Solution: By definition, $\mathbb{P}[h \text{ is rejected}] = \mathbb{P}[\widehat{R}_S(h) \geq \frac{3}{4}\epsilon]$. Since $R(h) \leq \epsilon/2$, $\mathbb{P}[h \text{ is rejected}] \leq \mathbb{P}[\widehat{R}_S(h) \geq \frac{3}{4}\epsilon \mid R(h) = \epsilon/2]$. By the Chernoff bounds, we can thus write

$$\begin{aligned}
\mathbb{P}[h \text{ is rejected}] &\leq \exp\left(-\frac{n}{3} \frac{\epsilon}{2} (1/2)^2\right) && (\text{Chernoff bound}) \\
&= \exp\left(-\frac{4}{3} \log \frac{2^{i+1}}{\delta}\right) && (\text{def. of } n) \\
&\leq \exp\left(-\log \frac{2^{i+1}}{\delta}\right) = \frac{\delta}{2^{i+1}}.
\end{aligned}$$

□

- (d) Algorithm $\mathcal{B}$ is defined as follows: we start with $i = 1$ and, at each round $i \geq 1$, we guess the parameter size s to be $\tilde{s} = \lfloor 2^{(i-1)/\log \frac{2}{\delta}} \rfloor$. We draw a sample S of size n (which depends on i) to test the hypothesis h_i returned by $\mathcal{A}$ when it is trained with a sample of size $S_{\mathcal{A}}(\epsilon/2, 1/2, \tilde{s})$, that is the sample complexity of $\mathcal{A}$ for a required precision $\epsilon/2$, confidence $1/2$, and size $\tilde{s}$ (we ignore the size of the representation of each example here). If h_i is accepted, the algorithm stops and returns h_i , otherwise it proceeds to the next iteration. Show that if at iteration i , the estimate $\tilde{s}$ is larger than or equal to s , then $\mathbb{P}[h_i \text{ is accepted}] \geq 3/8$.

Solution: The estimate $\tilde{s}$ is then an upper bound on s and thus, by definition of algorithm $\mathcal{B}$, $\mathbb{P}[R(h_i) \leq \epsilon/2] \geq 1/2$. If a hypothesis

h has error at least $\epsilon/2$ it is rejected with probability at most $\delta/2^{i+1} \leq \delta/4 \leq 1/4$, therefore, it is accepted with probability at least $3/4$. Thus, for $\tilde{s} \geq s$, $\mathbb{P}[h_i \text{ is accepted}] \geq 1/2 \times 1/4 = 3/8$. $\square$

- (e) Show that the probability that $\mathcal{B}$ does not halt after $j = \lceil \log \frac{2}{\delta} / \log \frac{8}{5} \rceil$ iterations with $\tilde{s} \geq s$ is at most $\delta/2$.

Solution: By the previous question, the probability that algorithm $\mathcal{B}$ fails to halt while $\tilde{s} \geq s$ is at most $1 - 3/8 = 5/8$. Thus, the probability that it does not halt after j iterations is at most $(5/8)^j \leq (5/8)^{\log \frac{2}{\delta} / \log \frac{8}{5}} = \exp \left(\log \frac{2}{\delta} / \log \frac{8}{5} \log \frac{5}{8} \right) = \delta/2$. $\square$

- (f) Show that for $i \geq \lceil 1 + (\log_2 s) \log \frac{2}{\delta} \rceil$, the inequality $\tilde{s} \geq s$ holds.

Solution: By definition,

$$\begin{aligned} \tilde{s} \geq s &\iff \lfloor 2^{(i-1)/\log \frac{2}{\delta}} \rfloor \geq s \\ &\iff 2^{(i-1)/\log \frac{2}{\delta}} \geq s \\ &\iff \frac{i-1}{\log \frac{2}{\delta}} \geq \log_2 s \\ &\iff i \geq 1 + (\log_2 s) \log \frac{2}{\delta} \\ &\iff i \geq \lceil 1 + (\log_2 s) \log \frac{2}{\delta} \rceil. \end{aligned}$$

$\square$

- (g) Show that with probability at least $1 - \delta$, algorithm $\mathcal{B}$ halts after at most $j' = \lceil 1 + (\log_2 s) \log \frac{2}{\delta} \rceil + j$ iterations and returns a hypothesis with error at most ϵ .

Solution: In view of the two previous questions, with probability at least $1 - \delta/2$, algorithm $\mathcal{B}$ halts after at most j' iterations. The probability that the hypothesis it returns be accepted while its error is greater than ϵ is at most $\delta/2^{j'+1} \leq \delta/2$. Thus, with probability $1 - \delta$, the algorithm halts and the hypothesis it returns has error at most ϵ . $\square$

Foundations of Machine Learning
 Department of Computer Science, NYU
 Homework assignment 2
 Due: March 1, 2005

1. Shattering coefficients of intervals in $\mathbb{R}$.

Let C be the set of intervals in $\mathbb{R}$. The VC dimension of H is 2 (shown in class). Compute its shattering coefficient $\Pi_C(m)$, $m \geq 0$. Compare your result with the general bound for shattering coefficients.

2. Bound on shattering coefficients.

Prove that the general bound given for shattering coefficients is tight: for any set X of $m > d$ elements, there exists a concept class C of VC dimension d such that $\Pi_C(m) = \sum_{i=0}^d \binom{m}{i}$.

3. VC dimension of a vector space of real functions.

Let F be a finite-dimensional vector space of real functions on $\mathbb{R}^n$, $\dim(F) = r < \infty$. Let H be the set of hypotheses:

$$H = \{ \{x : f(x) \geq 0\} : f \in F \}.$$

Show that d , the VC dimension of H , is finite and that $d \leq r$ [*Hint*: select an arbitrary set of $m = r + 1$ points and consider linear mapping $u : F \rightarrow \mathbb{R}^m$ defined by: $u(f) = (f(x_1), \dots, f(x_m))$].

4. Soft margin hyperplanes

The function of the slack variables used in the optimization problem for soft margin hyperplanes had the form: $\xi \mapsto \sum_{i=1}^m \xi_i$. Instead, we could use $\xi \mapsto (\sum_{i=1}^m \xi_i)^k$, with $k > 1$. Show that the solution can still be defined in terms of *support vectors*. Give the dual formulation of the problem.

5. SVM classification

- Randomly generate 5,000 examples from each of two Gaussian distributions with the same diagonal covariance matrix αId_{50} , $\alpha = 50$, centered one $(-1, \dots, -1)$ and $(1, \dots, 1)$ in $\mathbb{R}^{50}$. Examples from the first distribution are labeled with -1 , others with $+1$. Split the data into two equal sets of size 5,000, training and test [*hint*: the examples have been already generated for you. Download the files “negative.dat” and “positive.dat” from the usual place].

- Download and install the libsvm software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

- Select a family of kernels, polynomial or Gaussian. Use SVM classification for training set sizes 1,000, 2,000, 3,000, 4,000 and 5,000, and find the kernel parameters that minimize the test error. Plot the test error as a function of the parameters and mark the minimum.
- For a fixed parameter setting, plot the training and test errors as a function of the training set size and estimate the asymptotic test error.
- Compute the *Bayes error*, i.e., the error of the best possible classifier by using the density functions of the two classes. Compare with the results obtained using SVMs.

Foundations of Machine Learning
Department of Computer Science, NYU
Homework assignment 2 – Solution

- (1) [15 points] Shattering coefficients of intervals in $\mathbb{R}$.

Let $\{x_1, \dots, x_m\}$ be a set of m distinct ordered real numbers and let I be an interval. If $I \cap \{x_1, \dots, x_m\}$ contains k numbers, their indices are of the form $j, j+1, \dots, j+k-1$. How many different set of indices are possible? The answer is $m - k + 1$ since j can take values in $\{1, \dots, m - k + 1\}$. Thus, the total number of dichotomies for a set of size m is:

$$1 + \sum_{k=1}^m (m - k + 1) = 1 + \frac{m(m+1)}{2} = \binom{m}{0} + \binom{m}{1} + \binom{m}{2},$$

which matches the general bound on shattering coefficients.

- (2) [15 points] Bound on shattering coefficients.

Let X be an arbitrary set. Consider the set of all subsets of X of size less than or equal to d . The indicator functions of these sets form a concept class whose shattering coefficient is equal to the general upper bound.

- (3) [20 points] VC dimension of a vector space of real functions.

Show that no set of size $m = r + 1$ can be shattered by H . Let $x_1, \dots, x_m$ be m arbitrary points. Define the linear mapping $l : F \rightarrow \mathbb{R}^m$ defined by:

$$l(f) = (f(x_1), \dots, f(x_m))$$

Since the dimension of $\dim(F) = m - 1$, the rank of l is at most $m - 1$ and there exists $\alpha \in \mathbb{R}^m$ orthogonal to $l(F)$:

$$\forall f \in F, \sum_{i=1}^m \alpha_i f(x_i) = 0$$

We can assume that at least one α_i is negative. Then,

$$\forall f \in F, \sum_{i:\alpha_i \geq 0} \alpha_i f(x_i) = - \sum_{i:\alpha_i < 0} \alpha_i f(x_i)$$

Now, assume that there exists a set $\{x : f(x) \geq 0\}$ selecting exactly the x_i s on the left-hand side. Then all the terms on the left-hand side are non-negative, while those on the right-hand side are negative, which cannot be. Thus, $\{x_1, \dots, x_m\}$ cannot be shattered.

(4) [20 points] Soft margin hyperplanes

The corresponding dual problem is:

$$\max_{\alpha, \beta} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \frac{(\alpha_i + \beta_i)^{k/(k-1)}}{(kC)^{1/(k-1)}} \left(1 - \frac{1}{k}\right)$$

subject to:

$$\sum_{i=1}^m \alpha_i y_i = 0 \quad \alpha \geq 0 \quad \beta \geq 0.$$

(5) [30 points] SVM classification

- [(i)] The data was already generated.
- [(ii)] Downloading and installing the library should not have been a problem.
- [(iii)] [7.5 points] Plotting the test error as a function of the parameters should not have been a problem.
- [(iv)] [7.5 points] Plotting the training and test errors as a function of the size of the training data is also straightforward.
- [(v)] [15 points] The density functions of the two classes have the same covariance matrix, thus the best classifier h^* is the one based on the separating hyperplane halfway between the centers of the Gaussians passing through $\frac{\mu_1 + \mu_2}{2} = 0$. The corresponding error is the area where the two density functions overlap, which can be proved to be:

$$\Pr[\text{error}(h^*)] = \int_{\rho/2}^{\infty} \frac{1}{\sqrt{2\pi}} e^{-x^2} dx$$

with $\rho = (\mu_2 - \mu_1)^t \Sigma^{-1} (\mu_2 - \mu_1)$ and $\Sigma = 50I$. An approximate value of the integral is: $\Pr[\text{error}(h^*)] \approx .16$. This seems to be close to the empirical test error, but it must be a lower bound.

Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 2
Due: February 21, 2006

Problem 1: VC dimension

- (1) Show that the VC dimension of the class C of halfspaces over $\mathbb{R}^n$ is $n + 1$. To do that, proceed as follows.
 - (a) Show that $\text{VCdim}(C) \geq n + 1$.
 - (b) Use Radon's theorem: *Any set of $n + 2$ points $X \subset \mathbb{R}^n$ can be partitioned in two subsets X_1 and X_2 such that the convex hulls of X_1 and X_2 intersect.*
to show that $\text{VCdim}(C) \leq n + 1$.
 - (c) Prove Radon's theorem.
- (2) Consider now the class C_k of convex intersections of k halfspaces. Give lower and upper bounds estimates for $\text{VCdim}(C_k)$.
- (3) Let A and B be two sets of functions mapping from X into $\{0, 1\}$, and assume that both A and B have finite VC dimension, with $\text{VCdim}(A) = d_A$ and $\text{VCdim}(B) = d_B$. Let $C = A \cup B$ be the union of A and B .
 - (a) Prove that for all m , $\Pi_C(m) \leq \Pi_A(m) + \Pi_B(m)$.
 - (b) Use Sauer's lemma to show that for $m \geq d_A + d_B + 2$, $\Pi_C(m) < 2^m$, and give a bound on the VC dimension of C .

Problem 2: Sample complexity

A function $h : \{0, 1\}^n \rightarrow \{0, 1\}$ is *symmetric* if its value is uniquely determined by the number of 1's in the input. Let C denote the set of all symmetric functions.

- (a) Determine the VC dimension of C .
- (b) Give lower and upper bounds on the sample complexity of any consistent PAC learning algorithm for C .

- (c) Note that any hypothesis $h \in C$ can be represented by a vector $(y_0, y_1, \dots, y_n) \in \{0, 1\}^{n+1}$, where y_i is the value of h on examples having precisely i 1's. Devise a consistent learning algorithm for C based on that representation.

Problem 1: VC dimension [75 points]

(1) [25 points]

- (a) [5 points] It suffices to show the existence of a set of $n + 1$ points in $\mathbb{R}^n$ that can be shattered by halfspaces. Let x_0 be the origin and define x_i as the point whose i th coordinate is 1 and all others 0. Let $y_0, y_1, \dots, y_n \in \{-1, +1\}$ be an arbitrary set of labels for $x_0, \dots, x_n$. Let w be the vector whose i th coordinate is y_i . Then, the classifier defined by the hyperplane of equation $w \cdot x + y_0/2 = 0$ shatters $x_0, \dots, x_n$ since:

$$\begin{aligned} \text{sign}(w \cdot x_0 + y_0/2) &= \text{sign}(y_0 + y_0/2) = y_0, \text{ and} \\ \forall i \geq 1, \text{sign}(w \cdot x_i + y_0/2) &= \text{sign}(y_i + y_0/2) = y_i. \end{aligned} \quad (1)$$

- (b) [10 points] It suffices to show that no set of $n + 2$ points can be shattered by halfspaces. Let X be a set of $n + 2$ points. By Radon's theorem, it can be split into two sets X_1 and X_2 such that their convex hulls intersect.

Observe that when two sets of points X_1 and X_2 are separated by a hyperplane, their convex hulls are also separated by that hyperplane. Thus, X_1 and X_2 cannot be separated by a hyperplane and X is not shattered.

- (c) [10 points] Let $I_1 = \{i \in [1, n + 2] : x_i \in X_1\}$ and $I_2 = \{i \in [1, n + 2] : x_i \in X_2\}$. x is in the convex hull of X_1 and X_2 iff there exist $(\alpha_i)_{i \in I_1}$ and $(\alpha_i)_{i \in I_2}$ such that

$$\begin{aligned} x &= \sum_{i \in I_1} \alpha_i x_i \text{ with } \sum_{i \in I_1} \alpha_i = 1, \text{ and} \\ x &= \sum_{i \in I_2} \alpha_i x_i \text{ with } \sum_{i \in I_2} \alpha_i = 1. \end{aligned} \quad (2)$$

This leads to the following system of $n + 1$ equations in $n + 2$ unknown α_i :

$$\begin{cases} \sum_{i \in I_1} \alpha_i x_i - \sum_{i \in I_2} \alpha_i x_i = 0 \\ \sum_{i \in I_1} \alpha_i - \sum_{i \in I_2} \alpha_i = 0, \end{cases} \quad (3)$$

which has a non-trivial solution. This proves Radon's theorem.

- (2) [30 points] Let $m \geq 0$. Note the general fact that for any concept class $C = \{c_1 \cap c_2 : c_1 \in C_1, c_2 \in C_2\}$,

$$\Pi_C(m) \leq \Pi_{C_1}(m) \Pi_{C_2}(m). \quad (4)$$

Indeed, fix a set X of m points. Let $Y_1, \dots, Y_k$ be the traces of C_1 on X . By definition of $\Pi_{C_1}(X)$, $k \leq \Pi_{C_1}(X) \leq \Pi_{C_1}(m)$. By definition of $\Pi_{C_2}(Y_i)$, The traces of C_2 on a subset Y_i are at most $\Pi_{C_2}(Y_i) \leq \Pi_{C_2}(m)$. Thus, the traces of C on X are at most

$$k \Pi_{C_2}(Y_i) \leq \Pi_{C_1}(m) \Pi_{C_2}(m). \quad (5)$$

For the particular case of C_k , using Sauer's lemma, this implies that

$$\Pi_{C_k}(m) \leq (\Pi_{C_1}(m))^k \leq \left(\frac{em}{n+1} \right)^{k(n+1)}. \quad (6)$$

If $(em/(n+1))^{k(n+1)} < 2^m$, then the VC dimension of C_k is less than m . If the VC dimension of C_k is m , then $\Pi_{C_k}(m) = 2^m \leq (em/(n+1))^{k(n+1)}$. These inequalities give an upper bound and a lower bound on $\text{VCdim}(C_k)$. As an example, using the identity: $\forall x \in \mathbb{N} - \{3\}, \log_2(x) \leq x/2$, one can verify that:

$$\text{VCdim}(C_k) \leq 2(n+1)k \log(3k). \quad (7)$$

- (3) [20 points]

- (a) [5 points] When $C = A \cup B$, $\Pi_C(X) \leq \Pi_A(X) + \Pi_B(X)$ for any set X since dichotomies in $\Pi_C(X)$ can be generated by A or by B . Thus, for all m , $\Pi_C(m) \leq \Pi_A(m) + \Pi_B(m)$.
- (b) [15 points] For $m \geq d_A + d_B + 2$, by Sauer's lemma,

$$\begin{aligned} \Pi_C(m) &\leq \sum_{i=0}^{d_A} \binom{m}{i} + \sum_{i=0}^{d_B} \binom{m}{i} = \sum_{i=0}^{d_A} \binom{m}{i} + \sum_{i=0}^{d_B} \binom{m-i}{i} \\ &= \sum_{i=0}^{d_A} \binom{m}{i} + \sum_{i=m-d_B}^{d_B} \binom{m}{i} \end{aligned} \quad (8)$$

$$\leq \sum_{i=0}^{d_A} \binom{m}{i} + \sum_{i=d_A+2}^{d_B} \binom{m}{i} \quad (9)$$

$$< \sum_{i=0}^m \binom{m}{i} = 2^m. \quad (10)$$

Thus, the VC dimension of C is strictly less than $d_A + d_B + 2$:

$$\text{VCdim}(C) \leq d_A + d_B + 1. \quad (11)$$

Is this bound tight (can you show that for any d_A and d_B , there exist sets A and B such that equality holds)?

Problem 2: Sample complexity [25 points]

- (a) [15 points] For $i = 0, \dots, n$, let $x_i \in \{0, 1\}^n$ be defined by $x_i = (\underbrace{1, \dots, 1}_i, 0, \dots, 0)$. Then, $\{x_0, \dots, x_n\}$ can be shattered by C . Indeed, let $y_0, \dots, y_n \in \{0, 1\}$ be an arbitrary labeling of these points. Then, the function h defined by:

$$h(x) = y_i \quad (12)$$

for all x with i 1's is symmetric and $h(x_i) = y_i$. Thus, $\text{VCdim}(C) \geq n + 1$. Conversely, a set of $n + 2$ points cannot be shattered by C since at least two points would then have the same number of 1's and will not be distinguishable by C . Thus,

$$\text{VCdim}(C) = n + 1. \quad (13)$$

- (b) [5 points] Thus, in view of the theorems presented in class, a lower bound on the number of training examples needed to learn symmetric functions with accuracy $1 - \epsilon$ and confidence $1 - \delta$ is

$$\Omega\left(\frac{1}{\epsilon} \log \frac{1}{\delta} + \frac{n}{\epsilon}\right), \quad (14)$$

and an upper bound is:

$$O\left(\frac{1}{\epsilon} \log \frac{1}{\delta} + \frac{n}{\epsilon} \log \frac{1}{\epsilon}\right), \quad (15)$$

which is only within a factor $\frac{1}{\epsilon}$ of the lower bound.

- (c) [5 points] This is trivial. For a training data $(z_0, t_0), \dots, (z_m, t_m) \in \{0, 1\}^n \times \{0, 1\}$ define h as the symmetric function such that $h(z_i) = t_i$ for all $i = 0, \dots, m$.

Can you show that in view of the bounds given in (b), this algorithm is optimal?

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 2
Due: February 27, 2007

A. Bounds on VC dimension

1. Show that the VC dimension of the set of all closed balls in $\mathbb{R}^n$, that is sets of the form $\{x \in \mathbb{R}^n : \|x - x_0\|^2 \leq r\}$ for some $x_0 \in \mathbb{R}^n$ and $r \geq 0$ is less than or equal to $n + 2$.
2. Determine the VC dimension of the subsets of the real line formed by the union of k intervals.

B. VC dimension of intersection concepts

1. Let C_1 and C_2 be two concept classes. Show that for any concept class $C = \{c_1 \cap c_2 : c_1 \in C_1, c_2 \in C_2\}$,

$$\Pi_C(m) \leq \Pi_{C_1}(m) \Pi_{C_2}(m). \quad (1)$$

2. Let C be a concept class with VC dimension d and let C_s be the concept class formed by all intersections of s concepts from C , $s \geq 1$. Show that the VC dimension of C_s is bounded by $2ds \log_2(3s)$ (*Hint*: show that $\log_2(3x) < 9x/(2e)$ for any $x \geq 2$).

C. Infinite VC dimension

1. Show that if a concept class C has infinite VC dimension, then it is not PAC-learnable.
2. In the standard PAC-learning scenario, the learning algorithm receives all examples first and then computes its hypothesis. Within that setting, PAC-learning of concept classes with infinite VC dimension is not possible as seen in the previous question.

Imagine now a different scenario where the learning algorithm can alternate between drawing more examples and computation. The objective of this problem is to prove that PAC-learning can then be possible for some concept classes with infinite VC dimension.

To do so, consider for example the special case of the concept class C of all subsets of natural numbers.

- (a) Show that the VC dimension of C is infinite.
- (b) Professor Vitres has an idea for the first stage of a learning algorithm L PAC-learning C . In the first stage, L draws a sufficient number of points m such that the probability of drawing a point beyond the maximum value M observed be small, with high confidence. Can you complete Professor Vitres' idea by describing the second stage of the algorithm so that it PAC-learns C ? The description should be augmented with the proof that L can PAC-learn C .

A. Bounds on VC dimension

[30 points]

1. [15 points] Show that the VC dimension of the set of all closed balls in $\mathbb{R}^n$, that is sets of the form $\{x \in \mathbb{R}^n : \|x - x_0\|^2 \leq r\}$ for some $x_0 \in \mathbb{R}^n$ and $r \geq 0$ is less than or equal to $n + 2$.

Let $B(a, r)$ be the ball of radius r centered at $a \in \mathbb{R}^n$. Then $x \in B(a, r)$ iff

$$\sum_{i=1}^n \|x_i\|^2 - 2 \sum_{i=1}^n a_i x_i + \sum_{i=1}^n a_i^2 - r \leq 0, \quad (1)$$

which is equivalent to

$$\langle W, X \rangle + B \leq 0, \quad (2)$$

with $W = \begin{bmatrix} 1 \\ -2a_1 \\ \dots \\ -2a_n \end{bmatrix}$, $X = \begin{bmatrix} \sum_{i=1}^n \|x_i\|^2 \\ x_1 \\ \dots \\ x_n \end{bmatrix}$, and $B = \sum_{i=1}^n a_i^2 - r$.

The VC dimension of closed balls in $\mathbb{R}^n$ is thus at most equal to the VC dimension of hyperplanes in $\mathbb{R}^{n+1}$, that is $n + 2$.

2. [15 points] Determine the VC dimension of the subsets of the real line formed by the union of k intervals.

It is not hard to see that any $2k$ distinct points on the real line can be shattered using k intervals: it suffices to shatter each of the k pairs of consecutive points with an interval. Assume now that $2k + 1$ distinct points $x_1 < \dots < x_{2k+1}$ are given. For any $i \in [1, 2k + 1]$, label x_i with $(-1)^{i+1}$, that is alternatively label points with 1 or -1 . This leads to $k + 1$ points labeled positively and requires $2k + 1$ intervals to shatter the set since no interval can contain two consecutive points. Thus, no set of $2k + 1$ points can be shattered by k intervals and the VC dimension of the union of k intervals is $2k$.

B. VC dimension of intersection concepts

[30 points]

1. [15 points] Let C_1 and C_2 be two concept classes. Show that for any concept class $C = \{c_1 \cap c_2 : c_1 \in C_1, c_2 \in C_2\}$,

$$\Pi_C(m) \leq \Pi_{C_1}(m) \Pi_{C_2}(m). \quad (3)$$

Fix a set X of m points. Let $Y_1, \dots, Y_k$ be the set of intersections of the concepts of C_1 with X . By definition of $\Pi_{C_1}(X)$, $k \leq \Pi_{C_1}(X) \leq \Pi_{C_1}(m)$. By definition of $\Pi_{C_2}(Y_i)$, the intersection of the concepts of C_2 with Y_i are at most $\Pi_{C_2}(Y_i) \leq \Pi_{C_2}(m)$. Thus, the number of sets intersections of concepts of C with X is at most

$$k \Pi_{C_2}(Y_i) \leq \Pi_{C_1}(m) \Pi_{C_2}(m). \quad (4)$$

2. [15 points] Let C be a concept class with VC dimension d and let C_s be the concept class formed by all intersections of s concepts from C , $s \geq 1$. Show that the VC dimension of C_s is bounded by $2ds \log_2(3s)$ (*Hint*: show that $\log_2(3x) < 9x/(2e)$ for any $x \geq 2$).

In view of the result proved in the previous question, $\Pi_{C_s}(m) \leq (\Pi_{C_1}(m))^s$. By Sauer's lemma, this implies

$$\Pi_{C_s}(m) \leq \left(\frac{em}{d}\right)^{sd}. \quad (5)$$

If $\left(\frac{em}{d}\right)^{sd} < 2^m$, then the VC dimension of C_s is less than m . Thus, it suffices to show this inequality holds with $m = 2ds \log_2(3s)$. Plugging in that value for m and taking the $\log_2$ yield:

$$ds \log_2(2es \log_2(3s)) < 2ds \log_2(3s) \quad (6)$$

$$\Leftrightarrow \log_2(2es \log_2(3s)) < 2 \log_2(3s) = \log_2(9s^2) \quad (7)$$

$$\Leftrightarrow 2es \log_2(3s) < 9s^2 \quad (8)$$

$$\Leftrightarrow \log_2(3s) < \frac{9s}{2e}. \quad (9)$$

This last inequality holds for $s = 2$: $\log_2(6) \approx 2.6 < 9/(2e) \approx 3.3$. Since the functions corresponding to the left-hand-side grows more slowly than the one corresponding to the right-hand-side (compare derivatives for example), this implies that the inequality holds for all $s \geq 2$.

C. Infinite VC dimension

[65 points]

1. [15 points] Show that if a concept class C has infinite VC dimension, then it is not PAC-learnable.

By a theorem of (Ehrenfeucht et al., 1988) presented in class, any algorithm for PAC-learning a concept class C of VC dimension d with accuracy $1 - \epsilon$ and confidence $1 - \delta$ must use a sample size $m = \Omega(\frac{1}{\epsilon} \log \frac{1}{\delta} + \frac{d}{\epsilon})$. For an infinite VC-dimension, this lower bound requires an infinite number of points and thus implies that PAC-learning is not possible.

2. [50 points] In the standard PAC-learning scenario, the learning algorithm receives all examples first and then computes its hypothesis. Within that setting, PAC-learning of concept classes with infinite dimension is not possible as seen in the previous question.

Imagine now a different scenario where the learning algorithm can alternate between drawing more examples and computation. The objective of this problem is to prove that PAC-learning can then be possible for some concept classes with infinite VC dimension.

To do so, consider for example the special case of the concept class C of all subsets of natural numbers.

- (a) [0 point] Show that the VC dimension of C is infinite.

This is rather straightforward.

- (b) [50 points] Professor Vitres has an idea for the first stage of a learning algorithm L PAC-learning C . In the first stage, L draws a sufficient number of points m such that the probability of drawing a point beyond the maximum value M observed be small, with high confidence. Can you complete Professor Vitres' idea by describing the second stage of the algorithm so that it PAC-learns C ? The description should be augmented with the proof that L can PAC-learn C .

Here is a description of the algorithm. Let M be the maximum value observed after drawing m points and let p be the probability that a point greater than M be drawn. The probability that all points drawn be smaller than or equal to M is

$$(1 - p)^m \leq e^{-pm}. \quad (10)$$

Setting $\delta/2$ to match the upper bound, yields $\delta/2 = e^{-pm}$, that is

$$p = \frac{1}{m} \log \frac{2}{\delta}. \quad (11)$$

To bound p by $\epsilon/2$, we can impose the following

$$\frac{1}{m} \log \frac{2}{\delta} \leq \frac{\epsilon}{2}. \quad (12)$$

Thus, with confidence at least $1 - \delta/2$, the probability that a point greater than M be drawn is at most $\epsilon/2$ if L draws $m \geq \frac{2}{\epsilon} \log \frac{2}{\delta}$ points.

In the second stage, the problem is reduced to a finite VC dimension M . Since PAC-learning with $(\epsilon/2, \delta/2)$ is possible for a finite dimension, this guarantees the (ϵ, δ) -PAC-learning of the full algorithm.

Mehryar Mohri
 Foundations of Machine Learning
 Courant Institute of Mathematical Sciences
 Homework assignment 2
 Due: Mar 7th, 2008
 Credit: Ashish Rastogi, Afshin Rostamizadeh
 Ameet Talwalkar, and Eugene Weinstein.

1. Show that a finite concept class $\mathcal{C}$ has VC dimension at most $\log |\mathcal{C}|$.
2. Determine the VC dimension of the following concept classes:
 - (a) The class of all polygons with k vertices in the plane.
 - (b) The class of all circles in the plane.
 - (c) The class of union of k intervals on the real line.
3. [VC Dimension] Let F be a finite-dimensional vector space of real functions on $\mathbb{R}^n$, $\dim(F) = r < \infty$. Let H be the set of hypotheses:

$$H = \{\{x : f(x) \geq 0\} : f \in F\}.$$

Show that d , the VC dimension of H , is finite and that $d \leq r$ [*Hint*: select an arbitrary set of $m = r + 1$ points and consider the linear mapping $u : F \mapsto \mathbb{R}^m$ defined by: $u(f) = (f(x_1), \dots, f(x_m))$.]

4. [Regression] Consider the problem of learning a real valued function $h : \mathbb{R}^n \mapsto \mathbb{R}$ based on a training sample $S = \{(x_i, y_i), 1 \leq i \leq m\}$, $x_i \in \mathbb{R}^n$ and $y_i \in \mathbb{R}$. Consider $h(x) = w \cdot x$, where the weight vector $w \in \mathbb{R}^n$ is determined according to the solution of the following optimization problem:

$$\min_{w \in \mathbb{R}^n} \frac{1}{2} \|w\|_2^2 + \gamma \sum_{i=1}^m (w \cdot x_i - y_i)^2.$$

Let $\mathbf{X}$ be a $m \times n$ matrix where $\mathbf{X}_{i,j} = (x_i)_j$ and let $\mathbf{Y}$ be an m -dimensional column vector whose i th coordinate is y_i . Finally let $\mathbf{W}$ denote the n -dimensional column vector corresponding to the weight vector w .

- (a) Express the objective function above in terms of matrices $\mathbf{X}$ and the vectors $\mathbf{W}, \mathbf{Y}$, together with the tradeoff constant γ .

- (b) Determine the closed-form solution for the optimal weight vector $\mathbf{W}^*$ in terms of $\mathbf{X}, \mathbf{Y}, \gamma$ (let $\mathbf{I}$ denote the identity matrix). [Hint: you may use $\frac{\partial \|\mathbf{A}\|_2^2}{\partial \mathbf{A}} = 2\mathbf{A}$ for a matrix $\mathbf{A}$].
- (c) What is the time complexity of computing the optimal weight vector $\mathbf{W}$ as a function of the number of features n and the number of training points m . What is the complexity of computing $h(x)$ for a new point $x \in \mathbb{R}^n$?
- (d) The matrix $\mathbf{X}\mathbf{X}^T$ is called the Gram matrix $\mathbf{K}$. Using the observation that

$$\mathbf{X}^T (\mathbf{K} + \gamma \mathbf{I})^{-1} = (\mathbf{X}^T \mathbf{X} + \gamma \mathbf{I})^{-1} \mathbf{X}^T,$$

derive another expression for the optimal weight vector $\mathbf{W}$. What is the complexity of computing using this alternate closed-form expression?

1. [10 points] Show that a finite concept class $\mathcal{C}$ has VC dimension at most $\log |\mathcal{C}|$.

Solution: Proof by contraposition. Suppose that the VC dimension is $d > \log |\mathcal{C}|$. There must exist a set of d points such that every dichotomy on those d points is realized. The number of possible dichotomies is $2^d > 2^{\log |\mathcal{C}|} = |\mathcal{C}|$. But there are only $|\mathcal{C}|$ distinct concept classes.

2. [30 points] Determine the VC dimension of the following concept classes:
 - (a) The class of all polygons with k vertices in the plane.

Solution: In order to show that the VC-dimension of a class of concepts is d , we need to show that there exists a set of size d on which all dichotomies are realized, and that on all sets of size $d + 1$, there is some dichotomy that is not realized by the concept class. We know from class that the class of convex polygons with k vertices has VC dimension of $2k + 1$. Thus, we know that the VC dimension of our class is at least $2k + 1$. It can be shown that for all sets of size $2k + 2$ points, there is a labeling of these points that cannot be captured by (even) non-convex polygons with k vertices.

- (b) The class of all circles in the plane.

Solution: It is clear that any two points can be shattered by a circle. Any three non-colinear points can also be shattered. Now, given any four points, there are two cases. The first, that the convex hull of these four points is a triangle. If so, labelling the points on the triangle as positive and the point inside as negative is a dichotomy that cannot be realized by a circle. If the convex hull of the four points is a quadrilateral, then choosing the further of the two diagonally opposite points as positive and the other two as negative is a dichotomy that cannot be realized also. Finally, if the four points are colinear, there is a trivial dichotomy of alternate positive and negatives that cannot be realized. Thus, VC dimension of all circles in the plane is 3.

- (c) The class of union of k intervals on the real line.

Solution: Easy to check that a sequence of $2k + 1$ points on a line cannot be shattered, if successive points are labeled with alternate labels, starting with a positive label. Thus, VC dimension of the class of union of k intervals on the real line is $2k$.

3. [VC Dimension: 20 points] Let F be a finite-dimensional vector space of real functions on $\mathbb{R}^n$, $\dim(F) = r < \infty$. Let H be the set of hypotheses:

$$H = \{ \{x : f(x) \geq 0\} : f \in F \}.$$

Show that d , the VC dimension of H , is finite and that $d \leq r$ [Hint: select an arbitrary set of $m = r + 1$ points and consider the linear mapping $u : F \mapsto \mathbb{R}^m$ defined by: $u(f) = (f(x_1), \dots, f(x_m))$.]

Solution: Show that no set of size $m = r + 1$ can be shattered by H . Let $x_1, \dots, x_m$ be m arbitrary points. Define the linear mapping $l : F \mapsto \mathbb{R}^m$ defined by:

$$l(f) = (f(x_1), \dots, f(x_m)).$$

Since the dimension of $\dim(F) = m - 1$, the rank of l is at most $m - 1$ and there exists $\alpha \in \mathbb{R}^m$ orthogonal to $l(F)$:

$$\forall f \in F, \sum_{i=1}^m \alpha_i f(x_i) = 0.$$

We can assume that at least one α_i is negative. Then,

$$\forall f \in F, \sum_{i:\alpha_i \geq 0} \alpha_i f(x_i) = - \sum_{i:\alpha_i < 0} \alpha_i f(x_i).$$

Now, assume that there exists a set $\{x : f(x) \geq 0\}$ selecting exactly the x_i s on the left-hand side. Then all the terms on the left-hand side are non-negative, while those on the right-hand side are negative, which cannot be. Thus, $\{x_1, \dots, x_m\}$ cannot be shattered.

4. [Regression] Consider the problem of learning a real valued function $h : \mathbb{R}^n \mapsto \mathbb{R}$ based on a training sample $S = \{(x_i, y_i), 1 \leq i \leq m\}$, $x_i \in \mathbb{R}^n$ and $y_i \in \mathbb{R}$. Consider $h(x) = w \cdot x$, where the weight vector $w \in \mathbb{R}^n$ is determined according to the solution of the following optimization problem:

$$\min_{w \in \mathbb{R}^n} \frac{1}{2} \|w\|_2^2 + \gamma \sum_{i=1}^m (w \cdot x_i - y_i)^2.$$

Let X be a $m \times n$ matrix where $X_{i,j} = (x_i)_j$ and let Y be an m -dimensional column vector whose i th coordinate is y_i . Finally let W denote the n -dimensional column vector corresponding to the weight vector w .

- (a) [10 points] Express the objective function above in terms of matrices X and the vectors W, Y , together with the tradeoff constant γ .

Solution:

$$F = \frac{1}{2} W^T W + \gamma ((XW - Y)^T (XW - Y)).$$

- (b) [20 points] Determine the closed-form solution for the optimal weight vector W^* in terms of X, Y, γ (let I denote the identity matrix). [Hint: you may use $\frac{\partial \|A\|_2^2}{\partial A} = 2A$ for a matrix A .]

Solution:

$$\mathbf{W}^* = 2\gamma [\mathbf{I} + 2\gamma \mathbf{X}^T \mathbf{X}]^{-1} \mathbf{X}^T \mathbf{Y}.$$

- (c) [10 points] What is the time complexity of computing the optimal weight vector $\mathbf{W}$ as a function of the number of features n and the number of training points m . What is the complexity of computing $h(x)$ for a new point $x \in \mathbb{R}^n$?

Solution: The complexity of multiplying two matrices $\mathbf{A}, \mathbf{B}$ of dimensions $a \times b, b \times c$ respectively is $O(abc)$. The complexity of inverting a square matrix $\mathbf{A}$ of dimension $a \times a$ is $O(a^3)$ (faster algorithms exist, for e.g. Strassen's algorithm) but the more well-known $O(a^3)$ is accepted as a reference for this problem.

Based on these observations, the complexity of computing $\mathbf{I} + 2\gamma \mathbf{X}^T \mathbf{X}$ is $O(n^2 m)$. The complexity of inverting this matrix is $O(n^3)$. The complexity of multiplying the resulting $n \times n$ matrix with $\mathbf{X}^T \mathbf{Y}$ is $O(n^2 m)$. Thus, the overall complexity is $O(n^2(m + n))$.

- (d) [10 points] The matrix $\mathbf{X}\mathbf{X}^T$ is called the Gram matrix $\mathbf{K}$. Using the observation that

$$\mathbf{X}^T (\mathbf{K} + \gamma \mathbf{I})^{-1} = (\mathbf{X}^T \mathbf{X} + \gamma \mathbf{I})^{-1} \mathbf{X}^T,$$

derive another expression for the optimal weight vector $\mathbf{W}$. What is the complexity of computing using this alternate closed-form expression?

Solution: The optimal hyperplane $\mathbf{W}^*$ in the *dual* is given by:

$$\mathbf{W}^* = 2\gamma \mathbf{X}^T [\mathbf{I} + 2\gamma \mathbf{K}]^{-1} \mathbf{Y}.$$

The complexity of obtaining this solution is $O(m^2(m + n))$. Using this solution is more efficient when the number of sample points m is far smaller than the number of features n .

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 2
Due: March 17, 2009

A. Support Vector Machines

1. Download and install the `libsvm` software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

2. Download the `satimage` data set. Merge the training and validation set into one. We will refer to the resulting set as the training set from now on. Normalize all input vectors (training and test).
3. Consider the binary classification that consists of distinguishing class 6 from the rest. Use SVMs combined with polynomial kernels to solve this classification problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let the other parameters of polynomial kernels in `libsvm`, γ and c , be equal to their default values 1). Report the best value of the trade-off constant C measured on the validation set.

4. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation training and test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d .
5. How many of the support vectors lie on the margin hyperplanes?
6. In the standard two-group classification, errors on positive or negative points are counted the same way. Suppose we wish to penalize an error on a negative point (false positive error) $k > 0$ times more than an error on a positive point. Give the dual optimization problem corresponding to SVMs modified in this way.
7. Assume that k is an integer. Show how you could use `libsvm` without writing any additional code to find the solution of the modified SVMs just described.

8. Apply the modified SVMs to the classification task previously examined and compare with your previous SVMs results for different values of k : $k = 2, 4, 8, 16$.

B. Kernels

1. Let $K: X \times X \rightarrow \mathbb{R}$ be a PDS kernel, show that for all $x, y \in X$,

$$|K(x, y)| \leq \sqrt{K(x, x)K(y, y)}. \quad (1)$$

2. Show that the following kernels K are PDS:

- (a) $K(x, y) = \cos(x - y)$ over $\mathbb{R} \times \mathbb{R}$.
- (b) $K(x, y) = \cos(x^2 - y^2)$ over $\mathbb{R} \times \mathbb{R}$.
- (c) $K(x, y) = (x + y)^{-1}$ over $]0, +\infty[\times]0, +\infty[$.

3. Show that the following kernels K are NDS:

- (a) $K(x, y) = [\sin(x - y)]^2$ over $\mathbb{R} \times \mathbb{R}$.
- (b) $K(x, y) = \log(x + y)$ over $]0, +\infty[\times]0, +\infty[$.

4. Is the kernel K defined over $\mathbb{R}^n \times \mathbb{R}^n$ by $K(x, y) = \|x - y\|^{3/2}$ PDS? Is it NDS? Prove your responses.

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 2 - Solution
March 30, 2009

A. Support Vector Machines

The solution for this section is based on that of Chris Alberti and input from Ashish Rastogi.

1. Download and install the `libsvm` software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

2. [5 points] Download the `satimage` data set. Merge the training and validation set into one. We will refer to the resulting set as the training set from now on. Normalize all input vectors (training and test).

```
cat \
  satimage/satimage.scale.t \
  satimage/satimage.scale.val > data/train
libsvm-2.88/svm-scale \
  data/train > data/train.scaled
```

3. [10 points] Consider the binary classification that consists of distinguishing class 6 from the rest. Use SVMs combined with polynomial kernels to solve this classification problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let the other parameters of polynomial kernels in `libsvm`, γ and c , be equal to their default values 1). Report the best value of the trade-off constant C measured on the validation set.

Run training script:

```
python2.5 hw2A.py \
  --d 1,4,1 --log2c -8,6,.5 \
  --out train/train.scaled.out \
  data/train.scaled
```

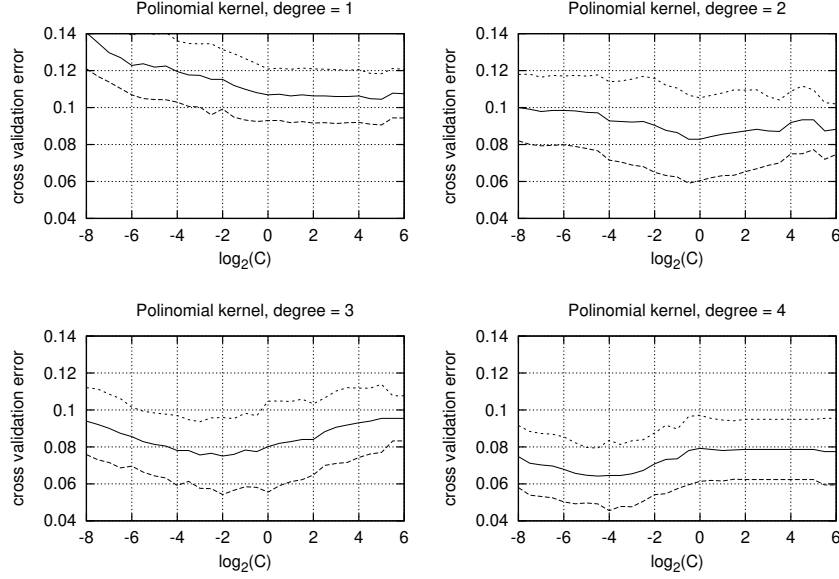


Figure 1: Average cross-validation error plus or minus one standard deviation for different values of the trade-off constant C and of the degree of the polynomial kernel

This script runs 10-fold cross validation, for different values of the degree d of the polynomial kernel and of the trade-off constant C . We test $d = 1, 2, 3, 4$ and $\log_2(C) = -8, -7.5, \dots, 5.5, 6$. We step the value of the trade-off constant logarithmically as suggested by:

<http://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf>

Which gives the cross-validation error plots shown in Figure 1.

The best values of trade-off constant C are:

$d = 1: C^* = 2^{(+5.0)} = 32$	$cv\text{-err} = 10.4\% \pm 1.4\%$
$d = 2: C^* = 2^{(+0.0)} = 1$	$cv\text{-err} = 8.3\% \pm 2.2\%$
$d = 3: C^* = 2^{(-2.0)} = .25$	$cv\text{-err} = 7.5\% \pm 2.1\%$
$d = 4: C^* = 2^{(-4.5)} = .0442$	$cv\text{-err} = 6.4\% \pm 1.5\%$

The best C measured on the validation set is $C^* = 2^{-4.5}$, with degree $d^* = 4$, which gives an average error rate of $6.4\% \pm 1.5\%$.

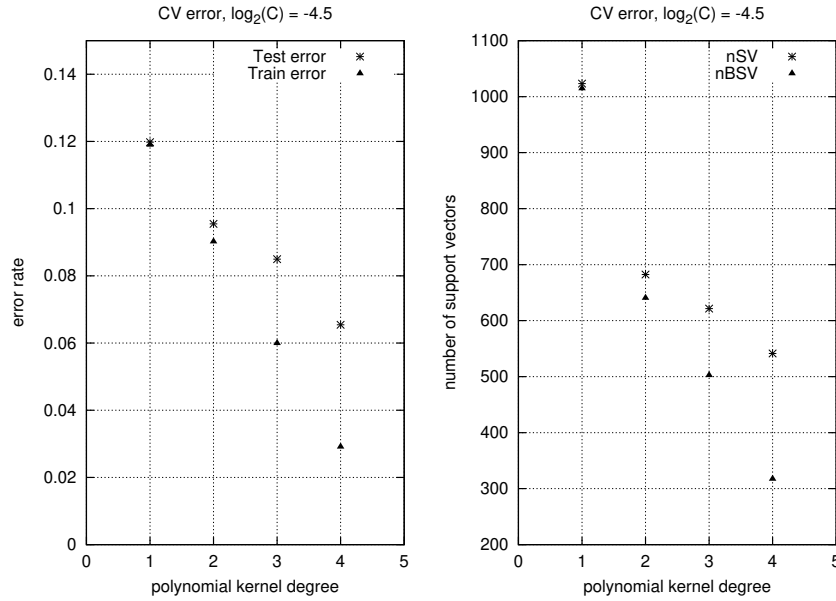


Figure 2: Average cross-validation error rates and average number of support vectors (nSV) and of bounded support vectors(nBSV) as a function of the degree of the polynomial kernel

4. [10 points] Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation training and test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d .

Run script again:

```
python2.5 hw2A.py \
  --d 1,4,1 --log2c=-4.5,-4.5,1 \
  data/train.scaled
```

The trade-off constant is fixed to $C^* = 2^{-4.5}$, and 10-fold cross validation is run for degrees 1 through 4. In Figure 2 we plot the resulting cross-validation training and test errors and the average number of support vectors (nSV is the number of support vectors, nBSV is the number of bounded support vectors, i.e. whose dual variable is equal to the trade-off constant).

5. [5 points] How many of the support vectors lie on the margin hyperplanes?

Support vectors always lie on the margin hyperplanes when their dual variable is smaller than C . This happens for all the support vectors (SV) that are not bounded (BSV). Our measurement gives the following averages:

d = 1: nSV - nBSV = 8.5
d = 2: nSV - nBSV = 41.8
d = 3: nSV - nBSV = 118.9
d = 4: nSV - nBSV = 223.8

6. [15 points] In the standard two-group classification, errors on positive or negative points are counted the same way. Suppose we wish to penalize an error on an negative point (false positive error) $k > 0$ times more than an error on a positive point. Give the dual optimization problem corresponding to SVMs modified in this way.

We can consider the more general problem of assigning a weight k_i to every sample that will multiply its misclassification penalty. The optimization problem becomes:

$$\begin{aligned} \text{minimize} \quad & \frac{1}{2} \|w\|^2 + \sum_{i=1}^m k_i \xi_i \\ \text{subject to} \quad & y_i(w x_i + b) \leq 1 - \xi_i, \xi_i \geq 0, \forall i \in 1, \dots, m \end{aligned}$$

Moving to the dual exactly as shown in class we obtain:

$$\begin{aligned} \text{maximize} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i \cdot x_j \\ \text{subject to} \quad & 0 \leq \alpha_i \leq C k_i, \forall i \in 1, \dots, m \end{aligned}$$

Setting $k_i = k$ for $y_i = -1$ and $k_i = 1$ for $y_i = 1$ gives the desired problem.

7. [10 points] Assume that k is an integer. Show how you could use libsvm without writing any additional code to find the solution of the modified SVMs just described.

If k is an integer we can repeat every negative training point in the data k times. False positives will thus get penalized k times as much as false negatives.

8. [10 points] Apply the modified SVMs to the classification task previously examined and compare with your previous SVMs results for different values of k : $k = 2, 4, 8, 16$.

Repeating the training for $k = 1, 2, 4, 8, 16$, we find the following results:

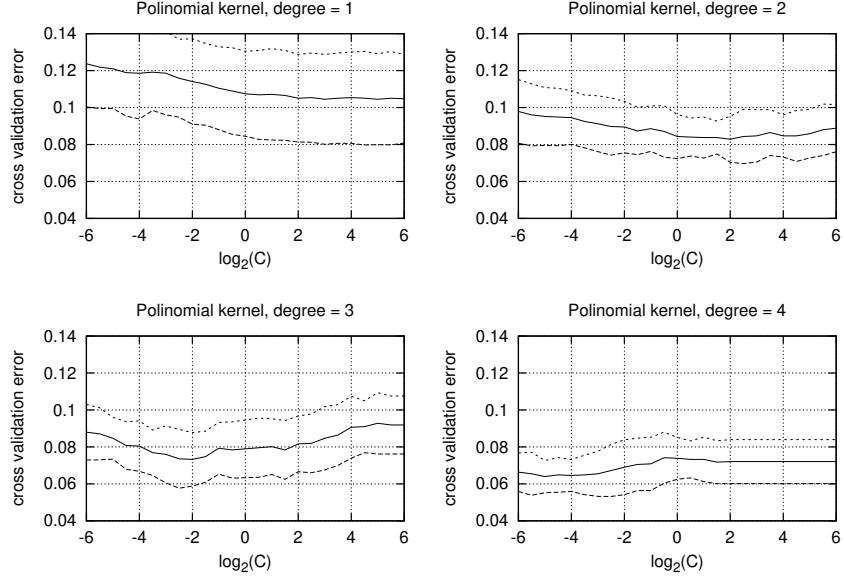


Figure 3: As in Figure 1, but false-positives are penalized twice as much as false-negatives.

$k = 1$: $(d^*, C^*) = (4, 2^{(-4.5)})$, $cv\text{-err} = 6.4\% \pm 1.5\%$
 $k = 2$: $(d^*, C^*) = (4, 2^{(-5.0)})$, $cv\text{-err} = 6.4\% \pm 0.9\%$
 $k = 4$: $(d^*, C^*) = (4, 2^{(-5.5)})$, $cv\text{-err} = 6.1\% \pm 1.7\%$
 $k = 8$: $(d^*, C^*) = (4, 2^{(-3.5)})$, $cv\text{-err} = 6.2\% \pm 1.0\%$
 $k = 16$: $(d^*, C^*) = (4, 2^{(-3.5)})$, $cv\text{-err} = 6.2\% \pm 1.4\%$

Plots equivalent to the ones in Figure 1 are given in Figure 3, Figure 4, Figure 5, Figure 6. We obtain the best average accuracy for $k = 4$.

B. Kernels

- [5 points] Let $K: X \times X \rightarrow \mathbb{R}$ be a PDS kernel, show that for all $x, y \in X$,

$$|K(x, y)| \leq \sqrt{K(x, x)K(y, y)}. \quad (1)$$

This follows directly the Cauchy-Schwarz inequality:

$$|\Phi(x)^\top \Phi(y)| \leq \|\Phi(x)\| \|\Phi(y)\|, \quad (2)$$

where Φ is a feature mapping associated to K .

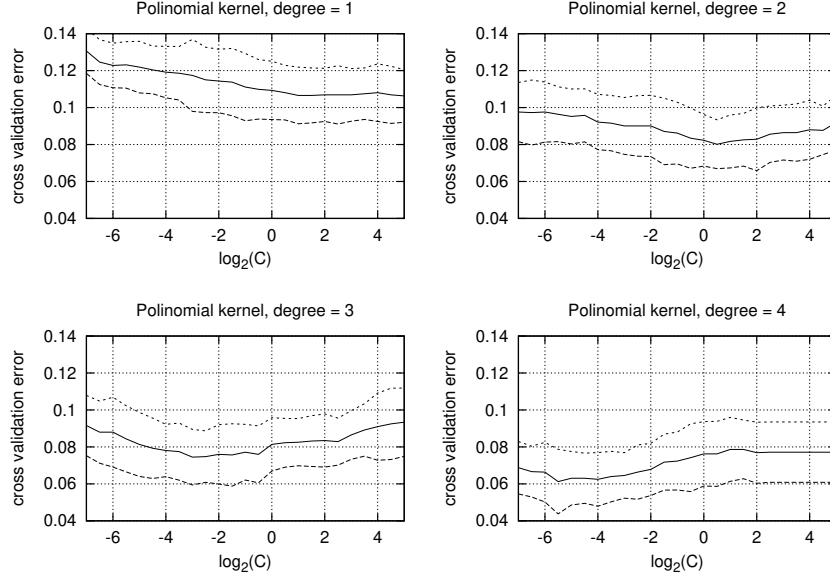


Figure 4: As in Figure 1, but false-positives are penalized 4 times as much as false-negatives.

2. Show that the following kernels K are PDS:

- (a) [10 points] $K(x, y) = \cos(x - y)$ over $\mathbb{R} \times \mathbb{R}$.

Since $\cos(x - y) = \cos x \cos y + \sin x \sin y$, $K(x, y)$ can be written as the dot product of the vectors

$$\Phi(x) = \begin{bmatrix} \cos x \\ \sin x \end{bmatrix} \quad \text{and} \quad \Phi(y) = \begin{bmatrix} \cos y \\ \sin y \end{bmatrix}, \quad (3)$$

thus it is PDS.

- (b) [10 points] $K(x, y) = \cos(x^2 - y^2)$ over $\mathbb{R} \times \mathbb{R}$.

This is a consequence of the fact that the kernel of the previous question is PDS since $\sum_{i,j} c_i c_j \cos(x_i^2 - x_j^2) = \sum_{i,j} c_i c_j \cos(x'_i - x'_j)$, with $x'_i = x_i^2$ for all i .

The solution for this question is similar to that of the previous question.

- (c) [bonus, 20 points] $K(x, y) = (x + y)^{-1}$ over $]0, +\infty[\times]0, +\infty[$.

For any $x_1, \dots, x_m \in \mathbb{R}$, $c_1, \dots, c_m \in \mathbb{R}$, and $a \geq 0$, let $f(a)$ be defined by

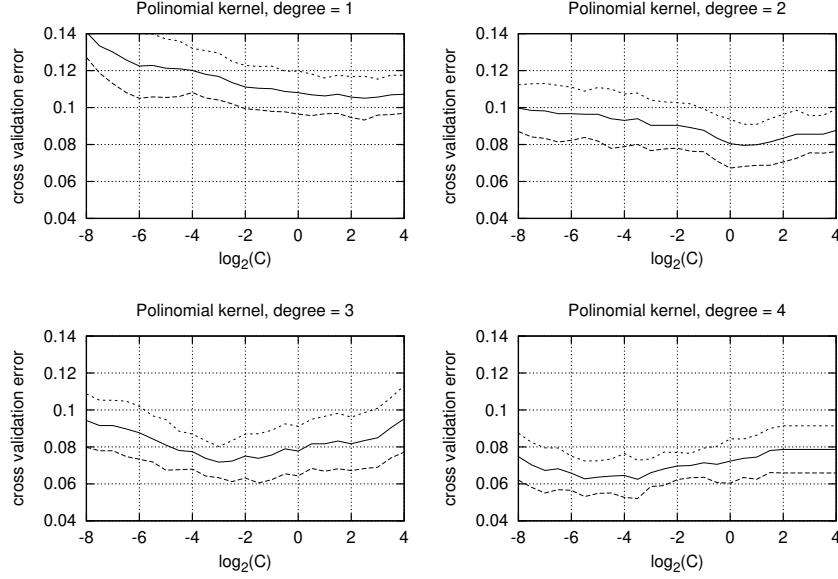


Figure 5: As in Figure 1, but false-positives are penalized 8 times as much as false-negatives.

$$f(a) = \sum_{i,j} c_i c_j K(x_i, x_j) a^{x_i + x_j}. \quad (4)$$

Then, $f'(a) = \sum_{i,j} c_i c_j a^{x_i + x_j - 1} = \frac{1}{a} \|(c_i a^{x_i})_i\|^2 \geq 0$. Therefore f is monotonically increasing. Thus, $f(1) \geq f(0)$, that is $\sum_{i,j} c_i c_j K(x_i, x_j) \geq 0$.

3. Show that the following kernels K are NDS:

(a) [10 points] $K(x, y) = \sin^2(x - y)$ over $\mathbb{R} \times \mathbb{R}$.

Note that $K(x, y) = 1 - \cos^2(x - y)$. Now, for any $x_1, \dots, x_m \in \mathbb{R}$ and $c_1, \dots, c_m \in \mathbb{R}$ such that $\sum_{i=1}^m c_i = 0$,

$$\sum_{i,j=1}^m c_i c_j K(x_i, x_j) = \sum_{i,j=1}^m c_i c_j - \sum_{i,j=1}^m c_i c_j \cos^2(x_i - x_j) \quad (5)$$

$$= - \sum_{i,j=1}^m c_i c_j \cos^2(x_i - x_j), \quad (6)$$

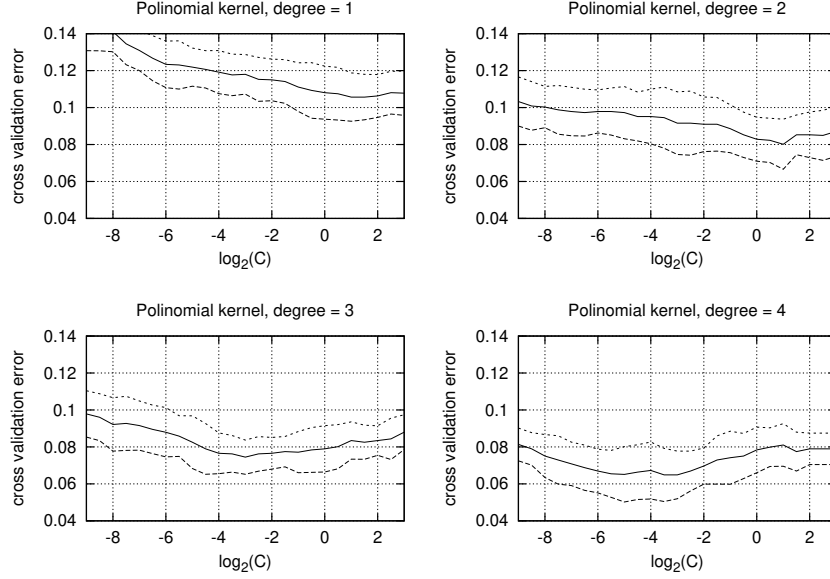


Figure 6: As in Figure 1, but false-positives are penalized 16 times as much as false-negatives.

since $\sum_{i,j=1}^m c_i c_j = (\sum_{i=1}^m c_i)^2 = 0$. It was already shown in a previous question that $K'(x, y) = \cos(x - y)$ defines a PDS kernel. Since the product of two PDS kernels is PDS, K'^2 is also PDS. This implies that $\sum_{i,j=1}^m c_i c_j \cos^2(x_i - x_j) \geq 0$, and $\sum_{i,j=1}^m c_i c_j K(x_i, x_j) \leq 0$, which can also be verified directly by expanding $\cos^2(x_i - x_j)$.

- (b) [bonus, 20 points] $K(x, y) = \log(x + y)$ over $]0, +\infty[\times]0, +\infty[$.
By a theorem of Lecture 5, K is NDS iff $\exp(-tK)$ is PDS for all $t > 0$. Here, for any $t > 0$, $\exp(-tK) = (x + y)^{-t}$. It was already shown in a previous question that $(x + y)^{-1}$, which corresponds to the case $t = 1$, is PDS. Since a product of PDS kernels is PDS, $(x + y)^{-n}$ also defines a PDS kernel for any positive integer n . To see the general case, it suffices to show that $(x + y)^{-t}$ is PDS for any $0 < t < 1$.

4. [bonus, 20 points] Is the kernel K defined over $\mathbb{R}^n \times \mathbb{R}^n$ by $K(x, y) = \|x - y\|^{3/2}$ PDS? Is it NDS? Prove your responses.

The kernel K is NDS. In fact, more generally, $\|x - y\|^p$ for $0 < p \leq 2$ defines an NDS kernel. This follows a general fact: if a kernel K is NDS, then K^α is NDS for any $0 < \alpha < 1$. Indeed, if K is NDS, then $\exp(-tK)$ is PDS for all $t > 0$. Then $-\exp(-tK)$ is NDS and so is $1 - \exp(-tK)$. Now, the following

formula holds for all $0 < \alpha < 1$:

$$K^\alpha = c_\alpha \int_0^\infty [1 - \exp(-tK)] \frac{dt}{t^{\alpha+1}}, \quad (7)$$

where c_α is a positive constant. Thus, K^α is NDS for all $0 < \alpha < 1$.

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 2
Due: March 19, 2010

A. VC Dimension

1. Let H and H' be two families of functions mapping from X to $\{0, 1\}$ with finite VC dimensions. Show that

$$\text{VCdim}(H \cup H') \leq \text{VCdim}(H) + \text{VCdim}(H') + 1. \quad (1)$$

Use that to determine the VC dimension of the hypothesis set formed by the union of axis-aligned rectangles and triangles in dimension 2.

2. For two sets A and B , let $A \Delta B$ denote the symmetric difference of A and B , that is $A \Delta B = (A \cup B) - (A \cap B)$. Let H be a non-empty family of subsets of X with finite VC dimension. Let A be an element of H and define $H \Delta A = \{X \Delta A : X \in H\}$. Show that

$$\text{VCdim}(H \Delta A) = \text{VCdim}(H). \quad (2)$$

B. SVMs

1. Download and install the `libsvm` software library from:
<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>.
2. Download the `ISOLET` data set from
<http://archive.ics.uci.edu/ml/datasets/ISOLET>. There are two files: `isolet1+2+3+4`, and `isolet5`. `isolet1+2+3+4` should be used for training and validation, `isolet5` for testing only.

The dataset corresponds to a number of people pronouncing each of the 26 letters in the alphabet. Each person pronounces the whole alphabet twice. We will consider the binary classification that consists of distinguishing the first 13 letters from the last 13 letters. Thus, assign label '-1' to the first 13, '+1' to the rest.
3. Normalize all input vectors (`isolet1+2+3+4` and `isolet5`): compute the scaling and offset on the `isolet1+2+3+4` so that each feature has zero mean and standard deviation 1, and apply the same scaling on the `isolet5` data.

4. Split the data `isolet1+2+3+4`, containing the alphabet spoken 240 times, into 10 folds, ensuring that the same speaker is only in one of the folds by splitting after each 24 pronunciations (modulo a few that are missing; check that all your files start with the label '1').
5. Let $x_1, \dots, x_m$ denote the sample formed by the ten folds. For each feature f , let $f_1, \dots, f_m$ denote its values for $x_1, \dots, x_m$, and let $y_1, \dots, y_m$ denote the labels. Compute the empirical correlation of each non-constant feature f with the labels

$$\hat{\rho}(f, y) = \frac{\hat{\sigma}_{fy}}{\sqrt{\hat{\sigma}_{ff}\hat{\sigma}_{yy}}},$$

where $\hat{\sigma}_{fy} = \frac{1}{m} \sum_{i=1}^m (f_i - \bar{f})(y_i - \bar{y})$, with $\bar{f}$ the average value of f , and $\bar{y}$ the average value of y . $\hat{\sigma}_{ff}$ and $\hat{\sigma}_{yy}$ are defined in a similar way. Sort all features in decreasing order of the absolute value of the correlation and save the correlation values.

6. We first consider a *Naive* algorithm. Given a kernel K , this algorithm assigns a label to a new point simply based on its similarity with respect to the set of positively labeled versus its similarity with the negatively labeled points of the training set. Thus, for any point x , if we denote by $\mathbf{y}$ the vector of the labels in the training set and use the notation $\mathbf{K}_x = \begin{bmatrix} K(x, x_1) \\ \vdots \\ K(x, x_m) \end{bmatrix}$, the label it assigns to x is

$$h(x) = \text{sgn}(\mathbf{K}_x \cdot \mathbf{y}).$$

Determine and report the performance of the Naive algorithm on the test set when using a polynomial kernel of degree d with $d = 1, 2, 3, 4$, when using a percentage p of the most correlated features, with $p = 100\%, 80\%, 40\%, 20\%$.

7. Determine and report the performance of SVMs for the same set of kernels and the same sets of features. To determine the trade-off parameter C , use 10-fold cross validation with the ten folds previously defined (let the other parameters of polynomial kernels in `libsvm`, γ and c , be equal to their default values 1). Give a plot comparing the performance of SVMs with that of the Naive algorithm for each value of p .
8. Now, first multiply each feature f by its empirical correlation $\hat{\rho}(f, y)$ and retrain SVMs with the full set of features. Report the test results obtained for the four values of d .
9. For each d and p , fix C to its best value obtained in question 7. For this value of C , for each d and p , train ten models, each time by excluding one

of the ten folds. Compute the average number of support vectors and the average test error for each d and p . Plot the average error as a function of the average fraction of support vectors. Discuss your results and compare with the leave-one-out theorem presented in class.

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Solution assignment 2
Solution of section B written by Cyril Allauzen.

A. VC Dimension

1. Let H and H' be two families of functions mapping from X to $\{0, 1\}$ with finite VC dimensions. Show that

$$\text{VCdim}(H \cup H') \leq \text{VCdim}(H) + \text{VCdim}(H') + 1. \quad (1)$$

Use that to determine the VC dimension of the hypothesis set formed by the union of axis-aligned rectangles and triangles in dimension 2.

The number of ways m particular points can be classified using $H \cup H'$ is at most the number of classifications using H plus the number of classifications using H' . This gives immediately the following inequality for growth functions for any $m \geq 0$:

$$\Pi_{H' \cup H}(m) \leq \Pi_H(m) + \Pi_{H'}(m). \quad (2)$$

Let $\text{VCdim}(H) = d$ and $\text{VCdim}(H') = d'$. Then, by Sauer's lemma,

$$\Pi_{H' \cup H}(m) \leq \sum_{i=0}^d \binom{m}{i} + \sum_{i=0}^{d'} \binom{m}{i}. \quad (3)$$

Using the identity $\binom{m}{i} = \binom{m}{m-i}$ and a change of variable, this can be rewritten as

$$\Pi_{H' \cup H}(m) \leq \sum_{i=0}^d \binom{m}{i} + \sum_{i=0}^{d'} \binom{m}{m-i} \leq \sum_{i=0}^d \binom{m}{i} + \sum_{i=m-d'}^m \binom{m}{i}.$$

Now, if $m - d' > d + 1$, that is $m \geq d + d' + 2$,

$$\Pi_{H' \cup H}(m) \leq \sum_{i=0}^m \binom{m}{i} - \binom{m}{d+1} = 2^m - \binom{m}{d+1} < 2^m.$$

Thus, the VC dimension of $H \cup H'$ cannot be greater than or equal to $d + d' + 2$, which implies $\text{VCdim}(H \cup H') \leq d + d' + 1$.

Now, by Lecture 3, the VC dimension of axis-aligned rectangles in dimension 2 is 4 and the VC dimension of triangles (3-gones) is 7. Thus, the VC dimension of the union of these sets is bounded by $4 + 7 + 1 = 12$.

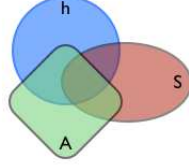


Figure 1: Illustration of $(h \Delta A) \cap S = (h \cap S) \Delta (A \cap S)$.

2. For two sets A and B , let $A \Delta B$ denote the symmetric difference of A and B , that is $A \Delta B = (A \cup B) - (A \cap B)$. Let H be a non-empty family of subsets of X with finite VC dimension. Let A be an element of H and define $H \Delta A = \{X \Delta A : X \in H\}$. Show that

$$\text{VCdim}(H \Delta A) = \text{VCdim}(H). \quad (4)$$

Fix a set S . We can show that the number of classifications of S using H is the same as when using $H \Delta A$. The set of classifications obtained using H can be identified with $\{S \cap h : h \in H\}$ and the set of classifications using $H \Delta A$ can be identified with $\{S \cap (h \Delta A) : h \in H\}$. Observe that for any $h \in H$,

$$S \cap (h \Delta A) = (S \cap h) \Delta (S \cap A). \quad (5)$$

Figure 1 helps illustrate this equality in a special case. Now, in view of this inequality, if $S \cap (h \Delta A) = S \cap (h' \Delta A)$ for $h, h' \in H$, then

$$(S \cap h) \Delta B = (S \cap h') \Delta B, \quad (6)$$

with $B = S \cap A$. Since two sets that have the same symmetric differences with respect to a set B must be equal, this implies

$$S \cap h = S \cap h'. \quad (7)$$

This shows that ϕ defined by

$$\begin{aligned} \phi: S \cap H &\rightarrow S \cap (H \Delta A) \\ S \cap h &\mapsto S \cap (h \Delta A) \end{aligned}$$

is a bijection and thus that the sets $S \cap H$ and $S \cap (H \Delta A)$ have the same cardinality.

B. SVMs

1. Download and install the `libsvm` software library from:
<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>.

2. Download the ISOLET data set from

<http://archive.ics.uci.edu/ml/datasets/ISOLET>. There are two files: `isolet1+2+3+4`, and `isolet5`. `isolet1+2+3+4` should be used for training and validation, `isolet5` for testing only.

The dataset corresponds to a number of people pronouncing each of the 26 letters in the alphabet. Each person pronounces the whole alphabet twice. We will consider the binary classification that consists of distinguishing the first 13 letters from the last 13 letters. Thus, assign label '-1' to the first 13, '+1' to the rest.

3. Normalize all input vectors (`isolet1+2+3+4` and `isolet5`): compute the scaling and offset on the `isolet1+2+3+4` so that each feature has zero mean and standard deviation 1, and apply the same scaling on the `isolet5` data.

Using `svm-scale` would not have been the right thing here.

4. Split the data `isolet1+2+3+4`, containing the alphabet spoken 240 times, into 10 folds, ensuring that the same speaker is only in one of the folds by splitting after each 24 pronunciations (modulo a few that are missing; check that all your files start with the label '1').

The key here is to ensure that each speaker is assigned to the same fold and to be careful with the missing datapoints. Using the `-v` option of the `svm-train` command-line utility would not do the right thing. We proceeded in two steps:

- (a) identify the speaker corresponding to each line of the data file and
- (b) randomly assign each speaker to one the folds and assign each line to random fold corresponding to the speaker for that line.

5. Let $x_1, \dots, x_m$ denote the sample formed by the ten folds. For each feature f , let $f_1, \dots, f_m$ denote its values for $x_1, \dots, x_m$, and let $y_1, \dots, y_m$ denote the labels. Compute the empirical correlation of each non-constant feature f with the labels

$$\hat{\rho}(f, y) = \frac{\hat{\sigma}_{fy}}{\sqrt{\hat{\sigma}_{ff}\hat{\sigma}_{yy}}},$$

where $\hat{\sigma}_{fy} = \frac{1}{m} \sum_{i=1}^m (f_i - \bar{f})(y_i - \bar{y})$, with $\bar{f}$ the average value of f , and $\bar{y}$ the average value of y . $\hat{\sigma}_{ff}$ and $\hat{\sigma}_{yy}$ are defined in a similar way. Sort all features in decreasing order of the absolute value of the correlation and save the correlation values.

The 10 features which correlations are the largest in absolute value are given below.

Feature	Correlation
480	-0.306521
214	-0.287193
182	-0.280878
215	-0.280177
183	-0.275566
213	-0.272187
181	-0.271201
184	-0.265968
216	-0.261865
188	-0.260272

6. We first consider a *Naive* algorithm. Given a kernel K , this algorithm assigns a label to a new point simply based on its similarity with respect to the set of positively labeled versus its similarity with the negatively labeled points of the training set. Thus, for any point x , if we denote by $\mathbf{y}$ the vector of the labels in the training set and use the notation $\mathbf{K}_x = \begin{bmatrix} K(x, x_1) \\ \vdots \\ K(x, x_m) \end{bmatrix}$, the label it assigns to x is

$$h(x) = \text{sgn}(\mathbf{K}_x \cdot \mathbf{y}).$$

Determine and report the performance of the Naive algorithm on the test set when using a polynomial kernel of degree d with $d = 1, 2, 3, 4$, when using a percentage p of the most correlated features, with $p = 100\%, 80\%, 40\%, 20\%$. Polynomial kernels are of the form

$$(\gamma \cdot \mathbf{x}^T \mathbf{y} + c_0)^d. \quad (8)$$

By default when using the `svm-train` command-line utility, the value of γ is $1/F$ where F denotes the number of features and the value of c_0 is 0. We chose to use these default values of γ and c_0 . The performance of the naive method on the test set is given below.

Accuracy		Percentage of features			
		20%	40%	80%	100%
Degree	1	66.39%	66.65%	67.60%	67.35%
	2	51.19%	55.23%	60.36%	64.34%
	3	68.31%	69.85%	72.80%	73.57%
	4	52.79%	56.76%	65.55%	70.43%

With $\gamma = 1/F$ and $c_0 = 0$, the performance of the even degree kernels is significantly worse than the one of the odd degree kernels. However, using $c_0 = 1$ would have made the performance of the even degree kernels match the one of the odd degree kernels.

7. Determine and report the performance of SVMs for the same set of kernels and the same sets of features. To determine the trade-off parameter C , use 10-fold cross validation with the ten folds previously defined (let the other parameters of polynomial kernels in `libsvm`, γ and c , be equal to their default values 1). Give a plot comparing the performance of SVMs with that of the Naive algorithm for each value of p .

Depending on the choice of γ and c_0 , the value of the best C can be quite different. We report here results obtained using $\gamma = 1/F$ and $c_0 = 0$.

% of features	Degree	C	Accuracy	
			(cross-validation)	(test set)
20%	1	0.4	78.17%	77.49%
	2	7.5	86.08%	85.57%
	3	7.5	87.72%	86.20%
	4	16	86.32%	84.99%
40%	1	3	81.27%	81.65%
	2	3	91.59%	90.70%
	3	8	92.59%	92.11%
	4	16	91.62%	89.80%
80%	1	10	85.50%	85.63%
	2	15	94.99%	94.36%
	3	4	95.71%	95.83%
	4	16	94.62%	93.77%
100%	1	4	85.94%	87.30%
	2	4	95.80%	94.87%
	3	6	96.09%	96.66%
	4	20	95.49%	94.55%

Figure 2 gives a plot comparing the performance of SVMs with that of the Naive algorithm for each value of p .

8. Now, first multiply each feature f by its empirical correlation $\hat{\rho}(f, y)$ and retrain SVMs with the full set of features. Report the test results obtained for the four values of d .

The results obtained here seem to depend heavily on the polynomial kernel parameters γ and c_0 . Ideally, one would like to determine the value of C using cross validation as done above. The results obtained using $C = 1$ are given below for three sets of polynomial kernel parameters.

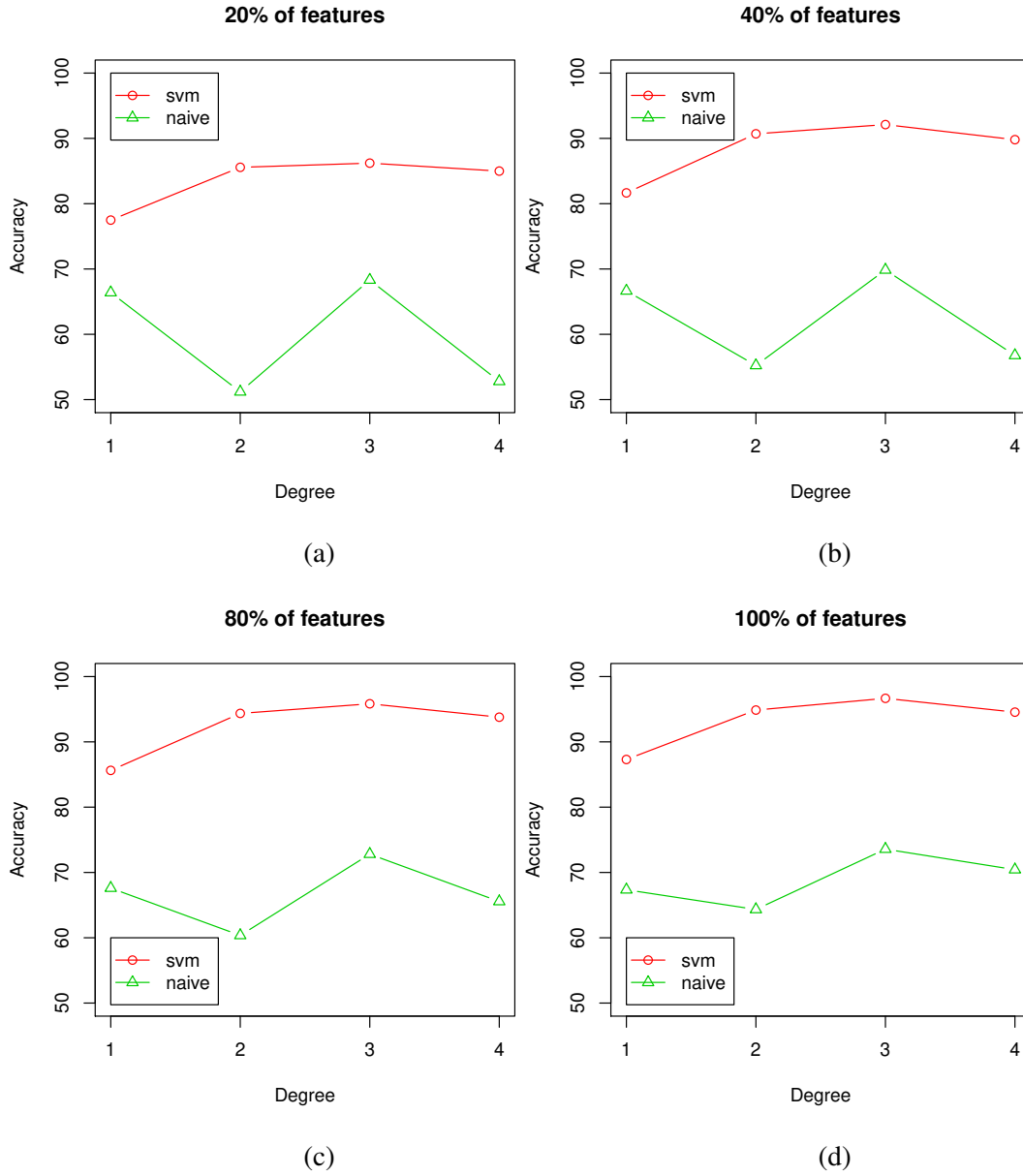


Figure 2: Accuracy on the test set of SVMs and the naive algorithms when using the (a) 20%, (c) 40%, (d) 80% and (d) 100% most correlated features

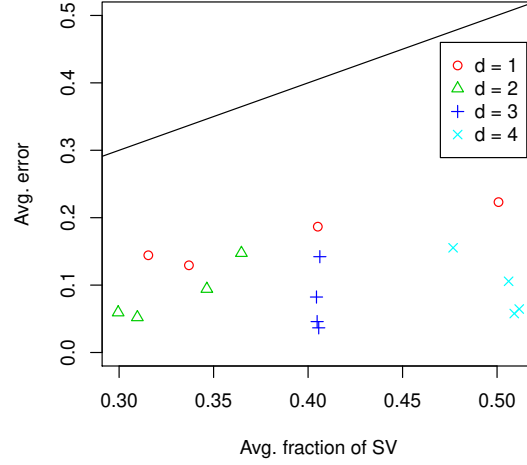


Figure 3: Average test error as a function of the fraction of support vector

$\gamma = 1/F, c_0 = 0$		$\gamma = 1/F, c_0 = 1$		$\gamma = 1, c_0 = 1$	
Degree	Accuracy	Degree	Accuracy	Degree	Accuracy
1	74.66%	1	74.66%	1	86.15%
2	50.03%	2	77.68%	2	93.33%
3	50.03%	3	79.03%	3	94.87%
4	50.03%	4	80.05%	4	94.55%

9. For each d and p , fix C to its best value obtained in question 7. For this value of C , for each d and p , train ten models, each time by excluding one of the ten folds. Compute the average number of support vectors and the average test error for each d and p . Plot the average error as a function of the average fraction of support vectors. Discuss your results and compare with the leave-one-out theorem presented in class.

The average test error and the average number of support vectors are given below for the different values of p and d .

% of features	Degree	Avg. accuracy	Avg. error	Avg. number of SV
20%	1	77.69%	22.31%	3123.8
	2	85.25%	14.75%	2275.1
	3	85.79%	14.21%	2534.0
	4	84.46%	15.54%	2973.9
40%	1	81.33%	18.67%	2527.3
	2	90.57%	9.43%	2161.0
	3	91.77%	8.23%	2522.9
	4	89.44%	10.56%	3157.1
80%	1	85.57%	14.43%	1967.8
	2	94.06%	5.84%	1868.3
	3	95.41%	4.59%	2525.1
	4	93.55%	6.45%	3192.2
100%	1	87.06%	12.94%	2101.9
	2	94.78%	5.22%	1931.7
	3	96.34%	3.66%	2530.3
	4	94.22%	5.78%	3175.8

Figure 3 shows the average test error as a function of the fraction of support vectors. Observe that the average test error appears to be bounded by the average fraction of support vectors, in line with what was suggested by the leave-one-out analysis presented in class.

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 2
February 25, 2013
Due: March 11, 2013

A. Rademacher complexity - properties

Let H be a hypothesis set reduced to two functions: $H = \{h_{-1}, h_{+1}\}$ and let $S = (x_1, \dots, x_m) \subseteq \mathcal{X}$ be a sample of size m .

1. Assume that h_{-1} is the constant function taking value -1 and h_{+1} the constant function taking the value $+1$. What is the VC-dimension d of H ? Upper bound the empirical Rademacher complexity $\mathfrak{R}_S(H)$ (*hint*: express $\mathfrak{R}_S(H)$ in terms of the absolute value of a sum of Rademacher variables and apply Jensen's inequality) and compare your bound with $\sqrt{d/m}$.
2. Assume that h_{-1} is the constant function taking value -1 and h_{+1} the function taking value -1 everywhere except at x_1 where it takes the value $+1$. What is the VC-dimension d of H ? Compute the empirical Rademacher complexity $\mathfrak{R}_S(H)$.

B. Rademacher complexity bound

Let G be a family of functions mapping from Z to $[0, 1]$. The general Rademacher complexity bound presented in class was based on the analysis of the function Φ defined by $\Phi(S) = \sup_{g \in G} \mathbb{E}[g] - \widehat{\mathbb{E}}_S[g]$ for any training sample $S = (z_1, \dots, z_m)$ of size m , with $\widehat{\mathbb{E}}_S[g] = \frac{1}{m} \sum_{i=1}^m g(z_i)$. Instead, apply McDiarmid's inequality to Ψ defined by $\Psi(S) = \sup_{g \in G} \mathbb{E}[g] - \widehat{\mathbb{E}}_S[g] - 2\mathfrak{R}_S(G)$ and try to obtain a slightly better generalization bound than the one obtained in class in terms of the empirical Rademacher complexity.

C. VC-dimension of union of k intervals.

What is the VC-dimension of subsets of the real line formed by the union of k intervals?

D. Generalization bound based on covering numbers.

Let H be a family of functions mapping $\mathcal{X}$ to a subset of real numbers $\mathcal{Y} \subseteq \mathbb{R}$. For any $\epsilon > 0$, the *covering number* $\mathcal{N}(H, \epsilon)$ of H for the L_∞ norm is the minimal $k \in \mathbb{N}$ such that H can be covered with k balls of radius ϵ , that is, there exists $\{h_1, \dots, h_k\} \subseteq H$ such that, for all $h \in H$, there exists $i \leq k$ with $\|h - h_i\|_\infty = \max_{x \in \mathcal{X}} |h(x) - h_i(x)| \leq \epsilon$. In particular, when H is a compact set, a finite covering can be extracted from a covering of H with balls of radius ϵ and thus $\mathcal{N}(H, \epsilon)$ is finite.

Covering numbers provide a measure of the complexity of a class of functions: the larger the covering number, the richer is the family of functions. The objective of this problem is to illustrate this by proving a learning bound in the case of the squared loss. Let D denote a distribution over $\mathcal{X} \times \mathcal{Y}$ according to which labeled examples are drawn. Then, the generalization error of $h \in H$ for the squared loss is defined by $R(h) = \mathbb{E}_{(x,y) \sim D} [(h(x) - y)^2]$ and its empirical error for a labeled sample $S = ((x_1, y_1), \dots, (x_m, y_m))$ by $\hat{R}(h) = \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i)^2$. We will assume that H is bounded, that is there exists $M > 0$ such that $|h(x) - y| \leq M$ for all $(x, y) \in \mathcal{X} \times \mathcal{Y}$. The following is the generalization bound proven in this problem:

$$\Pr_{S \sim D^m} \left[\sup_{h \in H} |R(h) - \hat{R}(h)| \geq \epsilon \right] \leq \mathcal{N}\left(H, \frac{\epsilon}{8M}\right) 2 \exp\left(\frac{-m\epsilon^2}{2M^4}\right). \quad (1)$$

The proof is based on the following steps.

1. Let $L_S = R(h) - \hat{R}(h)$, then show that for all $h_1, h_2 \in H$ and any labeled sample S , the following inequality holds:

$$|L_S(h_1) - L_S(h_2)| \leq 4M \|h_1 - h_2\|_\infty.$$

2. Assume that H can be covered by k subsets $B_1, \dots, B_k$, that is $H = B_1 \cup \dots \cup B_k$. Then, show that, for any $\epsilon > 0$, the following upper bound holds:

$$\Pr_{S \sim D^m} \left[\sup_{h \in H} |L_S(h)| \geq \epsilon \right] \leq \sum_{i=1}^k \Pr_{S \sim D^m} \left[\sup_{h \in B_i} |L_S(h)| \geq \epsilon \right].$$

3. Finally, let $k = \mathcal{N}(H, \frac{\epsilon}{8M})$ and let $B_1, \dots, B_k$ be balls of radius $\epsilon/(8M)$ centered at $h_1, \dots, h_k$ covering H . Use part (a) to show that for all $i \in [1, k]$,

$$\Pr_{S \sim D^m} \left[\sup_{h \in B_i} |L_S(h)| \geq \epsilon \right] \leq \Pr_{S \sim D^m} \left[|L_S(h_i)| \geq \frac{\epsilon}{2} \right],$$

and apply Hoeffding's inequality to prove (1).

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 2
February 25, 2013
Due: March 11, 2013

A. Rademacher complexity - properties

Let H be a hypothesis set reduced to two functions: $H = \{h_{-1}, h_{+1}\}$ and let $S = (x_1, \dots, x_m) \subseteq \mathcal{X}$ be a sample of size m .

1. Assume that h_{-1} is the constant function taking value -1 and h_{+1} the constant function taking the value $+1$. What is the VC-dimension d of H ? Upper bound the empirical Rademacher complexity $\mathfrak{R}_S(H)$ (*hint: express $\mathfrak{R}_S(H)$ in terms of the absolute value of a sum of Rademacher variables and apply Jensen's inequality*) and compare your bound with $\sqrt{d/m}$.

Solution: $\text{VCdim}(H) = 1$ since H can shatter one point and clearly at most one. Observe that

$$\sup_{h \in H} \sum_{i=1}^m \sigma_i h(x_i) = \sup_{h \in H} \left(\sum_{i=1}^m \sigma_i \right) h(x_1) = \left| \sum_{i=1}^m \sigma_i \right|. \quad (1)$$

Thus, by Jensen's inequality,

$$\begin{aligned} \mathfrak{R}_S(H) &= \frac{1}{m} \mathbb{E}_{\sigma} \left[\left| \sum_{i=1}^m \sigma_i \right| \right] \\ &\leq \frac{1}{m} \left[\mathbb{E}_{\sigma} \left[\left(\sum_{i=1}^m \sigma_i \right)^2 \right] \right]^{1/2} \\ &= \frac{1}{m} \left[\mathbb{E}_{\sigma} \left[\sum_{i=1}^m \sigma_i^2 \right] \right]^{1/2} \quad (\mathbb{E}[\sigma_i \sigma_j] = 0 \text{ for } i \neq j) \\ &= \frac{1}{\sqrt{m}}. \end{aligned}$$

By the Khintchine inequality, the upper bound is tight modulo the constant $1/\sqrt{2}$. The upper bound coincides with $\sqrt{d/m}$. $\square$

2. Assume that h_{-1} is the constant function taking value -1 and h_{+1} the function taking value -1 everywhere except at x_1 where it takes the value $+1$. What is the VC-dimension d of H ? Compute the empirical Rademacher complexity $\mathfrak{R}_S(H)$.

Solution: $\text{VCdim}(H) = 1$ since H can shatter x_1 and clearly at most one point. By definition,

$$\begin{aligned}\mathfrak{R}_S(H) &= \frac{1}{m} \mathbb{E}_{\sigma} \left[\sup_{h \in H} \sum_{i=1}^m \sigma_i h(x_i) \right] \\ &= \frac{1}{m} \mathbb{E}_{\sigma} \left[\sup_{h \in H} \sigma_1 h(x_1) - \sum_{i=2}^m \sigma_i \right] \\ &= \frac{1}{m} \mathbb{E}_{\sigma_1} \left[\sup_{h \in H} \sigma_1 h(x_1) \right] \quad (\mathbb{E}[\sigma_i] = 0) \\ &= \frac{1}{m} \mathbb{E}_{\sigma_1} [1] = \frac{1}{m}.\end{aligned}$$

Here $\mathfrak{R}_S(H)$ is a clearly more favorable quantity than $\sqrt{d/m} = \sqrt{1/m}$. $\square$

B. Rademacher complexity bound

Let G be a family of functions mapping from Z to $[0, 1]$. The general Rademacher complexity bound presented in class was based on the analysis of the function Φ defined by $\Phi(S) = \sup_{g \in G} \mathbb{E}[g] - \widehat{\mathbb{E}}_S[g]$ for any training sample $S = (z_1, \dots, z_m)$ of size m , with $\widehat{\mathbb{E}}_S[g] = \frac{1}{m} \sum_{i=1}^m g(z_i)$. Instead, apply McDiarmid's inequality to Ψ defined by $\Psi(S) = \sup_{g \in G} \mathbb{E}[g] - \widehat{\mathbb{E}}_S[g] - 2\mathfrak{R}_S(G)$ and try to obtain a slightly better generalization bound than the one obtained in class in terms of the empirical Rademacher complexity.

Solution: Let S' be a sample differing from S by one point, say z_m . Then, since a difference of suprema is upper bounded by the supremum of the differences, we

can write

$$\begin{aligned}
\Psi(S') - \Psi(S) &= \sup_{g \in G} (E[g] - \hat{E}_{S'}[g]) - \sup_{g \in G} (E[g] - \hat{E}_S[g]) + \frac{2}{m} E_{\sigma} \left[\sup_{g \in G} \sum_{i=1}^m \sigma_i g(z_i) - \sup_{g \in G} \sum_{i=1}^m \sigma_i g(z'_i) \right] \\
&\leq \sup_{g \in G} (E[g] - \hat{E}_{S'}[g]) - (E[g] - \hat{E}_S[g]) + \frac{2}{m} E_{\sigma} \left[\sup_{g \in G} \sum_{i=1}^m \sigma_i g(z_i) - \sum_{i=1}^m \sigma_i g(z'_i) \right] \\
&= \sup_{g \in G} \frac{1}{m} (g(z_m) - g(z'_m)) + 2 E_{\sigma} \left[\frac{1}{m} \sup_{g \in G} \sigma_m (g(z_m) - g(z'_m)) \right] \leq \frac{3}{m}.
\end{aligned}$$

Thus, by McDiarmid's inequality, $\Pr[\Psi(S) - E[\Psi(S)] > \epsilon] \leq \exp(-\frac{2}{9} m \epsilon^2)$.

Thus, for any $\delta > 0$, with probability at least $1 - \delta$,

$$\forall g \in G, \Psi(S) - E[\Psi(S)] \leq 3 \sqrt{\frac{\log \frac{1}{\delta}}{2m}}. \quad (2)$$

By definition, $E[\Psi(S)] = E[\Phi(S)] - 2\mathfrak{R}_m(G)$. In class, we showed that $E[\Phi(S)] \leq 2\mathfrak{R}_m(G)$. Thus, with probability at least $1 - \delta$, $\Psi(S) \leq \sqrt{\frac{\log \frac{1}{\delta}}{2m}}$, that is

$$\forall g \in G, E[g] \leq \hat{E}_S[g] + 2\hat{\mathfrak{R}}_S(G) + 3 \sqrt{\frac{\log \frac{1}{\delta}}{2m}}. \quad (3)$$

C. VC-dimension of union of k intervals.

What is the VC-dimension of subsets of the real line formed by the union of k intervals?

Solution:

The VC-dimension of this class is $2k$. It is not hard to see that any $2k$ distinct points on the real line can be shattered using k intervals; it suffices to shatter each of the k pairs of consecutive points with an interval. Assume now that $2k + 1$ distinct points $x_1 < \dots < x_{2k+1}$ are given. For any $i \in [1, 2k + 1]$, label x_i with $(-1)^{i+1}$, that is alternatively label points with 1 or -1 . This leads to $k + 1$ points labeled positively and requires $2k + 1$ intervals to shatter the set, since no interval can contain two consecutive points. Thus, no set of $2k + 1$ points can be shattered by k intervals, and the VC-dimension of the union of k intervals is $2k$. $\square$

D. Generalization bound based on covering numbers.

Let H be a family of functions mapping $\mathcal{X}$ to a subset of real numbers $\mathcal{Y} \subseteq \mathbb{R}$. For any $\epsilon > 0$, the *covering number* $\mathcal{N}(H, \epsilon)$ of H for the L_∞ norm is the minimal

$k \in \mathbb{N}$ such that H can be covered with k balls of radius ϵ , that is, there exists $\{h_1, \dots, h_k\} \subseteq H$ such that, for all $h \in H$, there exists $i \leq k$ with $\|h - h_i\|_\infty = \max_{x \in \mathcal{X}} |h(x) - h_i(x)| \leq \epsilon$. In particular, when H is a compact set, a finite covering can be extracted from a covering of H with balls of radius ϵ and thus $\mathcal{N}(H, \epsilon)$ is finite.

Covering numbers provide a measure of the complexity of a class of functions: the larger the covering number, the richer is the family of functions. The objective of this problem is to illustrate this by proving a learning bound in the case of the squared loss. Let D denote a distribution over $\mathcal{X} \times \mathcal{Y}$ according to which labeled examples are drawn. Then, the generalization error of $h \in H$ for the squared loss is defined by $R(h) = \mathbb{E}_{(x,y) \sim D}[(h(x) - y)^2]$ and its empirical error for a labeled sample $S = ((x_1, y_1), \dots, (x_m, y_m))$ by $\hat{R}(h) = \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i)^2$. We will assume that H is bounded, that is there exists $M > 0$ such that $|h(x) - y| \leq M$ for all $(x, y) \in \mathcal{X} \times \mathcal{Y}$. The following is the generalization bound proven in this problem:

$$\Pr_{S \sim D^m} \left[\sup_{h \in H} |R(h) - \hat{R}(h)| \geq \epsilon \right] \leq \mathcal{N}\left(H, \frac{\epsilon}{8M}\right) 2 \exp\left(\frac{-m\epsilon^2}{2M^4}\right). \quad (4)$$

The proof is based on the following steps.

1. Let $L_S = R(h) - \hat{R}(h)$, then show that for all $h_1, h_2 \in H$ and any labeled sample S , the following inequality holds:

$$|L_S(h_1) - L_S(h_2)| \leq 4M \|h_1 - h_2\|_\infty.$$

Solution: First split the term into two separate terms:

$$\begin{aligned} |L_S(h_1) - L_S(h_2)| &\leq |R(h_1) - R(h_2)| + |\hat{R}(h_1) - \hat{R}(h_2)| \\ &= \left| \mathbb{E}_{x,y} [(h_1(x) - y)^2 - (h_2(x) - y)^2] \right| + \left| \frac{1}{m} \sum_{i=1}^m (h_1(x_i) - y_i)^2 - (h_2(x_i) - y_i)^2 \right|. \end{aligned}$$

Then, expanding the term

$$\begin{aligned} (h_1(x) - y)^2 - (h_2(x) - y)^2 &= (h_1(x) - h_2(x))(h_1(x) + h_2(x) - 2y) \\ &= (h_1(x) - h_2(x))((h_1(x) - y) + (h_2(x) - y)) \leq \|h_1 - h_2\|_\infty 2M, \end{aligned}$$

allows us to bound both the empirical and true error, resulting in a total bound of $4M \|h_1 - h_2\|_\infty$. $\square$

2. Assume that H can be covered by k subsets $B_1, \dots, B_k$, that is $H = B_1 \cup \dots \cup B_k$. Then, show that, for any $\epsilon > 0$, the following upper bound holds:

$$\Pr_{S \sim D^m} \left[\sup_{h \in H} |L_S(h)| \geq \epsilon \right] \leq \sum_{i=1}^k \Pr_{S \sim D^m} \left[\sup_{h \in B_i} |L_S(h)| \geq \epsilon \right].$$

Solution: This follows by splitting the event into the union of several smaller events and then using the sum rule,

$$\begin{aligned} \Pr_S \left[\sup_{h \in H} |L_S(h)| \geq \epsilon \right] \\ = \Pr_S \left[\bigvee_{i=1}^k \sup_{h \in B_i} |L_S(h)| \geq \epsilon \right] \leq \sum_{i=1}^k \Pr_S \left[\sup_{h \in B_i} |L_S(h)| \geq \epsilon \right]. \end{aligned}$$

□

3. Finally, let $k = \mathcal{N}(H, \frac{\epsilon}{8M})$ and let $B_1, \dots, B_k$ be balls of radius $\epsilon/(8M)$ centered at $h_1, \dots, h_k$ covering H . Use part (a) to show that for all $i \in [1, k]$,

$$\Pr_{S \sim D^m} \left[\sup_{h \in B_i} |L_S(h)| \geq \epsilon \right] \leq \Pr_{S \sim D^m} \left[|L_S(h_i)| \geq \frac{\epsilon}{2} \right],$$

and apply Hoeffding's inequality to prove (4).

Solution: For any i let h_i be the center of ball B_i with radius $\frac{\epsilon}{8M}$. Note that for any $h \in H$ we have $|L_S(h) - L_S(h_i)| \leq 4M\|h - h_i\|_\infty \leq \epsilon/2$. Thus, if for any $h \in B_i$ we have $|L_S(h)| \geq \epsilon$ it must be the case that $|L_S(h_i)| \geq \epsilon/2$, which shows the inequality.

To complete the bound, we use Hoeffding's inequality applied to the random variables $(h(x_i) - y_i)^2/m \leq M^2/m$, which guarantees

$$\Pr_S \left[|L_S(h_i)| \geq \frac{\epsilon}{2} \right] \leq 2 \exp \left(\frac{-m\epsilon^2}{2M^4} \right).$$

□

Mehryar Mohri
Foundations of Machine Learning 2014
Courant Institute of Mathematical Sciences
Homework assignment 2
October 3, 2014
Due: October 17, 2014

A. VC-dimension of axis-aligned squares or triangles

1. What is the VC-dimension of axis-aligned squares in the plane?

Solution: It is not hard to see that the set of 3 points with coordinates $(1, 0)$, $(0, 1)$, and $(-1, 0)$ can shattered by axis-aligned squares: e.g., to label positively two of these points, use a square defined by the axes and with those two points as corners. Thus, the VC-dimension is at least 3. No set of 4 points can be fully shattered. To see this, let P_T be the highest point, P_B the lowest, P_L the leftmost, and P_R the rightmost, assuming for now that these can be defined in a unique way (no tie) – the cases where there are ties can be treated in a simpler fashion. Assume without loss of generality that the difference d_{BT} of y -coordinates between P_T and P_B is greater than the difference d_{LR} of x -coordinates between P_L and P_R . Then, P_T and P_B cannot be labeled positively while P_L and P_R are labeled negatively. Thus, the VC-dimension of axis-aligned squares in the plane is 3.

2. Consider right triangles in the plane with the sides adjacent to the right angle both parallel to the axes and with the right angle in the lower left corner. What is the VC-dimension of this family?

Solution: It is not hard to see that the set of 3 points with coordinates $(0, 0)$, $(-1, -1)$, $(-2, -2)$, and $(-3, -3)$ can shattered by such triangles. To see that no five points can be shattered, the same example or argument as for axis-aligned rectangles can be used: labeling all points positively except from the one within the interior of the convex hull is not possible (for the degenerate cases where no point is in the interior of the convex hull is simpler, this is even easier to see). Thus, the VC-dimension of this family of triangles is 4.

B. Growth function bound

1. Consider the family H of threshold functions over $\mathbb{R}^N$ defined by $\{\mathbf{x} = (x_1, \dots, x_N) \mapsto \text{sgn}(x_i - \theta) : i \in [1, N], \theta \in \mathbb{R}\}$, where $\text{sgn}(z) = +1$ if $z \geq 0$, $\text{sgn}(z) = -1$ otherwise. Give an explicit upper bound on the growth function $\Pi_H(m)$ of H that is in $O(mN)$.

Solution: For each feature, there at most $m + 1$ ways of selecting the threshold (between any two feature values or beyond or below all values). Thus, the total number of thresholds functions for a sample of size m is at most $(m + 1)N$. Thus, the growth function is upper bounded by $(m + 1)N$.

2. In class, we gave a bound on the Rademacher complexity of a family G in terms of the growth function (Lecture 3, slide 18). Show that a finer upper bound on the Rademacher complexity can be given in terms of $\mathbb{E}_S[\Pi(G, S)]$, where $\Pi(G, S)$ is the number of ways to label the points in sample S .

Solution: Following the proof given in class and using Jensen's inequality (at the last step), we can write:

$$\begin{aligned}
 \hat{\mathfrak{R}}_m(G) &= \mathbb{E}_{S, \sigma} \left[\sup_{g \in G} \frac{1}{m} \begin{bmatrix} \sigma_1 \\ \vdots \\ \sigma_m \end{bmatrix} \cdot \begin{bmatrix} g(z_1) \\ \vdots \\ g(z_m) \end{bmatrix} \right] \\
 &\leq \mathbb{E}_S \left[\frac{\sqrt{m} \sqrt{2 \log |\{(g(z_1), \dots, g(z_m)) : g \in G\}|}}{m} \right] \quad (\text{Massart's Lemma}) \\
 &= \mathbb{E}_S \left[\frac{\sqrt{m} \sqrt{2 \log \Pi(G, S)}}{m} \right] \\
 &\leq \frac{\sqrt{m} \sqrt{2 \log \mathbb{E}_S[\Pi(G, S)]}}{m} = \sqrt{\frac{2 \log \mathbb{E}_S[\Pi(G, S)]}{m}}.
 \end{aligned}$$

C. VC-dimension of neural networks

Let C be a concept class over $\mathbb{R}^r$ with VC-dimension d . A C -neural network with one intermediate layer is a concept defined over $\mathbb{R}^n$ that can be represented by a directed acyclic graph such as that of Figure 1, in which the input nodes are those at the bottom and in which each other node is labeled with a concept $c \in C$.

The output of the neural network for a given input vector $(x_1, \dots, x_n)$ is obtained as follows. First, each of the n input nodes is labeled with the corresponding value $x_i \in \mathbb{R}$. Next, the value at a node u in the higher layer and labeled with c is obtained by applying c to the values of the input nodes admitting an edge ending in u . Note that since c takes values in $\{0, 1\}$, the value at u is in $\{0, 1\}$. The value at the top or output node is obtained similarly by applying the corresponding concept to the values of the nodes admitting an edge to the output node.

1. Let H denote the set of all neural networks defined as above with $k \geq 2$ internal nodes. Show that the growth function $\Pi_H(m)$ can be upper bounded in terms of the product of the growth functions of the hypothesis sets defined at each intermediate layer.

Solution: Let $\Pi_u(m)$ denote the growth function at a node u in the intermediate layer. For a fixed set of values at the intermediate layer, using the concept class C the output node can generate at most $\Pi_C(m)$ distinct labelings. There are $\prod_u \Pi_u(m)$ possible sets of values at the intermediate layer since, by definition, for a sample of size m , at most $\Pi_u(m)$ distinct values are possible at each u . Thus, at most $\Pi_C(m) \times \prod_u \Pi_u(m)$ labelings can be generated by the neural network and $\Pi_H(m) \leq \Pi_C(m) \prod_u \Pi_u(m)$.

2. Use that to upper bound the VC-dimension of the C -neural networks (*hint*: you can use the implication $m = 2x \log_2(xy) \Rightarrow m > x \log_2(yx)$ valid for $m \geq 1$, and $x, y > 0$ with $xy > 4$).

Solution: For any intermediate node u , $\Pi_u(m) = \Pi_C(m)$. Thus, $\Pi_H(m) \leq \Pi_C(m)^k$. By Sauer's lemma, $\Pi_C(m)^k \leq \left(\frac{em}{d}\right)^d$, thus $\Pi_H(m) \leq \left(\frac{em}{d}\right)^{dk}$. Let $m = 2kd \log_2(ek)$. In view of the inequality given by the hint and $ek > 4$, this implies $m > dk \log_2\left(\frac{em}{d}\right)$, that is $2^m > \left(\frac{em}{d}\right)^{dk}$. Thus, the VC-dimension of H is less than

$$2kd \log_2(ek).$$

3. Let C be the family of concept classes defined by threshold functions $C = \{\text{sgn}(\sum_{j=1}^r w_j x_j) : \mathbf{w} \in \mathbb{R}^r\}$. Give an upper bound on the VC-dimension of H in terms of k and r .

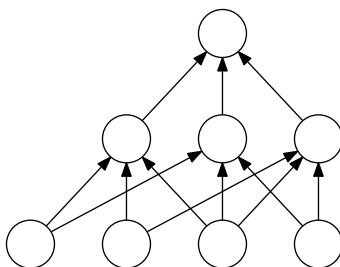


Figure 1: A neural network with one intermediate layer.

Solution: For threshold functions, the VC-dimension of C is r , thus, the VC-dimension of H is upper bounded by

$$2kr \log_2(ek).$$

Mehryar Mohri
 Foundations of Machine Learning 2015
 Courant Institute of Mathematical Sciences
 Homework assignment 2
 October 23, 2015
 Due: November 09, 2015
A. VC-dimension of convex combinations

1. Let H be a family of functions mapping from an input space $\mathcal{X}$ to $\{-1, +1\}$ and let T be a positive integer. Give an upper bound on the VC-dimension of the family of functions $\mathcal{F}_T$ defined by

$$\mathcal{F} = \left\{ \text{sgn} \left(\sum_{t=1}^T \alpha_t h_t \right) : h_t \in H, \alpha_t \geq 0, \sum_{t=1}^T \alpha_t \leq 1 \right\}.$$

(*Hint*: you can use Problem C. of (Foundations of Machine Learning, HW2, 2014, <http://www.cs.nyu.edu/~mohri/ml14/hw2.pdf> and its solution).

B. Growth function

1. A *linearly separable labeling* of a set X of vectors in $\mathbb{R}^d$ is a classification of X into two sets X^+ and X^- with $X^+ = \{\mathbf{x} \in X : \mathbf{w} \cdot \mathbf{x} > 0\}$ and $X^- = \{\mathbf{x} \in X : \mathbf{w} \cdot \mathbf{x} < 0\}$ for some $\mathbf{w} \in \mathbb{R}^d$.

Let $X = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ be a subset of $\mathbb{R}^d$.

- (a) Let $\{X^+, X^-\}$ be a dichotomy of X and let $\mathbf{x}_{m+1} \in \mathbb{R}^d$. Show that $\{X^+ \cup \{\mathbf{x}_{m+1}\}, X^-\}$ and $\{X^+, X^- \cup \{\mathbf{x}_{m+1}\}\}$ are linearly separable by a hyperplane going through the origin if and only if $\{X^+, X^-\}$ is linearly separable by a hyperplane going through the origin and $\mathbf{x}_{m+1}$.
- (b) Let $X = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ be a subset of $\mathbb{R}^d$ such that any k -element subset of X with $k \leq d$ is linearly independent. Then, the number of linearly separable labelings of X is $C(m, d) = 2 \sum_{k=0}^{d-1} \binom{m-1}{k}$. (*Hint*: prove by induction that $C(m+1, d) = C(m, d) + C(m, d-1)$).
- (c) Let $f_1, \dots, f_p$ be p functions mapping $\mathbb{R}^d$ to $\mathbb{R}$. Define $\mathcal{F}$ as the family of classifiers based on linear combinations of these

functions:

$$\mathcal{F} = \left\{ x \mapsto \operatorname{sgn} \left(\sum_{k=1}^p a_k f_k(x) \right) : a_1, \dots, a_p \in \mathbb{R} \right\}.$$

Define Ψ by $\Psi(x) = (f_1(x), \dots, f_p(x))$. Assume that there exists $x_1, \dots, x_m \in \mathbb{R}^d$ such that every p -subset of $\{\Psi(x_1), \dots, \Psi(x_m)\}$ is linearly independent. Then, show that

$$\Pi_{\mathcal{F}}(m) = 2 \sum_{i=0}^{p-1} \binom{m-1}{i}.$$

C. Support Vector Machines

1. Download and install the `libsvm` software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/> ,

and briefly consult the documentation to become more familiar with the tools.

2. Consider the `splice` data set

<http://www.cs.toronto.edu/~dave/data/splice/desc.html>.

Download the already formatted training and test files of a noisy version of that dataset from

http://www.cs.nyu.edu/~mohri/ml15/splice_noise_train.txt

http://www.cs.nyu.edu/~mohri/ml15/splice_noise_test.txt.

Use the `libsvm` scaling tool to scale the features of all the data. The scaling parameters should be computed only on the training data and then applied to the test data.

3. Consider the corresponding binary classification which consists of distinguishing two types of splice junctions in DNA sequences using about 60 features. Use SVMs combined with polynomial kernels to tackle this problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 3, 5$, plot the

average cross-validation error plus or minus one standard deviation as a function of C (let other parameters of polynomial kernels in `libsvm` be equal to their default values), varying C in powers of 5, starting from a small value $C = 5^{-k}$ to $C = 5^k$, for some value of k . k should be chosen so that you see a significant variation in training error, starting from a very high training error to a low training error. Expect longer training times with `libsvm` as the value of C increases.

4. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation error and the test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d . How many of the support vectors lie on the marginal hyperplanes? Plot the soft margin of the solution as a function of d .
5. Now, combine SVMs with Gaussian kernels to tackle the same task. Use cross-validation as before to determine the best value of C and σ , varying C in powers of 5, and σ in powers of 2 for a reasonable range so that you see a significant variation in training error, as before. Fix C and σ to the best values found via cross-validation. How does the test error of the solution compare to the best result obtained using polynomial kernels? What is the value of the soft margin?
6. Here, use as a kernel the sum of the best polynomial kernel (degree d^*) and the Gaussian kernel with the best parameter σ you found in the previous question. Use cross-validation as before to determine the best value of C . How does the test error of the solution compare to the best result obtained in the previous questions?

D. Kernels

Show that the following kernels are PDS.

1. Let n be a positive integer. K is defined by $K(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^N \cos^n(x_i^2 - y_i^2)$ for all $(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^N \times \mathbb{R}^N$.
2. Let σ be a positive real number. K is defined by $K(x, y) = e^{-\frac{\|\mathbf{x}-\mathbf{y}\|}{\sigma}}$ for all $(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^N \times \mathbb{R}^N$ (*Hint*: you could show that K is the normalized kernel of a kernel K' and show that K' is PDS using the following equality: $\|\mathbf{x} - \mathbf{y}\| = \frac{1}{2\Gamma(\frac{1}{2})} \int_0^{+\infty} \frac{1 - e^{-t\|\mathbf{x}-\mathbf{y}\|^2}}{t^{\frac{3}{2}}} dt$ valid for all $\mathbf{x}, \mathbf{y}$).

Mehryar Mohri
 Foundations of Machine Learning 2015
 Courant Institute of Mathematical Sciences
 Homework assignment 2
 October 23, 2015
 Due: November 09, 2015
A. VC-dimension of convex combinations

1. Let H be a family of functions mapping from an input space $\mathcal{X}$ to $\{-1, +1\}$ and let T be a positive integer. Give an upper bound on the VC-dimension of the family of functions $\mathcal{F}_T$ defined by

$$\mathcal{F} = \left\{ \text{sgn} \left(\sum_{t=1}^T \alpha_t h_t \right) : h_t \in H, \alpha_t \geq 0, \sum_{t=1}^T \alpha_t \leq 1 \right\}.$$

(*Hint*: you can use Problem C. of (Foundations of Machine Learning, HW2, 2014, <http://www.cs.nyu.edu/~mohri/ml14/hw2.pdf> and its solution).

Solution: Following the hint, we can think of this family of functions as a one hidden layer neural network, where the hidden layer is represented by the functions $h_t \in H$, and the top layer is a threshold function characterized by $(\alpha_1, \dots, \alpha_T)$. Denote this class of threshold functions by Δ_T . By problem C1 from FML2014-HW2, we can bound the growth function of $\mathcal{F}_T$ by:

$$\Pi_{\mathcal{F}_T}(m) \leq \Pi_{\Delta_T}(m) (\Pi_H(m))^T.$$

By problem C3 from FML2014-HW2, the VC dimension of Δ_T is at most T , and we may further denote the VC dimension of H by d . Applying Sauer's lemma to the growth function yields:

$$\Pi_{\Delta_T}(m) \leq \left(\frac{em}{T} \right)^T, \quad \Pi_H(m) \leq \left(\frac{em}{d} \right)^d.$$

Thus, we have that

$$\Pi_{\mathcal{F}_T}(m) \leq \left(\frac{em}{T} \right)^T \left(\frac{em}{d} \right)^{Td}.$$

Finally, we may apply the hint in problem C2 from FML2014-HW2 with $m = \max\{4T \log_2(2e), 2Td \log_2(eT)\}$ to see that

$$\left(\frac{em}{T} \right)^T \left(\frac{em}{d} \right)^{Td} < 2^{4T \log_2(2e) + 2Td \log_2(eT)},$$

so that the VC Dimension of $\mathcal{F}_T$ is bounded by:

$$2T(2\log_2(2e) + d\log_2(eT)).$$

Note that a coarser but relatively simpler bound would be to write:

$$\left(\frac{em}{T}\right)^T \left(\frac{em}{d}\right)^{Td} < (em)^{T(d+1)},$$

and to apply the hint in problem C2 from FML2014-HW2 with $m = 2T(d+1)\log_2(eT(d+1))$. Notice that this is actually asymptotically optimal in T and d up to log terms.

B. Growth function

1. A *linearly separable labeling* of a set X of vectors in $\mathbb{R}^d$ is a classification of X into two sets X^+ and X^- with $X^+ = \{\mathbf{x} \in X : \mathbf{w} \cdot \mathbf{x} > 0\}$ and $X^- = \{\mathbf{x} \in X : \mathbf{w} \cdot \mathbf{x} < 0\}$ for some $\mathbf{w} \in \mathbb{R}^d$.

Let $X = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ be a subset of $\mathbb{R}^d$.

- (a) Let $\{X^+, X^-\}$ be a dichotomy of X and let $\mathbf{x}_{m+1} \in \mathbb{R}^d$. Show that $\{X^+ \cup \{\mathbf{x}_{m+1}\}, X^-\}$ and $\{X^+, X^- \cup \{\mathbf{x}_{m+1}\}\}$ are linearly separable by a hyperplane going through the origin if and only if $\{X^+, X^-\}$ is linearly separable by a hyperplane going through the origin and $\mathbf{x}_{m+1}$.

Solution: $\{X^+ \cup \{\mathbf{x}_{m+1}\}, X^-\}$ and $\{X^+, X^- \cup \{\mathbf{x}_{m+1}\}\}$ are linearly separable by a hyperplane going through the origin if and only if there exists $\mathbf{w}_1 \in \mathbb{R}^d$ such that

$$\forall \mathbf{x} \in X^+, \mathbf{w}_1 \cdot \mathbf{x} > 0 \quad \forall \mathbf{x} \in X^-, \mathbf{w}_1 \cdot \mathbf{x} < 0, \text{ and } \mathbf{w}_1 \cdot \mathbf{x}_{m+1} > 0 \quad (1)$$

and there exists $\mathbf{w}_2 \in \mathbb{R}^d$ such that

$$\forall \mathbf{x} \in X^+, \mathbf{w}_2 \cdot \mathbf{x} > 0 \quad \forall \mathbf{x} \in X^-, \mathbf{w}_2 \cdot \mathbf{x} < 0, \text{ and } \mathbf{w}_2 \cdot \mathbf{x}_{m+1} < 0. \quad (2)$$

For any $\mathbf{w}_1, \mathbf{w}_2$, the function $f: (t \mapsto t\mathbf{w}_1 + (1-t)\mathbf{w}_2) \cdot \mathbf{x}_{m+1}$ is continuous over $[0, 1]$. (1) and (2) hold iff $f(0) < 0$ and $f(1) > 0$, that is iff there exists $\mathbf{w} = t_0\mathbf{w}_1 + (1-t_0)\mathbf{w}_2$ linearly separating $\{X^+, X^-\}$ and such at $\mathbf{w} \cdot \mathbf{x}_{m+1} = 0$. $\square$

- (b) Let $X = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ be a subset of $\mathbb{R}^d$ such that any k -element subset of X with $k \leq d$ is linearly independent. Then, the number of linearly separable labelings of X is $C(m, d) = 2 \sum_{k=0}^{d-1} \binom{m-1}{k}$. (*Hint*: prove by induction that $C(m+1, d) = C(m, d) + C(m, d-1)$).

Solution:

Repeating the formula, we obtain $C(m, d) = \sum_{k=0}^{m-1} \binom{m-1}{k} C(1, d-k)$. Since, $C(1, n) = 2$ if $n \geq 1$ and $C(1, n) = 0$ otherwise, the result follows. $\square$

- (c) Let $f_1, \dots, f_p$ be p functions mapping $\mathbb{R}^d$ to $\mathbb{R}$. Define $\mathcal{F}$ as the family of classifiers based on linear combinations of these functions:

$$\mathcal{F} = \left\{ x \mapsto \text{sgn} \left(\sum_{k=1}^p a_k f_k(x) \right) : a_1, \dots, a_p \in \mathbb{R} \right\}.$$

Define Ψ by $\Psi(x) = (f_1(x), \dots, f_p(x))$. Assume that there exists $x_1, \dots, x_m \in \mathbb{R}^d$ such that every p -subset of $\{\Psi(x_1), \dots, \Psi(x_m)\}$ is linearly independent. Then, show that

$$\Pi_{\mathcal{F}}(m) = 2 \sum_{i=0}^{p-1} \binom{m-1}{i}.$$

Solution:

This is a direct application of the result of the previous question. $\square$

C. Support Vector Machines

1. Download and install the `libsvm` software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/> ,

and briefly consult the documentation to become more familiar with the tools.

2. Consider the `splice` data set

<http://www.cs.toronto.edu/~dave/data/splice/desc.html>.

Download the already formatted training and test files of a noisy version of that dataset from

http://www.cs.nyu.edu/~mohri/ml15/splice_noise_train.txt
http://www.cs.nyu.edu/~mohri/ml15/splice_noise_test.txt.

Use the `libsvm` scaling tool to scale the features of all the data. The scaling parameters should be computed only on the training data and then applied to the test data.

3. Consider the corresponding binary classification which consists of distinguishing two types of splice junctions in DNA sequences using about 60 features. Use SVMs combined with polynomial kernels to tackle this problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 3, 5$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let other parameters of polynomial kernels in `libsvm` be equal to their default values), varying C in powers of 5, starting from a small value $C = 5^{-k}$ to $C = 5^k$, for some value of k . k should be chosen so that you see a significant variation in training error, starting from a very high training error to a low training error. Expect longer training times with `libsvm` as the value of C increases.

Solution:

Figure 1 shows the average cross-validation performance as a function of the regularization parameter C . Note that the algorithm starts to exhibit some over-fitting as C becomes very large. The performance for several choices of d and C are essentially indistinguishable; one suitable choice of optimal parameters is $C^* = 5^1$ and $d^* = 3$. $\square$

4. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation error and the test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d . How many of the support vectors lie on the marginal hyperplanes? Plot the soft margin of the solution as a function of d .

Solution:

The first plot in Figure 4 compares CV and test errors for C^* as a function of d . As expected test error is slightly higher than CV error.

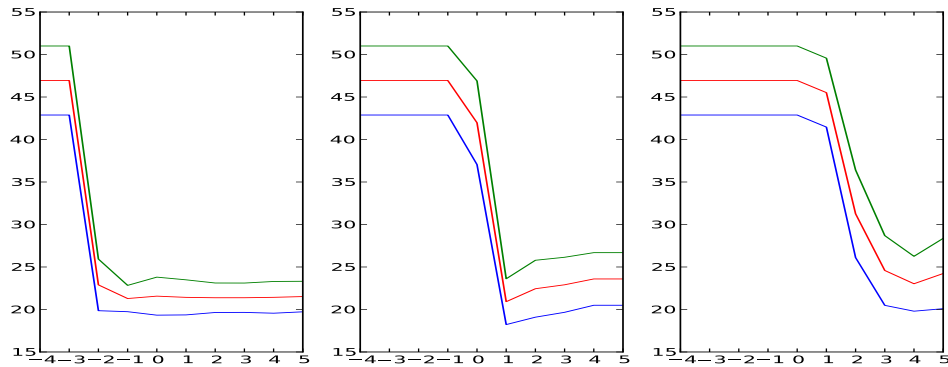


Figure 1: Average error (red) according to 10-fold cross-validation, with error-bars (green and blue) indicating one standard deviation. Left, middle and right panels correspond to $d = 1, 3, 5$ respectively. On the x -axis we have $\log_5 C$.

The second plot shows that the total number of support vectors and the number of support vectors on the marginal hyperplanes. The last plot shows the margin as a function of d . $\square$

5. Now, combine SVMs with Gaussian kernels to tackle the same task. Use cross-validation as before to determine the best value of C and σ , varying C in powers of 5, and σ in powers of 2 for a reasonable range so that you see a significant variation in training error, as before. Fix C and σ to the best values found via cross-validation. How does the test error of the solution compare to the best result obtained using polynomial kernels? What is the value of the soft margin?

Solution:

Figure 3 shows the average cross-validation performance as a function of the regularization parameter C and $g = 1/\sigma^2$. $C^* = 1$ and $g^* = 0.03125$. Figure 4 shows the test and validation error as a function of degree (top panel), the number of total and marginal support vectors (second panel) and the margin as function of g .

Note that the best test error for polynomial kernels is around 23% and with Gaussian kernels it is around 19%. $\square$

6. Here, use as a kernel the sum of the best polynomial kernel (degree

d^*) and the Gaussian kernel with the best parameter σ you found in the previous question. Use cross-validation as before to determine the best value of C . How does the test error of the solution compare to the best result obtained in the previous questions?

Solution: There are multiply ways to solve this problem. If you are using `libsvm` in `MATLAB`, then you can simply precompute the kernel matrix and use it to solve the problem. Otherwise, you can directly modify `libsvm` code (see `libsvm` FAQ on how to do this). In our case, we use Gaussian kernel with $g = 0.03125$ and polynomial kernel with $d = 3$. The test error is around 21% which is slightly worse than for Gaussian kernels, but better than for polynomial kernels.

D. Kernels

Show that the following kernels are PDS.

1. Let n be a positive integer. K is defined by $K(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^N \cos^n(x_i^2 - y_i^2)$ for all $(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^N \times \mathbb{R}^N$.

Solution: Since the product and sum of PDS kernels is PDS, it suffices to show that $k: x \mapsto \cos(x^2 - y^2)$ is PDS over $\mathbb{R} \times \mathbb{R}$. This is clear since

$$k(x, y) = \cos(x^2) \cos(y^2) + \sin(x^2) \sin(y^2) = \Phi(x) \cdot \Phi(y),$$

with

$$\Phi(x) = \begin{bmatrix} \cos(x^2) \\ \sin(x^2) \end{bmatrix}.$$

□

2. Let σ be a positive real number. K is defined by $K(x, y) = e^{-\frac{\|\mathbf{x}-\mathbf{y}\|}{\sigma}}$ for all $(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^N \times \mathbb{R}^N$ (*Hint:* you could show that K is the normalized kernel of a kernel K' and show that K' is PDS using the following equality: $\|\mathbf{x} - \mathbf{y}\| = \frac{1}{2\Gamma(\frac{1}{2})} \int_0^{+\infty} \frac{1 - e^{-t\|\mathbf{x}-\mathbf{y}\|^2}}{t^{\frac{3}{2}}} dt$ valid for all $\mathbf{x}, \mathbf{y}$).

Solution: It suffices to show that K is the normalized kernel associated to the kernel K' defined by

$$\forall (\mathbf{x}, \mathbf{y}) \in \mathbb{R}^N \times \mathbb{R}^N, K'(\mathbf{x}, \mathbf{y}) = e^{\phi(\mathbf{x}, \mathbf{y})}$$

where $\phi(\mathbf{x}, \mathbf{y}) = \frac{1}{\sigma} [\|\mathbf{x}\| + \|\mathbf{y}\| - \|\mathbf{x} - \mathbf{y}\|]$, and to show that K' is PDS. For the first part, observe that

$$\frac{K'(\mathbf{x}, \mathbf{y})}{\sqrt{K'(\mathbf{x}, \mathbf{x})K'(\mathbf{y}, \mathbf{y})}} = e^{\phi(\mathbf{x}, \mathbf{y}) - \frac{1}{2}\phi(\mathbf{x}, \mathbf{x}) - \frac{1}{2}\phi(\mathbf{y}, \mathbf{y})} = e^{-\frac{\|\mathbf{x} - \mathbf{y}\|}{\sigma}}.$$

To show that K' is PDS, it suffices to show that ϕ is PDS, since composition with a power series with non-negative coefficients (here exp) preserve the PDS property. Now, for any $c_1, \dots, c_n \in \mathbb{R}$, let $c_0 = -\sum_{i=1}^n c_i$, then, we can write

$$\begin{aligned} \sum_{i,j=1}^n c_i c_j \phi(\mathbf{x}_i, \mathbf{x}_j) &= \frac{1}{\sigma} \sum_{i,j=1}^n c_i c_j [\|\mathbf{x}_i\| + \|\mathbf{x}_j\| - \|\mathbf{x}_i - \mathbf{x}_j\|] \\ &= \frac{1}{\sigma} \left[-\sum_{i=1}^n c_0 c_i \|\mathbf{x}_i\| + -\sum_{i=1}^n c_0 c_j \|\mathbf{x}_j\| - \sum_{i,j=1}^n c_i c_j \|\mathbf{x}_i - \mathbf{x}_j\| \right] \\ &= -\frac{1}{\sigma} \sum_{i,j=0}^n c_i c_j \|\mathbf{x}_i - \mathbf{x}_j\|, \end{aligned}$$

with $\mathbf{x}_0 = 0$. Now, for any $z \in \mathbb{R}$, the following equality holds:

$$z^{\frac{1}{2}} = \frac{1}{2\Gamma(\frac{1}{2})} \int_0^{+\infty} \frac{1 - e^{-tz}}{t^{\frac{3}{2}}} dt.$$

Thus,

$$\begin{aligned} -\frac{1}{\sigma} \sum_{i,j=0}^n c_i c_j \|\mathbf{x}_i - \mathbf{x}_j\| &= \frac{1}{2\Gamma(\frac{1}{2})} \int_0^{+\infty} -\frac{1}{\sigma} \sum_{i,j=0}^n c_i c_j \frac{1 - e^{-t\|\mathbf{x}_i - \mathbf{x}_j\|}}{t^{\frac{3}{2}}} dt \\ &= \frac{1}{2\Gamma(\frac{1}{2})} \int_0^{+\infty} \frac{1}{\sigma} \frac{\sum_{i,j=0}^n c_i c_j e^{-t\|\mathbf{x}_i - \mathbf{x}_j\|}}{t^{\frac{3}{2}}} dt. \end{aligned}$$

Since a Gaussian kernel is PDS, the inequality $\sum_{i,j=0}^n c_i c_j e^{-t\|\mathbf{x}_i - \mathbf{x}_j\|} \geq 0$ holds and the right-hand side is non-negative. Thus, the inequality $-\frac{1}{\sigma} \sum_{i,j=0}^n c_i c_j \|\mathbf{x}_i - \mathbf{x}_j\| \geq 0$ holds, which shows that ϕ is PDS. $\square$

Alternatively, one can also apply the theorem on page 43 of the lecture slides on kernel methods to reduce the problem to showing that the norm $G(x, y) = \|x - y\|$ is a NDS function. This can be shown through a direct application of the definition of NDS together with the representation of the norm given in the hint.

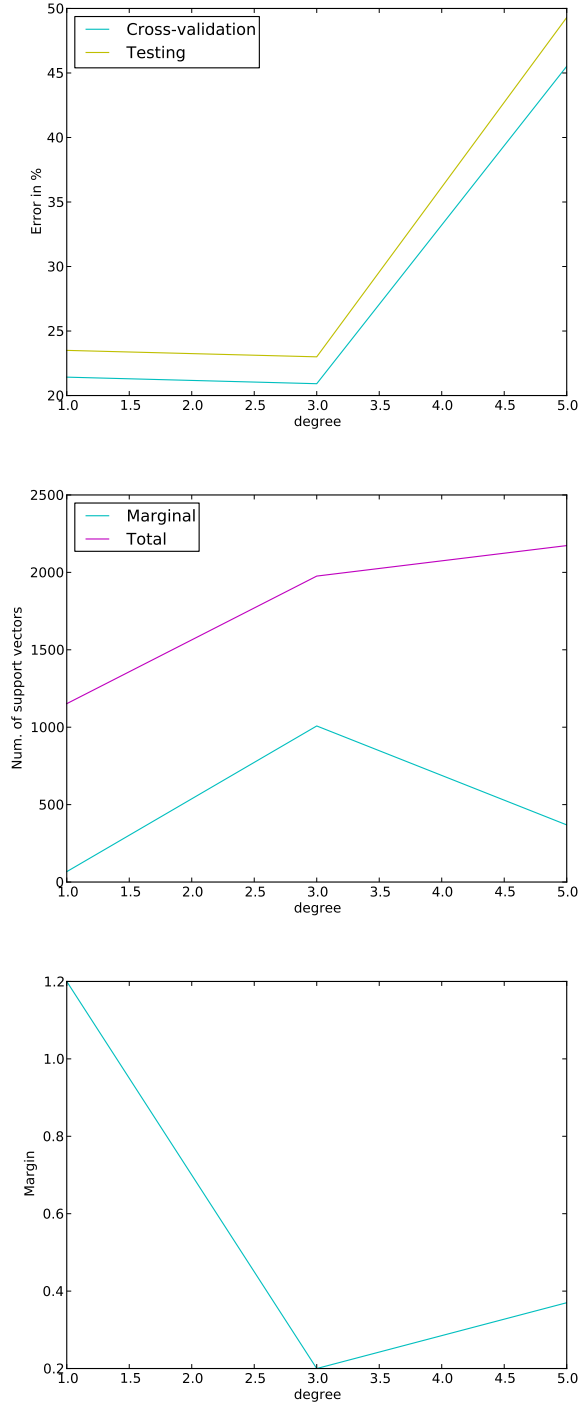


Figure 2: The test and validation error as a function of d (top panel), the number of total and marginal support vectors (second panel) and the margin as function of d .

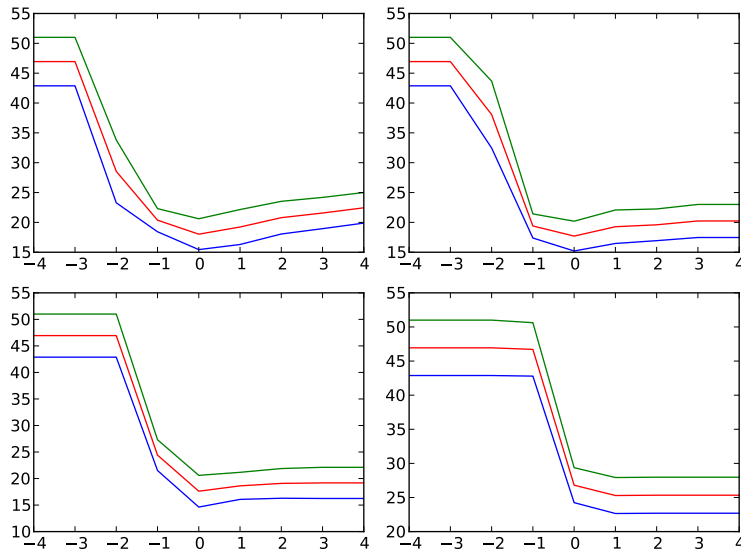


Figure 3: Average error (red) according to 10-fold cross-validation, with error-bars (green and blue) indicating one standard deviation. Top left, top right, bottom left and bottom right correspond to $g = 0.015625, 0.03125, 0.0625, 0.125$ respectively. Note that $g = 1/2\sigma^2$. On the x -axis we have $\log_5 C$.

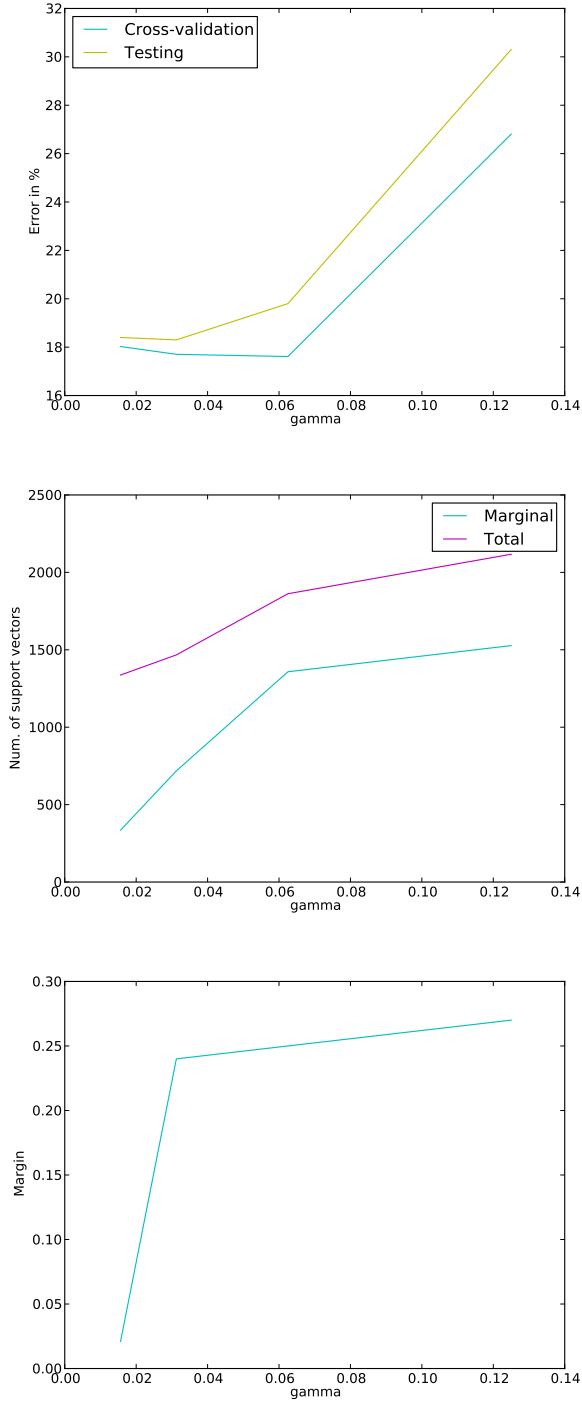


Figure 4: The test and validation error as a function of degree (top panel), the number of total and marginal support vectors (second panel) and the margin as function of g .

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 2
October 04, 2016
Due: October 18, 2016

A. Rademacher complexity

The definitions and notation are those introduced in the lectures slides.

1. What is the Rademacher complexity of a hypothesis set reduced to a single hypothesis? An alternative definition of the Rademacher is based on absolute values: $\mathfrak{R}'(H) = \frac{1}{m} \mathbb{E}_{\sigma, S}[\sup_{h \in H} |\sum_{i=1}^m \sigma_i h(x_i)|]$. Show the following upper bound for a hypothesis set reduced to a single hypothesis h :

$$\mathfrak{R}'(\{h\}) \leq \sqrt{\frac{\mathbb{E}_{x \sim D}[h^2(x)]}{m}}. \quad (1)$$

(*hint*: you can use the inequality $\mathbb{E}[|X|] \leq \sqrt{\mathbb{E}[X^2]}$ which is valid for any random variable X , by Jensen's inequality).

2. Fix $m \geq 1$. Prove the following identities for any $\alpha \in \mathbb{R}$ and any two hypothesis sets H and H' of functions mapping from $\mathcal{X}$ to $\mathbb{R}$:

$$\mathfrak{R}_m(\alpha H) = |\alpha| \mathfrak{R}_m(H) \quad (2)$$

$$\mathfrak{R}_m(H + H') \leq \mathfrak{R}_m(H) + \mathfrak{R}_m(H'). \quad (3)$$

$$\mathfrak{R}_m(\{\max(h, h') : h \in H, h' \in H'\}) \leq \mathfrak{R}_m(H) + \mathfrak{R}_m(H'), \quad (4)$$

where $\max(h, h')$ denotes the function $x \mapsto \max_{x \in \mathcal{X}}(h(x), h'(x))$ (*hint*: you could use the identity $\max(a, b) = \frac{1}{2}[a + b + |a - b|]$ valid for all $a, b \in \mathbb{R}$ and the contraction lemma (Lecture 4)).

B. VC-dimension

1. What is the VC-dimension of the family of subsets of the real line $[x, x + 1] \cup [x + 2, +\infty)$, with $x \in \mathbb{R}$?
2. VC-dimension of sine functions. Consider the hypothesis family of sine functions: $\{x \rightarrow \sin(\omega x) : \omega \in \mathbb{R}\}$.

- (a) Show that for any $x \in \mathbb{R}$ the points $x, 2x, 3x$ and $4x$ cannot be shattered by this family of sine functions.
- (b) Show that the VC-dimension of the family of sine functions is infinite. (*hint*: show that $\{2^{-m} : m \in \mathbb{N}\}$ can be fully shattered for any $m > 0$.)

C. Support Vector Machines

1. Download and install the `libsvm` software library from:

`http://www.csie.ntu.edu.tw/~cjlin/libsvm/`

2. Consider the `spambase` data set

`http://archive.ics.uci.edu/ml/datasets/Spambase`.

Download a shuffled version of that dataset from

`http://www.cs.nyu.edu/~mohri/ml16/spambase.data.shuffled`

Use the `libsvm` scaling tool to scale the features of all the data. Use the first 3450 examples for training, the last 1151 for testing. The scaling parameters should be computed only on the training data and then applied to the test data.‘;

3. Consider the binary classification that consists of predicting if the e-mail message is a spam using the 57 features. Use SVMs combined with polynomial kernels to tackle this binary classification problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let other parameters of polynomial kernels in `libsvm` be equal to their default values), varying C in powers of 2, starting from a small value $C = 2^{-k}$ to $C = 2^k$, for some value of k . k should be chosen so that you see a significant variation in training error, starting from a very high training error to a low training error. Expect longer training times with `libsvm` as the value of C increases.

4. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation error and the test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d . How many of the support vectors lie on the margin hyperplanes?

5. Suppose we replace in the primal optimization problem of SVMs the penalty term $\sum_{i=1}^m \xi_i = \|\boldsymbol{\xi}\|_1$ with $\|\boldsymbol{\xi}\|_\infty = \max_{i=1}^m \xi_i$. Give the associated dual optimization problem. Show that it differs from the standard dual optimization problem of SVMs only by the constraints, which can be expressed in terms of $\|\boldsymbol{\alpha}\|_1$.

Mehryar Mohri
 Foundations of Machine Learning
 Courant Institute of Mathematical Sciences
 Homework assignment 2
 October 04, 2016
 Due: October 18, 2016

A. Rademacher complexity

The definitions and notation are those introduced in the lectures slides.

1. What is the Rademacher complexity of a hypothesis set reduced to a single hypothesis? An alternative definition of the Rademacher is based on absolute values: $\mathfrak{R}'(H) = \frac{1}{m} \mathbb{E}_{\sigma, S} [\sup_{h \in H} |\sum_{i=1}^m \sigma_i h(x_i)|]$. Show the following upper bound for a hypothesis set reduced to a single hypothesis h :

$$\mathfrak{R}'(\{h\}) \leq \sqrt{\frac{\mathbb{E}_{x \sim D}[h^2(x)]}{m}}. \quad (1)$$

(*hint*: you can use the inequality $\mathbb{E}[|X|] \leq \sqrt{\mathbb{E}[X^2]}$ which is valid for any random variable X , by Jensen's inequality).

Solution: Let h be that single hypothesis. By definition,

$$\mathfrak{R}'(\{h\}) = \frac{1}{m} \mathbb{E}_{\sigma, S} \left[\sum_{i=1}^m \sigma_i h(x_i) \right] = \frac{1}{m} \mathbb{E}_S \left[\sum_{i=1}^m \mathbb{E}_{\sigma} [\sigma_i] h(x_i) \right] = 0,$$

since $\mathbb{E}_{\sigma} [\sigma_i] = 0$ for all $i \in [1, m]$. Using Jensen's inequality, with the

alternative definition the Rademacher complexity can be bounded as follows:

$$\begin{aligned}
\mathfrak{R}'(\{h\}) &= \frac{1}{m} \mathbb{E}_{\sigma, S} \left[\left| \sum_{i=1}^m \sigma_i h(x_i) \right| \right] \\
&= \frac{1}{m} \mathbb{E}_{\sigma, S} \left[\sqrt{\left| \sum_{i=1}^m \sigma_i h(x_i) \right|^2} \right] \\
&\leq \frac{1}{m} \sqrt{\mathbb{E}_{\sigma, S} \left[\left| \sum_{i=1}^m \sigma_i h(x_i) \right|^2 \right]} \quad (\text{by Jensen's inequality}) \\
&= \frac{1}{m} \sqrt{\mathbb{E}_{\sigma, S} \left[\sum_{i,j=1}^m \sigma_i \sigma_j h(x_i) h(x_j) \right]} \\
&= \frac{1}{m} \sqrt{\mathbb{E}_S \left[\sum_{i=1}^m h(x_i)^2 \right]} \quad (\mathbb{E}[\sigma_i \sigma_j] = 0 \text{ for } i \neq j) \\
&= \frac{1}{m} \sqrt{m \mathbb{E}_S [h(x_1)^2]} = \sqrt{\frac{\mathbb{E}_x [h^2(x)]}{m}}. \quad (\text{i.i.d. sample})
\end{aligned}$$

□

2. Fix $m \geq 1$. Prove the following identities for any $\alpha \in \mathbb{R}$ and any two hypothesis sets H and H' of functions mapping from $\mathcal{X}$ to $\mathbb{R}$:

$$\mathfrak{R}_m(\alpha H) = |\alpha| \mathfrak{R}_m(H) \quad (2)$$

$$\mathfrak{R}_m(H + H') \leq \mathfrak{R}_m(H) + \mathfrak{R}_m(H'). \quad (3)$$

$$\mathfrak{R}_m(\{\max(h, h') : h \in H, h' \in H'\}) \leq \mathfrak{R}_m(H) + \mathfrak{R}_m(H'), \quad (4)$$

where $\max(h, h')$ denotes the function $x \mapsto \max_{x \in \mathcal{X}}(h(x), h'(x))$ (*hint*: you could use the identity $\max(a, b) = \frac{1}{2}[a + b + |a - b|]$ valid for all $a, b \in \mathbb{R}$ and the contraction lemma (Lecture 4)).

Solution: If $\alpha \geq 0$, then

$$\sup_{h \in \alpha H} \sum_{i=1}^m \sigma_i h(x_i) = \sup_{h \in H} \sum_{i=1}^m \alpha \sigma_i h(x_i) = \alpha \sup_{h \in H} \sum_{i=1}^m \sigma_i h(x_i),$$

otherwise if $\alpha < 0$, then

$$\sup_{h \in \alpha H} \sum_{i=1}^m \sigma_i h(x_i) = \sup_{h \in \alpha H} \sum_{i=1}^m \alpha \sigma_i h(x_i) = (-\alpha) \sup_{h \in H} \sum_{i=1}^m (-\sigma_i) h(x_i).$$

Since σ_i and $-\sigma_i$ have the same distribution, this shows that $\mathfrak{R}_m(\alpha H) = |\alpha| \mathfrak{R}_m(H)$.

The second inequality actually holds with equality, and follows from:

$$\begin{aligned}
& \mathfrak{R}_m(H + H') \\
&= \frac{1}{m} \mathbb{E}_{\sigma, S} \left[\sup_{h \in H, h' \in H'} \sum_{i=1}^m \sigma_i (h(x_i) + h'(x_i)) \right] \\
&= \frac{1}{m} \mathbb{E}_{\sigma, S} \left[\sup_{h \in H, h' \in H'} \sum_{i=1}^m \sigma_i h(x_i) + \sup_{h \in H, h' \in H'} \sum_{i=1}^m \sigma_i h'(x_i) \right] \\
&= \frac{1}{m} \mathbb{E}_{\sigma, S} \left[\sup_{h \in H} \sum_{i=1}^m \sigma_i h(x_i) \right] + \frac{1}{m} \mathbb{E}_{\sigma, S} \left[\sup_{h' \in H'} \sum_{i=1}^m \sigma_i h'(x_i) \right].
\end{aligned}$$

For the third inequality, using the identity $\max(a, b) = \frac{1}{2}[a + b + |a - b|]$ valid for all $a, b \in \mathbb{R}$, and the sub-additivity of sup we can write:

$$\begin{aligned}
& \mathfrak{R}_m(\{\max(h, h') : h \in H, h' \in H'\}) \\
&= \frac{1}{2m} \mathbb{E}_{\sigma, S} \left[\sup_{h \in H, h' \in H'} \sum_{i=1}^m \sigma_i [h(x_i) + h'(x_i) + |h(x_i) - h'(x_i)|] \right] \\
&\leq \frac{1}{2} [\mathfrak{R}_m(H) + \mathfrak{R}_m(H')] + \frac{1}{2m} \mathbb{E}_{\sigma, S} \left[\sup_{h \in H, h' \in H'} \sum_{i=1}^m \sigma_i |h(x_i) - h'(x_i)| \right].
\end{aligned}$$

Since the absolute value function is 1-Lipschitz, by the contraction lemma, the third term can be bounded as follows

$$\begin{aligned}
& \frac{1}{2m} \mathbb{E}_{\sigma, S} \left[\sup_{h \in H, h' \in H'} \sum_{i=1}^m \sigma_i |h(x_i) - h'(x_i)| \right] \\
&\leq \frac{1}{2m} \mathbb{E}_{\sigma, S} \left[\sup_{h \in H, h' \in H'} \sum_{i=1}^m \sigma_i [h(x_i) - h'(x_i)] \right] \\
&\leq \frac{1}{2m} \mathbb{E}_{\sigma, S} \left[\sup_{h \in H, h' \in H'} \sum_{i=1}^m \sigma_i h(x_i) + \sup_{h \in H, h' \in H'} \sum_{i=1}^m -\sigma_i h'(x_i) \right] \\
&= \frac{1}{2m} \mathbb{E}_{\sigma, S} \left[\sup_{h \in H, h' \in H'} \sum_{i=1}^m \sigma_i h(x_i) \right] + \frac{1}{2m} \mathbb{E}_{\sigma, S} \left[\sup_{h \in H, h' \in H'} \sum_{i=1}^m -\sigma_i h'(x_i) \right] \\
&= \frac{1}{2} [\mathfrak{R}_m(H) + \mathfrak{R}_m(H')],
\end{aligned}$$

using the fact that σ_i and $-\sigma_i$ follow the same distribution. $\square$

B. VC-dimension

1. What is the VC-dimension of the family of subsets of the real line $[x, x + 1] \cup [x + 2, +\infty)$, with $x \in \mathbb{R}$?

Solution: The VC-dimension of this family is less than 4 since the labeling $+-+-$ is not possible to obtain using any hypothesis: for any set of 4 points $\{x_1, x_2, x_3, x_4\}$ with $x_1 < x_2 < x_3 < x_4$, the labeling of the first 3 points imposes $x_1 \in [x, x + 1]$, $x_2 \in (x + 1, x + 2)$, and $x_3 \in [x + 2, +\infty)$ for some $x \in \mathbb{R}$, which implies a positive labeling for x_4 .

The VC-dimension is equal to 3 since there is a set of 3 points that can be fully shattered. An example is $\{1/4, 9/8, 7/4\}$ with the following intervals for each labeling sequence:

+++	$[0, +1] \cup [2, +\infty)$
++-	$[1/4, 5/4] \cup [9/4, +\infty)$
+ - +	$[-1/2, +1/2] \cup [3/2, +\infty)$
+ - -	$[0, 1] \cup [2, +\infty)$
- + +	$[-1, 0] \cup [1, +\infty)$
- + -	$[1/2, 3/2] \cup [5/2, +\infty]$
- - +	$[3/2, 5/2] \cup [7/2, +\infty)$
- - -	$[2, 3] \cup [4, +\infty).$

□

2. VC-dimension of sine functions. Consider the hypothesis family of sine functions: $\{x \rightarrow \text{sign}(\sin(\omega x)) : \omega \in \mathbb{R}\}$.

- (a) Show that for any $x \in \mathbb{R}$ the points $x, 2x, 3x$ and $4x$ cannot be shattered by this family of sine functions.

Solution:

Fix $x \in \mathbb{R}$ and suppose there exists an ω that realizes the labeling $--+-$. Thus $\sin(\omega x) < 0$, $\sin(2\omega x) < 0$, $\sin(3\omega x) \geq 0$ and $\sin(4\omega x) < 0$. We will show that this implies $\sin^2(\omega x) < \frac{1}{2}$ and $\sin^2(\omega x) \geq \frac{3}{4}$, a contradiction.

Using the identity $\sin(2\theta) = 2 \sin(\theta) \cos(\theta)$ and the fact that $\sin(4\omega x) < 0$ we have

$$2 \sin(2\omega x) \cos(2\omega x) = \sin(4\omega x) < 0.$$

Since $\sin(2\omega x) < 0$ we can divide both sides of this inequality by $2\sin(2\omega x)$ to conclude $\cos(2\omega x) > 0$. Applying the identity $\cos(2\theta) = 1 - 2\sin^2(\theta)$ yields $1 - 2\sin^2(\omega x) > 0$, or $\sin^2(\omega x) < \frac{1}{2}$.

Using the identity $\sin(3\theta) = 3\sin(\theta) - 4\sin^3(\theta)$ and the fact that $\sin(3\omega x) \geq 0$ we have

$$3\sin(\omega x) - 4\sin^3(\omega x) = \sin(3\omega x) \geq 0$$

Since $\sin(\omega x) < 0$ we can divide both sides of this inequality by $\sin(\omega x)$ to conclude $3 - 4\sin^2(\omega x) \leq 0$, or $\sin^2(\omega x) \geq \frac{3}{4}$.

□

Interestingly, it is possible to find a set of four evenly spaced points that can be shattered by this family of sine functions, provided that the distance from the origin to the nearest point is allowed to differ from the distance between the points. For example, the points $\{1, 1.5, 2, 2.5\}$ are shattered by the frequencies

$$\omega \in \{-10, -9, -8, -7, -6, -5, -4, -1, 1, 2, 4, 5, 11, 14, 15, 23\}.$$

- (b) Show that the VC-dimension of the family of sine functions is infinite. (*hint*: show that $\{2^{-m} : m \in \mathbb{N}\}$ can be fully shattered for any $m > 0$.)

Solution:

For any $m > 0$, consider the set of points $(x_1, \dots, x_m)$ with arbitrary labels $(y_1, \dots, y_m) \in \{-1, +1\}^m$. Now, choose the parameter $\omega = \pi(1 + \sum_{i=1}^m 2^i y'_i)$ where $y'_i = \frac{1-y_i}{2}$. We show that this single parameter will always correctly classify the entire sample for any $m > 0$ and choice of labels. For any $j \in [1, m]$ we have

$$\begin{aligned} \omega x_j &= \omega 2^{-j} = \pi(2^{-j} + \sum_{i=1}^m 2^{i-j} y'_i) \\ &= \pi(2^{-j} + (\sum_{i=1}^{j-1} 2^{i-j} y'_i) + y'_j + (\sum_{i=1}^{m-j} 2^i y'_i)). \end{aligned}$$

The last term can be dropped from the sum since it only contributes multiples of 2π . Since $y'_i \in \{0, 1\}$ the remaining term, that is, $\pi(2^{-j} + (\sum_{i=1}^{j-1} 2^{i-j} y'_i) + y'_j) = \pi(\sum_{i=1}^{j-1} 2^{-i} y'_i + 2^{-j} + y'_j)$, can be upper and

lower bound as follows:

$$\pi\left(\sum_{i=1}^{j-1} 2^{-i} y'_i + 2^{-j} + y'_j\right) \leq \pi\left(\sum_{i=1}^j 2^{-i} + y'_j\right) < \pi(1 + y'_j),$$

$$\pi\left(\sum_{i=1}^{j-1} 2^{-i} y'_i + 2^{-j} + y'_j\right) > \pi y'_j.$$

Thus, if $y_j = 1$ we have $y'_j = 0$ and $0 < \omega x_j < \pi$, which implies $\text{sign}(\omega x_j) = 1$. Similarly, for $y_j = -1$ we have $\text{sign}(\omega x_j) = -1$. $\square$

C. Support Vector Machines

1. Download and install the `libsvm` software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

2. Consider the `spambase` data set

<http://archive.ics.uci.edu/ml/datasets/Spambase>.

Download a shuffled version of that dataset from

<http://www.cs.nyu.edu/~mohri/ml16/spambase.data.shuffled>

Use the `libsvm` scaling tool to scale the features of all the data. Use the first 3450 examples for training, the last 1151 for testing. The scaling parameters should be computed only on the training data and then applied to the test data.

3. Consider the binary classification that consists of predicting if the e-mail message is a spam using the 57 features. Use SVMs combined with polynomial kernels to tackle this binary classification problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let other parameters of polynomial kernels in `libsvm` be equal to their default values), varying C in powers of 2, starting from a small value $C = 2^{-k}$ to $C = 2^k$, for some value of k . k should be chosen so that you see a significant variation in training error, starting from a very high training

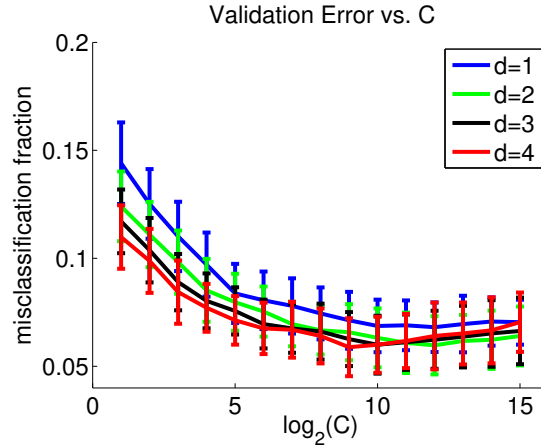


Figure 1: Average error according to 10-fold cross-validation, with error-bars indicating one standard deviation.

error to a low training error. Expect longer training times with `libsvm` as the value of C increases.

Solution: Figure 1 shows the average cross-validation performance as a function of the regularization parameter C . Note that the algorithm starts to exhibit some over-fitting as C becomes very large. The performance for several choices of d and C are essentially indistinguishable; one suitable choice of optimal parameters is $C^* = 2^9$ and $d^* = 4$.

4. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation error and the test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d . How many of the support vectors lie on the margin hyperplanes?

Solution: The first plot in Figure 2 shows that the test error decreases (slightly) with an increase in degree and also that the cross-validation error is (slightly) optimistic when compared to the test error on the held-out dataset.

The second plot shows that the total number of marginal support vectors increases with d (as does the dimension of the feature space) while the total number of overall support vectors decreases. This also implies that the number of support vectors due to mistakes is decreasing, which agrees with the

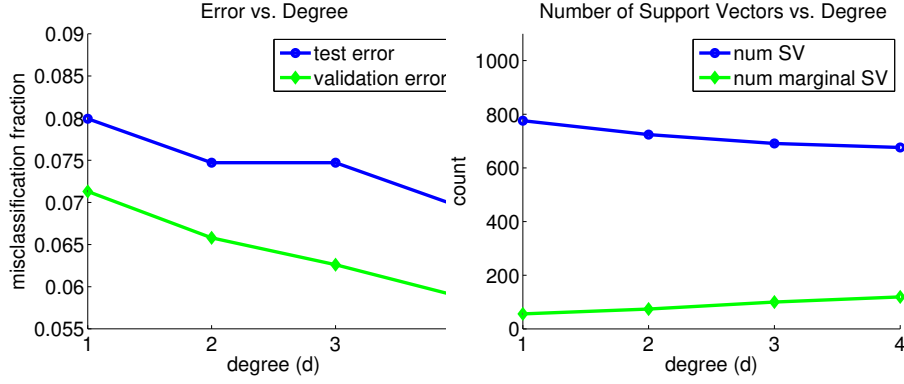


Figure 2: The test and validation error as a function of degree (left panel) as well as the number of total and marginal support vectors (right panel).

first plot.

- Suppose we replace in the primal optimization problem of SVMs the penalty term $\sum_{i=1}^m \xi_i = \|\xi\|_1$ with $\|\xi\|_\infty = \max_{i=1}^m \xi_i$. Give the associated dual optimization problem. Show that it differs from the standard dual optimization problem of SVMs only by the constraints, which can be expressed in terms of $\|\alpha\|_1$.

Solution: The optimization problem for this version of SVMs can be written as follows:

$$\begin{aligned}
 & \min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C\xi \\
 & \text{subject to} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi \forall i \in [1, m] \\
 & \quad \quad \quad \xi \geq 0.
 \end{aligned} \tag{5}$$

The corresponding Lagrange function can be written as

$$L = \frac{1}{2} \|\mathbf{w}\|^2 + C\xi - \sum_{i=1}^m \alpha_i [y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 + \xi] - \beta \xi.$$

Differentiating with respect to the primal variables gives:

$$\begin{aligned}\nabla_{\mathbf{w}} L = 0 &\implies \mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i \\ \nabla_b L = 0 &\implies \sum_{i=1}^m \alpha_i y_i = 0 \\ \nabla_{\xi} L = 0 &\implies \sum_{i=1}^m \alpha_i + \beta = C.\end{aligned}$$

Plugging in the first equality in L and using the second and third yields:

$$L = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j).$$

In view of the third equality, the condition $\beta \geq 0$ can be equivalently written as $\sum_{i=1}^m \alpha_i \leq C$. Thus, the equivalent dual optimization problem can be written as

$$\begin{aligned}\max_{\boldsymbol{\alpha}} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \\ \text{subject to} \quad & (\boldsymbol{\alpha} \geq 0) \wedge (\|\boldsymbol{\alpha}\|_1 \leq C) \wedge \left(\sum_{i=1}^m \alpha_i y_i = 0 \right).\end{aligned}$$

More generally, a $\|\cdot\|_p$ -constraint on $\boldsymbol{\xi}$ in the primal optimization problem leads to a $\|\cdot\|_q$ -constraint (dual norm constraint) on $\boldsymbol{\alpha}$ in the dual, where p and q are conjugate: $1/p + 1/q = 1$. $\square$

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 2
October 25, 2017
Due: November 08, 2017

A. Growth function

Growth function of stump functions. For any $x \in \mathbb{R}$ and $\theta \in \mathbb{R}$, let ϕ_θ denote the threshold function that assigns sign $+1$ to $x \leq \theta$, -1 otherwise: $\phi_\theta(x) = 21_{x \leq \theta} - 1$. Let $\mathcal{H}$ be the family of functions mapping $\mathbb{R}^N$ to $\{-1, +1\}$ defined by

$$\mathcal{H} = \left\{ \mathbf{x} \mapsto s \phi_\theta(x_i) : i \in [1, N], \theta \in \mathbb{R}, s \in \{-1, +1\} \right\},$$

where x_i is the i th coordinate of $\mathbf{x} \in \mathbb{R}^N$.

1. Show that the following upper bound holds for the growth function of $\mathcal{H}$:
 $\Pi_m(\mathcal{H}) \leq 2mN$.
2. Let $\mathcal{H}_2$ be the family of functions mapping $\mathbb{R}^N$ to $\{-1, +1\}$ defined by

$$\mathcal{H}_2 = \left\{ \mathbf{x} \mapsto s 1_{\phi_\theta(x_i)=+1} \phi_{\theta'}(x_j) + s' 1_{\phi_\theta(x_i)=-1} \phi_{\theta'}(x_j) : \right. \\ \left. i \neq j, i, j \in [1, N], \theta, \theta' \in \mathbb{R}, s, s' \in \{-1, +1\} \right\}.$$

Show that $\Pi_m(\mathcal{H}_2) = O(m^2 N^2)$. Give an explicit upper bound on $\Pi_m(\mathcal{H}_2)$.

B. VC-dimension

1. VC-dimension of circles in the plane.
 - (a) Show that the VC-dimension of the circles in the planes is 3.
 - (b) Let H_1 and H_2 be two families of functions mapping from $\mathcal{X}$ to $\{0, 1\}$ and $H = \{h_1 h_2 : h_1 \in H_1, h_2 \in H_2\}$ their product. Show that

$$\Pi_H(m) \leq \Pi_{H_1}(m) \Pi_{H_2}(m).$$

- (c) Give an upper bound on the VC-dimension of the family of intersections of k circles in the planes.

2. VC-dimension of Decision trees. A full binary tree is a tree in which each node is either a leaf or it is an internal node and admits exactly two child nodes.

- (a) Show that that a binary tree with n internal nodes has exactly $n + 1$ leaves (*Hint*: you can proceed by induction).
- (b) A binary decision tree is a full binary tree with each leaf labeled with $+1$ or -1 and each internal node labeled with a question.

A binary decision tree classifies a point as follows: starting with the root of the tree, if the internal node question applied to the point admits a positive answer, then the current node becomes the right child, otherwise it becomes the left child. This is repeated until a leaf node is reached. The label assigned to the point is then the sign of that leaf node.

Suppose the node questions are of the form $x_i > 0$, $i \in [1, N]$. Show that the VC-dimension of the set of binary decision trees with n nodes in dimension N is at most $(2n+1) \log_2(N+2)$ (*Hint*: bound the cardinality of the set). Use that to derive an upper bound on the Rademacher complexity of that set.

C. Support-Vector Machines

1. Download and install the `libsvm` software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

2. Consider the `spambase` data set

<http://archive.ics.uci.edu/ml/datasets/Spambase>.

Download a shuffled version of that dataset from

<http://www.cs.nyu.edu/~mohri/ml17/spambase.data.shuffled>

Use the `libsvm` scaling tool to scale the features of all the data. Use the first 3000 examples for training, the last 1601 for testing. The scaling parameters should be computed only on the training data and then applied to the test data.

3. Consider the binary classification that consists of predicting if the e-mail message is a spam using the 57 features. Use SVMs combined with polynomial kernels to tackle this binary classification problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let other parameters of polynomial kernels in `libsvm` be equal to their default values), varying C in powers of 2, starting from a small value $C = 2^{-k}$ to $C = 2^k$, for some value of k . k should be chosen so that you see a significant variation in training error, starting from a very high training error to a low training error. Expect longer training times with `libsvm` as the value of C increases.

4. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation error and the test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d . How many of the support vectors lie on the margin hyperplanes?
5. For any d , let K_d denote the polynomial kernel of degree d . Show that for any fixed integer $u > 0$, $G_u = \frac{2}{u(u+1)} \sum_{i \leq j \leq u} K_i K_j$ is a PDS kernel. Use SVMs combined with the polynomial kernel G_4 to tackle the same binary classification problem as in the previous questions: as in the previous questions, use ten-fold cross-validation to determine the best value of C ; report the ten-fold cross-validation error and the test error of the hypothesis obtained when training with G_u .

D. Rademacher complexity

Let $p > 2$ and let q be its conjugate: $1/p + 1/q = 1$. Let $\mathcal{H}_p$ be the family of linear functions defined over $\{\mathbf{x} \in \mathbb{R}^N : \|\mathbf{x}\|_p \leq r_p\}$ by

$$\mathcal{H} = \left\{ \mathbf{x} \mapsto \mathbf{w} \cdot \mathbf{x} : \|\mathbf{w}\|_q \leq \Lambda_q \right\},$$

for some $r_p > 0$ and $\Lambda_q > 0$. Give an upper bound on $\mathfrak{R}_m(\mathcal{H})$ in terms of Λ_q and r_p , assuming that for $p > 2$ the following inequality holds for all $z_1, \dots, z_m \in \mathbb{R}$: $\mathbb{E}_{\sigma} \left[\left| \sum_{i=1}^m \sigma_i z_i \right|^p \right] \leq \left[\frac{p}{2} \sum_{i=1}^m z_i^2 \right]^{\frac{p}{2}}$.

Mehryar Mohri
 Foundations of Machine Learning 2018
 Courant Institute of Mathematical Sciences
 Homework assignment 2
 October 07, 2018
 Due: October 21, 2018

A. Radmacher complexity

1. Consider the class of functions $\mathcal{H}$ mapping from $\mathbb{R}$ to $\{+1, -1\}$ such that

$$h(x) = \begin{cases} +1 & \text{for } x \in [a, b], \\ -1 & \text{otherwise,} \end{cases}$$

for some $a, b \in \mathbb{R}$. Give an upper bound on the growth function $\Pi_{\mathcal{H}}(m)$ and use it to derive an upper bound on the $\mathfrak{R}_m(\mathcal{H})$.

2. Prove that for any hypotheses class $\mathcal{H}$ and any function $h: \mathcal{X} \mapsto \mathbb{R}$, $\mathfrak{R}_m(\mathcal{H}) = \mathfrak{R}_m(\mathcal{H} + h)$.
3. Prove that if for two hypotheses classes $\mathcal{H}$ and $\mathcal{F}$ the inclusion $\mathcal{H} \subseteq \mathcal{F}$ holds, then the following inequality holds for any finite sample S : $\hat{\mathfrak{R}}_S(\mathcal{H}) \leq \hat{\mathfrak{R}}_S(\mathcal{F})$.
4. Let $\mathcal{H}_1$ be a family of functions mapping $\mathcal{X}$ to $\{0, 1\}$ and let $\mathcal{H}_2$ be a family of functions mapping $\mathcal{X}$ to $\{-1, +1\}$. Let $\mathcal{H} = \{h_1 h_2: h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}$. Show that the empirical Rademacher complexity of $\mathcal{H}$ for any sample S of size m can be bounded as follows:

$$\hat{\mathfrak{R}}_S(\mathcal{H}) \leq \hat{\mathfrak{R}}_S(\mathcal{H}_1) + \hat{\mathfrak{R}}_S(\mathcal{H}_2).$$

(*hint*: write $h_1 h_2$ in a way such that you can apply Talagrand's inequality.)

B. VC-dimension

1. What is the VC-dimension of axis-aligned squares in $\mathbb{R}^2$?
2. What is the VC-dimension of intersections of 2 axis-aligned squares in $\mathbb{R}^2$?

3. (Bonus) Let C be a concept class whose VC-dimension is 3. Show that the VC-dimension of intersections of k concepts from C is upper bounded by $6k \log_2(3k)$. (*hint*: use Sauer's lemma.)

C. Support Vector Machines

1. Download and install the libsvm software library from:

<https://www.csie.ntu.edu.tw/~cjlin/libsvm>

and briefly consult the documentation to become more familiar with the tools.

2. Consider the `svmguide1` dataset

<https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary/svmguide1>

Download a shuffled version of that dataset from

<http://www.cs.nyu.edu/~mohri/ml18/svmguide1.shuffled>

Use the `libsvm` scaling tool to scale the features of all the data. Use the first 2316 examples for training, the last 773 for testing. The scaling parameters should be computed only on the training data and then applied to the test data.

3. Consider the binary classification task in `svmguide1`, using the 4 features. Use SVMs combined with polynomial kernels to tackle this binary classification problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let other parameters of polynomial kernels in `libsvm` be equal to their default values), varying C in powers of 2, starting from a small value $C = 2^{-k}$ to $C = 2^k$, for some value of k . k should be chosen so that you see a significant variation in training error, starting from a very high training error to a low training error. Expect longer training times with `libsvm` as the value of C increases.

4. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the following results as a function of d :
 - (a) The average ten-fold cross-validation error, and the test error for the hypotheses obtained by running SVMs on the whole training set.
 - (b) The average number of support vectors, and the average number of support vectors lie on the margin hyperplanes.
5. SVMs are “sparse” in the sense that the number of support vectors is usually small compared to total number of observations. Suppose we explicitly maximize sparsity by penalizing the L_2 norm of the vector α that defines the weight vector w :

$$\begin{aligned}
 \min_{\alpha, b, \xi} \quad & \frac{1}{2} \|\alpha\|^2 + C \left(\sum_{i=1}^m \xi_i \right) \\
 \text{subject to} \quad & y_i \left(\left(\sum_{j=1}^m \alpha_j y_j x_j \right) \cdot x_i + b \right) \geq 1 - \xi_i, \\
 & \xi_i \geq 0, \alpha_i \geq 0, i \in [1, m].
 \end{aligned}$$

Show that the problem coincides with an instance of the primal optimization problem of SVMs, modulo the non-negativity constraint on α . You should indicate exactly how to view it as such.

Mehryar Mohri
Foundations of Machine Learning 2018
Courant Institute of Mathematical Sciences
Homework assignment 2
October 07, 2018
Due: October 21, 2018

A. Radmacher complexity

1. Consider the class of functions $\mathcal{H}$ mapping from $\mathbb{R}$ to $\{+1, -1\}$ such that

$$h(x) = \begin{cases} +1 & \text{for } x \in [a, b], \\ -1 & \text{otherwise,} \end{cases}$$

for some $a, b \in \mathbb{R}$. Give an upper bound on the growth function $\Pi_{\mathcal{H}}(m)$ and use it to derive an upper bound on the $\mathfrak{R}_m(\mathcal{H})$.

Solution:

$$\Pi_H(m) = \binom{n+1}{2} + 1 = \frac{1}{2}n^2 + \frac{1}{2}n + 1$$

2. Prove that for any hypotheses class $\mathcal{H}$ and any function $h: \mathcal{X} \mapsto \mathbb{R}$, $\mathfrak{R}_m(\mathcal{H}) = \mathfrak{R}_m(\mathcal{H} + h)$.

Solution: Expand the definition of empirical Radmacher complexity.

3. Prove that if for two hypotheses classes $\mathcal{H}$ and $\mathcal{F}$ the inclusion $\mathcal{H} \subseteq \mathcal{F}$ holds, then the following inequality holds for any finite sample S : $\widehat{\mathfrak{R}}_S(\mathcal{H}) \leq \widehat{\mathfrak{R}}_S(\mathcal{F})$.

Solution: Definition of Radmacher complexity and supremum over $\mathcal{H}$ is at least as supremum over $\mathcal{F}$.

4. Let $\mathcal{H}_1$ be a family of functions mapping $\mathcal{X}$ to $\{0, 1\}$ and let $\mathcal{H}_2$ be a family of functions mapping $\mathcal{X}$ to $\{-1, +1\}$. Let $\mathcal{H} = \{h_1 h_2: h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}$. Show that the empirical Rademacher complexity of $\mathcal{H}$ for any sample S of size m can be bounded as follows:

$$\widehat{\mathfrak{R}}_S(\mathcal{H}) \leq \widehat{\mathfrak{R}}_S(\mathcal{H}_1) + \widehat{\mathfrak{R}}_S(\mathcal{H}_2).$$

(*hint*: write h_1h_2 in a way such that you can apply Talagrand's inequality.)

Solution: Consider $\phi(x) = |x| - 1$. Then, one can verify that

$$h_1h_2 = \phi(h_1 + h_2).$$

As ϕ is an 1-Lipschitz function, by Talagrand's Contraction Lemma,

$$\hat{\mathfrak{R}}_S(\mathcal{H}) \leq \hat{\mathfrak{R}}_S(\mathcal{H}_1 + \mathcal{H}_2) \leq \hat{\mathfrak{R}}_S(\mathcal{H}_1) + \hat{\mathfrak{R}}_S(\mathcal{H}_2).$$

B. VC-dimension

1. What is the VC-dimension of axis-aligned squares in $\mathbb{R}^2$?

Solution: 3. First we prove there exists a 3-point set such that it can be fully shattered by axis-aligned squares. For example, suppose 3 points are vertices of an isosceles right triangle. It is easy to see that they can be fully shattered. We also need to prove no 4-points set could be fully shattered by axis-aligned squares. It is easy to see when 3 points are collinear, they can not be fully shattered (for example + - +). Suppose no 3 points are collinear and mark the 4 points clockwise as A, B, C, D. Assume $|AC| > |BD|$ and we can not generate this outcome: A+, B-, C+, D-.

2. What is the VC-dimension of intersections of 2 axis-aligned squares in $\mathbb{R}^2$?

Solution: 4. Same as axis-aligned rectangles.

3. (Bonus) Let C be a concept class whose VC-dimension is 3. Show that the VC-dimension of intersections of k concepts from C is upper bounded by $6k \log_2(3k)$. (*hint*: use Sauer's lemma.)

Solution: First we prove the following key property

Lemma 1. For two concept sets C_1, C_2 denote

$$C = \{c_1c_2 \mid c_1 \in C_1, c_2 \in C_2\}$$

we have $\Pi_C(m) \leq \Pi_{C_1}(m)\Pi_{C_2}(m)$.

Proof. For any set $\{x_1, \dots, x_m\} \subset \mathcal{X}$, it is easy to see that

$$\begin{aligned} & |\{(c_1(x_1)c_2(x_1), \dots, c_1(x_m)c_2(x_m)) \mid c_1 \in C_1, c_2 \in C_2\}| \\ & \leq |\{(c_1(x_1), \dots, c_1(x_m)) \mid c_1 \in C_1\}| |\{(c_2(x_1), \dots, c_2(x_m)) \mid c_2 \in C_2\}| \\ & \leq \Pi_{C_1}(m) \Pi_{C_2}(m) \end{aligned}$$

Taking max on the left hand side we close the proof $\square$

Back to this problem we denote C^k as the set of intersections of k concepts from C . Then by above lemma $\Pi_{C^k}(m) \leq (\Pi_C(m))^k$ for any $m \in \mathbb{N}$. We only need to prove that $(\Pi_C(m))^k < 2^m$ for $m = 6k \log_2(3k)$. By Sauer's lemma and the fact that $VC - \dim(C) = 3$ we get $\Pi_C(m) \leq (\frac{em}{3})^3$. Thus $(\Pi_C(m))^k \leq (\frac{em}{3})^{3k}$. We substitute m by $6k \log_2(3k)$ then the inequality turns out to be $2e \log_2(3k) < 9k$, which is trivially true.

C. Support Vector Machines

1. Download and install the libsvm software library from:

<https://www.csie.ntu.edu.tw/~cjlin/libsvm>

and briefly consult the documentation to become more familiar with the tools.

2. Consider the `svmguide1` dataset

<https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary/svmguide1>

Download a shuffled version of that dataset from

<http://www.cs.nyu.edu/~mohri/ml18/svmguide1.shuffled>

Use the `libsvm` scaling tool to scale the features of all the data. Use the first 2316 examples for training, the last 773 for testing. The scaling parameters should be computed only on the training data and then applied to the test data.

3. Consider the binary classification task in `svmguide1`, using the 4 features. Use SVMs combined with polynomial kernels to tackle this binary classification problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let other parameters of polynomial kernels in `libsvm` be equal to their default values), varying C in powers of 2, starting from a small value $C = 2^{-k}$ to $C = 2^k$, for some value of k . k should be chosen so that you see a significant variation in training error, starting from a very high training error to a low training error. Expect longer training times with `libsvm` as the value of C increases.

Solution: Figure 1 shows the average cross-validation performance as a function of the regularization parameter C from 2^{-6} to 2^6 . The performance for several choices of d and C are essentially indistinguishable; one suitable choice of optimal parameters is $C^* = 2^5$ and $d^* = 4$.

4. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the following results as a function of d :
 - (a) The average ten-fold cross-validation error, and the test error for the hypotheses obtained by running SVMs on the whole training set.
 - (b) The average number of support vectors, and the average number of support vectors lie on the margin hyperplanes.

Solution: The first plot in Figure 2 shows that the test error first decreases and then increases, as a function of increasing degree d . Also, the cross-validation error is (slightly) optimistic when compared to the test error on the held-out dataset, for larger values of d . The second plot shows that the total number of marginal support vectors slightly increases with d , while the total number of overall support vectors decreases first and then increases.

5. SVMs are “sparse” in the sense that the number of support vectors is usually small compared to total number of observations. Suppose we explicitly maximize sparsity by penalizing the L_2 norm of the vector

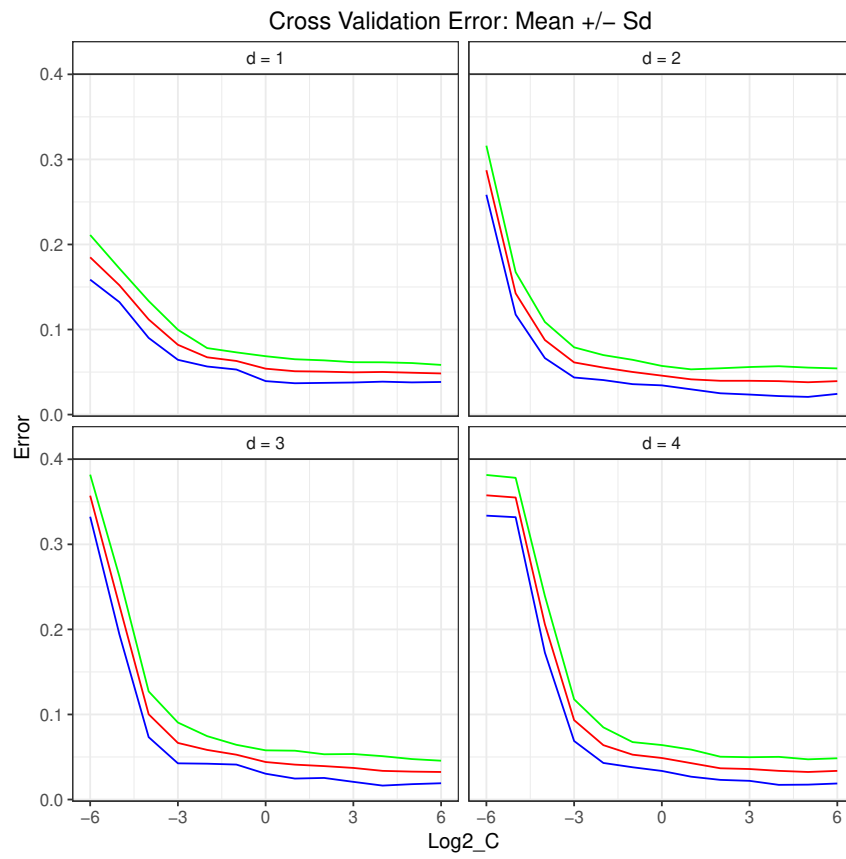


Figure 1: Average error according to 10-fold cross-validation, with error-bars indicating one standard deviation.

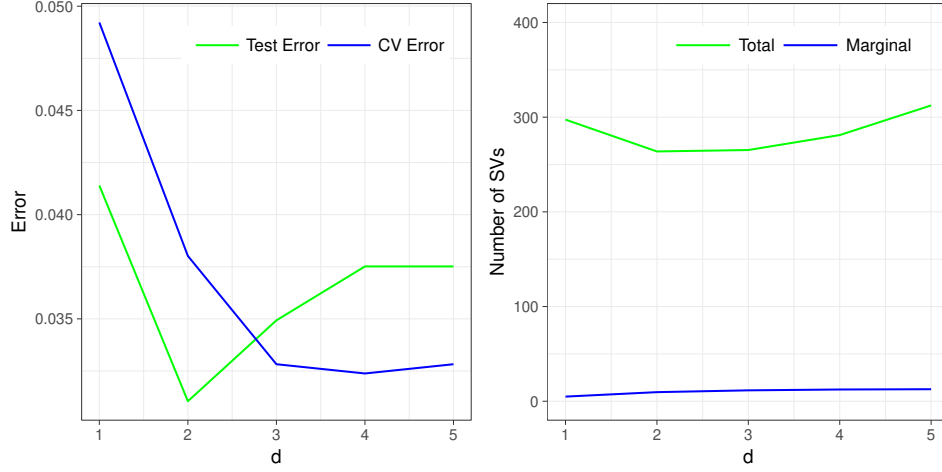


Figure 2: The test and validation error as a function of degree d (left panel) as well as the number of total and marginal support vectors (right panel).

α that defines the weight vector w :

$$\begin{aligned} \min_{\alpha, b, \xi} \quad & \frac{1}{2} \|\alpha\|^2 + C \left(\sum_{i=1}^m \xi_i \right) \\ \text{subject to} \quad & y_i \left(\left(\sum_{j=1}^m \alpha_j y_j x_j \right) \cdot x_i + b \right) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \alpha_i \geq 0, i \in [1, m]. \end{aligned}$$

Show that the problem coincides with an instance of the primal optimization problem of SVMs, modulo the non-negativity constraint on α . You should indicate exactly how to view it as such.

Solution: Let

$$x'_i = \left(y_1(x_1 \cdot x_i), \dots, y_m(x_m \cdot x_i) \right).$$

Then the optimization problem becomes

$$\begin{aligned} \min_{\alpha, b, \xi} \quad & \frac{1}{2} \|\alpha\|^2 + C \left(\sum_{i=1}^m \xi_i \right) \\ \text{subject to} \quad & y_i (\alpha \cdot x'_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \alpha_i \geq 0, i \in [1, m]. \end{aligned}$$

This is the standard formulation of the primal SVM optimization problem on samples $(x'_1, y_1), \dots, (x'_m, y_m)$, modulo the non-negativity constraints on α_i .

Another way is to show that the dual forms are equivalent. Define Lagrange variables $p_i \geq 0, q_i \geq 0, r_i \geq 0$. The Lagrangian is

$$\begin{aligned} L(\alpha, b, \xi, p, q, r) = & \frac{1}{2} \|\alpha\|^2 + C \left(\sum_{i=1}^m \xi_i \right) \\ & - \sum_{i=1}^m p_i \left\{ y_i \left[\left(\sum_{j=1}^m \alpha_j y_j x_j \right) \cdot x_i + b \right] - 1 + \xi_i \right\} \\ & - \sum_{i=1}^m q_i \xi_i - \sum_{i=1}^m r_i \alpha_i. \end{aligned}$$

Note that

$$\begin{aligned} & \sum_{i=1}^m p_i \left\{ y_i \left[\left(\sum_{j=1}^m \alpha_j y_j x_j \right) \cdot x_i + b \right] - 1 + \xi_i \right\} \\ = & \left(\sum_{i=1}^m p_i y_i x_i \right) \cdot \left(\sum_{i=1}^m \alpha_i y_i x_i \right) + \sum_{i=1}^m p_i y_i b - \sum_{i=1}^m p_i + \sum_{i=1}^m p_i \xi_i. \end{aligned}$$

To meet KKT conditions,

$$\nabla_{\alpha_i} L = \alpha_i - y_i x_i \cdot \left(\sum_{j=1}^m p_j y_j x_j \right) - r_i = 0 \Rightarrow \alpha_i = y_i x_i \cdot \left(\sum_{j=1}^m p_j y_j x_j \right) + r_i$$

$$\nabla_b L = - \sum_{i=1}^m p_i y_i = 0 \Rightarrow \sum_{i=1}^m p_i y_i = 0$$

$$\nabla_{\xi_i} L = C - p_i - q_i = 0 \Rightarrow p_i + q_i = C$$

And

$$\forall i \in [m], \quad \sum_{i=1}^m p_i \left\{ y_i \left[\left(\sum_{j=1}^m \alpha_j y_j x_j \right) \cdot x_i + b \right] - 1 + \xi_i \right\} = 0,$$

$$\forall i \in [m], \quad q_i \xi_i = 0, r_i \alpha_i = 0.$$

Plugging in the expression of α in L gives

$$\begin{aligned}
& L(\alpha, b, \xi, p, q, r) \\
&= \frac{1}{2} \|\alpha\|^2 + C \left(\sum_{i=1}^m \xi_i \right) - \sum_{i=1}^m \alpha_i (\alpha_i - r_i) \\
&\quad - \sum_{i=1}^m p_i y_i b + \sum_{i=1}^m p_i - \sum_{i=1}^m (p_i + q_i) \xi_i - \sum_{i=1}^m r_i \alpha_i \\
&= \frac{1}{2} \|\alpha\|^2 - \sum_{i=1}^m \alpha_i^2 + \sum_{i=1}^m p_i \\
&= -\frac{1}{2} \|\alpha\|^2 + \sum_{i=1}^m p_i
\end{aligned}$$

Note that

$$r_i \alpha_i = 0 \Rightarrow y_i x_i r_i \cdot \left(\sum_{j=1}^m p_j y_j x_j \right) + r_i^2 = 0.$$

Thus,

$$\begin{aligned}
& \|\alpha\|^2 \\
&= \sum_{i=1}^m \left[y_i x_i \cdot \left(\sum_{j=1}^m p_j y_j x_j \right) \right]^2 - \sum_{i=1}^m r_i^2 \\
&= \sum_{i=1}^m \left(\sum_{j,k=1}^m p_j y_j p_k y_k (x_i y_i \cdot x_j)(x_i y_i \cdot x_k) \right) - \sum_{i=1}^m r_i^2 \\
&= \sum_{j,k=1}^m p_j y_j p_k y_k \left(\sum_{i=1}^m (x_i y_i \cdot x_j)(x_i y_i \cdot x_k) \right) - \sum_{i=1}^m r_i^2 \\
&= \sum_{j,k=1}^m p_j p_k y_j y_k K(x_j, x_k) - \sum_{i=1}^m r_i^2,
\end{aligned}$$

where $K(x_j, x_k) = \sum_{i=1}^m (x_i y_i \cdot x_j)(x_i y_i \cdot x_k)$. Putting everything to-

gether, the dual optimization problem is

$$\begin{aligned} \max_{p,r} \quad & \sum_{i=1}^m p_i - \frac{1}{2} \sum_{i,j=1}^m p_i p_j y_i y_j K(x_i, x_j) + \frac{1}{2} \sum_{i=1}^m r_i^2 \\ \text{subject to} \quad & 0 \leq p_i \leq C \wedge r_i \geq 0 \wedge \sum_{i=1}^m p_i y_i = 0, i \in [m]. \end{aligned}$$

One can show that this dual form coincides with the dual form of the standard SVM primal problem with non-negative constraints on w , using kernel $K(\cdot, \cdot)$ instead of inner product. Note that if we let $x'_i = \left(y_1(x_1 \cdot x_i), \dots, y_m(x_m \cdot x_i) \right)$ as defined earlier, then $K(x_j, x_k) = (x'_j \cdot x'_k)$.

Mehryar Mohri
 Foundations of Machine Learning 2020
 Courant Institute of Mathematical Sciences
 Homework assignment 2
 Oct 08, 2020
 Due: Oct 28, 2020 [before class starts].

A. Rademacher complexity

1. Non-negativity of empirical Rademacher complexity: Show that for any hypothesis set $\mathcal{H}$ and sample S , we have $\hat{\mathfrak{R}}_S(\mathcal{H}) \geq 0$.
2. Empirical Rademacher complexity of products: let $\mathcal{H}_1$ and $\mathcal{H}_2$ be two hypothesis sets of functions mapping from the input space $\mathcal{X}$ to $\{0, 1\}$. Let $\mathcal{H} = \{h_1 h_2 : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}$. Show that the following inequality holds:

$$\hat{\mathfrak{R}}_S(\mathcal{H}) \leq \hat{\mathfrak{R}}_S(\mathcal{H}_1) + \hat{\mathfrak{R}}_S(\mathcal{H}_2).$$

Hint: you could use Talagrand's contraction lemma.

B. VC-dimension of neural networks

Let C be a concept class over $\mathbb{R}^r$ with VC-dimension d . A C -neural network with one intermediate layer is a concept defined over $\mathbb{R}^n$ that can be represented by a directed acyclic graph such as that of Figure 1, in which the input nodes are those at the bottom and in which each other node is labeled with a concept $c \in C$.

The output of the neural network for a given input vector $(x_1, \dots, x_n)$ is obtained as follows. First, each of the n input nodes is labeled with the corresponding value $x_i \in \mathbb{R}$. Next, the value at a node u in the higher layer and labeled with c is obtained by applying c to the values of the input nodes admitting an edge ending in u . Note that since c takes values in $\{0, 1\}$, the value at u is in $\{0, 1\}$. The value at the top or output node is obtained similarly by applying the corresponding concept to the values of the nodes admitting an edge to the output node.

1. Let H denote the set of all neural networks defined as above with $k \geq 2$ internal nodes. Show that the growth function $\Pi_H(m)$ can be upper bounded in terms of the product of the growth functions of the hypothesis sets defined at each intermediate layer.

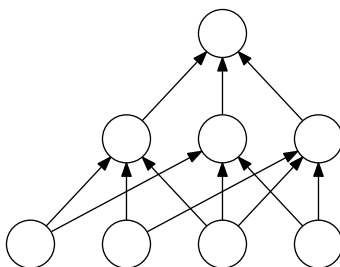


Figure 1: A neural network with one intermediate layer.

2. Use that to upper bound the VC-dimension of the C -neural networks (*Hint*: you can use the implication $m = 2x \log_2(xy) \Rightarrow m > x \log_2(ym)$ valid for $m \geq 1$, and $x, y > 0$ with $xy > 4$).
3. Let C be the family of concept classes defined by threshold functions $C = \{\text{sgn}(\sum_{j=1}^r w_j x_j) : \mathbf{w} \in \mathbb{R}^r\}$. Give an upper bound on the VC-dimension of H in terms of k and r .

C. Support Vector Machines (SVMs)

1. Download and install the `libsvm` software library from:

`http://www.csie.ntu.edu.tw/~cjlin/libsvm/`

2. Download the `abalone` data set:

`http://archive.ics.uci.edu/ml/datasets/Abalone`

Use the `libsvm` scaling tool to scale the features of all the data. Use the first 3133 examples for training, the last 1044 for testing. The scaling parameters should be computed only on the training data and then applied to the test data.

3. Download the `abalone` data set:

`http://archive.ics.uci.edu/ml/datasets/Abalone`

Use the `libsvm` scaling tool to scale the features of all the data. Use the first 3133 examples for training, the last 1044 for testing. The scaling parameters should be computed only on the training data and then applied to the test data.

4. Consider the binary classification that consists of distinguishing classes 1 through 9 from the rest. Use SVMs combined with polynomial kernels to tackle this binary classification problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let other parameters of polynomial kernels in `libsvm` be equal to their default values), varying C in powers of 2, starting from a small value $C = 2^{-k}$ to $C = 2^k$, for some value of k . k should be chosen so that you see a significant variation in training error, starting from a very high training error to a low training error. Expect longer training times with `libsvm` as the value of C increases.

5. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation error and the test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d . How many of the support vectors lie on the margin hyperplanes?
6. In class, we gave two types of argument in favor of the SVMs algorithm: one based on the sparsity of the support vectors, another based on the notion of margin. Suppose that instead of maximizing the margin, we choose instead to maximize sparsity by minimizing the norm p of the vector α that defines the weight vector $\mathbf{w}$, for some $p \geq 1$. For simplicity, fix $p = 2$. This gives the following optimization problem for a kernel function K :

$$\begin{aligned} \min_{\alpha, b} \quad & \frac{1}{2} \sum_{i=1}^m \alpha_i^2 + C \sum_{i=1}^m \xi_i \\ \text{subject to} \quad & y_i \left(\sum_{j=1}^m \alpha_j y_j K(x_i, x_j) + b \right) \geq 1 - \xi_i, i \in [1, m] \\ & \xi_i, \alpha_i \geq 0, i \in [1, m]. \end{aligned} \tag{1}$$

- (a) Show that modulo the non-negativity constraint on α , the problem coincides with an instance of the primal optimization problem of SVMs (indicate exactly how to view it as such).
- (b) Is the positive-definiteness of the kernel function K needed to ensure that this is a convex optimization problem? Justify your response.

- (c) Derive the dual optimization of problem of (1).
- (d) Suppose we omit the non-negativity constraint on α . Use `libsvm` to solve the problem. Plot the ten-fold cross-validation training and test errors for the hypotheses obtained based on the solution α as a function of d , for the best value of C measured on the validation set.

Mehryar Mohri
 Foundations of Machine Learning 2020
 Courant Institute of Mathematical Sciences
 Homework assignment 2
 Oct 08, 2020
 Due: Oct 28, 2020 [before class starts].

A. Rademacher complexity

1. Non-negativity of empirical Rademacher complexity: Show that for any hypothesis set $\mathcal{H}$ and sample S , we have $\widehat{\mathfrak{R}}_S(\mathcal{H}) \geq 0$.

Solution: By the sub-additivity of supremum, we can write:

$$\begin{aligned}\widehat{\mathfrak{R}}_S(\mathcal{H}) &= \frac{1}{m} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sup_{h \in \mathcal{H}} \sum_{i=1}^m \sigma_i h(x_i) \right] \\ &\geq \frac{1}{m} \sup_{h \in \mathcal{H}} \mathbb{E}_{\boldsymbol{\sigma}} \left[\sum_{i=1}^m \sigma_i h(x_i) \right] \\ &= \frac{1}{m} \sup_{h \in \mathcal{H}} \sum_{i=1}^m \mathbb{E}_{\boldsymbol{\sigma}} [\sigma_i] h(x_i) = 0.\end{aligned}$$

□

2. Empirical Rademacher complexity of products: let $\mathcal{H}_1$ and $\mathcal{H}_2$ be two hypothesis sets of functions mapping from the input space $\mathcal{X}$ to $\{0, 1\}$. Let $\mathcal{H} = \{h_1 h_2 : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}$. Show that the following inequality holds:

$$\widehat{\mathfrak{R}}_S(\mathcal{H}) \leq \widehat{\mathfrak{R}}_S(\mathcal{H}_1) + \widehat{\mathfrak{R}}_S(\mathcal{H}_2).$$

Hint: you could use Talagrand's contraction lemma.

Solution: Observe that for any $h_1 \in \mathcal{H}_1$ and $h_2 \in \mathcal{H}_2$, we can write $h_1 h_2 = (h_1 + h_2 - 1)1_{h_1 + h_2 - 1 \geq 0} = (h_1 + h_2 - 1)_+$. Since $x \mapsto (x - 1)_+$ is 1-Lipschitz over $[0, 2]$, by Talagrand's lemma, the following holds: $\widehat{\mathfrak{R}}_S(\mathcal{H}) \leq \widehat{\mathfrak{R}}_S(\mathcal{H}_1 + \mathcal{H}_2) \leq \widehat{\mathfrak{R}}_S(\mathcal{H}_1) + \widehat{\mathfrak{R}}_S(\mathcal{H}_2)$. □

B. VC-dimension of neural networks

Let C be a concept class over $\mathbb{R}^r$ with VC-dimension d . A C -neural network with one intermediate layer is a concept defined over $\mathbb{R}^n$ that can be

represented by a directed acyclic graph such as that of Figure 1, in which the input nodes are those at the bottom and in which each other node is labeled with a concept $c \in C$.

The output of the neural network for a given input vector $(x_1, \dots, x_n)$ is obtained as follows. First, each of the n input nodes is labeled with the corresponding value $x_i \in \mathbb{R}$. Next, the value at a node u in the higher layer and labeled with c is obtained by applying c to the values of the input nodes admitting an edge ending in u . Note that since c takes values in $\{0, 1\}$, the value at u is in $\{0, 1\}$. The value at the top or output node is obtained similarly by applying the corresponding concept to the values of the nodes admitting an edge to the output node.

1. Let H denote the set of all neural networks defined as above with $k \geq 2$ internal nodes. Show that the growth function $\Pi_H(m)$ can be upper bounded in terms of the product of the growth functions of the hypothesis sets defined at each intermediate layer.

Solution: Let $\Pi_u(m)$ denote the growth function at a node u in the intermediate layer. For a fixed set of values at the intermediate layer, using the concept class C the output node can generate at most $\Pi_C(m)$ distinct labelings. There are $\prod_u \Pi_u(m)$ possible sets of values at the intermediate layer since, by definition, for a sample of size m , at most $\Pi_u(m)$ distinct values are possible at each u . Thus, at most $\Pi_C(m) \times \prod_u \Pi_u(m)$ labelings can be generated by the neural network and $\Pi_H(m) \leq \Pi_C(m) \prod_u \Pi_u(m)$. $\square$

2. Use that to upper bound the VC-dimension of the C -neural networks (*Hint:* you can use the implication $m = 2x \log_2(xy) \Rightarrow m > x \log_2(yx)$ valid for $m \geq 1$, and $x, y > 0$ with $xy > 4$).

Solution: For any intermediate node u , $\Pi_u(m) = \Pi_C(m)$. Thus, for $\tilde{k} = k + 1$, $\Pi_H(m) \leq \Pi_C(m)^{\tilde{k}}$. By Sauer's lemma, $\Pi_C(m) \leq \left(\frac{em}{d}\right)^d$, thus $\Pi_H(m) \leq \left(\frac{em}{d}\right)^{d\tilde{k}}$. Let $m = 2\tilde{k}d \log_2(e\tilde{k})$. In view of the inequality given by the hint and $e\tilde{k} > 4$, this implies $m > d\tilde{k} \log_2\left(\frac{em}{d}\right)$, that is $2^m > \left(\frac{em}{d}\right)^{d\tilde{k}}$. Thus, the VC-dimension of H is less than

$$2\tilde{k}d \log_2(e\tilde{k}) = 2(k+1)d \log_2(e(k+1)).$$

$\square$

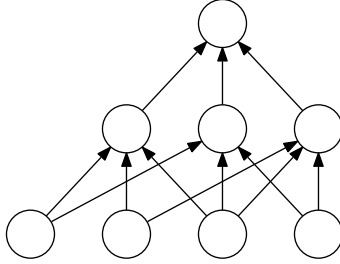


Figure 1: A neural network with one intermediate layer.

- Let C be the family of concept classes defined by threshold functions $C = \{\text{sgn}(\sum_{j=1}^r w_j x_j) : \mathbf{w} \in \mathbb{R}^r\}$. Give an upper bound on the VC-dimension of H in terms of k and r .

Solution: For threshold functions, the VC-dimension of C is r , thus, the VC-dimension of H is upper bounded by

$$2(k+1)r \log_2(e(k+1)).$$

□

C. Support Vector Machines (SVMs)

- Download and install the `libsvm` software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

- Download the `abalone` data set:

<http://archive.ics.uci.edu/ml/datasets/Abalone>

Use the `libsvm` scaling tool to scale the features of all the data. Use the first 3133 examples for training, the last 1044 for testing. The scaling parameters should be computed only on the training data and then applied to the test data.

Solution:

```
gunzip libsvm-3.0.tar.gz
tar xvf libsvm-3.0.tar
cd libsvm-3.0
make
```

□

3. Download the abalone data set:

<http://archive.ics.uci.edu/ml/datasets/Abalone>

Use the `libsvm` scaling tool to scale the features of all the data. Use the first 3133 examples for training, the last 1044 for testing. The scaling parameters should be computed only on the training data and then applied to the test data.

Solution:

```
# Rewrite the dataset in a libsvm compatible format,
# skipping the first feature which is the Abalone's age,
# and turning the Abalone's sex in 3 binary features.
awk -F, '
BEGIN {
    sex["M"] = "1:1 2:0 3:0";
    sex["F"] = "1:0 2:1 3:0";
    sex["I"] = "1:0 2:0 3:1";
}
{
    class = ($NF <= 9) ? -1 : 1;
    printf("%d %s", class, sex[$1]);
    for (i = 2; i <= NF - 1; ++i)
        printf(" %d:%s", i + 2, $i);
    printf("\n");
}' data/abalone.data.txt > output/dataset.txt

# Split in training and test set.
awk 'NR <= 3133 { print; }' data/dataset.txt \
    > output/train.txt
awk 'NR > 3133 { print; }' data/dataset.txt \
    > output/test.txt

# Scale training and test set.
libsvm-3.0/svm-scale -s output/scale.txt \
    output/train.txt > output/train.scaled.txt
libsvm-3.0/svm-scale -r output/scale.txt \
    output/test.txt > output/test.scaled.txt
```

□

4. Consider the binary classification that consists of distinguishing classes 1 through 9 from the rest. Use SVMs combined with polynomial kernels to tackle this binary classification problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let other parameters of polynomial kernels in `libsvm` be equal to their default values), varying C in powers of 2, starting from a small value $C = 2^{-k}$ to $C = 2^k$, for some value of k . k should be chosen so that you see a significant variation in training error, starting from a very high training error to a low training error. Expect longer training times with `libsvm` as the value of C increases.

Solution:

```
# Split the training data into ten equal-sized disjoint sets:
sort -R output/train.scaled.txt \
  > output/train.scaled.shuffled.txt

NUM_SAMPLES=$(cat output/train.scaled.shuffled.txt \
  | wc --lines)
for SPLIT in $(seq 1 10); do
  awk "NR < $NUM_SAMPLES * ($SPLIT - 1) / 10.0 \
    || NR >= $NUM_SAMPLES * $SPLIT / 10.0" \
    output/train.scaled.shuffled.txt \
    > output/train.$SPLIT.txt

  awk "NR >= $NUM_SAMPLES * ($SPLIT - 1) / 10.0 \
    && NR < $NUM_SAMPLES * $SPLIT / 10.0" \
    output/train.scaled.shuffled.txt \
    > output/dev.$SPLIT.txt
done

# Compute accuracy for different values of $C$ and
# for degrees 1 through 4 of the polynomial kernel.
for LOG2C in $(seq -10 10); do
  for DEGREE in 1 2 3 4; do
    for SPLIT in $(seq 1 10); do
      # Train models.
      C=$(python -c "print 2 ** $LOG2C")
      echo "c=$C "d=$DEGREE "split=$SPLIT

      libsvm-3.0/svm-train -t 1 -d $DEGREE -c $C \
        output/train.$SPLIT.txt \
        output/model.$LOG2C.$DEGREE.$SPLIT.txt \
        > output/train.$LOG2C.$DEGREE.$SPLIT.log.txt

      libsvm-3.0/svm-predict output/dev.1.txt \
        output/model.$LOG2C.$DEGREE.$SPLIT.txt \
        output/dev.$LOG2C.$DEGREE.$SPLIT.prediction.txt \
```

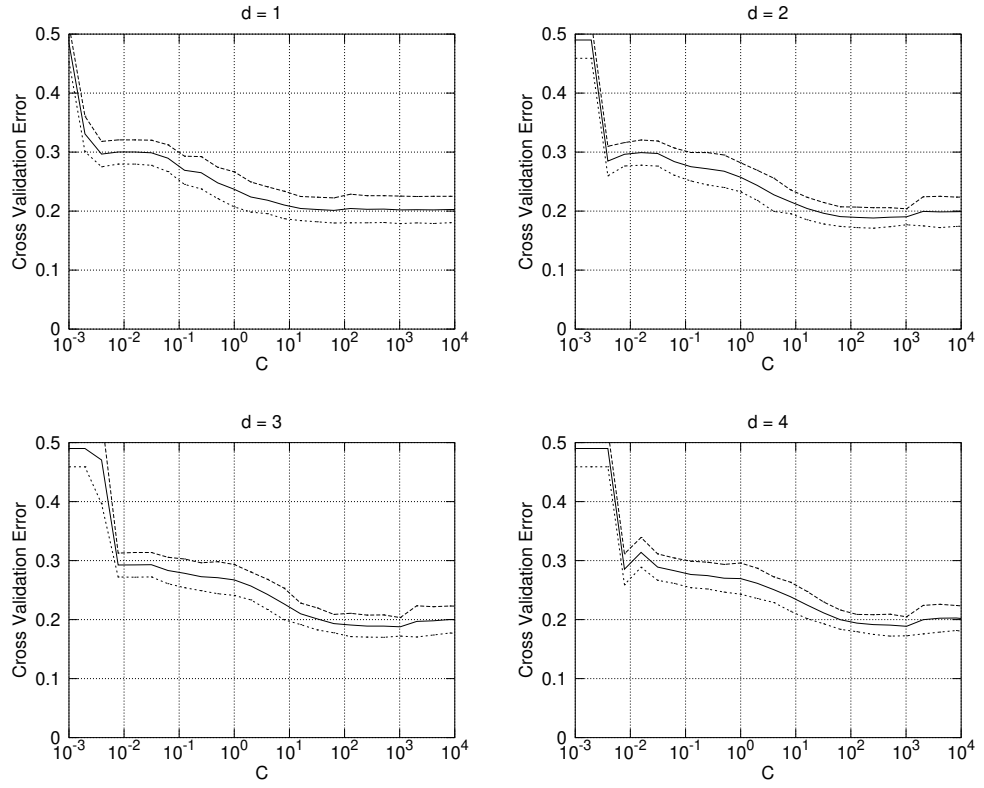


Figure 2: Mean cross validation error ± 1 standard deviation for different degrees d of the polynomial kernel, as a function of C .

```

        > output/dev.$LOG2C.$DEGREE.$SPLIT.log.txt
    done
done
done

# Compute mean and standard deviation
# of classification accuracy.
echo -n > output/dev.results.txt
for F in output/dev.*.log.txt; do
    echo $F $(cat $F) | \
        sed 's;.*\\.\\(.*\\)\\.\\(.*\\)\\.\\(.*\\)\\.log.* = \\(.*\\)%.*;\\1 \\2 \\3 \\4;' \
        | grep -v 'classification' \
        >> output/dev.results.txt;
done

awk '{
    acc = $4 / 100;
    accuracy_mean[$1" "$2] += acc / 10;
    accuracy_var[$1" "$2] += acc ^ 2 / (10 - 1);
}
END {
    for (cond in accuracy_mean) {
        mean = accuracy_mean[cond];
        std = sqrt(accuracy_var[cond] - mean ^ 2 * 10 / (10 - 1));
        print cond, mean, std;
    }
}' output/dev.results.txt \
    | sort -n -k 3 > output/dev.results.summary.txt

```

Plots of the results are shown in Figure 2. The best C and d from cross-validation are $(C^*, d^*) = (1024, 3)$. $\square$

5. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation error and the test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d . How many of the support vectors lie on the margin hyperplanes?

Solution:

Plots for this question are shown in Figure 3. The number of support vector on margin hyperplanes are obtained as $nSV - nBSV$ (number of support vectors - number of bounded support vectors). They are respectively 9, 26, 43, 48 for polynomial kernel degrees 1 through 4. $\square$

6. In class, we gave two types of argument in favor of the SVMs algorithm: one based on the sparsity of the support vectors, another based on the

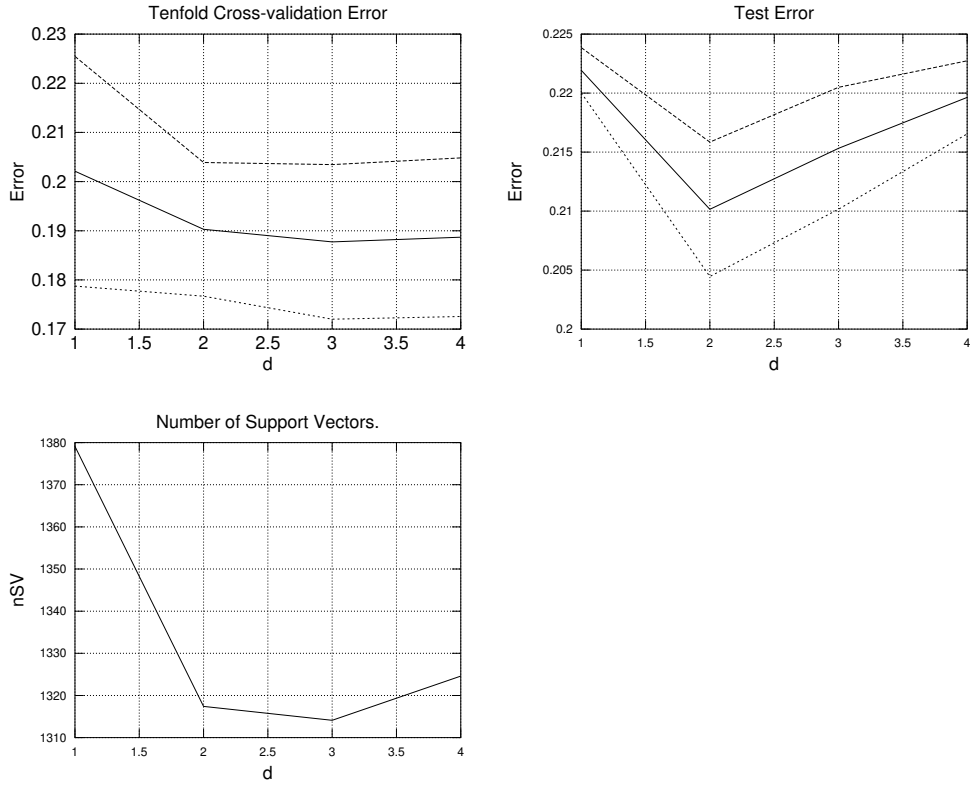


Figure 3: Tenfold cross-validation and test error, and average number of support vectors as a function of the degree d of the polynomial kernel.

notion of margin. Suppose that instead of maximizing the margin, we choose instead to maximize sparsity by minimizing the norm p of the vector α that defines the weight vector $\mathbf{w}$, for some $p \geq 1$. For simplicity, fix $p = 2$. This gives the following optimization problem for a kernel function K :

$$\begin{aligned} \min_{\alpha, b} \quad & \frac{1}{2} \sum_{i=1}^m \alpha_i^2 + C \sum_{i=1}^m \xi_i \\ \text{subject to} \quad & y_i \left(\sum_{j=1}^m \alpha_j y_j K(x_i, x_j) + b \right) \geq 1 - \xi_i, i \in [1, m] \\ & \xi_i, \alpha_i \geq 0, i \in [1, m]. \end{aligned} \tag{1}$$

- (a) Show that modulo the non-negativity constraint on α , the problem coincides with an instance of the primal optimization problem of SVMs (indicate exactly how to view it as such).

Solution: Let

$$x'_i = (y_1 K(x_i, x_1), \dots, y_m K(x_i, x_m)).$$

Then the optimization problem becomes

$$\begin{aligned} \min_{\alpha, b, \xi} \quad & \frac{1}{2} \|\alpha\|^2 + C \sum_{i=1}^m \xi_i \\ \text{subject to} \quad & y_i (\alpha \cdot x'_i + b) \geq 1 - \xi_i \\ & \xi_i, \alpha_i \geq 0, i \in [1, m] \end{aligned}$$

which is the standard formulation of the primal SVM optimization problem on samples x'_i , modulo the non-negativity constraints on α_i . $\square$

- (b) Is the positive-definiteness of the kernel function K needed to ensure that this is a convex optimization problem? Justify your response.

Solution: Modulo the non-negativity constraints on α_i , the optimization problem in (a) is known to be convex for any set of samples x'_i . Restricting the domain of the problem with additional convex constraints $\alpha_i \geq 0$ preserves convexity, so the problem is convex regardless of the positive-definiteness of K . $\square$

- (c) Derive the dual optimization of problem of (1).

Solution: The Lagrangian of (1) for all $\alpha_i \geq 0, \xi_i \geq 0, b, \alpha'_i \geq 0, \beta_i \geq 0, \gamma_i \geq 0, i \in [1, m]$ is

$$L = \frac{1}{2} \|\alpha\|^2 + C \sum_{i=1}^m \xi_i - \sum_{i=1}^m \alpha'_i (y_i (\alpha^T x'_i + b) - 1 + \xi_i) - \sum_{i=1}^m \beta_i \xi_i - \sum_{i=1}^m \gamma_i \alpha_i$$

and the KKT conditions are

$$\nabla_{\alpha} L = 0 \Leftrightarrow \alpha = \sum_{i=1}^m \alpha'_i y_i x'_i + \gamma$$

$$\nabla_b L = 0 \Leftrightarrow \sum_{i=1}^m \alpha'_i y_i = 0$$

$$\nabla_{\xi_i} L = 0 \Leftrightarrow \alpha'_i + \beta_i = C$$

and

$$\begin{aligned} \alpha'_i (y_i (\alpha^T x'_i + b) - 1 - \xi_i) &= 0 \\ \beta_i \xi_i &= 0 \\ \gamma_i \alpha_i &= 0. \end{aligned}$$

Using the KKT conditions on L we get

$$\begin{aligned} L &= \frac{1}{2} \left(\sum_{i=1}^m \alpha'_i y_i x'_i + \gamma \right)^T \left(\sum_{j=1}^m \alpha'_j y_j x'_j + \gamma \right) + C \sum_{i=1}^m \xi_i \\ &\quad - \sum_{i=1}^m \alpha'_i \left(y_i \left(\left(\sum_{j=1}^m \alpha'_j y_j x'_j + \gamma \right)^T x'_i + b \right) - 1 + \xi_i \right) \\ &\quad - \sum_{i=1}^m \cancel{\beta_i \xi_i} - \sum_{i=1}^m \cancel{\gamma_i \alpha_i} \\ &= -\frac{1}{2} \sum_{i=1}^m \alpha'_i y_i x_i'^T \left(\sum_{j=1}^m \alpha'_j y_j x'_j + \gamma \right) + \cancel{\frac{1}{2} \gamma^T \alpha} \\ &\quad + \sum_{i=1}^m C \xi_i - \alpha'_i (y_i b - 1 + \xi_i) \\ &= \sum_{i=1}^m \alpha'_i - \frac{1}{2} \sum_{i,j=1}^m \alpha'_i \alpha'_j y_i y_j x_i'^T (x'_j + \gamma) \\ &\quad + \sum_{i=1}^m \cancel{(C - \alpha'_i) \xi_i} - \sum_{i=1}^m \cancel{\alpha'_i y_i b}. \end{aligned}$$

Thus the dual optimization problem is

$$\begin{aligned} \max_{\alpha', \gamma} \quad & \sum_{i=1}^m \alpha'_i - \frac{1}{2} \sum_{i,j=1}^m \alpha'_i \alpha'_j y_i y_j x_i'^T (x_j' + \gamma) \\ \text{subject to} \quad & \sum_{i=1}^m \alpha'_i y_i = 0 \\ & 0 \leq \alpha'_i \leq C, \gamma_i \geq 0, i \in [1, m] \end{aligned}$$

□

- (d) Suppose we omit the non-negativity constraint on α . Use `libsvm` to solve the problem. Plot the ten-fold cross-validation training and test errors for the hypotheses obtained based on the solution α as a function of d , for the best value of C measured on the validation set.

Solution: New datasets of samples x_i' for different degrees of the polynomial kernel K can be computed e.g. with the following Matlab code.

```
load dataset.txt

train_set = dataset(1:3133, :);
test_set = dataset(3134:end, :);
train_labels = train_set(:, 1);
train_x = train_set(:, 2:end);
test_labels = test_set(:, 1);
test_x = test_set(:, 2:end);

dim = size(train_x, 2);
for degree = 1:4
    degree
    train_x_prime = ...
        (train_x * (repmat(train_labels, ...
                            [1 dim]) .* train_x)') ...
        .^ degree + 1;
    train_set_prime = [ train_labels, ...
                       train_x_prime ];
    save(['train.' num2str(degree) '.txt'], ...
        'train_set_prime');
    test_x_prime = ...
        (test_x * (repmat(train_labels, ...
                            [1 dim]) .* train_x)') ...
        .^ degree + 1;
    test_set_prime = [ test_labels, ...
                      test_x_prime ];
    save(['test.' num2str(degree) '.txt'], ...
```

```

                                'test_set_prime');
end
\end{lstlisting}

```

Training, cross-validation and testing can then be conducted exactly as in question 3. of this problem. The best value of C from cross-validation is $C^* = 1024$. Resulting error rates are shown in Figure 4.

□

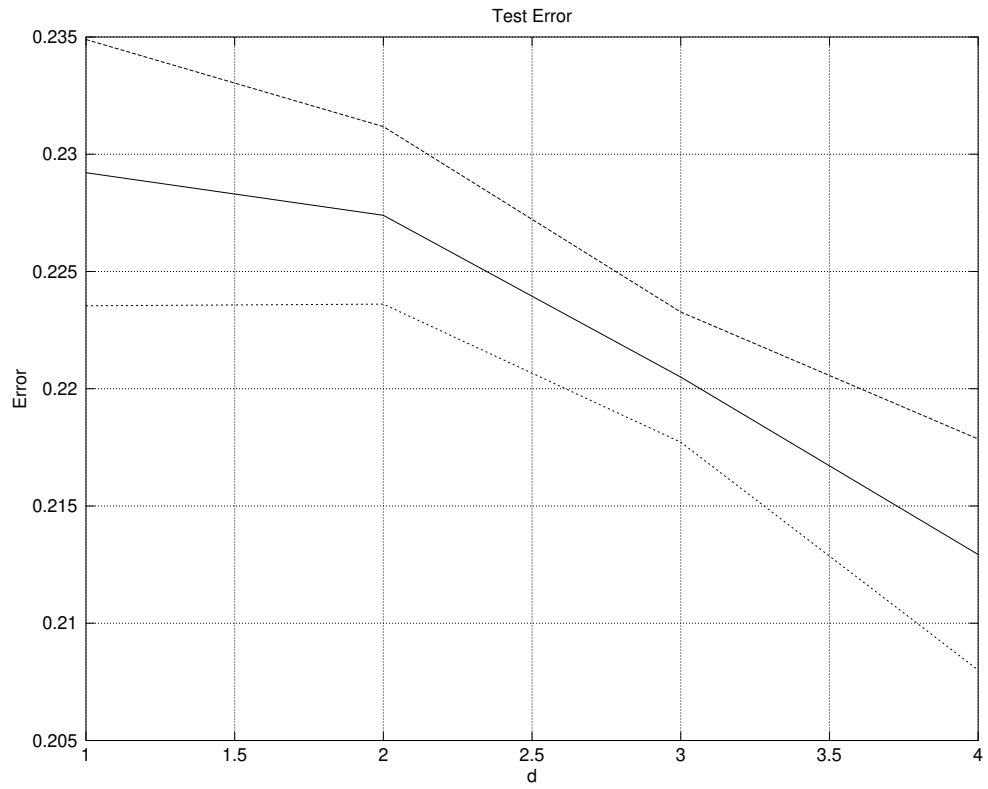


Figure 4: Test error $\pm$ one standard deviation for $C^* = 1024$ as a function of the degree d of the polynomial kernel, where training is done by solving the optimization problem described in question 5.

Foundations of Machine Learning
Department of Computer Science, NYU
Homework assignment 3
Due: May 3, 2005

1. Problem 1: Metrics and Kernels

Let X be a non-empty set and $L : X \times X \rightarrow \mathbb{R}$ be a negative definite symmetric kernel such that $L(x, x) = 0$ for all $x \in X$.

- (a) Show that there exists a Hilbert space H and a mapping $x \mapsto K_x$ from X to H such that:

$$L(x, y) = \|K_x - K_y\|^2.$$

Assume that $L(x, y) = 0 \Rightarrow x = y$. Show that $\sqrt{L}$ defines a metric on X .

- (b) Use this result to prove that the kernel $K(x, y) = \exp(-|x - y|^p)$, $x, y \in \mathbb{R}$, is not positive definite for $p > 2$.
- (c) The kernel $K(x, y) = \tanh(a(x \cdot y) + b)$ was shown to be equivalent to a two-layer neural network when combined with support vector machines. Show that K is not positive definite if $a < 0$ or $b < 0$. What can you conclude about the corresponding neural network when $a < 0$ or $b < 0$?

2. Problem 2: Boosting

This problem studies boosting-type algorithms defined with objective functions different from that of AdaBoost. We assume that the training data is given as m labeled examples $(x_1, y_1), \dots, (x_m, y_m) \in X \times \{-1, +1\}$. Let Φ be a strictly increasing convex and differentiable function over $\mathbb{R}$ such that: $\forall x \geq 0, \Phi(x) \geq 1$ and $\forall x < 0, \Phi(x) > 0$.

- (a) Consider the loss function $L(\alpha) = \sum_{i=1}^m \Phi(-y_i f(x_i))$ where f is a linear combination of base classifiers: $f = \sum_{t=1}^T \alpha_t h_t$ as with AdaBoost. The goal is to derive a new boosting algorithm using the objective function L . Characterize the best base classifier h_u to select at each round of boosting if we use coordinate descent.
- (b) Plot the following functions (1) misclassification loss $\Phi_1(-u) = 1_{u \leq 0}$; (2) least squared loss $\Phi_2(-u) = (1 - u)^2$; (3) SVM loss $\Phi_3(-u) = \max\{0, 1 - u\}$; and (4) logistic loss $\Phi_4(-u) = \log(1 + e^{-u})$. Do they satisfy all the hypotheses of the problem?

- (c) For each loss function verifying the hypotheses, derive the corresponding boosting algorithm, including the pseudocode. How does the algorithm differ from AdaBoost?

(1) [50 points] Problem 1: Metrics and Kernels

- (a) [25 points] By the theorem of lecture 5 (slide 36), if K is an NDS kernel, the kernel K' defined for any $x_0 \in X$ by:

$$K'(x, y) = \frac{1}{2}[L(x, x_0) + L(y, x_0) - L(x, y)]$$

is a PDS kernel ($K(x_0, x_0) = 0$). Let H be the reproducing hilbert space associated to K' . There exists a mapping $x \mapsto K_x$ from X to H such that $\forall x, y \in X, K'(x, y) = K_x(y)$. Then,

$$\begin{aligned} \|K_x - K_y\|^2 &= K'(x, x) + K'(y, y) - 2K'(x, y) \\ &= \frac{1}{2}[2L(x, x_0) - L(x, x)] + \\ &\quad \frac{1}{2}[2L(y, x_0) - L(y, y)] - \\ &\quad [L(x, x_0) + L(y, x_0) - L(x, y)] \\ &= L(x, y) \end{aligned}$$

It is then straightforward to show that $\sqrt{L}$ is a metric.

- (b) [20 points] Suppose that $K(x, y) = \exp(-|x - y|^p)$, $x, y \in \mathbb{R}$, is positive definite for $p > 2$. Then, for any $t > 0$, $\{x_1, \dots, x_n\} \subseteq X$, $\{c_1, \dots, c_n\} \subseteq \mathbb{R}$,

$$\sum_{i,j=1}^n c_i c_j \exp(-t|x_j - x_k|^p) = \sum_{i,j=1}^n c_i c_j \exp(-|t^{1/p}x_j - t^{1/p}x_k|^p) \geq 0$$

Then, by the theorem of lecture 5 (slide 36), K is an NDS kernel. But, $\sqrt{K}$ is not a metric for $p > 2$ since it does not verify the triangle inequality (take $x = 1$, $y = 2$, $z = 3$).

- (c) [5 points] If $a < 0$ or $b < 0$, $a\|x\|^2 + b < 0$ for some non-null vectors x . For such values, $K(x, x) = \tanh(a\|x\|^2 + b) < 0$. The kernel is thus not PDS and the SVM training may not converge to an optimal value. The equivalent neural network may also converge to a local minimum.

(2) [50 points] Problem 2: Boosting

- (a) [25 points] The direction e_u taken by coordinate descent after $T - 1$ rounds is the argmin_u of:

$$\begin{aligned}
 \left. \frac{dL(\alpha + \beta e_u)}{d\beta} \right|_{\beta=0} &= - \sum_{i=1}^m y_i h_u(x_i) \Phi'(-y_i f(x_i)) \\
 &\propto - \sum_{i=1}^m y_i h_u(x_i) \frac{\Phi'(-y_i f(x_i))}{\sum_{i=1}^m \Phi'(-y_i f(x_i))} \\
 &\propto - \sum_{i=1}^m y_i h_u(x_i) D_{T-1}(i) \\
 &= -(1 - 2\epsilon_u),
 \end{aligned}$$

where $D_{T-1}(i) = \frac{\Phi'(-y_i f(x_i))}{\sum_{i=1}^m \Phi'(-y_i f(x_i))}$. Thus, the base classifier h_u selected at each round is the one with the minimal error rate over the training data.

- (b) [10 points] Only the logistic loss (ϕ_4) verifies the hypotheses (assuming log with base 2).
- (c) [15 points] The step size β is the solution of:

$$\begin{aligned}
 \frac{dL(\alpha + \beta e_u)}{d\beta} &= - \sum_{i=1}^m y_i h_u(x_i) \Phi'_4(-y_i f(x_i) - \beta y_i h_u(x_i)) = 0 \\
 \Leftrightarrow - \sum_{i=1}^m y_i h_u(x_i) \frac{1}{1 + \exp(y_i f(x_i) + \beta y_i h_u(x_i))} &= 0.
 \end{aligned}$$

Using β as the step function, the algorithm differs from AdaBoost, only by the choice of the distribution update at each boosting round, here:

$$D_{t+1}(i) = \frac{1}{Z_t} \frac{1}{1 + \exp(y_i f(x_i))}.$$

Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 3
Due: March 14, 2006

Problem 1: Support Vector Machines

Let S be a training sample of size m .

- (1) Assume that S has been generated according to some probability distribution $D(x, y)$, where $(x, y) \in X \times \{-1, +1\}$.
 - (a) Define the Bayes classifier $h^* : X \rightarrow \{-1, +1\}$. Show that the kernel K^* defined by $K^*(x, x') = h^*(x)h^*(x')$ for any $x, x' \in X$ is positive definite symmetric. What is the dimension of the natural feature space associated to K^* ?
 - (b) Give the expression of the solution obtained using SVMs with this kernel. What is the number of support vectors? What is the value of the margin? What is the generalization error of the solution obtained? Under what condition is the data linearly separable?
 - (c) Let $h : X \rightarrow \mathbb{R}$ be an arbitrary real-valued function. Under what condition on h is the kernel K defined by $K(x, x') = h(x)h(x')$, $x, x' \in X$, positive definite symmetric?
- (2) Download the files “negative.dat” and “positive.dat” which contain an equal number of negative and positive examples. Download and install the libsvm software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

- (a) Let N_{SV} be the number of support vectors obtained when training an SVM. At most how many times do you need to train on a sample of size $m - 1$ in order to compute the leave-one-out error?
- (b) The leave-one-out error may have high variance and is costly to compute. Instead, you can do ten-fold cross validation: split the data in ten equal parts, ten times train on 9 and test on one, and compute the average error and standard deviation.
Use SVMs with polynomial kernels with samples of size 2000, 4000, 6000, 8000 and 10000, and plot the test error as a function

of the sample size for different values of the polynomial degree,
 $d = 1, \dots, 4$.

Problem 2: Kernel Methods

Let $X = \{a, c, g, t\}$. To classify DNA sequences using SVMs, we wish to define a kernel between sequences defined over X . We are given a finite set $I \subset X^*$ of non-coding regions (introns). For $x \in X^*$, denote by $|x|$ the length of x and by $F(x)$ the set of factors of x , i.e., the set of subsequences of x with contiguous symbols. For any two strings $x, y \in X^*$ define $K(x, y)$ by

$$K(x, y) = \sum_{z \in (F(x) \cap F(y)) - I} \rho^{|z|}, \quad (1)$$

where $\rho \geq 1$ is a real number.

- (1) Show that K is a rational kernel and that it is positive definite symmetric.
- (2) Give the time and space complexity of the computation of $K(x, y)$ with respect to the size s of a minimal automaton representing $X^* - I$.
- (3) Long common factors between x and y of length greater than or equal to n are likely to be important coding regions (exons). Modify the kernel K to assign weight $\rho_2^{|z|}$ to z when $|z| \geq n$, $\rho_1^{|z|}$ otherwise, where $1 \leq \rho_1 \ll \rho_2$. Show that the resulting kernel is still positive definite symmetric.

Problem 1: Support Vector Machines

[55 points]

(1) [20 points]

- (a) [5 points] By definition, given the distribution D , h^* is defined as:

$$h^* = \operatorname{argmin}_{h: X \rightarrow \{-1, +1\}} \operatorname{error}_D(h). \quad (1)$$

K^* is clearly positive definite symmetric since $K^*(x, x')$ is defined as the dot product (in dimension one) of the features vectors $h^*(x)$ and $h^*(x')$.

- (b) [15 points] The general expression of the solution is

$$h(x) = \operatorname{sgn}\left(\sum_{i=1}^m \alpha_i K^*(x, x_i) + b\right). \quad (2)$$

Here, it is easy to see both in the separable and non-separable case that the solution is simply:

$$h(x) = \operatorname{sgn}(K^*(x, x_+)), \quad (3)$$

where x_+ is such that $h^*(x_+) = +1$. One support vector is enough. The solution can be rewritten as

$$h(x) = h^*(x). \quad (4)$$

The generalization error of the solution is thus that of the Bayes classifier (it is optimal). The data is separable iff the Bayes error is zero.

- (c) [5 points] A kernel of this type is always positive definite symmetric since $K(x, x')$ is defined as a dot product of the feature vectors $h(x)$ and $h(x')$.

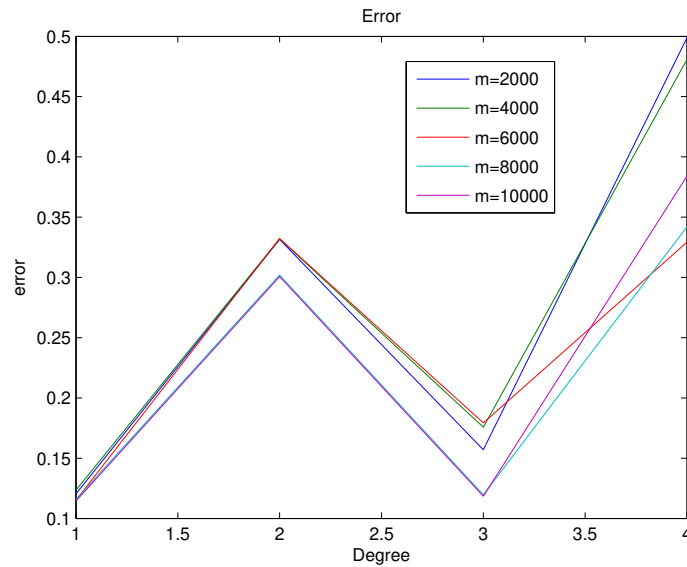


Figure 1: Error

(2) [35 points] [Thanks to Chien-I Liao for writing the solution for this section.]

(a) [10 points] We need to train once on m points. The test result and the number support vectors N_{SV} for m points are then known. Then, we just need to train and test N_{SV} SVMs on $m - 1$ points since the leave-one-out error when excluding a non-support vector point is identical to the original error.

(b) [25 points]

- First rescale all the data:


```
$ mv positive.dat old-positive.dat
$ ./svmscale old-positive.dat > positive.dat
$ mv negative.dat old-negative.dat
$ ./svmscale old-negative.dat > negative.dat
```
- Then write a program to split the data into 10 folds.
A sample C++ program could be found at
<http://cs.nyu.edu/~cil217/TA/split.cpp>.

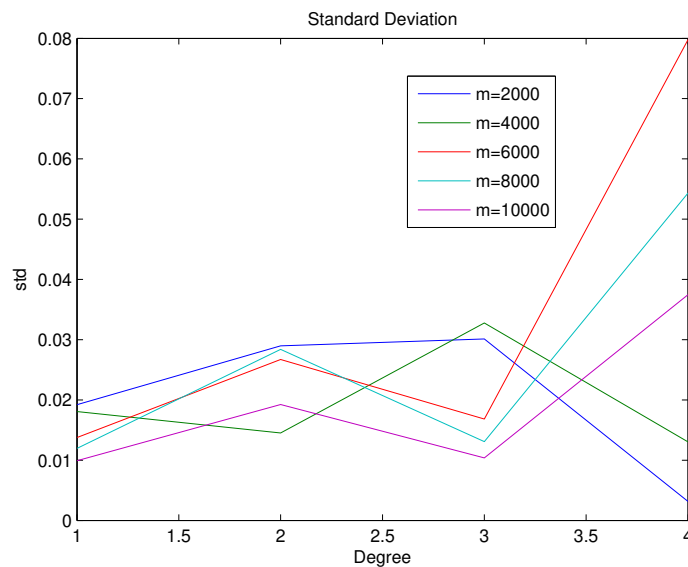


Figure 2: Standard Deviation

- Compile the code:
\$ g++ split.cpp -o split
- Then write a script to repeatedly run svm-train and svm-predict. A sample bash script could be found at http://cs.nyu.edu/~cil217/TA/train_test.sh
- Run the script:
\$ chmod 755 train_test.sh
\$./train_test.sh

Figures 1 and 2 show the result with default parameter setting.

Problem 2: Kernel Methods

[45 points]

- (1) [20 points] $X^* - I$ is a regular language and can be represented by a finite automaton. K can thus be defined by

$$\forall x, y \in X^*, \quad K(x, y) = [[T \circ T^{-1}]](x, y), \quad (5)$$

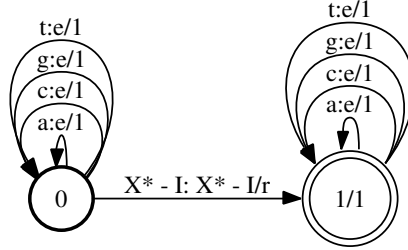


Figure 3: Weighted transducer T . e represents the empty string, and $r = \rho$. $X^* - I$ stands for a finite automaton accepting $X^* - I$.

where T is the weighted transducer shown in Figure 3. Thus, K is a rational kernel and in view of the theorem of Lecture 5, it is positive definite symmetric.

- (2) [10 points] Let M_{X^*-I} be the minimal automaton representing $X^* - I$. The transducer T of Figure 3 can be constructed using M_{X^*-I} . Then, $|T| = |M_{X^*-I}| + 8$. Using composition of weighted transducers, the running time complexity of the computation of the algorithm is:

$$O(|x||y||T \circ T^{-1}|) = O(|x||y||T|^2) = O(|x||y||M_{X^*-I}|^2). \quad (6)$$

- (3) [15 points] The set of strings Y over the alphabet X of length less than n form a regular language since they can be described by:

$$Y = \bigcup_{i=0}^{n-1} X^i. \quad (7)$$

Thus, $Y_1 = Y \cap (X^* - I)$ and $Y_2 = (X^* - I) - Y_1$ are also regular languages. It suffices to replace in the transducer T of Figure 3 the transition labeled with $X^* - I : X^* - I/\rho$ with two transitions:

- $Y_1 : Y_1/\rho_1$, and
- $Y_2 : Y_2/\rho_2$,

with the same origin and destination states and with Y_1 and Y_2 denoting finite automata representing them. The kernel is thus still rational and PDS since it is of the form $T' \circ T'^{-1}$.

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 3
Due: March 27, 2007

SVMs

1. Download and install the `libsvm` software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

2. Download the `yeast` data set. The data is already divided into ten folds. The classification problem consists of predicting the label of the last column (`CYT` vs. `non-CYT`).
3. Normalize input vectors. Use SVMs combined with polynomial kernels to solve this classification problem. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average error plus or minus one standard deviation as a function of C (let the other parameters of polynomial kernels in `libsvm`, γ and c , be equal to their default values 1). Report the best value of the trade-off constant C using ten-fold cross validation.
4. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation training and test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d .
5. How many of the support vectors lie on the margin hyperplanes? Find a way to compute the *soft* margin ρ and give its value for each d .
6. Derive the dual formulation of the SVM problem for a PDS kernel K when the error term is $\frac{C}{2} \sum_{i=1}^m \xi_i^2$ instead. Prove that there exists a PDS kernel K' such that this problem becomes equivalent to the dual optimization problem for SVMs in the *separable case*.
7. Show how to obtain the solution for this error term using `libsvm` without writing any new code. Find the best value of d and C using this error term and compare your results with what you found before.

Kernels

1. Let H be a Hilbert space with the corresponding dot product $\langle \cdot, \cdot \rangle$. Show that the kernel K defined over $H \times H$ by

$$K(x, y) = 1 - \langle x, y \rangle \quad (1)$$

is negative definite.

2. For any $p > 0$, let K_p be the kernel defined over $\mathbb{R}_+ \times \mathbb{R}_+$ by

$$K_p(x, y) = e^{-(x+y)^p}. \quad (2)$$

Show that K_p is positive definite symmetric (PDS) iff $p \leq 1$ (*hint*: you can use the fact that if K is NDS, then for any $0 < \alpha \leq 1$, K^α is also NDS).

3. To prevent fraud, a credit-card company decides to contact Professor Villebanque and provides him with a random list of several thousand fraudulent and non-fraudulent *events*. Events can be of very different nature, they may be for example transactions of various amounts, changes of address or card-holder information, or a request for a new card.

Professor Villebanque decides to use support vector machines combined with an appropriate kernel to help predict fraudulent events accurately. It is difficult for Professor Villebanque to define relevant features for such a diverse set of events. However, the risk department of the company has created a complicated method to estimate a probability $\Pr[U]$ for any event U . Thus, Professor Villebanque decides to make use of that information and comes up with the following kernel defined over all pairs of events (U, V) :

$$K(U, V) = \Pr[U \wedge V] - \Pr[U] \Pr[V]. \quad (3)$$

Help Professor Villebanque by showing that his kernel is positive definite symmetric, which guarantees the convergence of the algorithm.

Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 3 – Solution
March 13, 2007

SVMs

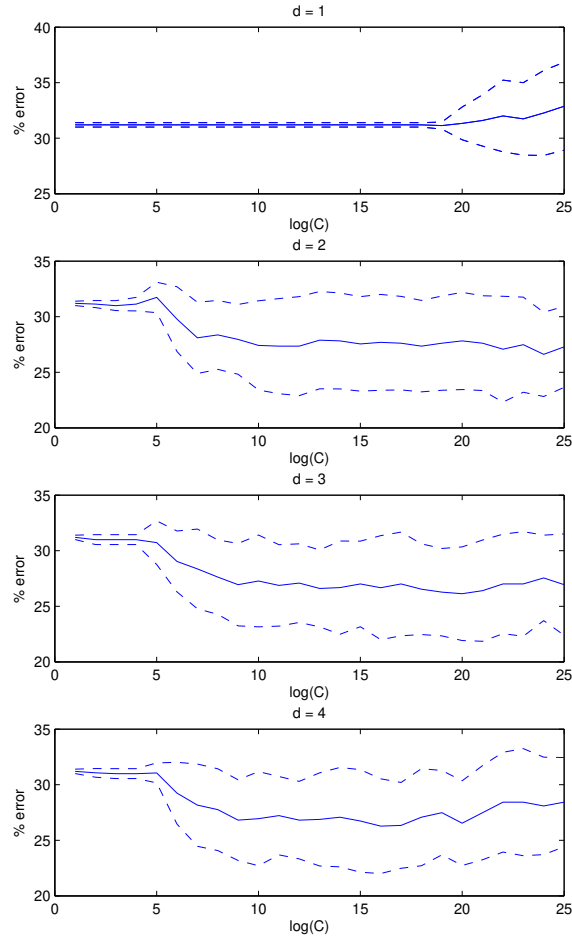
[60 points]

1. [0 point] Download and install the `libsvm` software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

2. [0 point] Download the `yeast` data set. The data is already divided into ten folds. The classification problem consists of predicting the label of the last column (`CYT` vs. `non-CYT`).
3. [15 points] Normalize input vectors. Use SVMs combined with polynomial kernels to solve this classification problem. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average error plus or minus one standard deviation as a function of C (let the other parameters of polynomial kernels in `libsvm`, γ and c , be equal to their default values 1). Report the best value of the trade-off constant C using ten-fold cross validation.

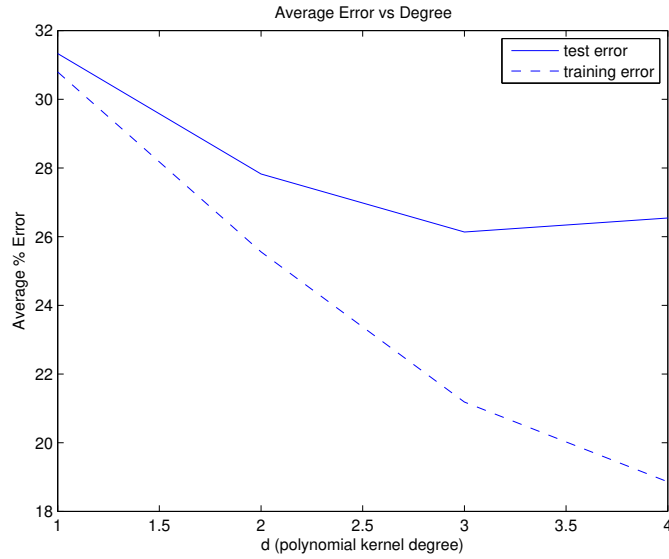
Below are example plots for the SVM error for various values of C and d . The data was scaled between $(-1, +1)$, and all other kernel parameters were left at their default values.



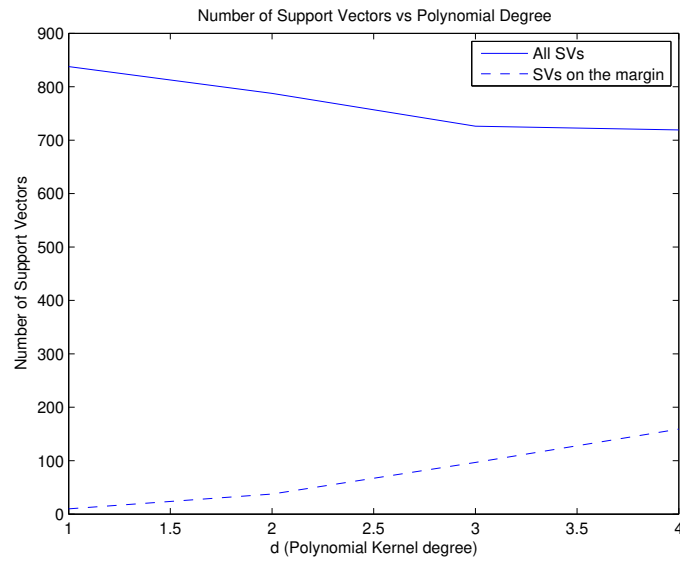
The best values of C and d are $C \approx 2^{20}$ and $d = 3$.

4. [5 points] Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation training and test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d .

Below is a graph showing the average training and test error for $C^* = 20$.



And the plot of average number of support vectors.



5. [15 points] How many of the support vectors lie on the margin hyperplanes? Find a way to compute the *soft* margin ρ and give its value for each d .

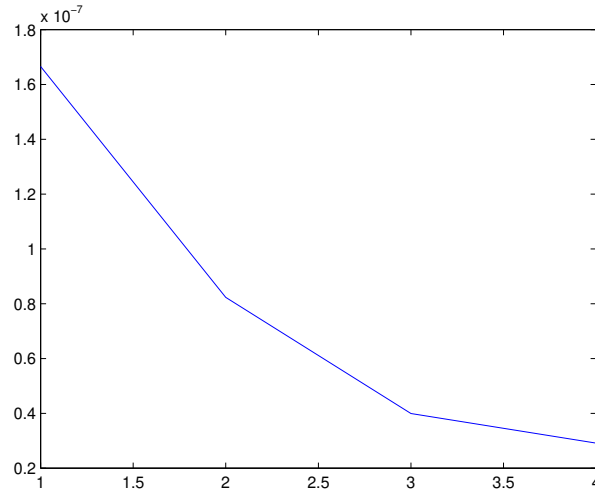
For the support vectors x_i lying on the margin hyperplanes, $\xi_i = 0$,

thus $y_i(w \cdot x_i + b) = 1$. In the absence of outliers, the hyperplane found is therefore an optimal (canonical) hyperplane defined by these support vectors. Thus, as in the separable case, we can compute the (soft) margin by:

$$\rho = 1/\|w'\| \text{ with } w' = \sum_{|\alpha_i| < C} \alpha_i y_i x_i, \quad (1)$$

since $\xi = 0$ is equivalent to $|\alpha_i| < C$.

Below we have a plot of the margin vs the kernel degree d .



The values plotted are:

C	d
0.1666×10^{-6}	1
0.0823×10^{-6}	2
0.0399×10^{-6}	3
0.0291×10^{-6}	4

6. [10 points] Derive the dual formulation of the SVM problem for a PDS kernel K when the error term is $\frac{C}{2} \sum_{i=1}^m \xi_i^2$ instead. Prove that there exists a PDS kernel K' such that this problem becomes equivalent to the dual optimization problem for SVMs in the *separable case*.

The dual formulation of the problem is

$$\begin{aligned} & \text{maximize} && \sum_{i=1}^m \alpha_i - \frac{1}{2} \left[\sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j K(x_i, x_j) + \frac{1}{C} \sum_{i=1}^m \alpha_i^2 \right] \\ & \text{subject to} && 0 \leq \alpha_i, \forall i \in [1, m]. \end{aligned} \quad (2)$$

Let K' be the kernel defined by $K'(x, y) = K(x, y) + \frac{1}{C} \mathbf{1}_{x=y}$. K' is a PDS kernel as a sum of two PDS kernels. Then, the optimization problem can be rewritten as the standard SVM problem in the separable case with the kernel K' :

$$\begin{aligned} & \text{maximize} && \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j K'(x_i, x_j) \\ & \text{subject to} && 0 \leq \alpha_i, \forall i \in [1, m]. \end{aligned} \quad (3)$$

7. [15 points] Show how to obtain the solution for this error term using `libsvm` without writing any new code. Find the best value of d and C using this error term and compare your results with what you found before.

Add m new features to each point as in the case of a trivial mapping. For point x_i only the i th new feature is non-zero and it is equal to u_i chosen so that $K'(x_i, x_i) = (x_i \cdot x_i + 1)^d + 1/C = (x_i \cdot x_i + u_i^2 + 1)^d$, that is

$$u_i = [(\|x_i\|^2 + 1 + 1/C)^{\frac{1}{d}} - (\|x_i\|^2 + 1)]^{\frac{1}{2}}. \quad (4)$$

For a linear kernel ($d = 1$), $u_i = 1/\sqrt{C}$.

Kernels

[40 points]

1. [5 points] Let H be a Hilbert space with the corresponding dot product $\langle \cdot, \cdot \rangle$. Show that the kernel K defined over $H \times H$ by

$$K(x, y) = 1 - \langle x, y \rangle \quad (5)$$

is negative definite.

Let $n \geq 1$, $\{c_1, \dots, c_n\} \subseteq \mathbb{R}$ and $\{x_1, \dots, x_n\} \subseteq H$ with $\sum_{i=1}^n c_i = 0$. Then,

$$\sum_{i,j=1}^n c_i c_j K(x_i, x_j) = \left(\sum_{i=1}^n c_i \right)^2 - \left\langle \sum_{i=1}^n c_i x_i, \sum_{j=1}^n c_j x_j \right\rangle = -\left\| \sum_{i=1}^n c_i x_i \right\|^2 \leq 0. \quad (6)$$

2. [15 points] For any $p > 0$, let K_p be the kernel defined over $\mathbb{R}_+ \times \mathbb{R}_+$ by

$$K_p(x, y) = e^{-(x+y)^p}. \quad (7)$$

Show that K_p is positive definite symmetric (PDS) iff $p \leq 1$ (*hint*: you can use the fact that if K is NDS, then for any $0 < \alpha \leq 1$, K^α is also NDS).

It is straightforward to see that the kernel K defined over $\mathbb{R}_+ \times \mathbb{R}_+$ by $K(x, y) = x + y$ is negative definite symmetric (NDS) and that $K(x, x) \geq 0$ for all $x \geq 0$. Thus, for any $0 < \alpha \leq 1$, K^α is also NDS. Thus, $\exp(-tK^\alpha)$ is PDS for all $t > 0$, in particular $t = 1$.

Assume now that K_p is PDS for some $p > 1$. Then, for all $\{c_1, \dots, c_n\} \subseteq \mathbb{R}$, $\{x_1, \dots, x_n\} \subseteq X$, and $t > 0$,

$$\sum_{i,j=1}^n c_i c_j e^{-t(x_i+x_j)^p} = \sum_{i,j=1}^n c_i c_j e^{-(t^{1/p}x_i+t^{1/p}x_j)^p} \geq 0. \quad (8)$$

Thus, $\Psi(x, y) = (x + y)^p$ is NDS. This contradicts

$$\sum_{i,j=1}^2 c_i c_j (x_i + x_j)^p = 2^p - 2 > 0 \quad (9)$$

for $x_1 = 0$, $x_2 = 1$, and $c_1 + c_2 = 0$ with $c_1 = 1$.

3. [20 points] To prevent fraud, a credit-card company decides to contact Professor Villebanque and provides him with a random list of several thousand fraudulent and non-fraudulent *events*. Events can be of very different nature, they may be for example transactions of various amounts, changes of address or card-holder information, or a request for a new card.

Professor Villebanque decides to use support vector machines combined with an appropriate kernel to help predict fraudulent events accurately. It is difficult for Professor Villebanque to define relevant features for such a diverse set of events. However, the risk department of the company has created a complicated method to estimate a probability $\Pr[U]$ for any event U . Thus, Professor Villebanque decides to make use of that information and comes up with the following kernel defined over all pairs of events (U, V) :

$$K(U, V) = \Pr[U \wedge V] - \Pr[U] \Pr[V]. \quad (10)$$

Help Professor Villebanque by showing that his kernel is positive definite symmetric, which guarantees the convergence of the algorithm.

Let $\langle \cdot, \cdot \rangle$ be the bilinear function defined over the space of all functions defined by:

$$\langle f, g \rangle = \mathbb{E}[(f(x) - \mathbb{E}[f(x)])(g(x) - \mathbb{E}[g(x)])]. \quad (11)$$

Since the covariance matrix of n random variables is positive semi-definite and symmetric, this defines a dot product. Observe that $\Pr[U] = \mathbb{E}[1_U]$ and $\Pr[U \wedge V] = \mathbb{E}[1_U 1_V]$, thus

$$K(U, V) = \langle 1_U, 1_V \rangle. \quad (12)$$

This shows that K is a positive definite symmetric kernel.

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 3
Due: March 28th, 2008
Credit: Ashish Rastogi, Afshin Rostamizadeh
Ameet Talwalkar, and Eugene Weinstein.

1. SVMs:

- (a) Download and install `libsvm` from

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

- (b) Download the `pendigits` data set. The task is to predict the digit label (0 – 9) based on the features computed over the digit image. The data is comma-delimited, with the last item being the label. Normalize the input data so that all feature values are between -1 and 1 .
- (c) Train and test a SVM using polynomial kernels and 10-fold cross validation. For each setting of the polynomial degree $d = 1, 2, 3, 4$, plot the average error as the data set size is changed from 50 to 1000 data points (keep the first n points of the data set).
- (d) Repeat the learning experiment with radial basis function (RBF) kernels. Use the script `grid.py` packaged with `libsvm` to do a sweep over the space of parameters (C, γ) , where C is the SVM learning parameter and γ is the coefficient in the RBF kernel. Report the values of C and γ that yield the highest accuracy under 10-fold cross validation. Also report the accuracy achieved.
- (e) Let (C^*, γ^*) be the best parameters found in the previous exercise. With C fixed at C^* , plot the 10-fold cross-validation accuracy as the γ parameter is varied.
- (f) Suppose you wish to use support vector machines to solve a learning problem where some training data points are more important than others. More formally, assume that each training point consists of a triplet (x_i, y_i, p_i) , where $0 \leq p_i \leq 1$ is the importance

of the i th point. Rewrite the primal SVM constrained optimization problem so that the penalty for mis-labeling a point x_i is scaled by the priority p_i . Then carry this modification through the derivation of the dual solution.

2. Kernels:

- (a) Given a data set $x_1, \dots, x_m$ and a kernel $k(x_i, x_j)$ with a Gram matrix K such that $K_{ij} = k(x_i, x_j)$. Give a map $\Phi(\cdot)$ such that if K is positive semidefinite then $k(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$.
- (b) Show the converse of the previous statement: that if there exists a mapping $\Phi(x)$, then the matrix K is positive semidefinite.
- (c) Let us define a *difference kernel* as $k(x, x') = ||x - x'||$ for $x, x' \in \mathbb{R}^n$. Show that this kernel is not positive definite symmetric (PDS).
- (d) The *cosine kernel* is defined as $k(x, x') = \cos \angle(x, x')$. Show that the cosine kernel is PDS.

1. SVMs:

- (a) Download and install `libsvm` from

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

- 5 points Download the `pendigits` data set. The task is to predict the digit label (0 – 9) based on the features computed over the digit image. The data is comma-delimited, with the last item being the label. Normalize the input data so that all feature values are between -1 and 1 .

The binary `svm-scale` should be used to normalize the data.

- 15 points Train and test a SVM using polynomial kernels and 10-fold cross validation. For each setting of the polynomial degree $d = 1, 2, 3, 4$, plot the average error as the data set size is changed from 50 to 1000 data points (keep the first n points of the data set).

The accuracy plot appears in Figure 1.

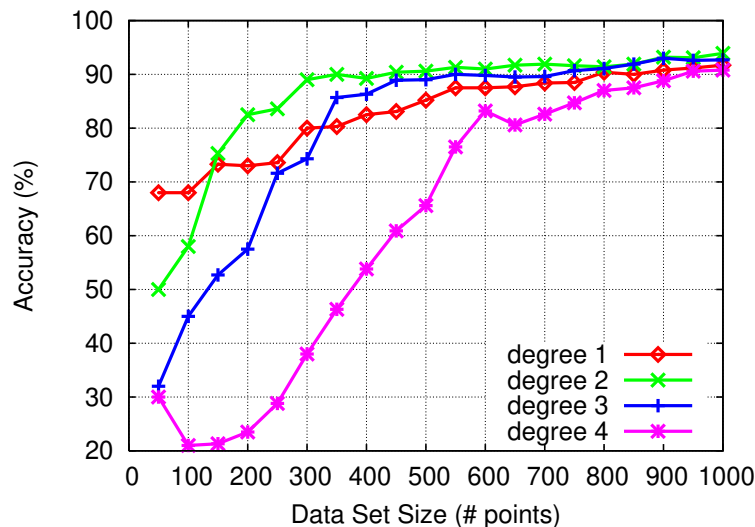


Figure 1: Accuracy achieved with polynomial kernels of varying degrees.

- 10 points Repeat the learning experiment with radial basis function (RBF) kernels. Use the script `grid.py` packaged with `libsvm` to do a sweep over the space of parameters (C, γ) , where C is the SVM learning parameter and γ is the coefficient in the RBF kernel. Report the values of C and γ that yield the highest accuracy under 10-fold cross validation. Also report the accuracy achieved.

The following command should yield a good sweep of the parameter space:

```
grid.py -log2g -5,5,1 -v 10 -log2c -5,5,1 data.txt
```

This highest accuracy achieved with this sweep is 99.0%, and the optimal parameter setting is $C = 4$ and $\gamma = 0.5$.

10 points Let (C^*, γ^*) be the best parameters found in the previous exercise. With C fixed at C^* , plot the 10-fold cross-validation accuracy as the γ parameter is varied. The plot appears in Figure 2.

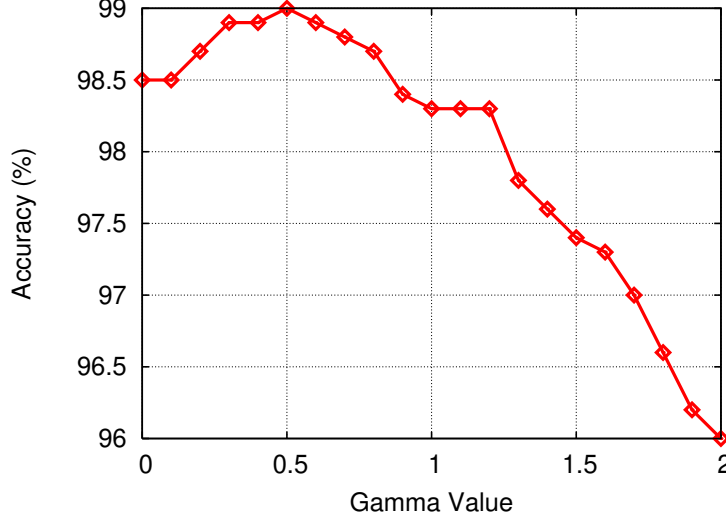


Figure 2: Accuracy achieved with RBF kernel for various settings of γ .

30 points Suppose you wish to use support vector machines to solve a learning problem where some training data points were more important than others. Assume each training point consists of a triplet (x_i, y_i, p_i) , where $0 \leq p_i \leq 1$ is the importance of the i th point. Rewrite the primal SVM constrained optimization problem so that the penalty for mis-labeling a point x_i is scaled by the priority p_i . Then carry this modification through the derivation of the dual solution.

The modified primal optimization problem can be written as

$$\begin{aligned} & \text{minimize} && \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i p_i \\ & \text{subject to} && y_i [w \cdot x_i + b] \geq 1 - \xi_i \end{aligned}$$

The Lagrangian holding for all $w, b, \alpha_i \geq 0, \beta_i \geq 0$ is then

$$\begin{aligned} L(w, b, \alpha) = & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i p_i \\ & - \sum_{i=1}^m \alpha_i [y_i (w \cdot x_i + b) - 1 + \xi_i] - \sum_{i=1}^m \beta_i \xi_i \end{aligned} \tag{1}$$

Then $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$ are the same as for the regular non-separable SVM optimization problem. We also have $\frac{\partial L}{\partial \xi_i} = Cp_i - \alpha_i - \beta_i$. Thus to satisfy the KKT conditions we have for all $i \in [1, m]$,

$$w = \sum_{i=1}^m \alpha_i y_i x_i \quad (2)$$

$$\sum_{i=1}^m \alpha_i y_i = 0 \quad (3)$$

$$\alpha_i + \beta_i = Cp_i \quad (4)$$

$$\alpha_i [y_i(w \cdot x_i + b) - 1 + \xi_i] = 0 \quad (5)$$

$$\beta_i \xi_i = 0 \quad (6)$$

Plugging Equation 2 into Equation 1, we get

$$\begin{aligned} L = & \frac{1}{2} \left\| \sum_{i=1}^m \alpha_i y_i x_i \right\|^2 + C \sum_{i=1}^m \xi_i p_i - \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \\ & - \sum_{i=1}^m \alpha_i y_i b + \sum_{i=1}^m \alpha_i - \sum \alpha_i \xi_i - \sum_{i=1}^m \beta_i \xi_i \end{aligned} \quad (7)$$

Using Equation 4, we can simplify:

$$L = \sum_{i=1}^m \alpha_i - \frac{1}{2} \left\| \sum_{i=1}^m \alpha_i y_i x_i \right\|^2$$

meaning that the objective function is the same as in the regular SVM problem. The difference is in the constraints on the optimization. Recall that our dual form holds for $\beta_i \geq 0$. Using again equation 4, our optimization problem is to maximize L subject to the constraints:

$$\forall i \in [1, m], 0 \leq \alpha_i \leq Cp_i \wedge \sum_{i=1}^m \alpha_i y_i = 0.$$

2. Kernels:

10 points Given a data set $x_1, \dots, x_m$ and a kernel $k(x_i, x_j)$ with a Gram matrix K such that $k(x_i, x_j) = K_{ij}$, show that a map $\Phi(\cdot)$ can be given such that if K is positive semidefinite then $k(x_i, x_j) = \Phi(x_i) \cdot \Phi(x_j)$.

Because K is positive semidefinite, it can be diagonalized as $K = S\Lambda S^\top$ where Λ is a diagonal matrix of K 's eigenvalues and S is the matrix of K 's eigenvectors. Further decomposing, we get $K = S\Lambda^{1/2}\Lambda^{1/2}S^\top$. We then have

$$k(x_i, x_j) = K_{ij} = (S\Lambda S^\top)_{ij} = (\Lambda^{1/2}S_i) \cdot (\Lambda^{1/2}S_j)$$

where S_i is the i th eigenvector of K . Thus the kernel map $\Phi(x_i) = \Lambda^{1/2}S_i$ clearly satisfies the desired condition.

10 points Show the converse of the previous statement: that if there exists a mapping $\Phi(x)$, then the matrix K is positive semidefinite.

For any $\alpha_1, \dots, \alpha_m \in \mathbb{R}$, we have

$$\sum_{i,j=1}^m \alpha_i \alpha_j K_{ij} = \left\| \sum_{i=1}^m \alpha_i \Phi(x_i) \right\|^2 \geq 0$$

10 points Let us define a *difference kernel* as $k(x, x') = \|x - x'\|^2$ for $x, x' \in \mathbb{R}^m$. Show that this kernel is not positive definite symmetric (PDS).

Consider the Gram matrix defined as $K_{ij} = k(x_i, x_j)$. It is clear that K will have all zeros on the diagonal. Hence $\text{tr}(K) = 0$. When $K \neq 0$, this means it must have at least one negative eigenvalue. Hence k is not PDS.

10 points The *cosine kernel* is defined as $k(x, x') = \cos \angle(x, x')$. Show that the cosine kernel is PDS.

Rewriting the cosine in terms of the dot product, we have

$$k(x, x') = \cos \angle(x, x') = \frac{x \cdot x'}{\|x\| \|x'\|}$$

Thus, the cosine kernel is just a scaling of the standard dot product, which is a PDS kernel. Hence, the cosine kernel is also PDS.

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 3
Due: May 08, 2009

A. Perceptron algorithm

Let S be a labeled sample of N points in $\mathbb{R}^N$ with

$$x_i = (\underbrace{(-1)^i, \dots, (-1)^i}_{i \text{ first components}}, (-1)^{i+1}, 0, \dots, 0) \quad \text{and} \quad y_i = (-1)^{i+1}. \quad (1)$$

- Show that the perceptron algorithm makes $\Omega(2^N)$ updates before finding a separating hyperplane, regardless of the order in which it receives the points.

B. Boosting

This problem considers an algorithm similar to AdaBoost but with a different objective function. Assume that the training data is given as m labeled examples $(x_1, y_1), \dots, (x_m, y_m) \in X \times \{-1, +1\}$. Let $\Phi: \mathbb{R} \rightarrow \mathbb{R}$ be the function defined by

$$\Phi(u) = \begin{cases} (1+u)^2 & \text{if } u \geq -1 \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

- Consider the objective function F defined by $F(\alpha) = \sum_{i=1}^m \Phi(-y_i f(x_i))$ where f is a linear combination of base classifiers: $f = \sum_{t=1}^T \alpha_t h_t$ as for AdaBoost. Show that F is convex and differentiable.
- Derive a new boosting algorithm using the objective function F . Characterize the best base classifier h_u to select at each round of boosting if we use coordinate descent.

Mehryar Mohri
 Foundations of Machine Learning
 Courant Institute of Mathematical Sciences
 Homework assignment 3
 Due: May 08, 2009

A. Perceptron algorithm

Let S be a labeled sample of N points in $\mathbb{R}^N$ with

$$x_i = (\underbrace{(-1)^i, \dots, (-1)^i}_{i \text{ first components}}, (-1)^{i+1}, 0, \dots, 0) \quad \text{and} \quad y_i = (-1)^{i+1}. \quad (1)$$

- [50 points] Show that the perceptron algorithm makes $\Omega(2^N)$ updates before finding a separating hyperplane, regardless of the order in which it receives the points.

Let w be the weight vector. Since each update is of the form $w \leftarrow w + y_i x_i$ and since the components of the sample points are integers, the components of w are also integers.

Let $n_1, \dots, n_N \in \mathbb{Z}$ denote the components of w . w correctly classifies all points iff $y_i(w \cdot x_i) > 0$ for $i = 1, \dots, m$, that is

$$\begin{cases} n_1 > 0 \\ n_1 - n_2 < 0 \\ -n_1 - n_2 + n_3 > 0 \\ \dots \\ (-1)^N(n_1 + n_2 + \dots + n_{N-1} - n_N) < 0 \end{cases} \Leftrightarrow \begin{cases} n_1 > 0 \\ n_2 > n_1 \\ n_3 > n_1 + n_2 \\ \dots \\ n_N > n_1 + n_2 + \dots + n_{N-1}. \end{cases}$$

These last inequalities show that the data is linearly separable with $w = (1, 2, \dots, 2^{N-1})$. They also imply that $n_1 \geq 1, n_2 \geq 2, n_3 \geq 4, \dots, n_N \geq 2^{N-1}$. Since each update can at most increment n_N by 1, the number of updates is at least $2^{N-1} = \Omega(2^N)$.

B. Boosting

This problem considers an algorithm similar to AdaBoost but with a different objective function. Assume that the training data is given as m labeled examples $(x_1, y_1), \dots, (x_m, y_m) \in X \times \{-1, +1\}$. Let $\Phi: \mathbb{R} \rightarrow \mathbb{R}$ be the function defined by

$$\Phi(u) = \begin{cases} (1+u)^2 & \text{if } u \geq -1 \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

[50 points]

- [20 points] Consider the objective function F defined by $F(\alpha) = \sum_{i=1}^m \Phi(-y_i f(x_i))$ where f is a linear combination of base classifiers: $f = \sum_{t=1}^T \alpha_t h_t$ as for AdaBoost. Show that F is convex and differentiable.

To show that F is differentiable it suffices to show that Φ is differentiable since F is a sum of functions obtained by composition of Φ with a linear function of α . Φ is continuously differentiable on the interval $] -\infty, -1[$ with $\Phi'(u) = 0$, and is continuously differentiable on the interval $] -1, +\infty[$ with $\Phi'(u) = 2(1+u)$ and $\lim_{x \rightarrow +\infty} \Phi'(u) = \lim_{x \rightarrow -\infty} \Phi'(u) = 0$, thus Φ is continuously differentiable over $\mathbb{R}$.

Since its differential is always non-negative, it is an increasing function. Φ is twice differentiable over $] -1, +\infty[$ with $\Phi''(u) = 2$ and $\Phi''(u) = 0$ on $] -\infty, -1[$ and is continuous at -1 , thus Φ is convex. F is thus convex as a sum of function obtained by composing an increasing and convex function with a linear function.

- [30 points] Derive a new boosting algorithm using the objective function F . Characterize the best base classifier h_u to select at each round of boosting if we use coordinate descent.
- [20 points] Let I denote the set of indices i for which $\Phi'(-y_i f(x_i)) \neq 0$: $I = \{i \in [1, m]: \Phi'(-y_i f(x_i)) \neq 0\}$. The direction e_u taken by coordinate descent after $T-1$ rounds is the argmin_u of:

$$\begin{aligned} \left. \frac{dF(\alpha + \beta e_u)}{d\beta} \right|_{\beta=0} &= - \sum_{i=1}^m y_i h_u(x_i) \Phi'(-y_i f(x_i)) \\ &\propto - \sum_{i=1}^m y_i h_u(x_i) \frac{\Phi'(-y_i f(x_i))}{\sum_{i \in I} \Phi'(-y_i f(x_i))} \\ &\propto - \sum_{i=1}^m y_i h_u(x_i) D_{T-1}(i) \\ &= -(1 - 2\epsilon_u), \end{aligned}$$

where $D_{T-1}(i) = \frac{\Phi'(-y_i f(x_i))}{\sum_{i \in I} \Phi'(-y_i f(x_i))}$, and $\epsilon_u = \Pr_{D_{T-1}}[h_u(x_i) \neq y_i]$. Thus, the base classifier h_u selected at each round is the one with the minimal ϵ_u .

– [10 points] The step size β is given by:

$$\begin{aligned} \frac{dF(\alpha + \beta e_u)}{d\beta} &= - \sum_{i=1}^m y_i h_u(x_i) \Phi'(-y_i f(x_i) - \beta y_i h_u(x_i)) = 0 \\ \Leftrightarrow \sum_{i \in I'}^m (1 - y_i f(x_i) - \beta y_i h_u(x_i)) &= 0, \end{aligned}$$

where $I' = \{i \in [1, m] : \Phi'(-y_i f(x_i) - \beta y_i h_u(x_i)) \neq 0\}$, that is $I' = \{i \in [1, m] : y_i f(x_i) + \beta y_i h_u(x_i) < 0\}$.

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 3
Due: April 19, 2010

A. Kernels

1. Show that the dimension of the feature space associated to the polynomial kernel of degree d , $K: \mathbb{R}^N \times \mathbb{R}^N \rightarrow \mathbb{R}$, $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + c)^d$, with $c > 0$, is

$$\binom{N + d}{d}. \quad (1)$$

Write K in terms of kernels $k_i: (\mathbf{x}, \mathbf{x}') \mapsto (\mathbf{x} \cdot \mathbf{x}')^i$, $i \in [0, d]$. What is the weight assigned to each k_i in that expression? How does it vary as a function of c .

2. For $\alpha \geq 0$, the kernel $K_\alpha: (\mathbf{x}, \mathbf{x}') \mapsto \sum_{k=1}^N \min(|x_k|^\alpha, |x'_k|^\alpha)$ over $\mathbb{R}^N \times \mathbb{R}^N$ is used in image classification. Show that K_α is PDS. To do that, you can proceed as follows.
 - (a) Use the fact that $(f, g) \mapsto \int_{t=0}^{+\infty} f(t)g(t)dt$ is an inner product over the set of measurable functions over $[0, +\infty)$ to show that $(x, x') \mapsto \min(x, x')$ is a PDS kernel (hint: associate an indicator function to x and another one to x').
 - (b) Use the previous question to show that K_1 is PDS and similarly K_α with other values of α .

B. Boosting

1. Assume that the main weak learner assumption of AdaBoost holds. Let h_t be the base learner selected at round t . Show that the base learner h_{t+1} selected at time t must be different from h_t .
2. Let the training sample be $S = ((x_1, y_1), \dots, (x_m, y_m))$. Suppose we wish to penalize differently errors made on x_i versus x_j . To do that, we associate some non-negative importance weight w_i to each point x_i and define the objective function $F(\boldsymbol{\alpha}) = \sum_{i=1}^m w_i e^{-y_i f(x_i)}$, where $f = \sum_t \alpha_t h_t$ with the notation already used in class. Show that this function is convex and differentiable and use it to derive a boosting-type algorithm (give a clear description of the algorithm similar to that of AdaBoost presented in class).

C. Perceptron

1. The margin bound on the maximum number of updates presented in class for the perceptron algorithm was given for the special case $\eta = 1$. Consider now the general perceptron update $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \eta y_t \mathbf{x}_t$, where $\eta > 0$. With the same assumptions as for the theorem presented in class, prove a bound on the maximum number of mistakes. How does η affect the bound?
2. Suppose each input vector $\mathbf{x}_t$, $t \in [1, T]$, coincides with the t th unit vector of $\mathbb{R}^T$. How many updates does it take the perceptron algorithm to converge? How does the number of updates (or mistakes) compare with the margin bound?

A. Kernels

1. Show that the dimension of the feature space associated to the polynomial kernel of degree d , $K: \mathbb{R}^N \times \mathbb{R}^N \rightarrow \mathbb{R}$, $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + c)^d$, with $c > 0$, is

$$\binom{N+d}{d}. \quad (1)$$

It is clear that K can be written as a linear combinations of all monomials $x_1^{k_1} \cdots x_N^{k_N}$ with $\sum_{j=1}^N k_j \leq d$. The dimension of the feature space is thus the number of such monomials, $f(N, d)$, that is the number of ways of adding N non-negative integers to obtain a sum of at most d . Note that any sum of $N-1$ integers less than or equal to d can be uniquely completely to be equal to d by adding one more term. This defines in fact a bijection between sums of $N-1$ integers with value at most d and sums of N integers with value equal to d . Thus, the number of sums of N integers exactly equal to d is $f(N-1, d)$.

Now, since a sum of N terms less than or equal to d is either equal to d or less than or equal to $d-1$,

$$f(N, d) = f(N-1, d) + f(N, d-1).$$

The result then follows by induction on $N+d$, using $f(1, 0) = f(0, 1) = 1$.

Write K in terms of kernels $k_i: (\mathbf{x}, \mathbf{x}') \mapsto (\mathbf{x} \cdot \mathbf{x}')^i$, $i \in [0, d]$. What is the weight assigned to each k_i in that expression? How does it vary as a function of c .

By the binomial identity, $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + c)^d = \sum_{i=0}^d \binom{d}{i} c^{d-i} (\mathbf{x} \cdot \mathbf{x}')^i$. The weight assigned to each k_i , $i \leq d$, is thus $\binom{d}{i} c^{d-i}$. Increasing c decreases the weight of k_i , particularly that of k_i s with larger i .

2. For $\alpha \geq 0$, the kernel $K_\alpha: (\mathbf{x}, \mathbf{x}') \mapsto \sum_{k=1}^N \min(|x_k|^\alpha, |x'_k|^\alpha)$ over $\mathbb{R}^N \times \mathbb{R}^N$ is used in image classification. Show that K_α is PDS. To do that, you can proceed as follows.

- (a) Use the fact that $(f, g) \mapsto \int_{t=0}^{+\infty} f(t)g(t)dt$ is an inner product over the set of measurable functions over $[0, +\infty)$ to show that $(x, x') \mapsto$

$\min(x, x')$ is a PDS kernel (hint: associate an indicator function to x and another one to x').

Observe that $\min(|u|^\alpha, |u'|^\alpha) = \int_0^{+\infty} 1_{t \in [0, |u|^\alpha]} 1_{t \in [0, |u'|^\alpha]} dt$, which shows that $(u, u') \mapsto \min(|u|^\alpha, |u'|^\alpha)$ is PDS.

- (b) Use the previous question to show that K_1 is PDS and similarly K_α with other values of α .

Since $K_\alpha(x, x') = \sum_{k=1}^N \min(|x_k|^\alpha, |x'_k|^\alpha)$, K_α is PDS as a sum of N PDS kernels.

B. Boosting

1. Assume that the main weak learner assumption of AdaBoost holds. Let h_t be the base learner selected at round t . Show that the base learner h_{t+1} selected at time t must be different from h_t .

By the weak learning assumption, there exists a hypothesis $h \in H$ whose D_{t+1} -error is less than half. Examine the empirical error of h_t for the distribution D_{t+1} . Since $Z_t = 2\sqrt{\epsilon_t(1 - \epsilon_t)}$ and $\alpha_t = \frac{1}{2} \log \frac{1 - \epsilon_t}{\epsilon_t}$,

$$\begin{aligned} \hat{R}_{D_{t+1}}(h_t) &= \sum_{i=1}^m \frac{D_t(i) e^{-\alpha_t y_i h_t(x_i)}}{Z_t} 1_{y_i h_t(x_i) < 0} \\ &= \sum_{y_i h_t(x_i) < 0}^m \frac{D_t(i) e^{\alpha_t}}{Z_t} \\ &= \frac{e^{\alpha_t}}{Z_t} \sum_{y_i h_t(x_i) < 0}^m D_t(i) \\ &= \frac{\sqrt{\frac{1 - \epsilon_t}{\epsilon_t}}}{2\sqrt{\epsilon_t(1 - \epsilon_t)}} \epsilon_t = \frac{1}{2}. \end{aligned}$$

This shows that h_t cannot be selected at round $t + 1$.

2. Let the training sample be $S = ((x_1, y_1), \dots, (x_m, y_m))$. Suppose we wish to penalize differently errors made on x_i versus x_j . To do that, we associate some non-negative importance weight w_i to each point x_i and define the objective function $F(\alpha) = \sum_{i=1}^m w_i e^{-y_i f(x_i)}$, where $f = \sum_t \alpha_t h_t$ with the notation already used in class. Show that this function is convex and differentiable and use it to derive a boosting-type algorithm (give a clear description of the algorithm similar to that of AdaBoost presented in class).

For all α , $F(\alpha) = \sum_{i=1}^m w_i e^{-y_i f(x_i)} = \sum_{i=1}^m w_i e^{-y_i \sum_{t=1}^T \alpha_t h_t(x_i)}$, with $w_i \geq 0$. F is convex as a non-negative linear combination of convex functions and is

clearly differentiable. Applying coordinate descent to this function leads to the same algorithm as AdaBoost with the only difference that

$$D_1(i) \leftarrow \frac{w_i}{\sum_{i=1}^m w_i}. \quad (2)$$

C. Perceptron

1. The margin bound on the maximum number of updates presented in class for the perceptron algorithm was given for the special case $\eta = 1$. Consider now the general perceptron update $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \eta y_t \mathbf{x}_t$, where $\eta > 0$. With the same assumptions as for the theorem presented in class, prove a bound on the maximum number of mistakes. How does η affect the bound?

The bound is unaffected as shown by the following using the same definitions and steps as in the lecture slides:

$$\begin{aligned} M\rho &\leq \frac{\mathbf{v} \cdot \sum_{t \in I} y_t \mathbf{x}_t}{\|\mathbf{v}\|} \\ &= \frac{\mathbf{v} \cdot \sum_{t \in I} (\mathbf{w}_{t+1} - \mathbf{w}_t)/\eta}{\|\mathbf{v}\|} \quad (\text{definition of updates}) \\ &= \frac{\mathbf{v} \cdot \mathbf{w}_{T+1}}{\eta \|\mathbf{v}\|} \\ &\leq \|\mathbf{w}_{T+1}\|/\eta \quad (\text{Cauchy-Schwarz ineq.}) \\ &= \|\mathbf{w}_{t_m} + \eta y_{t_m} \mathbf{x}_{t_m}\|/\eta \quad (t_m \text{ largest } t \text{ in } I) \\ &= \left[\|\mathbf{w}_{t_m}\|^2 + \eta^2 \|\mathbf{x}_{t_m}\|^2 + \underbrace{\eta y_{t_m} \mathbf{w}_{t_m} \cdot \mathbf{x}_{t_m}}_{\leq 0} \right]^{1/2} / \eta \\ &\leq \left[\|\mathbf{w}_{t_m}\|^2 + \eta^2 R^2 \right]^{1/2} / \eta \\ &\leq \left[M\eta^2 R^2 \right]^{1/2} / \eta = \sqrt{M}R. \quad (\text{applying the same to previous } ts \text{ in } I). \end{aligned}$$

2. Suppose each input vector $\mathbf{x}_t$, $t \in [1, T]$, coincides with the t th unit vector of $\mathbb{R}^T$. How many updates does it take the perceptron algorithm to converge? How does the number of updates (or mistakes) compare with the margin bound?

Clearly, it takes T updates and leads to $\mathbf{w} = \sum_{t=1}^T y_t \mathbf{x}_t$. Let $\mathbf{u} \in \mathbb{R}^T$ be a vector of norm 1 defining a separating hyperplane, thus $y_t \mathbf{u} \cdot \mathbf{x}_t = y_t u_t \geq 0$ for all $t \in [1, T]$. To obtain the maximum margin ρ , we seek a vector $\mathbf{u}$ maximizing the minimum of $y_t u_t$ with $y_t u_t \geq 0$ for all t and $\|\mathbf{u}\| = 1$. By symmetry, all $y_t u_t$ s are equal, thus $u_t = y_t / \sqrt{T}$ for all $t \in [1, T]$ and $\rho = 1/\sqrt{T}$. Thus, Novikoff's bound gives $R^2/\rho^2 = 1/(1/T) = T$.

Mehryar Mohri
 Foundations of Machine Learning
 Courant Institute of Mathematical Sciences
 Homework assignment 3
 April 14, 2011
 Due: May 02, 2011

A. Kernels

1. Show that for all $\lambda > 0$, the kernel K defined on $\mathbb{R} \times \mathbb{R}$ by

$$K(x, x') = \exp(-\lambda [\sin(x' - x)]^2), \quad (1)$$

for all $x, x' \in \mathbb{R}$ is PDS (*hint*: you could seek to rewrite $[\sin(x' - x)]^2$ as the square of the norm of the difference of two vectors).

2. Let $\Phi: X \rightarrow H$ be a feature mapping such that the dimension N of H is very high and let $K: X \times X \rightarrow \mathbb{R}$ be a PDS kernel defined by

$$K(x, x') = \mathbb{E}_{i \sim D} [\Phi(x)_i \Phi(x')_i], \quad (2)$$

where $[\Phi(x)]_i$ is the i th component of $\Phi(x)$ and similarly for $\Phi(x')$ and where D is a distribution over the indices i . We shall assume that $|[\Phi(x)]_i| \leq R$ for all $x \in X$ and $i \in [1, N]$.

Suppose that to compute $K(x, x')$ no other method is available than computing the inner product (2), which would require $O(N)$ time. One idea in that case is instead to compute an approximation of that kernel based on random selection of a subset I of the N components of $\Phi(x)$ and $\Phi(x')$ according to D , that is:

$$K'(x, x') = \frac{1}{n} \sum_{i \in I} [\Phi(x)]_i [\Phi(x')]_i, \quad (3)$$

where $|I| = n$.

- (a) Fix x and x' in X . Prove that

$$\Pr_{I \sim D^n} [|K(x, x') - K'(x, x')| > \epsilon] \leq 2e^{\frac{-n\epsilon^2}{2R^4}}. \quad (4)$$

(*hint*: use McDiarmid's inequality).

- (b) Let $\mathbf{K}$ and $\mathbf{K}'$ be the kernel matrices associated to K and K' . Show that for any $\epsilon, \delta > 0$, for $n > \frac{2R^4}{\epsilon^2} \log \frac{m(m+1)}{\delta}$, with probability at least $1 - \delta$, $|\mathbf{K}'_{ij} - \mathbf{K}_{ij}| \leq \epsilon$ for all $i, j \in [1, m]$.

B. Boosting

1. Let $\text{corr}(\mathbf{x}, \mathbf{x}')$ denote the inner product (or unnormalized correlation) of two vectors $\mathbf{x}$ and $\mathbf{x}'$. Prove that the distribution vector $(D_{t+1}(1), \dots, D_{t+1}(m))$ defined by AdaBoost and the vector of components $y_i h_t(x_i)$ are uncorrelated.
2. Fix $\epsilon \in (0, 1/2)$. Let the training sample be defined by m points in the plane with $\frac{m}{4}$ negative points all at coordinate $(1, 1)$, another set of $\frac{m}{4}$ negative points all at coordinate $(-1, -1)$, $\frac{m(1-\epsilon)}{4}$ positive points all at coordinate $(1, -1)$, and $\frac{m(1+\epsilon)}{4}$ positive points all at coordinate $(-1, +1)$. Describe the behavior of AdaBoost when run on this sample using boosting stumps, in particular, give the solution the algorithm returns after T rounds.

Mehryar Mohri
 Foundations of Machine Learning
 Courant Institute of Mathematical Sciences
 Homework assignment 3 - solution
 April 14, 2011
 Due: May 02, 2011

A. Kernels

1. Show that for all $\lambda > 0$, the kernel K defined on $\mathbb{R} \times \mathbb{R}$ by

$$K(x, x') = \exp(-\lambda[\sin(x' - x)]^2), \quad (1)$$

for all $x, x' \in \mathbb{R}$ is PDS (*hint*: you could seek to rewrite $[\sin(x' - x)]^2$ as the square of the norm of the difference of two vectors).

For all $x, x' \in \mathbb{R}$,

$$\begin{aligned} [\sin(x' - x)]^2 &= 1 - [\cos(x' - x)]^2 \\ &= 1 - [\cos x' \cos x + \sin x' \sin x]^2 \\ &= 1 - (\mathbf{u}(x') \cdot \mathbf{u}(x))^2, \end{aligned}$$

where $\mathbf{u}(x) = (\cos x, \sin x)^\top$ for all $x \in X$. Observe that $\|\mathbf{u}(x)\| = 1$ for all $x \in X$. Thus, $[\sin(x' - x)]^2 = \frac{1}{2}\|\mathbf{u}(x') - \mathbf{u}(x)\|^2$ and $K(x, x') = e^{-\frac{\lambda}{2}\|\mathbf{u}(x') - \mathbf{u}(x)\|^2}$. Since the Gaussian kernel is known to be PDS, K is also PDS (the fact that Gaussian kernels are PDS can be shown easily by observing that they are the normalized kernels associated to the kernel obtained by composing $\exp$ with standard inner products).

2. Let $\Phi: X \rightarrow H$ be a feature mapping such that the dimension N of H is very high and let $K: X \times X \rightarrow \mathbb{R}$ be a PDS kernel defined by

$$K(x, x') = \mathbb{E}_{i \sim D} [\Phi(x)_i \Phi(x')_i], \quad (2)$$

where $[\Phi(x)]_i$ is the i th component of $\Phi(x)$ and similarly for $\Phi(x')$ and where D is a distribution over the indices i . We shall assume that $|[\Phi(x)]_i| \leq R$ for all $x \in X$ and $i \in [1, N]$.

Suppose that to compute $K(x, x')$ no other method is available than computing the inner product (2), which would require $O(N)$ time. One idea in that case is instead to compute an approximation of that kernel based on random

selection of a subset I of the N components of $\Phi(x)$ and $\Phi(x')$ according to D , that is:

$$K'(x, x') = \frac{1}{n} \sum_{i \in I} [\Phi(x)]_i [\Phi(x')]_i, \quad (3)$$

where $|I| = n$.

(a) Fix x and x' in X . Prove that

$$\Pr_{I \sim D^n} [|K(x, x') - K'(x, x')| > \epsilon] \leq 2e^{-\frac{n\epsilon^2}{2R^4}}. \quad (4)$$

(*hint*: use McDiarmid's inequality).

By definition of K' , for all $x, x' \in X$, $E_D[K'(x, x')] = K(x, x')$. Replacing one indice $i \in I$ by i' affects $K'(x, x')$ by at most $\frac{1}{n} (|[\Phi(x)]_i [\Phi(x')]_i| + |[\Phi(x)]_{i'} [\Phi(x')]_{i'}|) \leq \frac{2R^2}{n}$. The result thus follows directly McDiarmid's inequality.

(b) Let $\mathbf{K}$ and $\mathbf{K}'$ be the kernel matrices associated to K and K' . Show that for any $\epsilon, \delta > 0$, for $n > \frac{2R^4}{\epsilon^2} \log \frac{m(m+1)}{\delta}$, with probability at least $1 - \delta$, $|\mathbf{K}'_{ij} - \mathbf{K}_{ij}| \leq \epsilon$ for all $i, j \in [1, m]$.

Since $\mathbf{K}$ and $\mathbf{K}'$ are symmetric, it suffices to prove the statement for the entries of these matrices that are above the diagonal. By the union bound and the previous question, the following holds:

$$\Pr_{I \sim D^n} [\exists i, j \in [1, m]: |\mathbf{K}'_{ij} - \mathbf{K}_{ij}| > \epsilon] \leq 2 \frac{m(m+1)}{2} e^{-\frac{n\epsilon^2}{2R^4}} = m(m+1) e^{-\frac{n\epsilon^2}{2R^4}}.$$

Setting $\delta > 0$ to match the upper bound leads directly to the inequality claimed.

B. Boosting

1. Let $\text{corr}(\mathbf{x}, \mathbf{x}')$ denote the inner product (or unnormalized correlation) of two vectors $\mathbf{x}$ and $\mathbf{x}'$. Prove that the distribution vector $(D_{t+1}(1), \dots, D_{t+1}(m))$ defined by AdaBoost and the vector of components $y_i h_t(x_i)$ are uncorrelated.

By definition, the unnormalized correlation is given by

$$\sum_{i=1}^m D_{t+1}(i) y_i h_t(x_i) = \sum_{i=1}^m \frac{D_t(i) e^{-\alpha_t y_i h_t(x_i)} y_i h_t(x_i)}{Z_t} = \frac{1}{Z_t} \frac{dZ_t}{d\alpha_t}, \quad (5)$$

since $Z_t = \sum_{i=1}^m D_t(i) e^{-\alpha_t y_i h_t(x_i)}$. Recall from lecture slides that α_t minimizes Z_t , thus $\frac{dZ_t}{d\alpha_t} = 0$. This shows that the distribution at round $t+1$ and the vector of margins at round t are uncorrelated.

2. Fix $\epsilon \in (0, 1/2)$. Let the training sample be defined by m points in the plane with $\frac{m}{4}$ negative points all at coordinate $(1, 1)$, another set of $\frac{m}{4}$ negative points all at coordinate $(-1, -1)$, $\frac{m(1-\epsilon)}{4}$ positive points all at coordinate $(1, -1)$, and $\frac{m(1+\epsilon)}{4}$ positive points all at coordinate $(-1, +1)$. Describe the behavior of AdaBoost when run on this sample using boosting stumps, in particular, give the solution the algorithm returns after T rounds.

It is not hard to see that the base hypotheses in this problem can be defined to be threshold functions based on the first or second axis, or constant functions (horizontal or vertical thresholds outside the convex hull of all the points).

The hypotheses selected by AdaBoost are therefore chosen from this set. It can be shown that the hypotheses selected in two consecutive rounds of AdaBoost are distinct. Furthermore, h_t and $-h_t$ cannot be selected in consecutive rounds since misclassified and correctly classified points by h_t are assigned the same distribution mass (see lecture slides). Thus, at each round a distinct hypothesis is chosen. The points at coordinate $(-1, -1)$ are misclassified by all these base hypothesis.

The algorithm should be stopped when the best ϵ_t found is $1/2$. It can be shown then that the error of the final classifier returned on the training set is $\frac{1}{4}(1 - \epsilon)$ since it misclassifies exactly the points at coordinate $(+1, -1)$.

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 3
April 5, 2013
Due: April 19, 2013

A. Kernels

1. Let $\mathcal{X}$ be a finite set. Show that the kernel K defined over $2^{\mathcal{X}}$, the set of subsets of $\mathcal{X}$, by

$$\forall A, B \in 2^{\mathcal{X}}, K(A, B) = \exp\left(-\frac{1}{2}|A \Delta B|\right),$$

where $A \Delta B$ is the symmetric difference of A and B is PDS (*hint*: you could use the fact that K is the result of the normalization of a kernel function K'). Note that this could define a similarity measure for documents based on the set of their common words, or n -grams, or gappy n -grams, or a similarity measure for images based on some patterns, or a similarity measure for graphs based on their common sub-graphs.

2. Let $\mathcal{X}$ be a finite set. Let K_0 be a PDS kernel over $\mathcal{X}$, show that K' defined by

$$\forall A, B \in 2^{\mathcal{X}}, K'(A, B) = \sum_{x \in A, x' \in B} K_0(x, x')$$

is a PDS kernel.

3. Show that K defined by $K(x, x') = \frac{1}{\sqrt{1 - (\mathbf{x} \cdot \mathbf{x}')}} for all $\mathbf{x}, \mathbf{x}' \in X = \{\mathbf{x} \in \mathbb{R}^N : \|\mathbf{x}\|_2 < 1\}$ is a PDS kernel. Bonus point: show that the dimension of the feature space associated to K is infinite (*hint*: one method to show that consists of finding an explicit expression of a feature mapping Φ).$

B. Support Vector Machines

1. Download and install the `libsvm` software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>,

and briefly consult the documentation to become more familiar with the tools.

2. Consider the `splice` data set

<http://www.cs.toronto.edu/~dave/data/splice/desc.html>.

Download the already formatted training and test files of that dataset from

<http://www.cs.nyu.edu/~mohri/ml13/splice.train.txt>

<http://www.cs.nyu.edu/~mohri/ml13/splice.test.txt>.

Use the `libsvm` scaling tool to scale the features of all the data. The scaling parameters should be computed only on the training data and then applied to the test data.

3. Consider the corresponding binary classification which consists of distinguishing two types of splice junctions in DNA sequences using about 60 features. Use SVMs combined with polynomial kernels to tackle this problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let other parameters of polynomial kernels in `libsvm` be equal to their default values), varying C in powers of 10, starting from a small value $C = 10^{-k}$ to $C = 10^k$, for some value of k . k should be chosen so that you see a significant variation in training error, starting from a very high training error to a low training error. Expect longer training times with `libsvm` as the value of C increases.

4. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation error and the test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d . How many of the support vectors lie on the margin hyperplanes?
5. Suppose we replace in the primal optimization problem of SVMs the penalty term $\sum_{i=1}^m \xi_i = \|\xi\|_1$ with $\|\xi\|_2^2$, that is we use the quadratic hinge loss instead. Give the associated dual optimization problem and compare it with the dual optimization problems of SVMs.
6. In class, we presented margin-based generalization bounds in support of the SVM algorithm based on the standard hinge loss. Can you derive similar margin-based generalization bounds when the quadratic hinge loss is used?

To do that, you could use instead of the margin loss function Φ_ρ defined in class the function Ψ_ρ defined by

$$\Psi_\rho(u) = \begin{cases} 1 & \text{if } u \leq 0 \\ \left(\frac{u}{\rho} - 1\right)^2 & \text{if } u \in [0, \rho] \\ 0 & \text{otherwise,} \end{cases}$$

and show that it is a Lipschitz function. Compare the empirical and complexity term of your generalization bound to those given in class using Φ_ρ .

C. Boosting

1. Let $\Psi: \mathbb{R} \rightarrow \mathbb{R}$ denote the function defined by $\Psi(u) = (1 - u)^2 1_{u \leq 1}$. Show that Ψ is an upper bound on the binary loss function and that it is convex and differentiable. Use Ψ to derive a boosting-style algorithm as in the case of the exponential function used in AdaBoost using coordinate descent. Describe the algorithm in detail.
2. Implement that algorithm with boosting stumps and apply the algorithm to the same data set as question B with the same training and test sets. Plot the average cross-validation error plus or minus one standard deviation as a function of the number of rounds of boosting T by selecting the value of this parameter out of $\{10, 10^2, \dots, 10^k\}$ for a suitable value of k , as in question B. Let T^* be the best value found for the parameter. Plot the error on the training and test set as a function of the number of rounds of boosting for $t \in [1, T^*]$. Compare your results with those obtained using SVMs in question B.

Mehryar Mohri
 Foundations of Machine Learning
 Courant Institute of Mathematical Sciences
 Homework assignment 3
 April 5, 2013
 Due: April 19, 2013

A. Kernels

1. Let $\mathcal{X}$ be a finite set. Show that the kernel K defined over $2^{\mathcal{X}}$, the set of subsets of $\mathcal{X}$, by

$$\forall A, B \in 2^{\mathcal{X}}, K(A, B) = \exp\left(-\frac{1}{2}|A \Delta B|\right),$$

where $A \Delta B$ is the symmetric difference of A and B is PDS (*hint*: you could use the fact that K is the result of the normalization of a kernel function K'). Note that this could define a similarity measure for documents based on the set of their common words, or n -grams, or gappy n -grams, or a similarity measure for images based on some patterns, or a similarity measure for graphs based on their common sub-graphs.

Solution: $k: (A, B) \mapsto |A \cap B|$ is a PDS kernel over $2^{\mathcal{X}}$ since $k(A, B) = \sum_{x \in \mathcal{X}} 1_A(x) 1_B(x) = \Phi(A) \cdot \Phi(B)$, where, for any subset A , $\Phi(A) \in \{0, 1\}^{|\mathcal{X}|}$ is the vector whose coordinate indexed by $x \in \mathcal{X}$ is 1 if $x \in A$, 0 otherwise. Since k is PDS, $K' = \exp(k)$ is also PDS (theorem presented in class: PDS property preserved by composition with power series). Since K is the result of the normalization of K' , it is also PDS. $\square$

2. Let $\mathcal{X}$ be a finite set. Let K_0 be a PDS kernel over $\mathcal{X}$, show that K' defined by

$$\forall A, B \in 2^{\mathcal{X}}, K'(A, B) = \sum_{x \in A, x' \in B} K_0(x, x')$$

is a PDS kernel.

Solution: Let Φ_0 be a feature mapping associated to k_0 , then $K'(A, B) = \sum_{x \in A, x' \in B} \Phi_0(x) \cdot \Phi_0(x') = \left(\sum_{x \in A} \Phi_0(x)\right) \cdot \left(\sum_{x' \in B} \Phi_0(x')\right) = \Psi(A) \cdot \Psi(B)$, where for any subset A , $\Psi(A) = \sum_{x \in A} \Phi_0(x)$. $\square$

3. Show that K defined by $K(x, x') = \frac{1}{\sqrt{1-(\mathbf{x} \cdot \mathbf{x}')}} for all $\mathbf{x}, \mathbf{x}' \in X = \{\mathbf{x} \in \mathbb{R}^N : \|\mathbf{x}\|_2 < 1\}$ is a PDS kernel. Bonus point: show that the dimension of the feature space associated to K is infinite (*hint*: one method to show that consists of finding an explicit expression of a feature mapping Φ).$

Solution: $f: x \mapsto \frac{1}{\sqrt{1-x}}$ admits the Taylor series expansion

$$f(x) = \sum_{n=0}^{\infty} \binom{1/2}{n} (-1)^n x^n$$

for $|x| < 1$, where $\binom{1/2}{n} = \frac{\frac{1}{2}(\frac{1}{2}-1)\cdots(\frac{1}{2}-n+1)}{n!}$. Observe that $\binom{1/2}{n}(-1)^n > 0$ for all $n \geq 0$, thus, the coefficients in the power series expansion are all positive. Since the radius of the convergence of the series is one and that by the Cauchy-Schwarz inequality $|\mathbf{x}' \cdot \mathbf{x}| \leq \|\mathbf{x}'\| \|\mathbf{x}\| < 1$ for $\mathbf{x}, \mathbf{x}' \in X$, by the closure property theorem presented in class, $(\mathbf{x}, \mathbf{x}') \mapsto f(\mathbf{x}' \cdot \mathbf{x})$ is a PDS kernel.

Now, let $a_n = \binom{1/2}{n}(-1)^n$. Then, for $\mathbf{x}, \mathbf{x}' \in X$,

$$\begin{aligned} f(\mathbf{x}' \cdot \mathbf{x}) &= \sum_{n=0}^{\infty} a_n \left(\sum_{i=1}^N x_i x'_i \right)^n \\ &= \sum_{n=0}^{\infty} a_n \sum_{s_1+\dots+s_N=n} \binom{n}{s_1, \dots, s_N} (x_{i_1} x'_{i_1})^{s_1} \cdots (x_{i_N} x'_{i_N})^{s_N} \\ &= \sum_{s_1, \dots, s_N \geq 0} a_{s_1+\dots+s_N} \binom{s_1+\dots+s_N}{s_1, \dots, s_N} (x_{i_1} x'_{i_1})^{s_1} \cdots (x_{i_N} x'_{i_N})^{s_N}, \end{aligned}$$

where the sums can be permuted since the series is absolutely summable. Thus, we can write $f(\mathbf{x}' \cdot \mathbf{x}) = \Phi(\mathbf{x}) \cdot \Phi(\mathbf{x}')$ with

$$\Phi(\mathbf{x}) = \left(\sqrt{a_{s_1+\dots+s_N} \binom{s_1+\dots+s_N}{s_1, \dots, s_N}} x_{i_1}^{s_1} \cdots x_{i_N}^{s_N} \right)_{s_1, \dots, s_N \geq 0}.$$

Φ is a mapping to an infinite-dimensional space. □

B. Support Vector Machines

1. Download and install the `libsvm` software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>,

and briefly consult the documentation to become more familiar with the tools.

2. Consider the `splice` data set

<http://www.cs.toronto.edu/~delve/data/splice/desc.html>.

Download the already formatted training and test files of that dataset from

<http://www.cs.nyu.edu/~mohri/ml13/splice.train.txt>

<http://www.cs.nyu.edu/~mohri/ml13/splice.test.txt>.

Use the `libsvm` scaling tool to scale the features of all the data. The scaling parameters should be computed only on the training data and then applied to the test data.

Solution:

```
# Scale training and test set.
libsvm-3.0/svm-scale -s output/scale.txt \
    output/splice.train.txt > output/splice.train.scaled.txt
libsvm-3.0/svm-scale -r output/scale.txt \
    output/splice.test.txt > output/splice.test.scaled.txt
```

3. Consider the corresponding binary classification which consists of distinguishing two types of splice junctions in DNA sequences using about 60 features. Use SVMs combined with polynomial kernels to tackle this problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let other parameters of polynomial kernels in `libsvm` be equal to their default values), varying C in powers of 10, starting from a small value $C = 10^{-k}$ to $C = 10^k$, for some value of k . k should be chosen so that you see a significant variation in training error, starting from a very high training error to a low training error. Expect longer training times with `libsvm` as the value of C increases.

Solution: Figures in this solution section are courtesy of Jinho Jang.

Figure 1 displays the cross validation error for polynomial kernels of degree $d \in \{1, 2, 3, 4\}$ as a function of the regularization parameter C . A direct

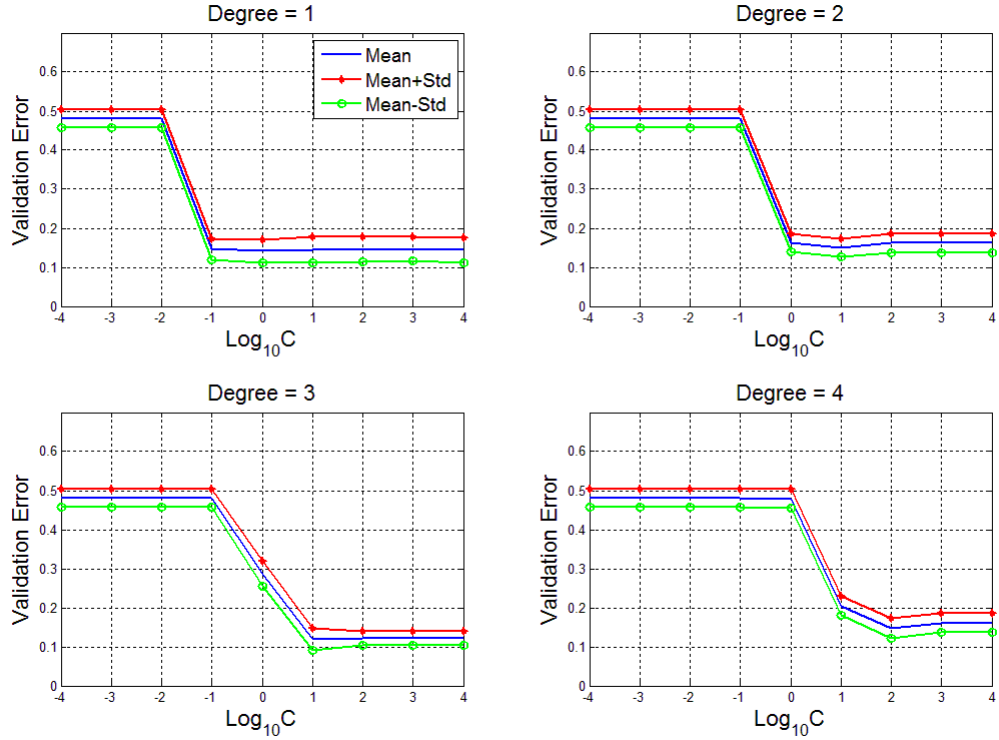


Figure 1: Validation error for SVMs on the DNA data with polynomial kernels of varying degrees $d = \{1, 2, 3, 4\}$ and regularization parameter $C = 10^k$; $k \in \{-4, -3, \dots, 4\}$.

comparison of the mean CV error appears in Figure 2. Generally, we do not see dramatic differences in performance between values of $C > 10^2$ and $d = \{1, 2, 4\}$. As a curiosity, Figure 3 demonstrates that for $C = 10$, we eventually see an overfitting phenomenon as we increase the degree beyond the scope of the assignment.

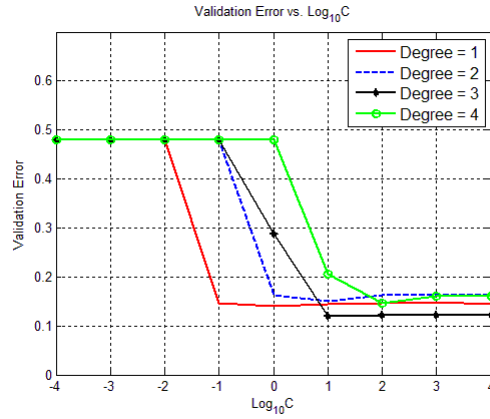


Figure 2: A direct comparison of the polynomial degrees and the validation error for SVMs on the DNA data with polynomial kernels of varying degrees $d = \{1, 2, 3, 4\}$ and regularization parameter $C = 10^k$; $k \in \{-4, -3, \dots, 4\}$. We see that once $C \geq 10^2$, there are not dramatic differences between the settings.

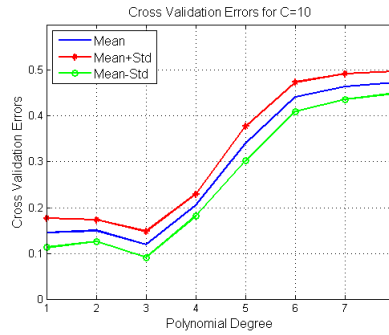


Figure 3: Validation errors for further polynomial degrees beyond those suggested in the problem statement. The algorithm begins to overfit as the polynomial degree increases.

Figure 4: The test and validation error as a function of degree (left panel) as well as the number of total and marginal support vectors (right panel).

4. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation error and the test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d . How many of the support vectors lie on the margin hyperplanes?

Solution: The first plot in Figure 4 shows that the test error decreases (slightly) with an increase in degree and also that the cross-validation error is (slightly) optimistic when compared to the test error on the held-out dataset.

The second plot shows that the total number of marginal support vectors increases with d (as does the dimension of the feature space) while the total number of overall support vectors decreases. This also implies that the number of support vectors due to mistakes is decreasing, which agrees with the first plot. $\square$

5. Suppose we replace in the primal optimization problem of SVMs the penalty term $\sum_{i=1}^m \xi_i = \|\boldsymbol{\xi}\|_1$ with $\|\boldsymbol{\xi}\|_2^2$, that is we use the quadratic hinge loss instead. Give the associated dual optimization problem and compare it with the dual optimization problems of SVMs.

Solution: The optimization problem for this version of SVMs can be written as follows:

$$\begin{aligned} \min_{\mathbf{w}, b, \boldsymbol{\xi}} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i^2 \\ \text{subject to} \quad & \forall i \in [1, m], y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i \wedge \xi_i \geq 0. \end{aligned}$$

The objective function is convex as a sum of the two convex function $\mathbf{w} \mapsto \|\mathbf{w}\|^2$ and $\boldsymbol{\xi} \mapsto \|\boldsymbol{\xi}\|^2$ and is infinitely differentiable. The constraints are affine functions. Slater's condition holds, the KKT theorem applies, and strong duality holds. The corresponding Lagrange function can be written as

$$L = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i^2 - \sum_{i=1}^m \alpha_i [y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 + \xi_i] - \sum_{i=1}^m \beta_i \xi_i,$$

with dual variables $\alpha_i, \beta_i \geq 0$. Differentiating with respect to the primal

variables gives:

$$\begin{aligned}\nabla_{\mathbf{w}} L &= \mathbf{w} - \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i = 0 \implies \mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i \\ \nabla_b L &= -\sum_{i=1}^m \alpha_i y_i = 0 \implies \sum_{i=1}^m \alpha_i y_i = 0 \\ \nabla_{\xi_i} L &= 2C\xi_i - \alpha_i - \beta_i = 0 \implies \alpha_i + \beta_i = 2C\xi_i.\end{aligned}$$

Plugging in these equalities in L yields:

$$\begin{aligned}L &= -\frac{1}{2}\|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i - C \sum_{i=1}^m \xi_i^2 \\ &= \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) - \frac{1}{2C} \sum_{i=1}^m (\alpha_i + \beta_i)^2.\end{aligned}$$

Thus, this gives the following dual optimization problem:

$$\begin{aligned}\max_{\alpha, \beta} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) - \frac{1}{2C} \sum_{i=1}^m (\alpha_i + \beta_i)^2 \\ \text{subject to} \quad & (\alpha \geq 0) \wedge (\beta \geq 0) \wedge \left(\sum_{i=1}^m \alpha_i y_i = 0 \right).\end{aligned}$$

The maximization over β clearly gives $\beta = 0$. This corresponds to the fact that the conditions $\xi_i \geq 0$ were in fact not necessary. In view of that, the dual optimization problem becomes

$$\begin{aligned}\max_{\alpha} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) - \frac{1}{2C} \sum_{i=1}^m \alpha_i^2 \\ \text{subject to} \quad & (\alpha \geq 0) \wedge \left(\sum_{i=1}^m \alpha_i y_i = 0 \right).\end{aligned}$$

Let $\mathbf{K} \in \mathbb{R}^{m \times m}$ denote the kernel matrix associated to the sample, $\mathbf{K}_{ij} = \mathbf{x}_i \cdot \mathbf{x}_j$, and $\tilde{\mathbf{K}}$ the kernel matrix defined by $\tilde{\mathbf{K}} = \mathbf{K} + \frac{1}{2C} \mathbf{I}$. Then, the problem can be equivalently written as

$$\begin{aligned}\max_{\alpha} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \tilde{\mathbf{K}}_{ij} \\ \text{subject to} \quad & (\alpha \geq 0) \wedge \left(\sum_{i=1}^m \alpha_i y_i = 0 \right).\end{aligned}$$

The dual problem therefore coincides with that of SVMs in the separable case modulo the change of the kernel matrix. $\square$

6. In class, we presented margin-based generalization bounds in support of the SVM algorithm based on the standard hinge loss. Can you derive similar margin-based generalization bounds when the quadratic hinge loss is used? To do that, you could use instead of the margin loss function Φ_ρ defined in class the function Ψ_ρ defined by

$$\Psi_\rho(u) = \begin{cases} 1 & \text{if } u \leq 0 \\ \left(\frac{u}{\rho} - 1\right)^2 & \text{if } u \in [0, \rho] \\ 0 & \text{otherwise,} \end{cases}$$

and show that it is a Lipschitz function. Compare the empirical and complexity term of your generalization bound to those given in class using Φ_ρ .

Solution: Ψ_ρ is differentiable over $]0, \rho[$ and its differential at u is $\Psi'_\rho(u) = \frac{2}{\rho}(\frac{u}{\rho} - 1)$, thus $|\Psi'_\rho(u)| \leq \frac{2}{\rho}$. This can be used to show straightforwardly that Ψ_ρ is $\frac{2}{\rho}$ -Lipschitz. In view of that, using the same proof techniques as those used to prove the general margin bound of slide 34, lecture 4, we can conclude that for any $\delta > 0$, with probability at least $1 - \delta$,

$$R(h) \leq \frac{1}{m} \sum_{i=1}^m \Psi_\rho(y_i h(x_i)) + \frac{4}{\rho} \mathfrak{R}_m(H) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}.$$

Note that $\Psi_\rho = \Phi_\rho^2$ and that $\Psi_\rho \leq \Phi_\rho$, thus the empirical term in this bound is always smaller than the one in the bound presented in class. But, the complexity term is multiplied by a factor of 2: $\frac{4}{\rho} \mathfrak{R}_m(H)$ instead of $\frac{2}{\rho} \mathfrak{R}_m(H)$. $\square$

C. Boosting

1. Let $\Psi: \mathbb{R} \rightarrow \mathbb{R}$ denote the function defined by $\Psi(u) = (1 - u)^2 1_{u \leq 1}$. Show that Ψ is an upper bound on the binary loss function and that it is convex and differentiable. Use Ψ to derive a boosting-style algorithm as in the case of the exponential function used in AdaBoost using coordinate descent. Describe the algorithm in detail.

Solution: Since $\Psi \geq 0$, for $u > 0$, the inequality $\Psi(u) \geq 1_{u \leq 0}(u)$ holds. For $u \leq 0$, $(1 - u)^2 \geq (1 - 0)^2 = 1_{u \leq 0}(u)$. Thus, Ψ is an upper bound on $1_{u \leq 0}$.

The functions $u \mapsto (1 - u)^2$ and $u \mapsto 0$ are both continuously differentiable and admit the same differential value, 0, at $u = 1$. Ψ is thus continuously differentiable. For $u \leq 1$, $\Psi'(u) = -2(1 - u)$ and for $u \geq 1$, $\Psi'(u) = 0$, thus Ψ' is non-decreasing and Ψ is convex.

We adopt the notation already used in class. The objective function becomes

$$F(\alpha) = \sum_{i=1}^m \Psi(y_i f(x_i)) = \sum_{i=1}^m \Psi\left(y_i \sum_{t=1}^T \alpha_t h_t(x_i)\right).$$

We denote by f_t the function defined after t rounds of boosting: $f_t = \sum_{s=1}^t \alpha_s h_s$. For convenience, we also define $f_0 = 0$. Note that since $\Psi' \leq 0$, $\sum_{i=1}^m \Psi'(y_i f_t(x_i)) = 0$ implies that $\Psi'(y_i f_t(x_i)) = 0$ for all $i \in [1, m]$, that is $y_i f_t(x_i) \geq 1$. f_t is then correctly classifying all points with margin 1. If that occurs, we can stop boosting. In the following, we will therefore assume that $\sum_{i=1}^m \Psi'(y_i f_t(x_i)) \neq 0$ for all $t \in [1, T]$ where T is the minimum of a predefined number of rounds and the last round at which this condition holds.

The original distribution D_1 is the uniform distribution over the sample: $D_1(i) = \frac{1}{m}$ for all $i \in [1, m]$. The distribution D_t at the end of the $(t - 1)$ th round, $t \in [2, T]$ is defined by

$$D_t(i) = \frac{\Psi'(y_i f_{t-1}(x_i))}{\sum_{i=1}^m \Psi'(y_i f_{t-1}(x_i))}.$$

Let ϵ_t denote the coordinate descent direction at time t , which is determined by $\mathbf{e}_t = \operatorname{argmin}_{\eta} \frac{dF(\alpha_{t-1} + \eta \mathbf{e}_t)}{d\eta} \Big|_{\eta=0}$. By definition,

$$F(\alpha_{t-1} + \eta \mathbf{e}_t) = \sum_{i=1}^m \Psi\left(-y_i \sum_{s=1}^{t-1} \alpha_s h_s(x_i) - \eta y_i h_t(x_i)\right).$$

Then, we can write

$$\begin{aligned} \frac{dF(\alpha_{t-1} + \eta \mathbf{e}_t)}{d\eta} \Big|_{\eta=0} &= \sum_{i=1}^m y_i h_t(x_i) \Psi'\left(y_i \sum_{s=1}^{t-1} \alpha_s h_s(x_i)\right) \\ &= \sum_{i=1}^m y_i h_t(x_i) D_t(i) \left[\sum_{i=1}^m \Psi'(y_i f_{t-1}(x_i)) \right] \\ &= [(1 - \epsilon_t) - \epsilon_t] \left[\sum_{i=1}^m \Psi'(y_i f_{t-1}(x_i)) \right] \\ &= [2\epsilon_t - 1] \left[- \sum_{i=1}^m \Psi'(y_i f_{t-1}(x_i)) \right], \end{aligned}$$

```

SQUARELOSSBOOST( $S = ((x_1, y_1), \dots, (x_m, y_m))$ )
1  for  $i \leftarrow 1$  to  $m$  do
2       $D_1(i) \leftarrow \frac{1}{m}$ 
3  for  $t \leftarrow 1$  to  $T$  do
4       $h_t \leftarrow$  base classifier in  $H$  with small error  $\epsilon_t = \Pr_{D_t}[h_t(x_i) \neq y_i]$ 
5       $\alpha_t \leftarrow \eta \triangleright$  obtained using search using expression of  $\eta$ .
6       $f_t \leftarrow f_{t-1} + \alpha_t h_t$ 
7      for  $i \leftarrow 1$  to  $m$  do
8           $D_{t+1}(i) \leftarrow \frac{\Psi'(y_i f_t(x_i))}{\sum_{i=1}^m \Psi'(y_i f_t(x_i))}$ 
9  return  $h = \text{sign}(f_T)$ 

```

Figure 5: Pseudocode of the boosting algorithm derived.

where $\epsilon_t = \Pr_{i \sim D_t}[h_t(x_i) \neq y_i]$. Since $\Psi' \leq 0$, the base classifier h_t chosen at round t is thus the one with the smallest weighted error ϵ_t according to the distribution D_t .

To determine the step, we search η such that:

$$\frac{dF(\alpha_{t-1} + \eta \mathbf{e}_t)}{d\eta} = 0 \Leftrightarrow \sum_{i=1}^m y_i h_t(x_i) \Psi'(y_i f_{t-1}(x_i) + \eta y_i h_t(x_i)) = 0.$$

Let $J(\eta)$ be defined by $J(\eta) = \{i : y_i f_{t-1}(x_i) + \eta y_i h_t(x_i) < 1\}$. Using $(y_i h_t(x_i))^2 = 1$ The condition can then be rewritten as

$$\begin{aligned} \frac{dF(\alpha_{t-1} + \eta \mathbf{e}_t)}{d\eta} = 0 &\Leftrightarrow \sum_{i=1}^m y_i h_t(x_i) (1 - y_i f_{t-1}(x_i) - \eta y_i h_t(x_i)) = 0 \\ &\Leftrightarrow \eta = \frac{1}{|J(\eta)|} \sum_{i \in J(\eta)} y_i h_t(x_i) - h_t(x_i) f_{t-1}(x_i) \\ &\Leftrightarrow \eta = \frac{\sum_{i \in J(\eta)} h_t(x_i) (y_i - f_{t-1}(x_i))}{|J(\eta)|}. \end{aligned}$$

This does not provide a closed form solution, but the best step η_t at time t can be determined using this formula by varying η . The pseudocode of the algorithm is given in Figure 1.

2. Implement that algorithm with boosting stumps and apply the algorithm to the same data set as question B with the same training and test sets. Plot

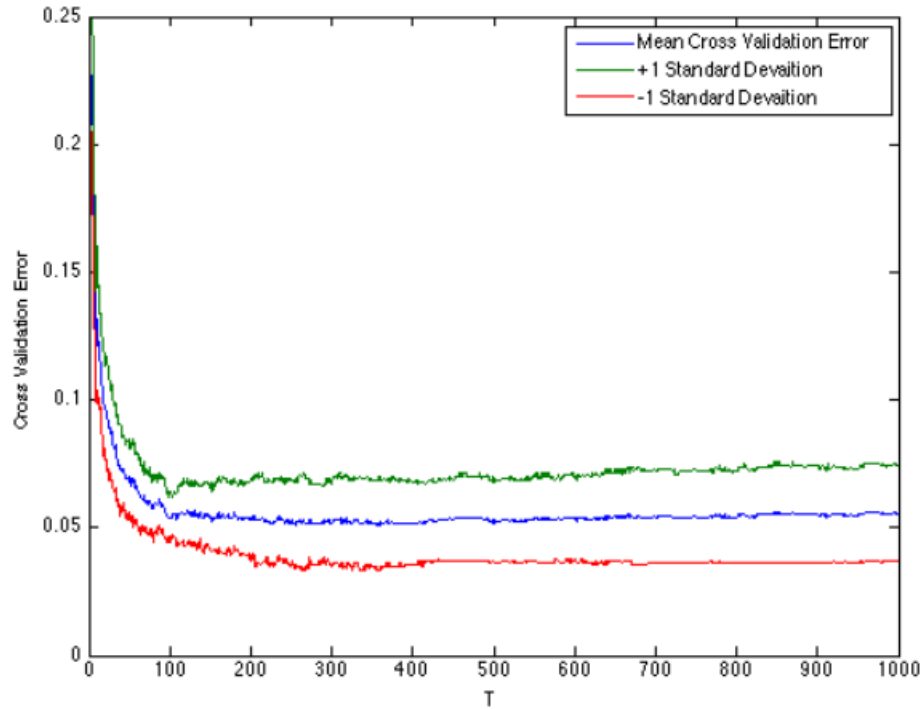


Figure 6: Cross validation error shown with ± 1 standard deviation.

the average cross-validation error plus or minus one standard deviation as a function of the number of rounds of boosting T by selecting the value of this parameter out of $\{10, 10^2, \dots, 10^k\}$ for a suitable value of k , as in question B. Let T^* be the best value found for the parameter. Plot the error on the training and test set as a function of the number of rounds of boosting for $t \in [1, T^*]$. Compare your results with those obtained using SVMs in question B.

Solution:

In figure 6 we see a typical plot of the average cross-validation performance shown with ± 1 standard deviation. Note that the error decreases exponentially and eventually levels out after roughly 150 iterations. In figure ?? we see the training and test error up to and after $T^* = 150$ rounds. Note that the test error eventually levels off, while the training error continues to decrease

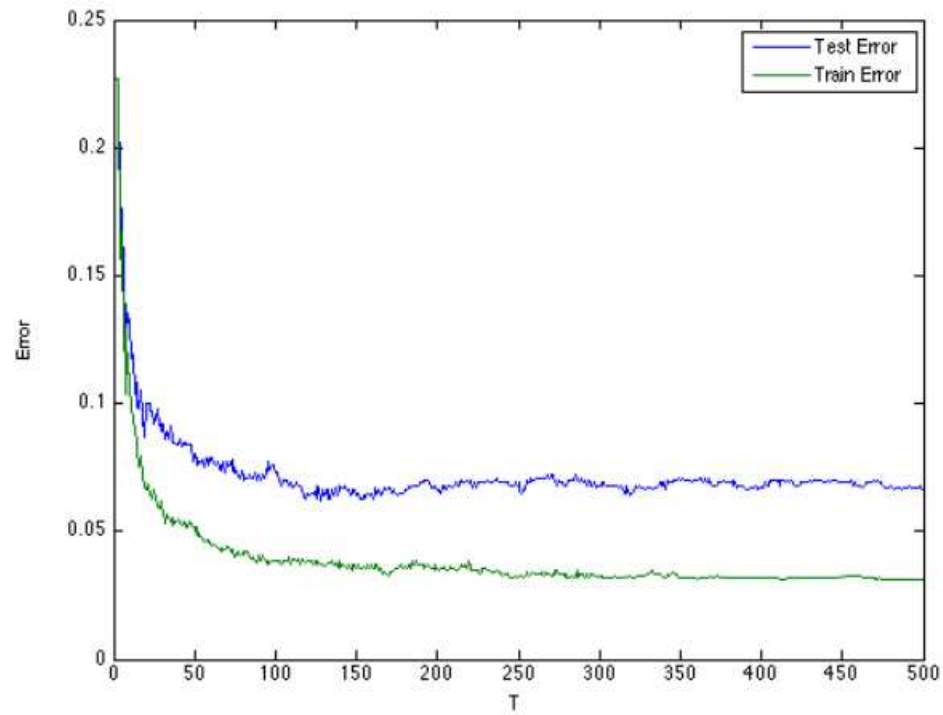


Figure 7: Test and training error for boosting vs number of rounds.

towards zero. Overall, the AdaBoost algorithm performs only slightly better than SVM algorithm increasing accuracy by about 3%. $\square$

Mehryar Mohri
Foundations of Machine Learning 2014
Courant Institute of Mathematical Sciences
Homework assignment 3
October 28, 2014
Due: November 14, 2014

A. Kernels

1. Show that the kernel K defined by

$$\forall (x, y) \in \mathbb{R}^N \times \mathbb{R}^N, \quad K(x, y) = \frac{1}{1 + \frac{\|x-y\|^2}{\sigma^2}}, \quad (1)$$

where $\sigma > 0$ is a parameter, is PDS (*hint*: the function $x \mapsto \int_0^{+\infty} e^{-sx} e^{-s} ds$ defined for all $x \geq 0$ could be useful for the proof).

2. Show that the kernel K defined by

$$\forall (x, y) \in \mathbb{R}^N \times \mathbb{R}^N, \quad K(x, y) = \exp \left(\frac{\sum_{i=1}^N \min(|x_i|, |y_i|)}{\sigma^2} \right), \quad (2)$$

where $\sigma > 0$ is a parameter, is PDS (*hint*: the function $(x_0, y_0) \mapsto \int_0^{+\infty} 1_{t \in [0, |x_0|]} 1_{t \in [0, |y_0|]} dt$ defined over $\mathbb{R} \times \mathbb{R}$ could be useful for the proof).

B. Support Vector Machines

1. Download and install the `libsvm` software library from:

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/> ,

and briefly consult the documentation to become more familiar with the tools.

2. Consider the `splice` data set

<http://www.cs.toronto.edu/~delve/data/splice/desc.html>.

Download the already formatted training and test files of a noisy version of that dataset from

http://www.cs.nyu.edu/~mohri/ml14/splice_noise_train.txt
http://www.cs.nyu.edu/~mohri/ml14/splice_noise_test.txt.

Use the `libsvm` scaling tool to scale the features of all the data. The scaling parameters should be computed only on the training data and then applied to the test data.

3. Consider the corresponding binary classification which consists of distinguishing two types of splice junctions in DNA sequences using about 60 features. Use SVMs combined with polynomial kernels to tackle this problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let other parameters of polynomial kernels in `libsvm` be equal to their default values), varying C in powers of 5, starting from a small value $C = 5^{-k}$ to $C = 5^k$, for some value of k . k should be chosen so that you see a significant variation in training error, starting from a very high training error to a low training error. Expect longer training times with `libsvm` as the value of C increases.

4. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation error and the test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d . How many of the support vectors lie on the marginal hyperplanes? Plot the soft margin of the solution as a function of d .
5. Now, combine SVMs with Gaussian kernels to tackle the same task. Use cross-validation as before to determine the best value of C and σ , varying C in powers of 5, and σ in powers of 2 for a reasonable range so that you see a significant variation in training error, as before. Fix C and σ to the best values found via cross-validation. How does the test error of the solution compare to the best result obtained using polynomial kernels? What is the value of the soft margin?

C. Boosting

As discussed in class, AdaBoost can be viewed as coordinate descent applied to an exponential objective function. Here, we consider an alternative ensemble method algorithm, HingeBoost, that consists of applying coordinate

descent to an objective function based on the hinge loss. Using the same notation as in class, consider the function F defined for all $\alpha \in \mathbb{R}^N$ by

$$F(\alpha) = \sum_{i=1}^m \max \left(0, 1 - y_i \sum_{j=1}^N \alpha_j h_j(x_i) \right), \quad (3)$$

where the h_j s are base classifiers belonging to a hypothesis set H of functions taking values -1 or $+1$.

1. Show that F is convex and admits a right- and left-derivative along any direction.
2. For any $j \in [1, N]$, let $\mathbf{e}_j$ denote the direction corresponding to the base hypothesis h_j . Let α_t denote the vector of coefficients $\alpha_{t,j}$, $j \in [1, N]$ obtained after $t \geq 0$ iterations of coordinate descent and $f_t = \sum_{j=1}^N \alpha_{t,j} h_j$ the predictor obtained after t iterations.
Give the expression of the right-derivative $F'_+(\alpha_{t-1}, \mathbf{e}_j)$ and the left-derivative $F'_-(\alpha_{t-1}, \mathbf{e}_j)$ after $t - 1$ iterations in terms of f_{t-1} .
3. For any $j \in [1, N]$, define the maximum directional derivative $\delta F(\alpha_{t-1}, \mathbf{e}_j)$ at α_{t-1} as follows:

$$\delta F(\alpha_{t-1}, \mathbf{e}_j) = \begin{cases} 0 & \text{if } F'_-(\alpha_{t-1}, \mathbf{e}_j) \leq 0 \leq F'_+(\alpha_{t-1}, \mathbf{e}_j) \\ F'_+(\alpha_{t-1}, \mathbf{e}_j) & \text{if } F'_-(\alpha_{t-1}, \mathbf{e}_j) \leq F'_+(\alpha_{t-1}, \mathbf{e}_j) \leq 0 \\ F'_-(\alpha_{t-1}, \mathbf{e}_j) & \text{if } 0 \leq F'_-(\alpha_{t-1}, \mathbf{e}_j) \leq F'_+(\alpha_{t-1}, \mathbf{e}_j). \end{cases}$$

The direction $\mathbf{e}_j$ considered by the coordinate descent considered here is the one maximizing $|\delta F(\alpha_{t-1}, \mathbf{e}_j)|$. Once the best direction j is selected, the step η can be determined by minimizing $F(\alpha_{t-1} + \eta \mathbf{e}_j)$ using a grid search. Give the pseudocode of HingeBoost.

4. Bonus question: implement HingeBoost and AdaBoost using as base classifiers boosting stumps and compare their performance on the data set of Problem B. The number of rounds of boosting T can be determined via cross-validation varying T in powers of 10. Report the test errors of each algorithm and compare them with those obtained for SVMs in problem B.

Mehryar Mohri
Foundations of Machine Learning 2014
Courant Institute of Mathematical Sciences
Homework assignment 3
October 28, 2014
Due: November 14, 2014

A. Kernels

1. Show that the kernel K defined by

$$\forall (x, y) \in \mathbb{R}^N \times \mathbb{R}^N, \quad K(x, y) = \frac{1}{1 + \frac{\|x-y\|^2}{\sigma^2}}, \quad (1)$$

where $\sigma > 0$ is a parameter, is PDS (*hint*: the function $x \mapsto \int_0^{+\infty} e^{-sx} e^{-s} ds$ defined for all $x \geq 0$ could be useful for the proof).

Solution: For $x \geq 0$, the integral $\int_0^{+\infty} e^{-sx} e^{-s} ds$ is well defined and

$$\int_0^{+\infty} e^{-sx} e^{-s} ds = \int_0^{+\infty} e^{-s(1+x)} ds = \left[-\frac{e^{-s(1+x)}}{1+x} \right]_0^{+\infty} = \frac{1}{1+x}. \quad (2)$$

Since the Gaussian kernel, $(x, y) \mapsto e^{-\frac{\|x-y\|^2}{\sigma^2}}$ is PDS for any $\sigma \neq 0$, the kernel $(x, y) \mapsto \int_0^{+\infty} e^{-\frac{s\|x-y\|^2}{\sigma^2}} e^{-s} ds$ is also PDS since for any $x_1, \dots, x_m$ in $\mathbb{R}^N$ and $c_1, \dots, c_m$ in $\mathbb{R}$,

$$\sum_{i,j=1}^m c_i c_j e^{-\frac{s\|x_i-x_j\|^2}{\sigma^2}} \geq 0 \Rightarrow \int_0^{+\infty} \sum_{i,j=1}^m c_i c_j e^{-\frac{s\|x_i-x_j\|^2}{\sigma^2}} e^{-s} ds \geq 0. \quad (3)$$

By (2), $\int_0^{+\infty} e^{-\frac{s\|x-y\|^2}{\sigma^2}} e^{-s} ds = \frac{1}{1 + \frac{\|x-y\|^2}{\sigma^2}}$ for all x, y in $\mathbb{R}^N$, which concludes the proof. $\square$

2. Show that the kernel K defined by

$$\forall (x, y) \in \mathbb{R}^N \times \mathbb{R}^N, \quad K(x, y) = \exp \left(\frac{\sum_{i=1}^N \min(|x_i|, |y_i|)}{\sigma^2} \right), \quad (4)$$

where $\sigma > 0$ is a parameter, is PDS (*hint*: the function $(x_0, y_0) \mapsto \int_0^{+\infty} 1_{t \in [0, |x_0|]} 1_{t \in [0, |y_0|]} dt$ defined over $\mathbb{R} \times \mathbb{R}$ could be useful for the proof).

Solution: Observe that for all $x, y \in \mathbb{R}$,

$$\int_0^{+\infty} 1_{t \in [0, |x|]} 1_{t \in [0, |y|]} dt = \min\{|x|, |y|\}. \quad (5)$$

Thus, $(x, y) \mapsto \min(|x|, |y|)$ is a PDS kernel over $\mathbb{R} \times \mathbb{R}$ since for any $x_1, \dots, x_m$ in $\mathbb{R}^N$ and $c_1, \dots, c_m$ in $\mathbb{R}$,

$$\begin{aligned} & \sum_{i,j=1}^m c_i c_j \min\{|x_i|, |x_j|\} \\ &= \int_0^{+\infty} \sum_{i,j=1}^m c_i c_j \sum_{k=1}^N 1_{t \in [0, |x_i|]} 1_{t \in [0, |x_j|]} dt \\ &= \int_0^{+\infty} \left\| \begin{bmatrix} c_1 1_{t \in [0, |x_1|]} \\ \vdots \\ c_m 1_{t \in [0, |x_m|]} \end{bmatrix} \right\|^2 dt \geq 0. \end{aligned}$$

Thus, $(x, y) \mapsto \sum_{i=1}^N \min(|x_i|, |y_i|)$ is a PDS kernel over $\mathbb{R}^N \times \mathbb{R}^N$ as a sum of PDS kernels. Its composition with $x \mapsto e^{\frac{x}{\sigma^2}}$ with admits a power series with an infinite radius of convergence and non-negative coefficients is thus also PDS, which concludes the proof. $\square$

B. Support Vector Machines

1. Download and install the `libsvm` software library from:

`http://www.csie.ntu.edu.tw/~cjlin/libsvm/` ,

and briefly consult the documentation to become more familiar with the tools.

2. Consider the `splice` data set

`http://www.cs.toronto.edu/~dave/data/splice/desc.html`.

Download the already formatted training and test files of a noisy version of that dataset from

`http://www.cs.nyu.edu/~mohri/ml14/splice\_noise\_train.txt`
`http://www.cs.nyu.edu/~mohri/ml14/splice\_noise\_test.txt`.

Use the `libsvm` scaling tool to scale the features of all the data. The scaling parameters should be computed only on the training data and then applied to the test data.

3. Consider the corresponding binary classification which consists of distinguishing two types of splice junctions in DNA sequences using about 60 features. Use SVMs combined with polynomial kernels to tackle this problem.

To do that, randomly split the training data into ten equal-sized disjoint sets. For each value of the polynomial degree, $d = 1, 2, 3, 4$, plot the average cross-validation error plus or minus one standard deviation as a function of C (let other parameters of polynomial kernels in `libsvm` be equal to their default values), varying C in powers of 5, starting from a small value $C = 5^{-k}$ to $C = 5^k$, for some value of k . k should be chosen so that you see a significant variation in training error, starting from a very high training error to a low training error. Expect longer training times with `libsvm` as the value of C increases.

Solution:

Figure 1 shows the average cross-validation performance as a function of the regularization parameter C . Note that the algorithm starts to exhibit some over-fitting as C becomes very large. The performance for several choices of d and C are essentially indistinguishable; one suitable choice of optimal parameters is $C^* = 5^1$ and $d^* = 3$. $\square$

4. Let (C^*, d^*) be the best pair found previously. Fix C to be C^* . Plot the ten-fold cross-validation error and the test errors for the hypotheses obtained as a function of d . Plot the average number of support vectors obtained as a function of d . How many of the support vectors lie on the marginal hyperplanes? Plot the soft margin of the solution as a function of d .

Solution:

The first plot in Figure 4 compares CV and test errors for C^* as a function of g . As expected test error is slightly higher than CV error.

The second plot shows that the total number of support vectors and the number of support vectors on the marginal hyperplanes. The last plot shows the margin as a function of g . $\square$

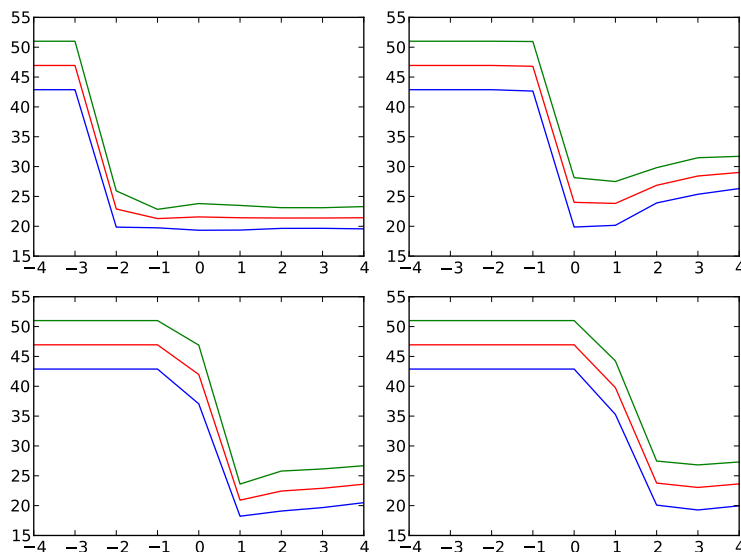


Figure 1: Average error (red) according to 10-fold cross-validation, with error-bars (green and blue) indicating one standard deviation. Top left, top right, bottom left and bottom right correspond to $d = 1, 2, 3, 4$ respectively. On the x -axis we have $\log_5 C$.

5. Now, combine SVMs with Gaussian kernels to tackle the same task. Use cross-validation as before to determine the best value of C and σ , varying C in powers of 5, and σ in powers of 2 for a reasonable range so that you see a significant variation in training error, as before. Fix C and σ to the best values found via cross-validation. How does the test error of the solution compare to the best result obtained using polynomial kernels? What is the value of the soft margin?

Solution:

Figure 3 shows the average cross-validation performance as a function of the regularization parameter C . $C^* = 1$ and $g^* = 0.05$. $\square$

C. Boosting

As discussed in class, AdaBoost can be viewed as coordinate descent applied to an exponential objective function. Here, we consider an alternative en-

semble method algorithm, HingeBoost, that consists of applying coordinate descent to an objective function based on the hinge loss. Using the same notation as in class, consider the function F defined for all $\alpha \in \mathbb{R}^N$ by

$$F(\alpha) = \sum_{i=1}^m \max \left(0, 1 - y_i \sum_{j=1}^N \alpha_j h_j(x_i) \right), \quad (6)$$

where the h_j s are base classifiers belonging to a hypothesis set H of functions taking values -1 or $+1$.

1. Show that F is convex and admits a right- and left-derivative along any direction.

Solution:

Since the hinge loss is convex, its composition with affine function of α is also convex and F is convex as a sum of convex functions.

For the existence of one-sided directional derivatives, one can use the fact that any convex function has one-sided directional derivatives or alternatively, that our specific function is the sum of piecewise affine functions, which are also known to have one-sided directional derivatives (think of one-dimensional hinge loss).

□

2. For any $j \in [1, N]$, let $\mathbf{e}_j$ denote the direction corresponding to the base hypothesis h_j . Let α_t denote the vector of coefficients $\alpha_{t,j}$, $j \in [1, N]$ obtained after $t \geq 0$ iterations of coordinate descent and $f_t = \sum_{j=1}^N \alpha_{t,j} h_j$ the predictor obtained after t iterations. Give the expression of the right-derivative $F'_+(\alpha_{t-1}, \mathbf{e}_j)$ and the left-derivative $F'_-(\alpha_{t-1}, \mathbf{e}_j)$ after $t-1$ iterations in terms of f_{t-1} .

Solution:

Distinguishing different cases depending on the value of $y_i f_{t-1}(x_i) = 1$, it is straightforward to derive the following expressions for all $j \in [1, N]$:

$$F'_+(\alpha_{t-1}, \mathbf{e}_j) = \sum_{i=1}^m -y_i h_j(x_i) [1_{y_i f_{t-1}(x_i) < 1} + 1_{(y_i h_j(x_i) < 0) \wedge (y_i f_{t-1}(x_i) = 1)}]$$

$$F'_-(\alpha_{t-1}, \mathbf{e}_j) = \sum_{i=1}^m -y_i h_j(x_i) [1_{y_i f_{t-1}(x_i) < 1} + 1_{(y_i h_j(x_i) > 0) \wedge (y_i f_{t-1}(x_i) = 1)}].$$

The key here is that when $y_i f_{t-1}(x_i) \neq 1$, each term in the sum will be either 0 or the affine function independent of $y_i h_j(x_i)$. On the other hand, when $y_i f_{t-1}(x_i) = 1$, the sign of $y_i h_j(x_i)$ determines whether the finite differences will extend into the 0 portion of the affine portion of the term. $\square$

3. For any $j \in [1, N]$, define the maximum directional derivative $\delta F(\alpha_{t-1}, \mathbf{e}_j)$ at α_{t-1} as follows:

$$\delta F(\alpha_{t-1}, \mathbf{e}_j) = \begin{cases} 0 & \text{if } F'_-(\alpha_{t-1}, \mathbf{e}_j) \leq 0 \leq F'_+(\alpha_{t-1}, \mathbf{e}_j) \\ F'_+(\alpha_{t-1}, \mathbf{e}_j) & \text{if } F'_-(\alpha_{t-1}, \mathbf{e}_j) \leq F'_+(\alpha_{t-1}, \mathbf{e}_j) \leq 0 \\ F'_-(\alpha_{t-1}, \mathbf{e}_j) & \text{if } 0 \leq F'_-(\alpha_{t-1}, \mathbf{e}_j) \leq F'_+(\alpha_{t-1}, \mathbf{e}_j). \end{cases}$$

The direction $\mathbf{e}_j$ considered by the coordinate descent considered here is the one maximizing $|\delta F(\alpha_{t-1}, \mathbf{e}_j)|$. Once the best direction j is selected, the step η can be determined by minimizing $F(\alpha_{t-1} + \eta \mathbf{e}_j)$ using a grid search. Give the pseudocode of HingeBoost.

Solution: The pseudocode of the HingeBoost algorithm is given in Figure 5. $\square$

4. Bonus question: implement HingeBoost and AdaBoost using as base classifiers boosting stumps and compare their performance on the data set of Problem B. The number of rounds of boosting T can be determined via cross-validation varying T in powers of 10. Report the test errors of each algorithm and compare them with those obtained for SVMs in problem B.

Solution: The test errors of each algorithm should be approximately:

- SVM with Gaussian kernel: 18%
- SVM with Polynomial kernel: 20%
- AdaBoost: 17%
- HingeBoost: 19%

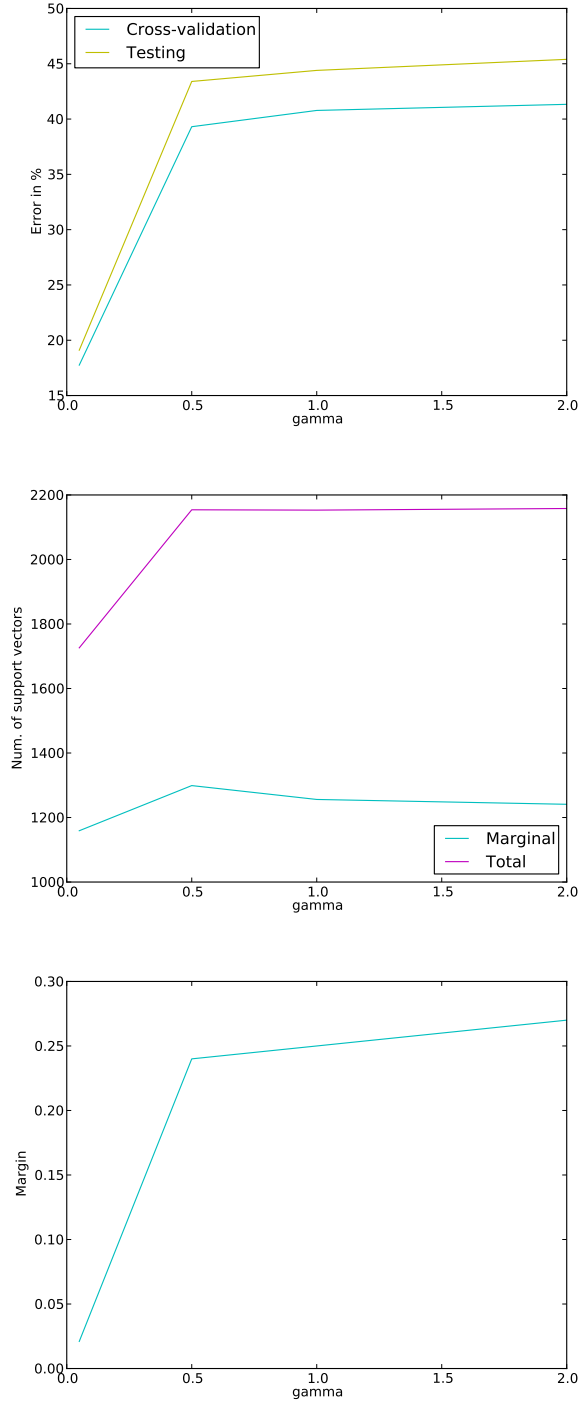


Figure 2: The test and validation error as a function of g (top panel), the number of total and marginal support vectors (second panel) and the margin as function of g .

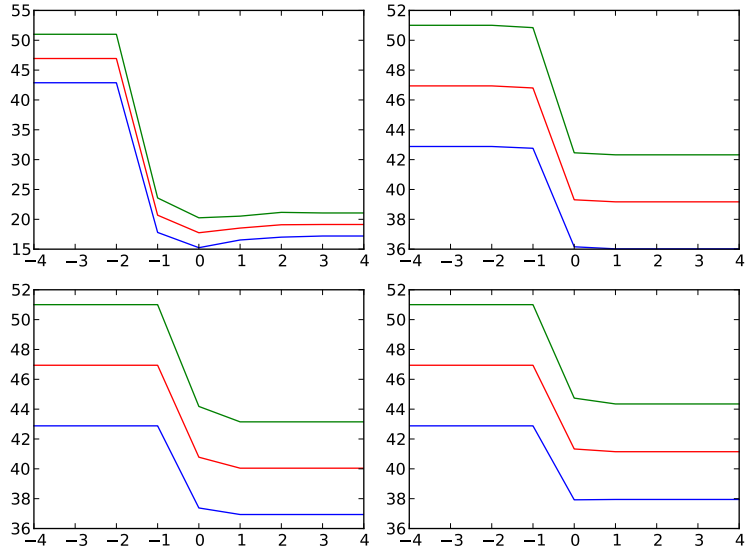


Figure 3: Average error (red) according to 10-fold cross-validation, with error-bars (green and blue) indicating one standard deviation. Top left, top right, bottom left and bottom right correspond to $g = 0.05, 0.5, 1, 2$ respectively. Note that $g = 1/2\sigma^2$. On the x -axis we have $\log_5 C$.

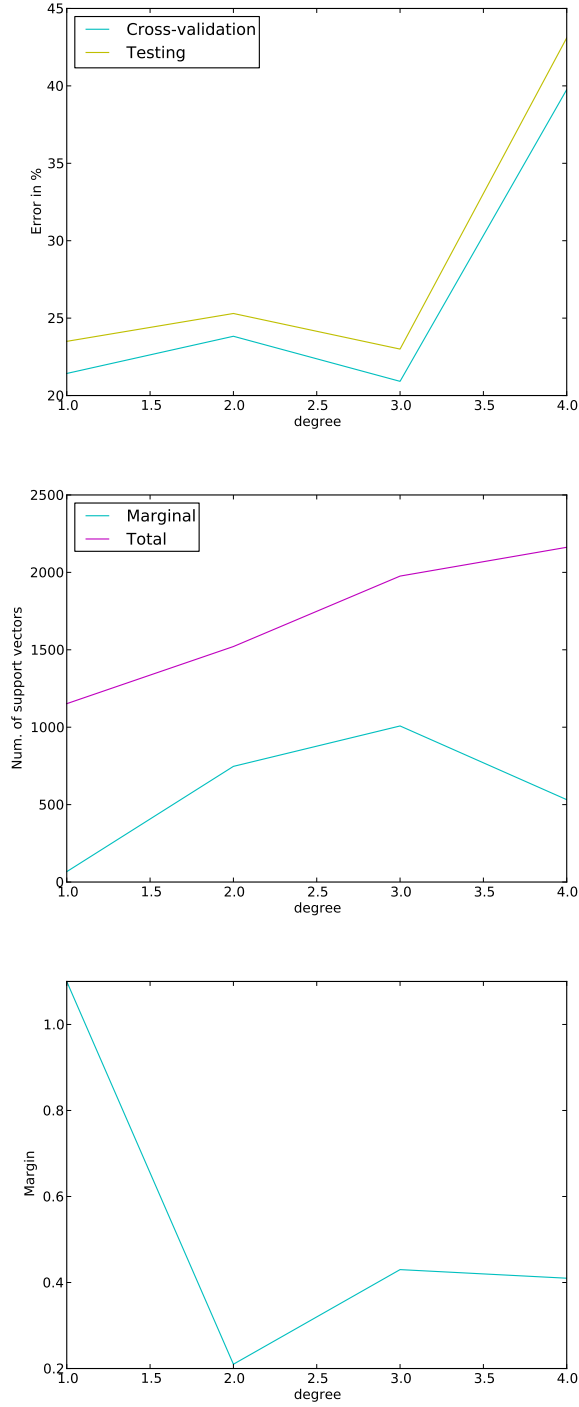


Figure 4: The test and validation error as a function of degree (top panel), the number of total and marginal support vectors (second panel) and the margin as function of d .

```

HINGEBOOST( $S = ((x_1, y_1), \dots, (x_m, y_m))$ )
1   $f \leftarrow 0$ 
2  for  $j \leftarrow 1$  to  $N$  do
3       $r \leftarrow \sum_{i=1}^m -y_i h_j(x_i) [1_{y_i f(x_i) < 1} + 1_{(y_i h_j(x_i) < 0) \wedge (y_i f(x_i) = 1)}]$ 
4       $l \leftarrow \sum_{i=1}^m -y_i h_j(x_i) [1_{y_i f(x_i) < 1} + 1_{(y_i h_j(x_i) > 0) \wedge (y_i f(x_i) = 1)}]$ 
5      if  $(l \leq 0) \wedge (r \geq 0)$  then
6           $d[j] \leftarrow 0$ 
7      elseif  $(l \leq r)$  then
8           $d[j] \leftarrow r$ 
9      else  $d[j] \leftarrow l$ 
10 for  $t \leftarrow 1$  to  $T$  do
11      $k \leftarrow \operatorname{argmin}_{j \in [1, N]} |d[j]|$ 
12      $\eta \leftarrow \operatorname{argmin}_{\eta \geq 0} G(f + \eta h_k) \triangleright \text{line search}$ 
13      $f \leftarrow f + \eta h_k$ 
14 return  $f$ 

```

Figure 5: Pseudocode of the HingeBoost algorithm. The function G is defined for any f by $G(f) = \sum_{i=1}^m \max(0, f(x_i))$.

Mehryar Mohri
Foundations of Machine Learning 2015
Courant Institute of Mathematical Sciences
Homework assignment 3
November 24, 2015
Due: December 07, 2015

A. Boosting-type Algorithm

1. Show that for all $u \in \mathbb{R}$ and integer $p > 1$, $1_{u \leq 0} \leq \Phi_p(-u)$ where $\Phi_p(u) = \max((1+u)^p, 0)$. Show that Φ_p is convex and differentiable.
2. Use Φ_p to derive a boosting-type algorithm using coordinate descent. You should give a full description of your algorithm, including the pseudocode, details for the choice of the step and direction, as well as a generalization bound.

B. L_2 -Regularized Maxent

This problem studies L_2 -regularized Maxent. We will use the notation introduced in class and will denote by J_S the dual objective function minimized given a sample S :

$$J_S(\mathbf{w}) = \frac{\lambda}{2} \|\mathbf{w}\|_2^2 + \mathbb{E}_{x \sim S} [-\log \mathbf{p}_{\mathbf{w}}[x]],$$

where $\lambda > 0$ is a regularization parameter. We will assume that the feature vector is bounded: $\|\Phi(x)\|_2 \leq r$ for all $x \in \mathcal{X}$, for some $r > 0$.

1. Use McDiarmid's inequality to prove that for any $\delta > 0$, with probability at least $1 - \delta$, the following inequality holds:

$$\left\| \mathbb{E}_{x \sim \mathcal{D}} [\Phi(x)] - \mathbb{E}_{x \sim S} [\Phi(x)] \right\|_2 \leq \sqrt{\frac{2r^2}{m}} \left(1 + \sqrt{\log \frac{1}{\delta}} \right).$$

2. Let $\hat{\mathbf{w}}$ be the L_2 -regularized maxent solution for a sample S and $\mathbf{w}_{\mathcal{D}}$ the solution for an infinite sample:

$$\hat{\mathbf{w}} = \operatorname{argmin}_{\mathbf{w} \in \mathbb{R}^N} J_S(\mathbf{w}) \quad \text{and} \quad \mathbf{w}_{\mathcal{D}} = \operatorname{argmin}_{\mathbf{w} \in \mathbb{R}^N} J_{\mathcal{D}}(\mathbf{w}).$$

where $J_{\mathcal{D}}(\mathbf{w}) = \frac{\lambda}{2}\|\mathbf{w}\|_2^2 + \mathbb{E}_{x \sim \mathcal{D}}[-\log \mathbf{p}_{\mathbf{w}}[x]]$. Use the definition of $\hat{\mathbf{w}}$ and $\mathbf{w}_{\mathcal{D}}$ as minimizers (use gradients) to prove that the following inequality holds:

$$\|\hat{\mathbf{w}} - \mathbf{w}_{\mathcal{D}}\|_2 \leq \frac{\left\| \mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] \right\|_2}{\lambda}.$$

3. For any $\mathbf{w}$ and any distribution $\mathcal{Q}$ define $\mathcal{L}_{\mathcal{Q}}(\mathbf{w})$ by $\mathcal{L}_{\mathcal{Q}}(\mathbf{w}) = \mathbb{E}_{x \sim \mathcal{Q}}[-\log \mathbf{p}_{\mathbf{w}}[x]]$. Show that

$$\mathcal{L}_{\mathcal{D}}(\hat{\mathbf{w}}) - \mathcal{L}_{\mathcal{D}}(\mathbf{w}_{\mathcal{D}}) \leq (\hat{\mathbf{w}} - \mathbf{w}_{\mathcal{D}}) \cdot \left[\mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] \right] + \frac{\lambda}{2}\|\mathbf{w}_{\mathcal{D}}\|_2^2 - \frac{\lambda}{2}\|\hat{\mathbf{w}}\|_2^2.$$

4. Use that to show that the following inequality holds for any $\mathbf{w}$:

$$\mathcal{L}_{\mathcal{D}}(\hat{\mathbf{w}}) \leq \frac{1}{\lambda} \left\| \mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] \right\|_2^2 + \mathcal{L}_{\mathcal{D}}(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2.$$

5. Conclude by proving that for any $\delta > 0$, with probability at least $1 - \delta$, the following inequality holds:

$$\mathcal{L}_{\mathcal{D}}(\hat{\mathbf{w}}) \leq \inf_{\mathbf{w} \in \mathbb{R}^N} \mathcal{L}_{\mathcal{D}}(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2 + \frac{2r^2}{\lambda m} \left(1 + \sqrt{\log \frac{1}{\delta}} \right)^2.$$

C. Randomized Halving

In class, we showed that, in the realizable scenario (at least one expert is always correct), the number of mistakes made by Halving is upper bounded by $\log_2 N$. Here, we consider for the same realizable scenario a randomized version of Halving defined as follows.

As for Halving, let H_t denote the set of remaining experts at the beginning of round t , with $H_1 = H$ the full set of N experts. At each round, let r_t be the fraction of experts in H_t predicting 1. Then, the prediction $\hat{y}_t$ made by the algorithm is 1 with probability

$$p_t = \left[\frac{1}{2} \log_2 \frac{1}{1 - r_t} \right] 1_{r_t \leq \frac{3}{4}} + 1_{r_t > \frac{3}{4}},$$

0 with probability $1 - p_t$. The true label y_t is then received and H_{t+1} is derived from H_t by removing all experts who made a mistake.

1. Write the pseudocode of the algorithm.

2. Define the potential function $\Phi_t = \log_2 |H_t|$. Let $\mu_t = 1_{y_t \neq \hat{y}_t}$, prove that for all $t \geq 1$, $E[\mu_t] \leq \frac{\Phi_t - \Phi_{t+1}}{2}$.
3. Show that the expected number of mistakes made by randomized Halving is at most $\frac{1}{2} \log_2 N$.
4. (Bonus question) Prove that no randomized algorithm makes fewer than $\lfloor \frac{1}{2} \log_2 N \rfloor$ mistakes, in expectation.

Mehryar Mohri
Foundations of Machine Learning 2015
Courant Institute of Mathematical Sciences
Homework assignment 3
November 24, 2015
Due: December 07, 2015

A. Boosting-type Algorithm

1. Show that for all $u \in \mathbb{R}$ and integer $p > 1$, $1_{u \leq 0} \leq \Phi_p(-u)$ where $\Phi_p(u) = \max((1+u)^p, 0)$. Show that Φ_p is convex and differentiable.

Solution: We first show that $1_{u \leq 0} \leq \Phi_p(-u)$ for all u . Observe that for $u > 0$, $1_{u \leq 0} = 0 \leq \Phi_p(-u)$ by definition of Φ_p . For $u \leq 0$, $-u \geq 0$ and $\Phi_p(-u) = (1-u)^p \geq 1 = 1_{u \leq 0}$, which proves the desired statement.

Now we show that $\Phi_p(u)$ is differentiable and convex. We consider two cases: p is even and p is odd. If p is even then $\Phi_p(u) = (1+u)^p$ for all u since $(1+u)^p \geq 0$ for all u . Therefore, $\Phi'_p(u) = p(1+u)^{p-1}$ and $\Phi''_p(u) = p(p-1)(1+u)^{p-2}$. Moreover, $p(p-1)(1+u)^{p-2} > 0$ since $p-2$ is even and $p-1 > 0$. This shows that $\Phi_p(u)$ is differentiable and convex in this case.

Now if p is odd then $\Phi_p(u) = 0$ for $u \in (-\infty, -1]$ and $\Phi_p(u) = (1+u)^p$ for $u \in (-1, \infty)$. Therefore, $\Phi'_p(u) = 0$ on $(-\infty, -1)$ and $\Phi'_p(u) = p(1+u)^{p-1}$ for $u \in (-1, \infty)$. To show that Φ_p is differentiable at -1 we consider left and right derivatives:

$$\begin{aligned} \lim_{u \uparrow -1} \frac{\Phi_p(u) - \Phi_p(-1)}{u + 1} &= \lim_{u \uparrow -1} \frac{0}{u + 1} = 0 \\ \lim_{u \downarrow -1} \frac{\Phi_p(u) - \Phi_p(-1)}{u + 1} &= \lim_{u \downarrow -1} \frac{(1+u)^p}{1+u} = \lim_{u \downarrow -1} (1+u)^{p-1} = 0 \end{aligned}$$

Similarly, we observe that $\Phi''_p(u) = 0$ on $(-\infty, -1)$ and $\Phi''_p(u) = p(p-1)(1+u)^{p-2}$ for $u \in (-1, \infty)$. Using the same arguments as for the first derivative and the fact that $p \geq 3$ since p is odd we get

$$\begin{aligned} \lim_{u \uparrow -1} \frac{\Phi'_p(u) - \Phi'_p(-1)}{u + 1} &= \lim_{u \uparrow -1} \frac{0}{u + 1} = 0 \\ \lim_{u \downarrow -1} \frac{\Phi'_p(u) - \Phi'_p(-1)}{u + 1} &= \lim_{u \downarrow -1} \frac{p(1+u)^{p-1}}{1+u} = \lim_{u \downarrow -1} p(1+u)^{p-2} = 0 \end{aligned}$$

and hence $\Phi_p''(-1) = 0$. It follows that $\Phi_p''(u) \geq 0$ for all u and Φ_p is convex.

2. Use Φ_p to derive a boosting-type algorithm using coordinate descent. You should give a full description of your algorithm, including the pseudocode, details for the choice of the step and direction, as well as a generalization bound.

Solution: We assume that we have access to N weak learners $h_1, \dots, h_N$ and the goal is to learn an ensemble hypothesis $g = \sum_{j=1}^N \alpha_j h_j$ and predict according to $\text{sgn}(g)$. Observe that

$$\frac{1}{m} \sum_{i=1}^m 1_{\text{sgn}(g(x_i)) \neq y_i} = \frac{1}{m} \sum_{i=1}^m 1_{y_i g(x_i) \leq 0} \leq \frac{1}{m} \sum_{i=1}^m \Phi_p(-y_i g(x_i))$$

by previous part of this question and our boosting-type algorithm that we provide consists of applying coordinate descent to this convex and differentiable objective. If $F(\boldsymbol{\alpha}_t) = \frac{1}{m} \sum_{i=1}^m \Phi_p(-y_i f_t(x_i))$ and $f_t = \sum_{s=1}^t \alpha_s h_s$ is the solution after t iterations, then at iteration $t+1$ our algorithm picks the direction

$$h_k = \underset{h_j: j=1, \dots, N}{\text{argmin}} \left. F'(\boldsymbol{\alpha}_t + \eta \mathbf{e}_j) \right|_{\eta=0}.$$

That is,

$$h_k = \underset{h_j: j=1, \dots, N}{\text{argmax}} \sum_{i=1}^m y_i h_j(x_i) (1 - y_i f_t(x_i))^{p-1}. \quad (1)$$

Once the direction is determined, the step size α_{t+1} is set by solving

$$F'(\boldsymbol{\alpha}_t + \eta \mathbf{e}_k) = 0 \quad (2)$$

for η . This solution can be found using line search. The pseudocode for this algorithm is given in Algorithm 1. Note that $M_t(i)$ s are used to avoid computing $y_i f_t(x_i)$ from scratch at every iteration t .

As an ensemble method this algorithm enjoys generalization bound of Corollary 6.1 from the textbook.

Algorithm 1 Boosting-type Algorithm.

Inputs: sample $((x_1, y_1), \dots, (x_m, y_m))$.

for $i = 1$ **to** m **do**

$M_1(i) \leftarrow 0$

end for

for $t = 1$ **to** T **do**

$h_t \leftarrow$ solution of (1)

$\alpha_t \leftarrow$ solution of (2)

for $i = 1$ **to** m **do**

$M_{t+1}(i) \leftarrow M_t(i) + y_i \alpha_t h_t(x_i)$

end for

end for

$g \leftarrow \sum_{t=1}^T \alpha_t h_t$

return: $h = \text{sgn}(g)$.

B. L_2 -Regularized Maxent

This problem studies L_2 -regularized Maxent. We will use the notation introduced in class and will denote by J_S the dual objective function minimized given a sample S :

$$J_S(\mathbf{w}) = \frac{\lambda}{2} \|\mathbf{w}\|_2^2 + \mathbb{E}_{x \sim S} [-\log \mathbf{p}_{\mathbf{w}}[x]],$$

where $\lambda > 0$ is a regularization parameter. We will assume that the feature vector is bounded: $\|\Phi(x)\|_2 \leq r$ for all $x \in \mathcal{X}$, for some $r > 0$.

1. Use McDiarmid's inequality to prove that for any $\delta > 0$, with probability at least $1 - \delta$, the following inequality holds:

$$\left\| \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] - \mathbb{E}_{x \sim S}[\Phi(x)] \right\|_2 \leq \sqrt{\frac{2r^2}{m}} \left(1 + \log \sqrt{\frac{1}{\delta}} \right).$$

Solution:

For any sample S , define $\Gamma(S) = \|\mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] - \mathbb{E}_{x \sim S}[\Phi(x)]\|_2$. Let S' be a sample differing from S by one point, say x_m in S and x'_m in S' , then, we can write

$$|\Gamma(S') - \Gamma(S)| \leq \left\| \mathbb{E}_{x \sim S'}[\Phi(x)] - \mathbb{E}_{x \sim S}[\Phi(x)] \right\|_2 \leq \frac{1}{m} \|\Phi(x'_m) - \Phi(x_m)\|_2 \leq \frac{2r}{m}.$$

Thus, for any $\delta > 0$, with probability at least $1 - \delta$,

$$\Gamma(S) \leq \mathbb{E}_{S \sim \mathcal{D}^m} [\Gamma(S)] + \sqrt{\frac{2r^2}{m} \log \frac{1}{\delta}}.$$

Recall that $\mathbb{E}_{x \sim S}[\Phi(x)] = \frac{1}{m} \sum_{i=1}^m \Phi(x_i)$, and denote $X_i = \frac{1}{m} \left[\mathbb{E}_{x \in \mathcal{D}}[\Phi(x)] - \Phi(x_i) \right]$, so that $\sum_{i=1}^m X_i = \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] - \mathbb{E}_{x \sim S}[\Phi(x)]$.

Then, by Jensen's inequality,

$$\begin{aligned} \mathbb{E}_{S \sim \mathcal{D}^m} [\Gamma(S)] &= \mathbb{E}_{S \sim \mathcal{D}^m} \left[\left\| \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] - \mathbb{E}_{x \sim S}[\Phi(x)] \right\|_2 \right] \\ &\leq \sqrt{\mathbb{E}_{S \sim \mathcal{D}^m} \left[\left\| \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] - \mathbb{E}_{x \sim S}[\Phi(x)] \right\|_2^2 \right]} \\ &= \sqrt{\mathbb{E}_{S \sim \mathcal{D}^m} \left[\frac{1}{m^2} \sum_{i,j=1}^m \mathbf{X}_i \cdot \mathbf{X}_j \right]} \\ &= \sqrt{\frac{1}{m^2} \sum_{i=1}^m \mathbb{E}[\|\mathbf{X}_i\|^2]} \quad (\text{for } i \neq j, \mathbb{E}[\mathbf{X}_i \cdot \mathbf{X}_j] = \mathbb{E}[\mathbf{X}_i] \cdot \mathbb{E}[\mathbf{X}_j] = 0) \\ &= \sqrt{\frac{1}{m} \mathbb{E}[\|\mathbf{X}_1\|^2]} \quad (x_i \text{ s drawn i.i.d.}) \\ &= \sqrt{\frac{\mathbb{E}[\|\mathbf{X}_1\|^2] + \mathbb{E}[\|\mathbf{X}_2\|^2]}{2m}} \\ &= \sqrt{\frac{\mathbb{E}[\|\mathbf{X}_1 - \mathbf{X}_2\|^2]}{2m}} \quad (\mathbb{E}[\mathbf{X}_1 \cdot \mathbf{X}_2] = \mathbb{E}[\mathbf{X}_1] \cdot \mathbb{E}[\mathbf{X}_2] = 0) \\ &= \sqrt{\frac{(2r)^2}{2m}} = \sqrt{\frac{2r^2}{m}}. \end{aligned}$$

2. Let $\hat{\mathbf{w}}$ be the L_2 -regularized maxent solution for a sample S and $\mathbf{w}_{\mathcal{D}}$ the solution for an infinite sample:

$$\hat{\mathbf{w}} = \underset{\mathbf{w} \in \mathbb{R}^N}{\operatorname{argmin}} J_S(\mathbf{w}) \quad \text{and} \quad \mathbf{w}_{\mathcal{D}} = \underset{\mathbf{w} \in \mathbb{R}^N}{\operatorname{argmin}} J_{\mathcal{D}}(\mathbf{w}).$$

where $J_{\mathcal{D}}(\mathbf{w}) = \frac{\lambda}{2} \|\mathbf{w}\|_2^2 + \mathbb{E}_{x \sim \mathcal{D}} [-\log \mathbf{p}_{\mathbf{w}}[x]]$. Use the definition of $\hat{\mathbf{w}}$ and $\mathbf{w}_{\mathcal{D}}$ as minimizers (use gradients) to prove that the following inequality holds:

$$\|\hat{\mathbf{w}} - \mathbf{w}_{\mathcal{D}}\|_2 \leq \frac{\left\| \mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] \right\|_2}{\lambda}.$$

Solution:

Define function Q for all $\mathbf{w}$ by $Q(\mathbf{w}) = \log Z = \log \left(\sum_x \exp(\mathbf{w} \cdot \Phi(x)) \right)$. Q is convex as a composition of the log-sum function with an affine function and we can write for any $\mathbf{w}$:

$$\begin{aligned} J_S(\mathbf{w}) &= \frac{\lambda}{2} \|\mathbf{w}\|_2^2 - \mathbf{w} \cdot \mathbb{E}_{x \sim S}[\Phi(x)] + Q(\mathbf{w}) \\ J_D(\mathbf{w}) &= \frac{\lambda}{2} \|\mathbf{w}\|_2^2 - \mathbf{w} \cdot \mathbb{E}_{x \sim D}[\Phi(x)] + Q(\mathbf{w}). \end{aligned}$$

Since the gradient of the objective function is zero at the minimum, we can write

$$\begin{aligned} \nabla J_S(\hat{\mathbf{w}}) &= 0 = \lambda \hat{\mathbf{w}} - \mathbb{E}_{x \sim S}[\Phi(x)] + \nabla Q(\hat{\mathbf{w}}) \\ \nabla J_D(\mathbf{w}_D) &= 0 = \lambda \mathbf{w}_D - \mathbb{E}_{x \sim D}[\Phi(x)] + \nabla Q(\mathbf{w}_D). \end{aligned}$$

Taking the difference yields:

$$\lambda(\mathbf{w}_D - \hat{\mathbf{w}}) = \mathbb{E}_{x \sim D}[\Phi(x)] - \mathbb{E}_{x \sim S}[\Phi(x)] + \nabla Q(\hat{\mathbf{w}}) - \nabla Q(\mathbf{w}_D).$$

Taking the inner product of each side with $\mathbf{w}_D - \hat{\mathbf{w}}$ gives:

$$\begin{aligned} \lambda \|\mathbf{w}_D - \hat{\mathbf{w}}\|_2^2 &= (\mathbf{w}_D - \hat{\mathbf{w}}) \cdot \left[\mathbb{E}_{x \sim D}[\Phi(x)] - \mathbb{E}_{x \sim S}[\Phi(x)] \right] \\ &\quad + (\nabla Q(\hat{\mathbf{w}}) - \nabla Q(\mathbf{w}_D)) \cdot (\mathbf{w}_D - \hat{\mathbf{w}}) \\ &\leq (\mathbf{w}_D - \hat{\mathbf{w}}) \cdot \left[\mathbb{E}_{x \sim D}[\Phi(x)] - \mathbb{E}_{x \sim S}[\Phi(x)] \right] \\ &\leq \|\mathbf{w}_D - \hat{\mathbf{w}}\|_2 \left\| \mathbb{E}_{x \sim D}[\Phi(x)] - \mathbb{E}_{x \sim S}[\Phi(x)] \right\|_2. \quad (\text{Cauchy-Schwarz ineq.}) \end{aligned}$$

where we used $(\nabla Q(\hat{\mathbf{w}}) - \nabla Q(\mathbf{w}_D)) \cdot (\mathbf{w}_D - \hat{\mathbf{w}}) \leq 0$, which holds by the convexity of Q .

3. For any $\mathbf{w}$ and any distribution $\mathcal{Q}$ define $\mathcal{L}_{\mathcal{Q}}(\mathbf{w})$ by $\mathcal{L}_{\mathcal{Q}}(\mathbf{w}) = \mathbb{E}_{x \sim \mathcal{Q}}[-\log \mathbf{p}_{\mathbf{w}}[x]]$. Show that

$$\mathcal{L}_D(\hat{\mathbf{w}}) - \mathcal{L}_D(\mathbf{w}_D) \leq (\hat{\mathbf{w}} - \mathbf{w}_D) \cdot \left[\mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim D}[\Phi(x)] \right] + \frac{\lambda}{2} \|\mathbf{w}_D\|_2^2 - \frac{\lambda}{2} \|\hat{\mathbf{w}}\|_2^2.$$

Solution:

$$\begin{aligned}
& \mathcal{L}_{\mathcal{D}}(\widehat{\mathbf{w}}) - \mathcal{L}_{\mathcal{D}}(\mathbf{w}_{\mathcal{D}}) \\
&= \mathcal{L}_{\mathcal{D}}(\widehat{\mathbf{w}}) - \mathcal{L}_S(\widehat{\mathbf{w}}) + \mathcal{L}_S(\widehat{\mathbf{w}}) - \mathcal{L}_{\mathcal{D}}(\mathbf{w}_{\mathcal{D}}) \\
&= \widehat{\mathbf{w}} \cdot \left[\mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] \right] + \mathcal{L}_S(\widehat{\mathbf{w}}) - \mathcal{L}_{\mathcal{D}}(\mathbf{w}_{\mathcal{D}}) \\
&= \widehat{\mathbf{w}} \cdot \left[\mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] \right] + \mathcal{L}_S(\widehat{\mathbf{w}}) + \frac{\lambda}{2} \|\widehat{\mathbf{w}}\|_2^2 - \frac{\lambda}{2} \|\widehat{\mathbf{w}}\|_2^2 - \mathcal{L}_{\mathcal{D}}(\mathbf{w}_{\mathcal{D}}) \\
&\leq \widehat{\mathbf{w}} \cdot \left[\mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] \right] + \mathcal{L}_S(\mathbf{w}_{\mathcal{D}}) + \frac{\lambda}{2} \|\mathbf{w}_{\mathcal{D}}\|_2^2 - \frac{\lambda}{2} \|\widehat{\mathbf{w}}\|_2^2 - \mathcal{L}_{\mathcal{D}}(\mathbf{w}_{\mathcal{D}}) \\
&= (\widehat{\mathbf{w}} - \mathbf{w}_{\mathcal{D}}) \cdot \left[\mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] \right] + \frac{\lambda}{2} \|\mathbf{w}_{\mathcal{D}}\|_2^2 - \frac{\lambda}{2} \|\widehat{\mathbf{w}}\|_2^2.
\end{aligned}$$

4. Use that to show that the following inequality holds for any $\mathbf{w}$:

$$\mathcal{L}_{\mathcal{D}}(\widehat{\mathbf{w}}) \leq \frac{1}{\lambda} \left\| \mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] \right\|_2^2 + \mathcal{L}_{\mathcal{D}}(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2.$$

Solution:

In view of the previous inequality, for any $\mathbf{w}$, we can write

$$\begin{aligned}
& \mathcal{L}_{\mathcal{D}}(\widehat{\mathbf{w}}) \\
&= (\widehat{\mathbf{w}} - \mathbf{w}_{\mathcal{D}}) \cdot \left[\mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] \right] + \mathcal{L}_{\mathcal{D}}(\mathbf{w}_{\mathcal{D}}) + \frac{\lambda}{2} \|\mathbf{w}_{\mathcal{D}}\|_2^2 - \frac{\lambda}{2} \|\widehat{\mathbf{w}}\|_2^2 \\
&\leq (\widehat{\mathbf{w}} - \mathbf{w}_{\mathcal{D}}) \cdot \left[\mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] \right] + \mathcal{L}_{\mathcal{D}}(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 - \frac{\lambda}{2} \|\widehat{\mathbf{w}}\|_2^2 \\
&\leq \frac{1}{\lambda} \left\| \mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] \right\|_2^2 + \mathcal{L}_{\mathcal{D}}(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 - \frac{\lambda}{2} \|\widehat{\mathbf{w}}\|_2^2 \\
&\leq \frac{1}{\lambda} \left\| \mathbb{E}_{x \sim S}[\Phi(x)] - \mathbb{E}_{x \sim \mathcal{D}}[\Phi(x)] \right\|_2^2 + \mathcal{L}_{\mathcal{D}}(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2.
\end{aligned}$$

5. Conclude by proving that for any $\delta > 0$, with probability at least $1 - \delta$, the following inequality holds:

$$\mathcal{L}_{\mathcal{D}}(\widehat{\mathbf{w}}) \leq \inf_{\mathbf{w} \in \mathbb{R}^N} \mathcal{L}_{\mathcal{D}}(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 + \frac{2r^2}{\lambda m} \left(1 + \sqrt{\log \frac{1}{\delta}} \right)^2.$$

Solution:

This follows immediately the application of the inequality derived in 1).

C. Randomized Halving

In class, we showed that, in the realizable scenario (at least one expert is always correct), the number of mistakes made by Halving is upper bounded by $\log_2 N$. Here, we consider for the same realizable scenario a randomized version of Halving defined as follows.

As for Halving, let H_t denote the set of remaining experts at the beginning of round t , with $H_1 = H$ the full set of N experts. At each round, let r_t be the fraction of experts in H_t predicting 1. Then, the prediction $\hat{y}_t$ made by the algorithm is 1 with probability

$$p_t = \left\lceil \frac{1}{2} \log_2 \frac{1}{1 - r_t} \right\rceil 1_{r_t \leq \frac{3}{4}} + 1_{r_t > \frac{3}{4}},$$

0 with probability $1 - p_t$. The true label y_t is then received and H_{t+1} is derived from H_t by removing all experts who made a mistake.

1. Write the pseudocode of the algorithm.
2. Define the potential function $\Phi_t = \log_2 |H_t|$. Let $\mu_t = 1_{y_t \neq \hat{y}_t}$, prove that for all $t \geq 1$, $\mathbb{E}[\mu_t] \leq \frac{\Phi_t - \Phi_{t+1}}{2}$.

Solution:

If $y_t = 0$, then $\mathbb{E}[\mu_t] = p_t$. In round t , $r_t |H_t|$ experts make a mistake and are removed. Thus, $|H_{t+1}| = (1 - r_t) |H_t|$ and we can write

$$\frac{1}{2}(\Phi_t - \Phi_{t+1}) = \frac{1}{2} \log_2 \frac{|H_t|}{|H_{t+1}|} = \frac{1}{2} \log_2 \frac{1}{1 - r_t} \geq \min \left(\frac{1}{2} \log_2 \frac{1}{1 - r_t}, 1 \right).$$

Observe that for $r_t > \frac{3}{4}$, $\frac{1}{2} \log_2 \frac{1}{1 - r_t} > \frac{1}{2} \log_2 \frac{1}{1 - 3/4} = \frac{1}{2} \log_2 4 = 1$. Thus, $\min \left(\frac{1}{2} \log_2 \frac{1}{1 - r_t}, 1 \right) = p_t$ and $\frac{1}{2}(\Phi_t - \Phi_{t+1}) \geq p_t$.

If $y_t = 1$, then $\mathbb{E}[\mu_t] = 1 - p_t$. In round t , $(1 - r_t) |H_t|$ experts make a mistake and are removed. Thus, $|H_{t+1}| = r_t |H_t|$ and we can write

$$\frac{1}{2}(\Phi_t - \Phi_{t+1}) = \frac{1}{2} \log_2 \frac{1}{r_t} = -\frac{1}{2} \log_2 r_t = 1 - \frac{1}{2} \log_2(4r_t).$$

Thus, for $r_t > \frac{3}{4}$, $\frac{1}{2}(\Phi_t - \Phi_{t+1}) > 1 - \frac{1}{2} \log_2(3) > 0 = 1 - 1 = 1 - r_t$. For $r_t \leq \frac{3}{4}$, using the fact that $x(x - 1) \leq 1/4$ for all $x \in [0, 1]$, we can

write

$$\begin{aligned} 1 - \frac{1}{2} \log_2(4r_t) &= 1 - \frac{1}{2} \log_2 \left(\frac{4r_t(1-r_t)}{1-r_t} \right) \\ &\geq 1 - \frac{1}{2} \log_2 \left(\frac{1}{1-r_t} \right) \\ &= 1 - p_t. \end{aligned}$$

3. Show that the expected number of mistakes made by randomized Halving is at most $\frac{1}{2} \log_2 N$.

Solution:

In view of the previous questions,

$$\sum_{t \geq 1} \mathbb{E}[\mu_t] \leq \frac{1}{2} \sum_{t \geq 1} \Phi_t - \Phi_{t+1} \leq \frac{1}{2} \Phi_1 = \frac{1}{2} \log_2 N.$$

4. (Bonus question) Prove that no randomized algorithm makes fewer than $\lfloor \frac{1}{2} \log_2 N \rfloor$ mistakes, in expectation.

Mehryar Mohri
 Foundations of Machine Learning
 Courant Institute of Mathematical Sciences
 Homework assignment 3
 October 31, 2016
 Due: A. November 11, 2016; B. November 22, 2016

A. Boosting

1. Implement AdaBoost with boosting stumps and apply the algorithm to the `spambase` data set of HW2 with the same training and test sets. Plot the average cross-validation error plus or minus one standard deviation as a function of the number of rounds of boosting T by selecting the value of this parameter out of $\{10, 10^2, \dots, 10^k\}$ for a suitable value of k , as in HW2. Let T^* be the best value found for the parameter. Plot the error on the training and test set as a function of the number of rounds of boosting for $t \in [1, T^*]$. Compare your results with those obtained using SVMs in HW2.
2. Consider the following variant of the classification problem where, in addition to the positive and negative labels $+1$ and -1 , points may be labeled with 0 . This can correspond to cases where the true label of a point is unknown, a situation that often arises in practice, or more generally to the fact that the learning algorithm incurs no loss for predicting -1 or $+1$ for such a point. Let $\mathcal{X}$ be the input space and let $\mathcal{Y} = \{-1, 0, +1\}$. As in standard binary classification, the loss of $f: \mathcal{X} \rightarrow \mathbb{R}$ on a pair $(x, y) \in \mathcal{X} \times \mathcal{Y}$ is defined by $1_{yf(x) < 0}$.

Consider a sample $S = ((x_1, y_1), \dots, (x_m, y_m)) \in (\mathcal{X} \times \mathcal{Y})^m$ and a hypothesis set H of base functions taking values in $\{-1, 0, +1\}$. For a base hypothesis $h_t \in H$ and a distribution D_t over indices $i \in [1, m]$, define ϵ_t^s for $s \in \{-1, 0, +1\}$ by $\epsilon_t^s = \mathbb{E}_{i \sim D_t} [1_{y_i h_t(x_i) = s}]$.

- (a) Derive a boosting-style algorithm for this setting in terms of ϵ_t^s s, using the same objective function as that of AdaBoost. You should carefully justify the definition of the algorithm.
- (b) What is the weak-learning assumption in this setting?
- (c) Write the full pseudocode of the algorithm.
- (d) Give an upper bound on the training error of the algorithm as a function of the number of rounds of boosting and ϵ_t^s s.

B. On-line learning

The objective of this problem is to show how another regret minimization algorithm can be defined and studied. Let L be a loss function convex in its first argument and taking values in $[0, M]$.

We will adopt the notation used in the lectures and assume $N > e^2$. Additionally, for any expert $i \in [1, N]$, we denote by $r_{t,i}$ the instantaneous regret of that expert at time $t \in [1, T]$, $r_{t,i} = L(\hat{y}_t, y_t) - L(y_{t,i}, y_t)$, and by $R_{t,i}$ his cumulative regret up to time t : $R_{t,i} = \sum_{s=1}^t r_{s,i}$. For convenience, we also define $R_{0,i} = 0$ for all $i \in [1, N]$. For any $x \in \mathbb{R}$, $(x)_+$ denotes $\max(x, 0)$, that is the positive part of x , and for $\mathbf{x} = (x_1, \dots, x_N)^\top \in \mathbb{R}^N$, $(\mathbf{x})_+ = ((x_1)_+, \dots, (x_N)_+)^\top$.

Let $\alpha > 2$ and consider the algorithm that predicts at round $t \in [1, T]$ according to $\hat{y}_t = \frac{\sum_{i=1}^N w_{t,i} y_{t,i}}{\sum_{i=1}^N w_{t,i}}$, with the weight $w_{t,i}$ defined based on the α th power of the regret up to time $(t-1)$: $w_{t,i} = (R_{t-1,i})_+^{\alpha-1}$. The potential function we use to analyze the algorithm is based on the function Φ defined over $\mathbb{R}^N$ by $\Phi: \mathbf{x} \mapsto \|(\mathbf{x})_+\|_\alpha^2 = \left[\sum_{i=1}^N (x_i)_+^\alpha \right]^{\frac{2}{\alpha}}$.

1. Show that Φ is twice differentiable over $\mathbb{R}^N - B$, where B is defined as follows:

$$B = \{\mathbf{u} \in \mathbb{R}^N : (\mathbf{u})_+ = 0\}.$$

2. For any $t \in [1, T]$, let $\mathbf{r}_t$ denote the vector of instantaneous regrets, $\mathbf{r}_t = (r_{t,1}, \dots, r_{t,N})^\top$, and similarly $\mathbf{R}_t = (R_{t,1}, \dots, R_{t,N})^\top$. We define the potential function as $\Phi(\mathbf{R}_t) = \|(\mathbf{R}_t)_+\|_\alpha^2$. Compute $\nabla \Phi(\mathbf{R}_{t-1})$ for $\mathbf{R}_{t-1} \notin B$ and show that $\nabla \Phi(\mathbf{R}_{t-1}) \cdot \mathbf{r}_t \leq 0$ (*hint*: use the convexity of the loss with respect to the first argument).
3. (Bonus question) Prove the inequality $\mathbf{r}^\top [\nabla^2 \Phi(\mathbf{u})] \mathbf{r} \leq 2(\alpha - 1) \|\mathbf{r}\|_\alpha^2$ valid for all $\mathbf{r} \in \mathbb{R}^N$ and $\mathbf{u} \in \mathbb{R}^N - B$ (*hint*: write the Hessian $\nabla^2 \Phi(\mathbf{u})$ as a sum of a diagonal matrix and a positive semi-definite matrix multiplied by $(2 - \alpha)$. Also, use Hölder's inequality generalizing Cauchy-Schwarz: for any $p > 1$ and $q > 1$ with $\frac{1}{p} + \frac{1}{q} = 1$ and $\mathbf{u}, \mathbf{v} \in \mathbb{R}^N$, $|\mathbf{u} \cdot \mathbf{v}| \leq \|\mathbf{u}\|_p \|\mathbf{v}\|_q$).
4. Using the answers to the two previous questions and Taylor's formula, show that for all $t \geq 1$, $\Phi(\mathbf{R}_t) - \Phi(\mathbf{R}_{t-1}) \leq (\alpha - 1) \|\mathbf{r}_t\|_\alpha^2$, if $\gamma \mathbf{R}_{t-1} + (1 - \gamma) \mathbf{R}_t \notin B$ for all $\gamma \in [0, 1]$.
5. Suppose there exists $\gamma \in [0, 1]$ such that $(1 - \gamma) \mathbf{R}_{t-1} + \gamma \mathbf{R}_t \in B$. Show that $\Phi(\mathbf{R}_t) \leq (\alpha - 1) \|\mathbf{r}_t\|_\alpha^2$.
6. Using the two previous questions, derive an upper bound on $\Phi(\mathbf{R}_T)$ expressed in terms of T , N , and M .

7. Show that $\Phi(\mathbf{R}_T)$ admits as a lower bound the square of the regret R_T of the algorithm.
8. Using the two previous questions give an upper bound on the regret R_T . For what value of α is the bound the most favorable? Give a simple expression of the upper bound on the regret for a suitable approximation of that optimal value.

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 3
October 31, 2016
Due: A. November 11, 2016; B. November 22, 2016

A. Boosting

1. Implement AdaBoost with boosting stumps and apply the algorithm to the `spambase` data set of HW2 with the same training and test sets. Plot the average cross-validation error plus or minus one standard deviation as a function of the number of rounds of boosting T by selecting the value of this parameter out of $\{10, 10^2, \dots, 10^k\}$ for a suitable value of k , as in HW2. Let T^* be the best value found for the parameter. Plot the error on the training and test set as a function of the number of rounds of boosting for $t \in [1, T^*]$. Compare your results with those obtained using SVMs in HW2.

Solution:

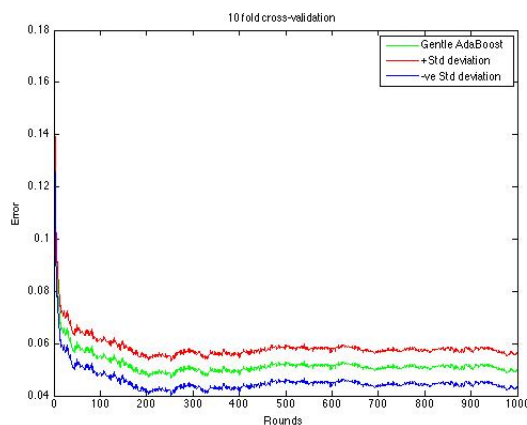


Figure 1: Cross validation error shown with ± 1 standard deviation (Figure courtesy of Chaitanya Rudra).

In figure 1 we see a typical plot of the average cross-validation performance shown with ± 1 standard deviation. Note that the error decreases exponen-

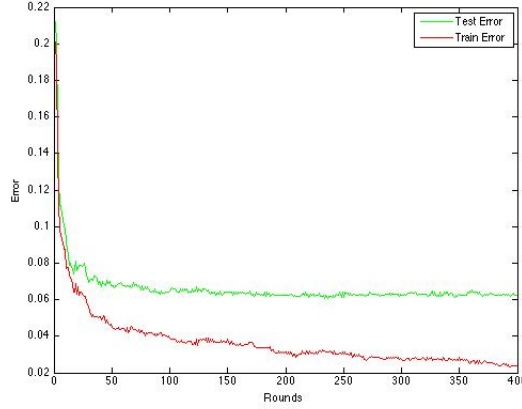


Figure 2: Test and training error (Figure courtesy of Chaitanya Rudra).

tially and eventually levels out after roughly 400 iterations. In figure 2 we see the training and test error after $T^* = 400$ rounds. Note that the test error eventually levels off, while the training error continues to decrease towards zero. Overall, the AdaBoost algorithm performs only slightly better than SVM algorithm increasing accuracy by about 1%.

2. Consider the following variant of the classification problem where, in addition to the positive and negative labels $+1$ and -1 , points may be labeled with 0 . This can correspond to cases where the true label of a point is unknown, a situation that often arises in practice, or more generally to the fact that the learning algorithm incurs no loss for predicting -1 or $+1$ for such a point. Let $\mathcal{X}$ be the input space and let $\mathcal{Y} = \{-1, 0, +1\}$. As in standard binary classification, the loss of $f: \mathcal{X} \rightarrow \mathbb{R}$ on a pair $(x, y) \in \mathcal{X} \times \mathcal{Y}$ is defined by $1_{yf(x) < 0}$.

Consider a sample $S = ((x_1, y_1), \dots, (x_m, y_m)) \in (\mathcal{X} \times \mathcal{Y})^m$ and a hypothesis set H of base functions taking values in $\{-1, 0, +1\}$. For a base hypothesis $h_t \in H$ and a distribution D_t over indices $i \in [1, m]$, define ϵ_t^s for $s \in \{-1, 0, +1\}$ by $\epsilon_t^s = \mathbb{E}_{i \sim D_t} [1_{y_i h_t(x_i) = s}]$.

- (a) Derive a boosting-style algorithm for this setting in terms of ϵ_t^s s, using the same objective function as that of AdaBoost. You should carefully justify the definition of the algorithm.

Solution: Say a ‘boosting-style algorithm’ is just AdaBoost with a pos-

sibly different step size α_t . Recall these definitions from the description of AdaBoost: The final hypothesis is $f(x) = \sum_t \alpha_t h_t(x)$ and the normalization constant in round t is $Z_t = \sum_i D_t(i) \exp(-\alpha_t y_i h_t(x_i))$. We proved in class that

$$\frac{1}{m} \sum_i 1_{y_i f(x_i) < 0} \leq \frac{1}{m} \sum_i \exp(-y_i f(x_i)) = \prod_t Z_t$$

and that AdaBoost's step size can be derived by minimizing this objective in each round t . Taking that same approach, observe that

$$Z_t = \sum_i D_t(i) \exp(-\alpha_t y_i h_t(x_i)) = \epsilon_t^0 + \epsilon_t^- \exp(\alpha_t) + \epsilon_t^+ \exp(-\alpha_t).$$

Differentiating the right-hand side with respect to α_t and setting equal to zero shows that Z_t is minimized by letting $\alpha_t = \frac{1}{2} \log \left(\frac{\epsilon_t^+}{\epsilon_t^-} \right)$.

- (b) What is the weak-learning assumption in this setting?

Solution: One possible assumption is $\frac{\epsilon_t^+ - \epsilon_t^-}{\sqrt{1 - \epsilon_t^0}} \geq \gamma > 0$. Informally, this assumption says that the difference between the accuracy and error of each weak hypothesis is non-negligible relative to the fraction of examples on which the hypothesis makes any prediction at all. In part (d) we will prove that this assumption suffices to drive the training error to zero.

- (c) Write the full pseudocode of the algorithm.

Solution:

1. Given: Training examples $((x_1, y_1), \dots, (x_m, y_m))$.
2. Initialize D_1 to the uniform distribution on training examples.
3. for $t = 1, \dots, T$:
 - a. $h_t \leftarrow$ base classifier in H .
 - b. $\alpha_t \leftarrow \frac{1}{2} \log \left(\frac{\epsilon_t^+}{\epsilon_t^-} \right)$.
 - c. For each $i = 1, \dots, m$: $D_{t+1}(i) \leftarrow \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$, where $Z_t \leftarrow \sum_i D_t(i) \exp(-\alpha_t y_i h_t(x_i))$ is the normalization constant.
 - i. $f \leftarrow \sum_{t=1}^T \alpha_t h_t$.
4. Return: $\text{sign}(f)$.

- (d) Give an upper bound on the training error of the algorithm as a function of the number of rounds of boosting and ϵ_t^s s.

Solution: Plug in the value of α_t from part (a) into $Z_t = \epsilon_t^0 + \epsilon_t^- \exp(\alpha_t) + \epsilon_t^+ \exp(-\alpha_t)$ to obtain $Z_t = \epsilon_t^0 + 2\sqrt{\epsilon_t^- \epsilon_t^+}$. Therefore

$$\frac{1}{m} \sum_i 1_{y_i f(x_i) < 0} \leq \prod_t Z_t = \prod_t \left(\epsilon_t^0 + 2\sqrt{\epsilon_t^- \epsilon_t^+} \right).$$

Moreover, if the weak learning assumption from part (b) is satisfied then

$$\begin{aligned} \epsilon_t^0 + 2\sqrt{\epsilon_t^- \epsilon_t^+} &= \epsilon_t^0 + \sqrt{(1 - \epsilon_t^0)^2 - (\epsilon_t^+ - \epsilon_t^-)^2} \\ &= \epsilon_t^0 + (1 - \epsilon_t^0) \sqrt{1 - \frac{(\epsilon_t^+ - \epsilon_t^-)^2}{(1 - \epsilon_t^0)^2}} \\ &\leq \sqrt{1 - \frac{(\epsilon_t^+ - \epsilon_t^-)^2}{1 - \epsilon_t^0}} \\ &\leq \sqrt{1 - \gamma^2}. \end{aligned}$$

The first equality follows from $(\epsilon_t^+ + \epsilon_t^-)^2 - (\epsilon_t^+ - \epsilon_t^-)^2 = 4\epsilon_t^+ \epsilon_t^-$ (just multiply and gather terms) and $\epsilon_t^+ + \epsilon_t^- = 1 - \epsilon_t^0$. The first inequality follows from the fact that square root is concave on $[0, \infty)$, and thus $\lambda\sqrt{x} + (1 - \lambda)\sqrt{y} \leq \sqrt{\lambda x + (1 - \lambda)y}$ for $\lambda \in [0, 1]$. The last inequality follows from the weak learning assumption.

Therefore we have $\frac{1}{m} \sum_i 1_{y_i f(x_i) < 0} \leq \left(\sqrt{1 - \gamma^2} \right)^T \leq \exp\left(-\frac{\gamma^2 T}{2}\right)$, where we used $1 + x \leq \exp(x)$.

B. On-line learning

The objective of this problem is to show how another regret minimization algorithm can be defined and studied. Let L be a loss function convex in its first argument and taking values in $[0, M]$.

We will adopt the notation used in the lectures and assume $N > e^2$. Additionally, for any expert $i \in [1, N]$, we denote by $r_{t,i}$ the instantaneous regret of that expert at time $t \in [1, T]$, $r_{t,i} = L(\hat{y}_t, y_t) - L(y_{t,i}, y_t)$, and by $R_{t,i}$ his cumulative regret up to time t : $R_{t,i} = \sum_{s=1}^t r_{s,i}$. For convenience, we also define $R_{0,i} = 0$ for all $i \in [1, N]$. For any $x \in \mathbb{R}$, $(x)_+$ denotes $\max(x, 0)$, that is the positive part of x , and for $\mathbf{x} = (x_1, \dots, x_N)^\top \in \mathbb{R}^N$, $(\mathbf{x})_+ = ((x_1)_+, \dots, (x_N)_+)^\top$.

Let $\alpha > 2$ and consider the algorithm that predicts at round $t \in [1, T]$ according to $\hat{y}_t = \frac{\sum_{i=1}^n w_{t,i} y_{t,i}}{\sum_{i=1}^n w_{t,i}}$, with the weight $w_{t,i}$ defined based on the α th power of the regret up to time $(t-1)$: $w_{t,i} = (R_{t-1,i})_+^{\alpha-1}$. The potential function we use to analyze the algorithm is based on the function Φ defined over $\mathbb{R}^N$ by $\Phi: \mathbf{x} \mapsto \|(\mathbf{x})_+\|_\alpha^2 = [\sum_{i=1}^N (x_i)_+^\alpha]^{\frac{2}{\alpha}}$.

1. Show that Φ is twice differentiable over $\mathbb{R}^N - B$, where B is defined as follows:

$$B = \{\mathbf{u} \in \mathbb{R}^N : (\mathbf{u})_+ = 0\}.$$

Solution: For $\mathbf{x} \notin B$, we can write $\Phi(x) = \phi_2(\sum_{i=1}^N \phi_1(x_i))$, where $x_1, \dots, x_N$ denote the components of $\mathbf{x}$ and $\phi_1: \mathbb{R} \rightarrow \mathbb{R}$ and $\phi_2: (0, \infty_+) \rightarrow \mathbb{R}$ are the functions defined by

$$\forall u \in \mathbb{R}, \phi_1(u) = (u)_+^\alpha \quad (1)$$

$$\forall u \in (0, \infty_+), \phi_2(u) = u^{\frac{2}{\alpha}}. \quad (2)$$

It is not hard to see that both ϕ_1 and ϕ_2 are twice (continuously) differentiable, which shows, by composition, the same property for Φ over $\mathbb{R}^N - B$. $\square$

2. For any $t \in [1, T]$, let $\mathbf{r}_t$ denote the vector of instantaneous regrets, $\mathbf{r}_t = (r_{t,1}, \dots, r_{t,N})^\top$, and similarly $\mathbf{R}_t = (R_{t,1}, \dots, R_{t,N})^\top$. We define the potential function as $\Phi(\mathbf{R}_t) = \|(\mathbf{R}_t)_+\|_\alpha^2$. Compute $\nabla \Phi(\mathbf{R}_{t-1})$ for $\mathbf{R}_{t-1} \notin B$ and show that $\nabla \Phi(\mathbf{R}_{t-1}) \cdot \mathbf{r}_t \leq 0$ (*hint: use the convexity of the loss with respect to the first argument*).

Solution: For any $i \in [1, N]$,

$$\frac{\partial \Phi(\mathbf{R}_{t-1})}{\partial R_{t-1,i}} = \frac{2}{\alpha} \alpha (R_{t-1,i})_+^{\alpha-1} \left[\sum_{i=1}^N (R_{t-1,i})_+^\alpha \right]^{\frac{2-\alpha}{\alpha}} = 2w_{t,i} \|(\mathbf{R}_{t-1})_+\|_\alpha^{2-\alpha}. \quad (3)$$

Thus, we can write

$$\begin{aligned} \text{sign}(\nabla \Phi(\mathbf{R}_t) \cdot \mathbf{r}_t) &= \text{sign} \left(\sum_{i=1}^N w_{t,i} r_{t,i} \right) \\ &= \text{sign} \left(\sum_{i=1}^N w_{t,i} (L(\hat{y}_t, y_t) - L(y_{t,i}, y_t)) \right) \\ &= \text{sign} \left(\sum_{i=1}^N \frac{w_{t,i}}{\sum_{i=1}^N w_{t,i}} (L(\hat{y}_t, y_t) - L(y_{t,i}, y_t)) \right). \end{aligned}$$

Now, by the convexity of the first argument of L , the following holds:

$$\begin{aligned} \sum_{i=1}^N \frac{w_{t,i}}{\sum_{i=1}^N w_{t,i}} (L(\hat{y}_t, y_t) - L(y_{t,i}, y_t)) \\ = L(\hat{y}_t, y_t) - \sum_{i=1}^N \frac{w_{t,i}}{\sum_{i=1}^N w_{t,i}} L(y_{t,i}, y_t) \leq 0, \end{aligned}$$

since $\hat{y}_t = \frac{\sum_{i=1}^n w_{t,i} y_{t,i}}{\sum_{i=1}^n w_{t,i}}$. $\square$

3. (Bonus question) Prove the inequality $\mathbf{r}^\top [\nabla^2 \Phi(\mathbf{u})] \mathbf{r} \leq 2(\alpha - 1) \|\mathbf{r}\|_\alpha^2$ valid for all $\mathbf{r} \in \mathbb{R}^N$ and $\mathbf{u} \in \mathbb{R}^N - B$ (hint: write the Hessian $\nabla^2 \Phi(\mathbf{u})$ as a sum of a diagonal matrix and a positive semi-definite matrix multiplied by $(2 - \alpha)$. Also, use Hölder's inequality generalizing Cauchy-Schwarz: for any $p > 1$ and $q > 1$ with $\frac{1}{p} + \frac{1}{q} = 1$ and $\mathbf{u}, \mathbf{v} \in \mathbb{R}^N$, $|\mathbf{u} \cdot \mathbf{v}| \leq \|\mathbf{u}\|_p \|\mathbf{v}\|_q$).

Solution: As already seen, for any $i \in [1, N]$, we have

$$\frac{\partial \Phi(\mathbf{u})}{\partial u_i} = 2(u_i)_+^{\alpha-1} \|(\mathbf{u})_+\|_\alpha^{2-\alpha}. \quad (4)$$

For $j \neq i$, we obtain:

$$\frac{\partial^2 \Phi(\mathbf{u})}{\partial u_j \partial u_i} = 2(2 - \alpha)(u_j)_+^{\alpha-1} (u_i)_+^{\alpha-1} \|(\mathbf{u})_+\|_\alpha^{2(1-\alpha)}. \quad (5)$$

We also have

$$\frac{\partial^2 \Phi(\mathbf{u})}{\partial u_i^2} = 2(\alpha - 1)(u_i)_+^{\alpha-2} \|(\mathbf{u})_+\|_\alpha^{2-\alpha} + 2(2 - \alpha)(u_i)_+^{2(\alpha-1)} \|(\mathbf{u})_+\|_\alpha^{2(1-\alpha)}. \quad (6)$$

Consider the diagonal matrix $\mathbf{D} = \text{diag}((u_1)_+^{\alpha-2}, \dots, (u_N)_+^{\alpha-2})$ and the matrix $\mathbf{M}$ defined by $\mathbf{M}_{ij} = (u_j)_+^{\alpha-1} (u_i)_+^{\alpha-1}$. In view of the previous expressions, we can write

$$\nabla^2 \Phi(\mathbf{u}) = 2(\alpha - 1) \|(\mathbf{u})_+\|_\alpha^{2-\alpha} \mathbf{D} + 2(2 - \alpha) \|(\mathbf{u})_+\|_\alpha^{2(1-\alpha)} \mathbf{M}. \quad (7)$$

$\mathbf{M}$ is a positive semi-definite matrix since $\mathbf{M} = \mathbf{v}\mathbf{v}^\top$ with $\mathbf{v} = (u_1^{\alpha-1}, \dots, u_N^{\alpha-1})^\top$. Thus, for any $\mathbf{r}$, $\mathbf{r}^\top \mathbf{M} \mathbf{r} \geq 0$. Since $2 - \alpha < 0$, we can write

$$\begin{aligned} \mathbf{r}^\top \nabla^2 \Phi(\mathbf{u}) \mathbf{r} &\leq 2(\alpha - 1) \|(\mathbf{u})_+\|_\alpha^{2-\alpha} \mathbf{r}^\top \mathbf{D} \mathbf{r} \\ &= 2(\alpha - 1) \|(\mathbf{u})_+\|_\alpha^{2-\alpha} \sum_{i=1}^N (u_i)_+^{\alpha-2} r_{t,i}^2. \end{aligned}$$

By Hölder's inequality, the following holds:

$$\sum_{i=1}^N (u_i)_+^{\alpha-2} r_{t,i}^2 \leq \left[\sum_{i=1}^N ((u_i)_+^{\alpha-2})^{\frac{\alpha}{\alpha-2}} \right]^{\frac{\alpha-2}{\alpha}} \left[\sum_{i=1}^N (r_i^2)^{\frac{\alpha}{2}} \right]^{\frac{2}{\alpha}} = \|(\mathbf{u})_+\|_{\alpha}^{\alpha-2} \|\mathbf{r}\|_{\alpha}^2.$$

This implies that $\mathbf{r}^\top \nabla^2 \Phi(\mathbf{u}) \mathbf{r} \leq 2(\alpha - 1) \|\mathbf{r}\|_{\alpha}^2$. $\square$

4. Using the answers to the two previous questions and Taylor's formula, show that for all $t \geq 1$, $\Phi(\mathbf{R}_t) - \Phi(\mathbf{R}_{t-1}) \leq (\alpha - 1) \|\mathbf{r}_t\|_{\alpha}^2$, if $\gamma \mathbf{R}_{t-1} + (1 - \gamma) \mathbf{R}_t \notin B$ for all $\gamma \in [0, 1]$.

Solution: By assumption, the segment $[\mathbf{R}_{t-1}, \mathbf{R}_t]$ does not meet B and thus Φ is twice differentiable over the interior $(\mathbf{R}_{t-1}, \mathbf{R}_t)$. Thus, by Taylor's formula, there exists $\mathbf{u} \in (\mathbf{R}_{t-1}, \mathbf{R}_t)$ such that

$$\begin{aligned} \Phi(\mathbf{R}_t) - \Phi(\mathbf{R}_{t-1}) &= \nabla(\mathbf{R}_{t-1}) \cdot \mathbf{r}_t + \frac{1}{2} \mathbf{r}_t^\top \nabla^2(\mathbf{u}) \mathbf{r}_t \leq \frac{1}{2} \mathbf{r}_t^\top \nabla^2(\mathbf{u}) \mathbf{r}_t \\ &\leq (\alpha - 1) \|\mathbf{r}_t\|_{\alpha}^2, \end{aligned}$$

where we used the inequalities obtained in the two previous questions. $\square$

5. Suppose there exists $\gamma \in [0, 1]$ such that $(1 - \gamma) \mathbf{R}_{t-1} + \gamma \mathbf{R}_t \in B$. Show that $\Phi(\mathbf{R}_t) \leq (\alpha - 1) \|\mathbf{r}_t\|_{\alpha}^2$.

Solution: For that γ , by definition, we have $(\mathbf{R}_{t-1} + \gamma \mathbf{r}_t)_+ = \mathbf{0}$. Observe that for any two scalars c and d , $(c + d)_+ \leq c_+ + d_+$ and therefore that

$$(\mathbf{R}_{t-1} + \mathbf{r}_t)_+ \leq (\mathbf{R}_{t-1} + \gamma \mathbf{r}_t)_+ + (1 - \gamma)(\mathbf{r}_t)_+ = (1 - \gamma)(\mathbf{r}_t)_+ \leq (\mathbf{r}_t)_+. \quad (8)$$

$\mathbf{u} \mapsto \|\mathbf{u}\|_{\alpha}^2$ is a non-decreasing function of each component u_i , thus $\Phi(\mathbf{R}_t) = \Phi(\mathbf{R}_{t-1} + \mathbf{r}_t) \leq \|(\mathbf{r}_t)_+\|_{\alpha}^2$. Since $\alpha > 2$, this implies $\Phi(\mathbf{R}_t) \leq (\alpha - 1) \|(\mathbf{r}_t)_+\|_{\alpha}^2$. $\square$

6. Using the two previous questions, derive an upper bound on $\Phi(\mathbf{R}_T)$ expressed in terms of T , N , and M .

Solution: In view of the two previous questions, the inequality $\Phi(\mathbf{R}_t) - \Phi(\mathbf{R}_{t-1}) \leq (\alpha - 1) \|\mathbf{r}_t\|_{\alpha}^2$ holds in all cases. Summing up these inequalities for all $t \in [1, T]$, we obtain

$$\Phi(\mathbf{R}_T) = \Phi(\mathbf{R}_T) - \Phi(\mathbf{R}_0) = \sum_{t=1}^T \Phi(\mathbf{R}_t) - \Phi(\mathbf{R}_{t-1}) \leq (\alpha - 1) \sum_{t=1}^T \|\mathbf{r}_t\|_{\alpha}^2.$$

Since $|r_{t,i}| \leq M$ for all t and i , this yields $\Phi(\mathbf{R}_T) \leq (\alpha - 1) N^{\frac{2}{\alpha}} M^2 T$. $\square$

7. Show that $\Phi(\mathbf{R}_T)$ admits as a lower bound the square of the regret R_T of the algorithm.

Solution: By definition of the regret,

$$R_T = \max_{i \in [1, N]} R_{T,i} \leq \|\mathbf{R}_T\|_\alpha = \sqrt{\Phi(\mathbf{R}_T)}.$$

Note that this implies that $\Phi(\mathbf{R}_T) \geq (R_T)_+^2$, which is not exactly the statement given in the question. However, it suffices for proving upper bounds on the regret, which is the motivation behind our construction of Φ .

8. Using the two previous questions give an upper bound on the regret R_T . For what value of α is the bound the most favorable? Give a simple expression of the upper bound on the regret for a suitable approximation of that optimal value.

Solution: In view of the inequalities of the two previous questions, we have

$$R_T \leq \sqrt{(\alpha - 1)N^{\frac{2}{\alpha}}M^2T}.$$

For $N > e^2$, the function $\alpha \mapsto (\alpha - 1)N^{\frac{2}{\alpha}}$ reaches its minimum over $(2, +\infty)$ at $(\log N + \sqrt{\log^2 N - \log N})$, which is approximately $2 \log N$. Plugging in that value yields the bound

$$R_T \leq M\sqrt{(2 \log N - 1)eT}.$$

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 3
November 16, 2017
Due: Dec 01, 2017

A. Kernels

Show that the following kernels K are PDS:

1. For all integers $n > 0$, $K(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^N \cos^n(x_i^2 - y_i^2)$ over $\mathbb{R}^N \times \mathbb{R}^N$.
2. $K(x, y) = \min(x, y) - xy$ over $[0, 1] \times [0, 1]$.
3. $\forall \sigma > 0$, $K(x, y) = e^{-\frac{\|\mathbf{x}-\mathbf{y}\|}{\sigma}}$ over $\mathbb{R}^N \times \mathbb{R}^N$.

B. Boosting

In class, we showed that AdaBoost can be viewed as coordinate descent applied to a convex upper bound on the empirical error. Here, we consider instead an algorithm seeking to minimize the empirical margin loss. For any $0 \leq \rho < 1$, using the same notation as in class, let $\hat{R}_\rho(f) = \frac{1}{m} \sum_{i=1}^m 1_{y_i f(x_i) \leq \rho}$ denote the empirical margin loss of a function f of the form $f = \frac{\sum_{t=1}^T \alpha_t h_t}{\sum_{t=1}^T \alpha_t}$ for a labeled sample $S = ((x_1, y_1), \dots, (x_m, y_m))$.

1. Show that $\hat{R}_\rho(f)$ can be upper bounded as follows:

$$\hat{R}_\rho(f) \leq \frac{1}{m} \sum_{i=1}^m \exp \left(-y_i \sum_{t=1}^T \alpha_t h_t(x_i) + \rho \sum_{t=1}^T \alpha_t \right).$$

2. For any $\rho > 0$, let G_ρ be the objective function defined for all $\alpha \geq 0$ by

$$G_\rho(\alpha) = \frac{1}{m} \sum_{i=1}^m \exp \left(-y_i \sum_{j=1}^N \alpha_j h_j(x_i) + \rho \sum_{j=1}^N \alpha_j \right),$$

with $h_j \in H$ for all $j \in [1, N]$, with the notation used in class in the boosting lecture. Show that G_ρ is convex and differentiable.

3. Derive a boosting-style algorithm $\mathcal{A}_\rho$ by applying (maximum) coordinate descent to G_ρ . You should justify in detail the derivation of the algorithm, in particular the choice of the base classifier selected at each round and that of the step. Compare both to their counterparts in AdaBoost.
4. What is the equivalent of the weak learning assumption for $\mathcal{A}_\rho$ (*Hint*: use non-negativity of the step value)?
5. Give the full pseudocode of the algorithm $\mathcal{A}_\rho$. What can you say about the $\mathcal{A}_0$ algorithm?
6. Implement the $\mathcal{A}_\rho$ algorithm. The algorithm admits two parameters: the number of rounds T and ρ . Experiment with this algorithm to tackle the same classification problem as the one described in the second homework assignment, with the same choice of training and test sets and the same cross-validation set-up. Use a grid search to determine the best values of ρ (e.g., $\rho \in \{2^{-10}, 2^{-9}, \dots, 2^{-1}\}$ and $T \in \{100, 200, 500, 1000\}$) and report the test error obtained. Compare your result with the one obtained by running AdaBoost using the same set-up. Also, compare these results with those obtained in the second homework assignment using SVMs.
7. For both the $\mathcal{A}_\rho$ results and AdaBoost, plot the cumulative margins after 500 rounds of boosting, that is plot the fraction of training points with margin less than or equal to θ as a function of $\theta \in [0, 1]$.
8. Bound on $\widehat{R}_\rho(f)$.
 - (a) Prove the upper bound $\widehat{R}_\rho(f) \leq \exp\left(\sum_{t=1}^T \alpha_t \rho\right) \prod_{t=1}^T Z_t$, where the normalization factors Z_t are defined as in the case of AdaBoost in class (with α_t the step chosen by $\mathcal{A}_\rho$ at round t).
 - (b) Give the expression of Z_t as a function of ρ and ϵ_t , where ϵ_t is the weighted error of the hypothesis found by $\mathcal{A}_\rho$ at round t (defined in the same way as for AdaBoost in class). Use that to prove the following upper bound

$$\widehat{R}_\rho(f) \leq \exp\left(-\sum_{t=1}^T D\left(\frac{1-\rho}{2} \parallel \epsilon_t\right)\right),$$

where $D(p \parallel q)$ denotes the binary relative entropy of p and q : $D(p \parallel q) = p \log \frac{p}{q} + (1-p) \log \frac{1-p}{1-q}$, for any $p, q \in [0, 1]$.

- (c) Assume that for all $t \in [1, T]$, $\frac{1-\rho}{2} - \epsilon_t > \gamma > 0$. Use the result of the previous question to show that

$$\hat{R}_\rho(f) \leq \exp(-2\gamma^2 T).$$

(*Hint:* you can use Pinsker's inequality: $D(p||q) \geq 2(p - q)^2$ for all $p, q \in [0, 1]$). Show that for $T > \frac{\log m}{2\gamma^2}$, all points have margin at least ρ .

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 3
November 16, 2017
Due: Dec 01, 2017

A. Kernels

Show that the following kernels K are PDS:

1. For all integers $n > 0$, $K(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^N \cos^n(x_i^2 - y_i^2)$ over $\mathbb{R}^N \times \mathbb{R}^N$.

Solution: Since $\cos(x-y) = \cos x \cos y + \sin x \sin y$, $K(x, y)$ can be written as the dot product of the vectors

$$\Phi(x) = \begin{bmatrix} \cos x \\ \sin x \end{bmatrix} \quad \text{and} \quad \Phi(y) = \begin{bmatrix} \cos y \\ \sin y \end{bmatrix};$$

thus it is PDS.

This is a consequence of the fact that the kernel $K(x, y) = \cos(x - y)$ is PDS since $\sum_{ij} c_i c_j \cos(x_i^2 - x_j^2) = \sum_{ij} c_i c_j \cos(x'_i - x'_j)$, with $x'_i = x_i^2$ for all i .

2. $K(x, y) = \min(x, y) - xy$ over $[0, 1] \times [0, 1]$.

Solution: Let $\langle \cdot, \cdot \rangle$ denote the inner product over the set of all measurable functions on $[0, 1]$:

$$\langle f, g \rangle = \int_0^1 f(t)g(t)dt.$$

Note that $k_1(x, y) = \int_0^1 1_{t \in [0, x]} 1_{t \in [0, y]} dt = \langle 1_{[0, x]}, 1_{[0, y]} \rangle$, and $k_2(x, y) = \int_0^1 1_{t \in [x, 1]} 1_{t \in [y, 1]} dt = \langle 1_{[x, 1]}, 1_{[y, 1]} \rangle$. Since they are defined as inner products, both k_1 and k_2 are PDS. Note that $k_1(x, y) = \min(x, y)$ and $k_2(x, y) = 1 - \max(x, y)$. Observe that $K = k_1 k_2$, thus K is PDS as a product of two PDS kernels.

3. $\forall \sigma > 0, K(x, y) = e^{-\frac{\|\mathbf{x} - \mathbf{y}\|}{\sigma}}$ over $\mathbb{R}^N \times \mathbb{R}^N$.

Solution: It suffices to show that K is the normalized kernel associated to the kernel K' defined by

$$\forall (\mathbf{x}, \mathbf{y}) \in \mathbb{R}^N \times \mathbb{R}^N, K'(\mathbf{x}, \mathbf{y}) = e^{\phi(\mathbf{x}, \mathbf{y})}$$

where $\phi(\mathbf{x}, \mathbf{y}) = \frac{1}{\sigma} [\|\mathbf{x}\| + \|\mathbf{y}\| - \|\mathbf{x} - \mathbf{y}\|]$, and to show that K' is PDS. For the first part, observe that

$$\frac{K'(\mathbf{x}, \mathbf{y})}{\sqrt{K'(\mathbf{x}, \mathbf{x})K'(\mathbf{y}, \mathbf{y})}} = e^{\phi(\mathbf{x}, \mathbf{y}) - \frac{1}{2}\phi(\mathbf{x}, \mathbf{x}) - \frac{1}{2}\phi(\mathbf{y}, \mathbf{y})} = e^{-\frac{\|\mathbf{x} - \mathbf{y}\|}{\sigma}}.$$

To show that K' is PDS, it suffices to show that ϕ is PDS, since composition with a power series with non-negative coefficients (here $\exp$) preserve the PDS property. Now, for any $c_1, \dots, c_n \in \mathbb{R}$, let $c_0 = -\sum_{i=1}^n c_i$, then, we can write

$$\begin{aligned} \sum_{i,j=1}^n c_i c_j \phi(\mathbf{x}_i, \mathbf{x}_j) &= \frac{1}{\sigma} \sum_{i,j=1}^n c_i c_j [\|\mathbf{x}_i\| + \|\mathbf{x}_j\| - \|\mathbf{x}_i - \mathbf{x}_j\|] \\ &= \frac{1}{\sigma} \left[-\sum_{i=1}^n c_0 c_i \|\mathbf{x}_i\| + -\sum_{i=1}^n c_0 c_j \|\mathbf{x}_j\| - \sum_{i,j=1}^n c_i c_j \|\mathbf{x}_i - \mathbf{x}_j\| \right] \\ &= -\frac{1}{\sigma} \sum_{i,j=0}^n c_i c_j \|\mathbf{x}_i - \mathbf{x}_j\|, \end{aligned}$$

with $\mathbf{x}_0 = 0$. Now, for any $z \in \mathbb{R}$, the following equality holds:

$$z^{\frac{1}{2}} = \frac{1}{2\Gamma(\frac{1}{2})} \int_0^{+\infty} \frac{1 - e^{-tz}}{t^{\frac{3}{2}}} dt.$$

Thus,

$$\begin{aligned} -\frac{1}{\sigma} \sum_{i,j=0}^n c_i c_j \|\mathbf{x}_i - \mathbf{x}_j\| &= \frac{1}{2\Gamma(\frac{1}{2})} \int_0^{+\infty} -\frac{1}{\sigma} \sum_{i,j=0}^n c_i c_j \frac{1 - e^{-t\|\mathbf{x}_i - \mathbf{x}_j\|}}{t^{\frac{3}{2}}} dt \\ &= \frac{1}{2\Gamma(\frac{1}{2})} \int_0^{+\infty} \frac{1}{\sigma} \frac{\sum_{i,j=0}^n c_i c_j e^{-t\|\mathbf{x}_i - \mathbf{x}_j\|}}{t^{\frac{3}{2}}} dt. \end{aligned}$$

Since a Gaussian kernel is PDS, the inequality $\sum_{i,j=0}^n c_i c_j e^{-t\|\mathbf{x}_i - \mathbf{x}_j\|} \geq 0$ holds and the right-hand side is non-negative. Thus, the inequality

$$-\frac{1}{\sigma} \sum_{i,j=0}^n c_i c_j \|\mathbf{x}_i - \mathbf{x}_j\| \geq 0$$

holds, which shows that ϕ is PDS. $\square$

Alternatively, one can also apply the theorem on page 43 of the lecture slides on kernel methods to reduce the problem to showing that the norm $G(x, y) = \|x - y\|$ is a NDS function. This can be shown through a direct application of the definition of NDS together with the representation of the norm given in the hint.

B. Boosting

In class, we showed that AdaBoost can be viewed as coordinate descent applied to a convex upper bound on the empirical error. Here, we consider instead an algorithm seeking to minimize the empirical margin loss. For any $0 \leq \rho < 1$, using the same notation as in class, let $\hat{R}_\rho(f) = \frac{1}{m} \sum_{i=1}^m 1_{y_i f(x_i) \leq \rho}$ denote the empirical margin loss of a function f of the form $f = \frac{\sum_{t=1}^T \alpha_t h_t}{\sum_{t=1}^T \alpha_t}$ for a labeled sample $S = ((x_1, y_1), \dots, (x_m, y_m))$.

1. Show that $\hat{R}_\rho(f)$ can be upper bounded as follows:

$$\hat{R}_\rho(f) \leq \frac{1}{m} \sum_{i=1}^m \exp \left(-y_i \sum_{t=1}^T \alpha_t h_t(x_i) + \rho \sum_{t=1}^T \alpha_t \right).$$

Solution: The following shows how $\hat{R}_\rho(f)$ can be upper-bounded:

$$\begin{aligned} \hat{R}_\rho(f) &= \frac{1}{m} \sum_{i=1}^m 1_{y_i f(x_i) \leq \rho} = \frac{1}{m} \sum_{i=1}^m 1_{y_i \sum_{t=1}^T \alpha_t h_t(x_i) - \rho \sum_{t=1}^T \alpha_t \leq 0} \\ &\leq \frac{1}{m} \sum_{i=1}^m \exp \left(-y_i \sum_{t=1}^T \alpha_t h_t(x_i) + \rho \sum_{t=1}^T \alpha_t \right). \end{aligned}$$

□

2. For any $\rho > 0$, let G_ρ be the objective function defined for all $\alpha \geq 0$ by

$$G_\rho(\alpha) = \frac{1}{m} \sum_{i=1}^m \exp \left(-y_i \sum_{j=1}^N \alpha_j h_j(x_i) + \rho \sum_{j=1}^N \alpha_j \right),$$

with $h_j \in H$ for all $j \in [1, N]$, with the notation used in class in the boosting lecture. Show that G_ρ is convex and differentiable.

Solution: Since $\exp$ is convex and that composition with an affine function (of α) preserves convexity, each term of the objective is a convex function and G_ρ is convex as a sum of convex functions. The differentiability follows directly that of $\exp$. □

3. Derive a boosting-style algorithm $\mathcal{A}_\rho$ by applying (maximum) coordinate descent to G_ρ . You should justify in detail the derivation of the algorithm, in

particular the choice of the base classifier selected at each round and that of the step. Compare both to their counterparts in AdaBoost.

Solution: By definition of G_ρ , for any direction $\mathbf{e}_t$ we can write

$$G_\rho(\boldsymbol{\alpha}_{t-1} + \eta \mathbf{e}_t) = \frac{1}{m} \sum_{i=1}^m \exp \left(-y_i \sum_{s=1}^{t-1} \alpha_s h_s(x_i) + \rho \sum_{s=1}^{t-1} \alpha_s \right) e^{-y_i \eta h_t(x_i) + \rho \eta}.$$

Let D_1 be the uniform distribution, that is $D_1(i) = \frac{1}{m}$ for all $i \in [1, m]$ and for any $t \in [2, T]$, define D_t by

$$D_t(i) = \frac{D_{t-1}(i) \exp(-y_i \alpha_{t-1} h_{t-1}(x_i))}{Z_{t-1}},$$

with $Z_{t-1} = \sum_{i=1}^m D_{t-1}(i) \exp(-y_i \alpha_{t-1} h_{t-1}(x_i))$. Observe that $D_t(i) = \frac{\exp \left(-y_i \sum_{s=1}^{t-1} \alpha_s h_s(x_i) \right)}{m \prod_{s=1}^{t-1} Z_s}$. Thus,

$$\begin{aligned} & \left. \frac{dG_\rho(\boldsymbol{\alpha}_{t-1} + \eta \mathbf{e}_t)}{d\eta} \right|_0 \\ &= \frac{1}{m} \sum_{i=1}^m [-y_i h_t(x_i) + \rho] \exp \left(-y_i \sum_{s=1}^{t-1} \alpha_s h_s(x_i) + \rho \sum_{s=1}^{t-1} \alpha_s \right) \\ &= \sum_{i=1}^m [-y_i h_t(x_i) + \rho] D_t(i) \left(\prod_{s=1}^{t-1} Z_s \right) e^{\rho \sum_{s=1}^{t-1} \alpha_s} \\ &= \left[\sum_{i=1}^m -y_i h_t(x_i) D_t(i) + \rho \right] \prod_{s=1}^{t-1} Z_s e^{\rho \sum_{s=1}^{t-1} \alpha_s} \\ &= (-(1 - \epsilon_t) + \epsilon_t + \rho) \left(\prod_{s=1}^{t-1} Z_s \right) e^{\rho \sum_{s=1}^{t-1} \alpha_s} \\ &= (2\epsilon_t - 1 + \rho) \left(\prod_{s=1}^{t-1} Z_s \right) e^{\rho \sum_{s=1}^{t-1} \alpha_s}, \end{aligned}$$

where $\epsilon_t = \sum_{i=1}^m D_t(i) 1_{y_i h_t(x_i) > 0}$. The direction selected is the one minimizing $(2\epsilon_t - 1 + \rho)$ that is ϵ_t . Thus, the algorithm selects at each round the base classifier with the smallest weighted error, as in the case of AdaBoost.

To determine the step η selected at round t , we solve the following equation

$$\begin{aligned}
\frac{dG_\rho(\boldsymbol{\alpha}_{t-1} + \eta \mathbf{e}_t)}{d\eta} &= 0 \\
\Leftrightarrow \sum_{i=1}^m D_t(i) Z_t e^{-y_i \eta h_t(x_i) + \rho \eta} [-y_i h_t(x_i) + \rho] &= 0 \\
\Leftrightarrow \sum_{y_i h_t(x_i) > 0} D_t(i) Z_t e^{\eta(-1+\rho)} (-1 + \rho) + \sum_{y_i h_t(x_i) < 0} D_t(i) Z_t e^{\eta(1+\rho)} (1 + \rho) &= 0 \\
\Leftrightarrow (1 - \epsilon_t) e^{\eta(-1+\rho)} (-1 + \rho) + \epsilon_t e^{\eta(1+\rho)} (1 + \rho) &= 0 \\
\Leftrightarrow (1 - \epsilon_t) e^{-\eta} (-1 + \rho) + \epsilon_t e^{\eta} (1 + \rho) &= 0 \\
\Leftrightarrow e^{2\eta} = \frac{(1 - \rho)(1 - \epsilon_t)}{(1 + \rho)\epsilon_t} \\
\Leftrightarrow \eta = \frac{1}{2} \log \frac{(1 - \epsilon_t)}{\epsilon_t} - \frac{1}{2} \log \frac{(1 + \rho)}{(1 - \rho)}.
\end{aligned}$$

Thus, the algorithm differs from AdaBoost only by the choice of the step, which it chooses more conservatively: the step size is smaller than that of AdaBoost by the additive constant $\frac{1}{2} \log \frac{(1+\rho)}{(1-\rho)}$. $\square$

4. What is the equivalent of the weak learning assumption for $\mathcal{A}_\rho$ (*Hint: use non-negativity of the step value*)?

Solution: For the coordinate descent algorithm to make progress at each round, the step size selected along the descent direction must be non-negative, that is

$$\begin{aligned}
\frac{(1 - \rho)(1 - \epsilon_t)}{(1 + \rho)\epsilon_t} > 1 &\Leftrightarrow (1 - \rho)(1 - \epsilon_t) > \rho\epsilon_t + \epsilon_t \\
&\Leftrightarrow \epsilon_t < \frac{1 - \rho}{2}.
\end{aligned}$$

Thus, the error of the base classifier chosen must be at least $\rho/2$ better than one half. $\square$

5. Give the full pseudocode of the algorithm $\mathcal{A}_\rho$. What can you say about the $\mathcal{A}_0$ algorithm?

Solution: The normalization factor Z_t can be expressed in terms of ϵ_t and ρ

```

 $\mathcal{A}_\rho(S = ((x_1, y_1), \dots, (x_m, y_m)))$ 
1  for  $i \leftarrow 1$  to  $m$  do
2       $D_1(i) \leftarrow \frac{1}{m}$ 
3  for  $t \leftarrow 1$  to  $T$  do
4       $h_t \leftarrow$  base classifier in  $H$  with small error  $\epsilon_t = \mathbb{P}_{i \sim D_t}[h_t(x_i) \neq y_i]$ 
5       $\alpha_t \leftarrow \frac{1}{2} \log \frac{(1-\epsilon_t)}{\epsilon_t} - \frac{1}{2} \log \frac{(1+\rho)}{(1-\rho)}$ 
6       $Z_t \leftarrow 2 \left[ \frac{\epsilon_t(1-\epsilon_t)}{1-\rho^2} \right]^{\frac{1}{2}} \triangleright$  normalization factor
7      for  $i \leftarrow 1$  to  $m$  do
8           $D_{t+1}(i) \leftarrow \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ 
9   $f \leftarrow \sum_{t=1}^T \alpha_t h_t$ 
10 return  $h = \text{sgn}(f)$ 

```

Figure 1: Algorithm $\mathcal{A}_\rho$ for $H \subseteq \{-1, +1\}^{\mathcal{X}}$.

using its definition:

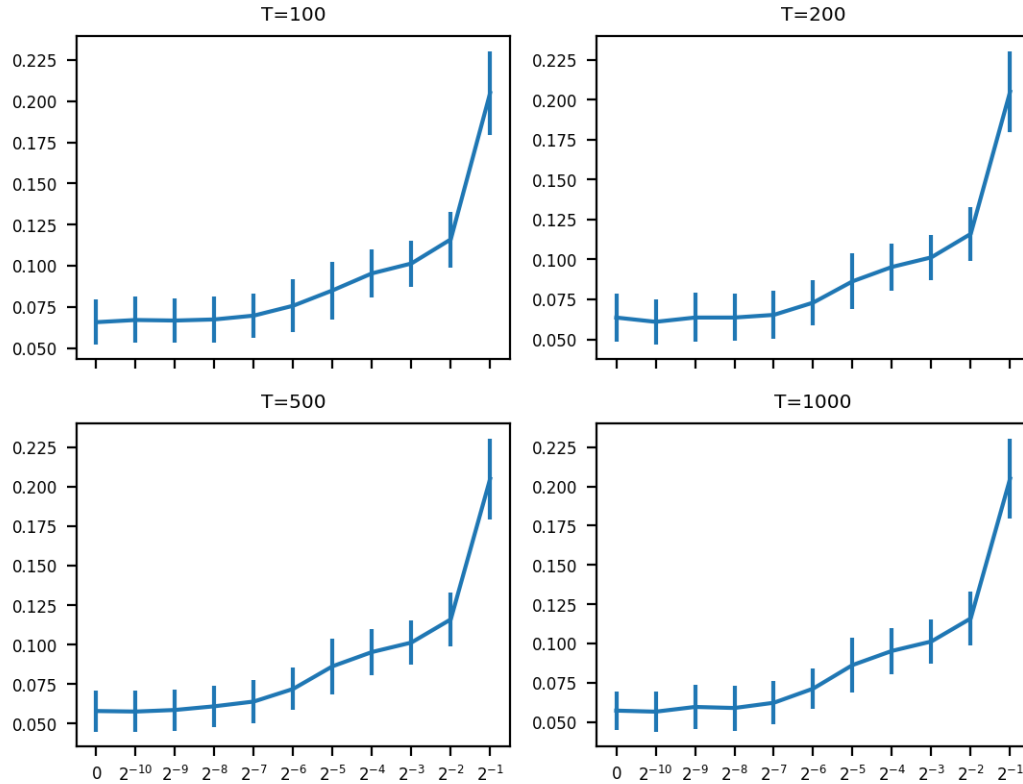
$$\begin{aligned}
 Z_t &= \sum_{i=1}^m D_t(i) \exp(-y_i \alpha_t h_t(x_i)) \\
 &= e^{-\alpha_t} (1 - \epsilon_t) + e^{\alpha_t} \epsilon_t \\
 &= \sqrt{\frac{1+\rho}{1-\rho}} (1 - \epsilon_t) \epsilon_t + \sqrt{\frac{1-\rho}{1+\rho}} (1 - \epsilon_t) \epsilon_t \\
 &= \sqrt{\epsilon_t (1 - \epsilon_t)} \left[\sqrt{\frac{1+\rho}{1-\rho}} + \sqrt{\frac{1-\rho}{1+\rho}} \right] \\
 &= \sqrt{\epsilon_t (1 - \epsilon_t)} \left[\frac{2}{\sqrt{1 - \rho^2}} \right] \\
 &= 2 \sqrt{\frac{\epsilon_t (1 - \epsilon_t)}{1 - \rho^2}}.
 \end{aligned}$$

The pseudocode of the algorithm is then given in Figure 1. $\mathcal{A}_0$ coincides exactly with AdaBoost. $\square$

6. Implement the $\mathcal{A}_\rho$ algorithm. The algorithm admits two parameters: the number of rounds T and ρ . Experiment with this algorithm to tackle the same classification problem as the one described in the second homework

assignment, with the same choice of training and test sets and the same cross-validation set-up. Use a grid search to determine the best values of ρ (e.g., $\rho \in \{2^{-10}, 2^{-9}, \dots, 2^{-1}\}$ and $T \in \{100, 200, 500, 1000\}$) and report the test error obtained. Compare your result with the one obtained by running AdaBoost using the same set-up. Also, compare these results with those obtained in the second homework assignment using SVMs.

Solution: The following figures show the cross-validation results within one standard deviation for various values of ρ and numbers of boosting rounds. Note that $\rho = 0$ corresponds to AdaBoost.

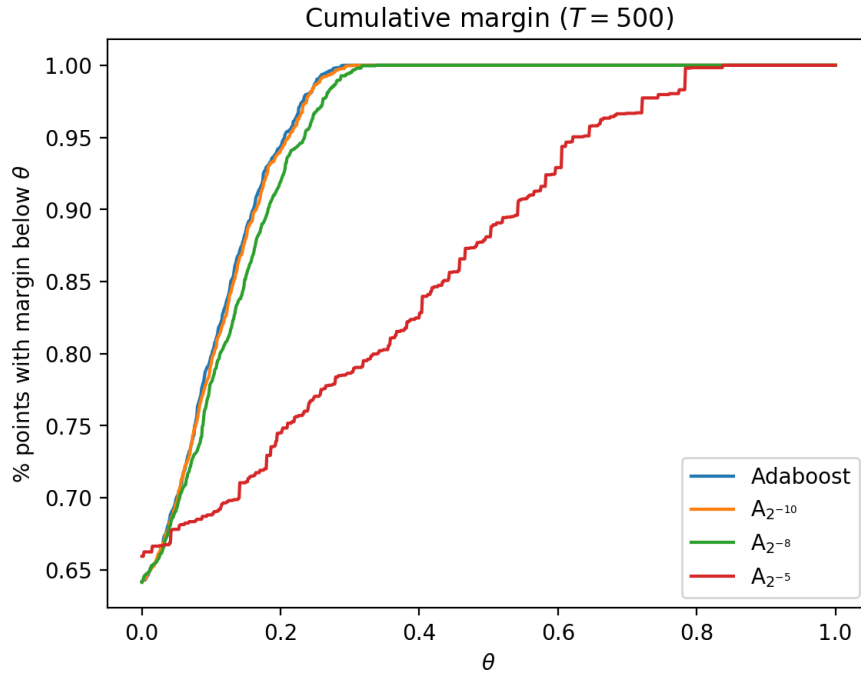


The lowest cross-validation error (0.057) occurs for margin $\rho = 2^{-10}$ and 1000 boosting rounds. The following table shows the training/test errors for $\mathcal{A}_{\rho=2^{-10}}$ and Adaboost (with $T = 1000$) and SVM from last homework.

Algorithm	$\mathcal{A}_{\rho=2^{-10}}$	AdaBoost	SVM
10-CV Error	0.0567	0.0573	0.0673
Test Error	0.0531	0.05	0.06

7. For both the $\mathcal{A}_\rho$ results and AdaBoost, plot the cumulative margins after 500 rounds of boosting, that is plot the fraction of training points with margin less than or equal to θ as a function of $\theta \in [0, 1]$.

Solution: In the graph below, x-axis is θ and y-axis is the fraction of points with margin less than θ . Different colors correspond to different $\mathcal{A}_\rho$: red line is $\mathcal{A}_{\rho=2^{-5}}$ and blue line is AdaBoost($\mathcal{A}_0$). As expected, for $\rho > 0$, $\mathcal{A}_\rho$ achieves a larger margin for more points compared to AdaBoost. $\mathcal{A}_{2^{-10}}$ (Orange line) is almost identical to AdaBoost.



8. Bound on $\hat{R}_\rho(f)$.

- (a) Prove the upper bound $\hat{R}_\rho(f) \leq \exp\left(\sum_{t=1}^T \alpha_t \rho\right) \prod_{t=1}^T Z_t$, where the normalization factors Z_t are defined as in the case of AdaBoost in class (with α_t the step chosen by $\mathcal{A}_\rho$ at round t).

Solution:

In view of the definition of D_t and the bound derived in question 1), we can write

$$\begin{aligned}\widehat{R}_\rho(f) &\leq \frac{1}{m} \sum_{i=1}^m \exp \left(-y_i \sum_{t=1}^T \alpha_t h_t(x_i) + \rho \sum_{t=1}^T \alpha_t \right) \\ &= \frac{1}{m} \sum_{i=1}^m \left(m \prod_{t=1}^T Z_t \right) D_t(i) \exp \left(\rho \sum_{t=1}^T \alpha_t \right) \\ &= \exp \left(\rho \sum_{t=1}^T \alpha_t \right) \left(\prod_{t=1}^T Z_t \right).\end{aligned}$$

□

- (b) Give the expression of Z_t as a function of ρ and ϵ_t , where ϵ_t is the weighted error of the hypothesis found by $\mathcal{A}_\rho$ at round t (defined in the same way as for AdaBoost in class). Use that to prove the following upper bound

$$\widehat{R}_\rho(f) \leq \exp \left(- \sum_{t=1}^T D \left(\frac{1-\rho}{2} \parallel \epsilon_t \right) \right),$$

where $D(p \parallel q)$ denotes the binary relative entropy of p and q : $D(p \parallel q) = p \log \frac{p}{q} + (1-p) \log \frac{1-p}{1-q}$, for any $p, q \in [0, 1]$.

Solution: Define u by $u = \frac{1-\rho}{1+\rho}$. The expression of Z_t was already given above. Plugging in that expression in the bound of the previous question and using the expression of α_t gives

$$\begin{aligned}\widehat{R}_\rho(f) &\leq \left(\prod_t e^{\alpha_t} \right)^\rho \left(\prod_{t=1}^T \sqrt{\epsilon_t(1-\epsilon_t)} (u^{\frac{1}{2}} + u^{-\frac{1}{2}}) \right) \\ &= \left(\sqrt{\frac{1-\rho}{1+\rho}} \right)^{\rho T} \left(\prod_t \sqrt{\frac{1-\epsilon_t}{\epsilon_t}} \right)^\rho \left(\prod_{t=1}^T \sqrt{\epsilon_t(1-\epsilon_t)} (u^{\frac{1}{2}} + u^{-\frac{1}{2}}) \right) \\ &= \left(u^{\frac{1+\rho}{2}} + u^{-\frac{1-\rho}{2}} \right)^T \prod_{t=1}^T \sqrt{\epsilon_t^{1-\rho} (1-\epsilon_t)^{1+\rho}}.\end{aligned}$$

Observe that

$$\begin{aligned}
u^{\frac{1+\rho}{2}} + u^{-\frac{1-\rho}{2}} &= \left(\frac{1-\rho}{1+\rho}\right)^{\frac{1+\rho}{2}} + \left(\frac{1+\rho}{1-\rho}\right)^{\frac{1-\rho}{2}} \\
&= \frac{(1-\rho) + (1+\rho)}{(1+\rho)^{\frac{1+\rho}{2}} (1-\rho)^{\frac{1-\rho}{2}}} \\
&= \frac{2}{(1+\rho)^{\frac{1+\rho}{2}} (1-\rho)^{\frac{1-\rho}{2}}} \\
&= \frac{1}{\left(\frac{1+\rho}{2}\right)^{\frac{1+\rho}{2}} \left(\frac{1-\rho}{2}\right)^{\frac{1-\rho}{2}}}.
\end{aligned}$$

We also have

$$\begin{aligned}
&\log \left[\sqrt{\epsilon_t^{1-\rho} (1-\epsilon_t)^{1+\rho}} \right] \\
&= \frac{1-\rho}{2} \log(\epsilon_t) + \frac{1+\rho}{2} \log(1-\epsilon_t) \\
&= -D\left(\frac{1-\rho}{2} \parallel \epsilon_t\right) + \frac{1-\rho}{2} \log\left(\frac{1-\rho}{2}\right) + \frac{1+\rho}{2} \log\left(\frac{1+\rho}{2}\right) \\
&= -D\left(\frac{1-\rho}{2} \parallel \epsilon_t\right) + \log\left(\left(\frac{1+\rho}{2}\right)^{\frac{1+\rho}{2}} \left(\frac{1-\rho}{2}\right)^{\frac{1-\rho}{2}}\right).
\end{aligned}$$

Combining these two inequalities gives

$$\hat{R}_\rho(f) \leq \exp\left(-\sum_{t=1}^T D\left(\frac{1-\rho}{2} \parallel \epsilon_t\right)\right).$$

□

- (c) Assume that for all $t \in [1, T]$, $\frac{1-\rho}{2} - \epsilon_t > \gamma > 0$. Use the result of the previous question to show that

$$\hat{R}_\rho(f) \leq \exp(-2\gamma^2 T).$$

(Hint: you can use Pinsker's inequality: $D(p \parallel q) \geq 2(p-q)^2$ for all $p, q \in [0, 1]$). Show that for $T > \frac{\log m}{2\gamma^2}$, all points have margin at least ρ .

Solution: By Pinsker's inequality, we have $D\left(\frac{1-\rho}{2} \parallel \epsilon_t\right) \geq 2\left[\frac{1-\rho}{2} - \epsilon_t\right]^2$. Thus, we can write

$$\hat{R}_\rho(f) \leq \exp(-2\gamma^2 T).$$

Thus, if the upper bound is less than $1/m$, then $\widehat{R}_\rho(f) = 0$ and every training point has margin at least ρ . The inequality $\exp(-2\gamma^2 T) < 1/m$ is equivalent to $T > \frac{\log m}{2\gamma^2}$. $\square$

Mehryar Mohri
 Foundations of Machine Learning 2018
 Courant Institute of Mathematical Sciences
 Homework assignment 3
 11/10, 2018
 Due: 11/26, 2018

A. Kernel PCA

Read the *Dimensionality Reduction* Chapter 12 in the course textbook Foundations of ML with a focus on PCA and Kernel PCA. Sections 12.1 and 12.2 are recommended. In this problem we will analyze a hypothesis set based on KPCA projection. Let $K(x, y)$ be a kernel function, $\Phi_K(x)$ be its corresponding feature map and $S = \{x_1, \dots, x_m\}$ be a sample of m points. When Π is the rank- r KPCA projection, we define the (regularized) hypothesis set of linear separators in the RKHS $\mathbb{H}$ of kernel K as

$$H = \left\{ x \rightarrow \langle w, \Pi \Phi_K(x) \rangle_{\mathbb{H}} : \|w\|_{\mathbb{H}} \leq 1 \right\}. \quad (1)$$

This hypothesis set essentially means that the input data is projected onto a smaller dimensional subspace of the RKHS before fitting a separation hyperplane. This problem will show that we can use the eigenvectors and eigenvalues of the sample kernel matrix to give a closed form expression for the functions $h \in H$ without a need for explicit representation of the RKHS itself.

Let $\mathbf{K}$ be the sample kernel matrix for kernel K evaluated on m points of sample S , that is $\mathbf{K}_{i,j} = K(x_i, x_j)$. Let $\lambda_1, \dots, \lambda_r$ are the top r (nonzero) eigenvalues of $\mathbf{K}$ with the corresponding eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_r$. Denote the j -th element of vector $\mathbf{v}_i$ as $[\mathbf{v}_i]_j$. Follow the subproblems below to derive the explicit representation of $h \in H$.

1. Assume that the feature maps $\Phi_K(x)$ are centered on sample S and recall that the sample covariance operator is $\Sigma = \sum_{i=1}^m \frac{1}{m} \Phi_K(x_i) \Phi_K(x_i)^\top$. Prove that $h(x) = \sum_{i=1}^r \alpha_i \langle \mathbf{u}_i, \Phi_K(x) \rangle_{\mathbb{H}}$ for some $\alpha_i \in \mathbb{R}$, where $\mathbf{u}_1, \dots, \mathbf{u}_r$ are the eigenvectors of Σ corresponding to its top r eigenvalues.
2. Prove that $\mathbf{u}_i = \mathbf{X} \frac{\mathbf{v}_i}{\sqrt{\lambda_i}}$, where $\mathbf{X} = [\Phi_K(x_1), \dots, \Phi_K(x_m)]$

- Using the result above, prove that any function $h \in H$ can be represented as

$$h(x) = \sum_{i=1}^r \sum_{j=1}^m \frac{\alpha_i}{\sqrt{\lambda_i}} K(x_j, x) [v_i]_j,$$

for some $\alpha_i \in \mathbb{R}$.

- Bonus question: derive the Rademacher complexity bound on the hypothesis set H defined in this problem.

B. Multi-class boosting

Lecture 10 introduces the AdaBoost.MH algorithm, which is AdaBoost for multi-class classification. (Consult with Lecture 10's slides if you are unfamiliar with multi-class learning setting.) AdaBoost.MH is defined by objective function $F(\alpha)$:

$$F(\alpha) = \sum_{l=1}^k \sum_{i=1}^m e^{-y_i[l] \sum_{t=1}^n \alpha_t h_t(x_i, l)},$$

where $y_i \in \mathcal{Y} = \{-1, +1\}^k$, and $y_i[l]$ denotes the l -th coordinate of y_i for any $i \in [m]$ and $l \in [k]$. The base classifiers come from $H = \{h : \mathcal{X} \times [k] \rightarrow \{-1, +1\}\}$. Consider an alternative objective function for the same problem:

$$G(\alpha) = \sum_{i=1}^m e^{-\frac{1}{k} \sum_{l=1}^k y_i[l] \sum_{t=1}^n \alpha_t h_t(x_i, l)}.$$

- Compare $G(\alpha)$ with $F(\alpha)$. Show that $F(\alpha) \geq kG(\alpha)$.
- Let $g_n(x_i, l) = \sum_{t=1}^n \alpha_t h_t(x_i, l)$. Assume that $|g_n(x_i, l)| \leq 1$ for all $x_i \in \mathcal{X}, l \in [k]$. Show that $kG(\alpha)$ is a convex function upper bounding the multi-label multi-class error:

$$\sum_{i=1}^m \sum_{l=1}^k 1_{y_i[l] \neq \text{sgn}(g_n(x_i, l))} \leq kG(\alpha).$$

- Derive an algorithm defined by the application of coordinate descent to $G(\alpha)$. You should give a full description of your algorithm, including the pseudocode, details for the choice of the step and direction, as well as a generalization bound.

Mehryar Mohri
 Foundations of Machine Learning 2018
 Courant Institute of Mathematical Sciences
 Homework assignment 3
 11/10, 2018
 Due: 11/26, 2018

A. Kernel PCA

Read the *Dimensionality Reduction* Chapter 12 in the course textbook Foundations of ML with a focus on PCA and Kernel PCA. Sections 12.1 and 12.2 are recommended. In this problem we will analyze a hypothesis set based on KPCA projection. Let $K(x, y)$ be a kernel function, $\Phi_K(x)$ be its corresponding feature map and $S = \{x_1, \dots, x_m\}$ be a sample of m points. When Π is the rank- r KPCA projection, we define the (regularized) hypothesis set of linear separators in the RKHS $\mathbb{H}$ of kernel K as

$$H = \left\{ x \rightarrow \langle w, \Pi \Phi_K(x) \rangle_{\mathbb{H}} : \|w\|_{\mathbb{H}} \leq 1 \right\}. \quad (1)$$

This hypothesis set essentially means that the input data is projected onto a smaller dimensional subspace of the RKHS before fitting a separation hyperplane. This problem will show that we can use the eigenvectors and eigenvalues of the sample kernel matrix to give a closed form expression for the functions $h \in H$ without a need for explicit representation of the RKHS itself.

Let $\mathbf{K}$ be the sample kernel matrix for kernel K evaluated on m points of sample S , that is $\mathbf{K}_{i,j} = K(x_i, x_j)$. Let $\lambda_1, \dots, \lambda_r$ are the top r (nonzero) eigenvalues of $\mathbf{K}$ with the corresponding eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_r$. Denote the j -th element of vector $\mathbf{v}_i$ as $[\mathbf{v}_i]_j$. Follow the subproblems below to derive the explicit representation of $h \in H$.

1. Assume that the feature maps $\Phi_K(x)$ are centered on sample S and recall that the sample covariance operator is $\Sigma = \sum_{i=1}^m \frac{1}{m} \Phi_K(x_i) \Phi_K(x_i)^\top$. Prove that $h(x) = \sum_{i=1}^r \alpha_i \langle \mathbf{u}_i, \Phi_K(x) \rangle_{\mathbb{H}}$ for some $\alpha_i \in \mathbb{R}$, where $\mathbf{u}_1, \dots, \mathbf{u}_r$ are the eigenvectors of Σ corresponding to its top r eigenvalues.

Solution: This is a direct application of the orthonormal basis $\mathbf{u}_1, \dots, \mathbf{u}_r$.

$$\begin{aligned} h(x) &= \langle w, \mathbf{U}_k \mathbf{U}_k^\top \Phi_k(x) \rangle_{\mathbb{H}} \\ &= \langle w, \sum_{i=1}^r \mathbf{u}_i \mathbf{u}_i^\top \Phi_k(x) \rangle_{\mathbb{H}} \\ &= \sum_{i=1}^r \langle w, \mathbf{u}_i \rangle_{\mathbb{H}} \langle \mathbf{u}_i, \Phi_k(x) \rangle_{\mathbb{H}} \end{aligned}$$

Denoting $\alpha_i = \langle w, \mathbf{u}_i \rangle_{\mathbb{H}}$, we obtain the solution.

2. Prove that $\mathbf{u}_i = \mathbf{X} \frac{\mathbf{v}_i}{\sqrt{\lambda_i}}$, where $\mathbf{X} = [\Phi_K(x_1), \dots, \Phi_K(x_m)]$

Solution: For more details see Ch12, Section 12.2 of the textbook. The eigenvalue-eigenvector equation for Σ is

$$\Sigma \mathbf{u}_i = \gamma_i \mathbf{u}_i$$

Substituting $\Sigma = \frac{1}{m} \mathbf{X} \mathbf{X}^\top$ and $\mathbf{u}_i = \mathbf{X} w_i$ for some $w_i \in \mathbb{R}^m$ since u_i belongs to the span of $\mathbf{X} = [\Phi_K(x_1), \dots, \Phi_K(x_m)]$. Also multiplying by $\mathbf{X}^\top$ from the left, we get.

$$\frac{1}{m} (\mathbf{X}^\top \mathbf{X}) (\mathbf{X}^\top \mathbf{X}) w_i = \gamma_i (\mathbf{X}^\top \mathbf{X}) w_i$$

Divide both sides by m .

$$\left(\frac{1}{m} \mathbf{K} \right)^2 w_i = \frac{\gamma_i}{m} \mathbf{K} w_i$$

It can be shown that the solution to the equation above is $w_i = \frac{\mathbf{v}_i}{\sqrt{\lambda_i}}$, which directly leads to $\mathbf{u}_i = \mathbf{X} \frac{\mathbf{v}_i}{\sqrt{\lambda_i}}$.

3. Using the result above, prove that any function $h \in H$ can be represented as

$$h(x) = \sum_{i=1}^r \sum_{j=1}^m \frac{\alpha_i}{\sqrt{\lambda_i}} K(x_j, x) [v_i]_j,$$

for some $\alpha_i \in \mathbb{R}$.

Solution:

$$\begin{aligned}\langle \mathbf{u}_i, \Phi_K(x) \rangle_{\mathbf{H}} &= \Phi_K^\top(x) \mathbf{X} \frac{\mathbf{v}_i}{\sqrt{\lambda_i}} \\ &= \frac{1}{\sqrt{\lambda_i}} \sum_{j=1}^m K(x_j, x) [\mathbf{v}_i]_j\end{aligned}$$

Substituting the above in the result from part 1 provides the final expression for $h(x)$.

4. Bonus question: derive the Rademacher complexity bound on the hypothesis set H defined in this problem.

Solution: Use the standard techniques for deriving generalization bounds described in this course, as well as Cauchy-Schwarz inequality and Jensen's inequality. For example, one can derive an upper bound $O\left(\sqrt{\frac{\text{Tr}(K)}{m}}\right)$ and even tighter one $O\left(\sqrt{\frac{\sum_{i=1}^r \lambda_i}{m}}\right)$.

B. Multi-class boosting

Lecture 10 introduces the AdaBoost.MH algorithm, which is AdaBoost for multi-class classification. (Consult with Lecture 10's slides if you are unfamiliar with multi-class learning setting.) AdaBoost.MH is defined by objective function $F(\alpha)$:

$$F(\alpha) = \sum_{l=1}^k \sum_{i=1}^m e^{-y_i[l] \sum_{t=1}^n \alpha_t h_t(x_i, l)},$$

where $y_i \in \mathcal{Y} = \{-1, +1\}^k$, and $y_i[l]$ denotes the l -th coordinate of y_i for any $i \in [m]$ and $l \in [k]$. The base classifiers come from $H = \{h : \mathcal{X} \times [k] \rightarrow \{-1, +1\}\}$. Consider an alternative objective function for the same problem:

$$G(\alpha) = \sum_{i=1}^m e^{-\frac{1}{k} \sum_{l=1}^k y_i[l] \sum_{t=1}^n \alpha_t h_t(x_i, l)}.$$

1. Compare $G(\alpha)$ with $F(\alpha)$. Show that $F(\alpha) \geq kG(\alpha)$.

Solution: Since e^{-x} is a convex function, by Jensen's inequality

$$\frac{1}{k} \sum_{l=1}^k e^{-y_i[l] \sum_{t=1}^n \alpha_t h_t(x_i, l)} \geq e^{-\frac{1}{k} \sum_{l=1}^k y_i[l] \sum_{t=1}^n \alpha_t h_t(x_i, l)}$$

thus $F(\alpha) \geq kG(\alpha)$

2. Let $g_n(x_i, l) = \sum_{t=1}^n \alpha_t h_t(x_i, l)$. Assume that $|g_n(x_i, l)| \leq 1$ for all $x_i \in \mathcal{X}, l \in [k]$. Show that $kG(\alpha)$ is a convex function upper bounding the multi-label multi-class error:

$$\sum_{i=1}^m \sum_{l=1}^k 1_{y_i[l] \neq \text{sgn}(g_n(x_i, l))} \leq kG(\alpha).$$

Solution: Since the exponential is linear in α and e^{-x} is convex, $G(\alpha)$ is convex.

We have

$$\frac{1}{k} \sum_{l=1}^k 1_{y_i[l] \neq \text{sgn}(g_n(x_i, l))} = \frac{1}{k} \sum_{l=1}^k 1_{y_i[l] g_n(x_i, l) \leq 0} \leq 1 - \frac{1}{k} \sum_{l=1}^k y_i[l] g_n(x_i, l).$$

The last inequality holds because

$$1_{y_i[l] g_n(x_i, l) \leq 0} + y_i[l] g_n(x_i, l) \leq 1,$$

where we use the fact that $|g_n(x_i, l)| \leq 1$ and thus $y_i[l] g_n(x_i, l) \leq 1$. Finally,

$$1 - \frac{1}{k} \sum_{l=1}^k y_i[l] g_n(x_i, l) \leq e^{-\frac{1}{k} \sum_{l=1}^k y_i[l] g_n(x_i, l)},$$

which concludes the proof.

3. Drive an algorithm defined by the application of coordinate descent to $G(\alpha)$. You should give a full description of your algorithm, including the pseudocode, details for the choice of the step and direction, as well as a generalization bound.

Solution: Define $G_i(\alpha) = e^{-\frac{1}{k} \sum_{l=1}^k y_i[l] \sum_{j=1}^n \alpha_j h_j(x_i, l)}$ then $G(\alpha) = \sum_{i=1}^m G_i(\alpha)$. we denote $\alpha_t = (\alpha_1, \dots, \alpha_t, 0, \dots, 0)$

For descent direction,

$$\frac{d}{d\eta} G(\alpha_t + \eta e_{t+1}) = -\frac{1}{k} \sum_{i=1}^m \sum_{l=1}^k y_i[l] h_{t+1}(x_i, l) G_i(\alpha_t + \eta e_{t+1})$$

thus

$$\begin{aligned}
\frac{d}{d\eta}G(\boldsymbol{\alpha}_t + \eta e_{t+1})|_{\eta=0} &= -\frac{1}{k} \sum_{i=1}^m \sum_{l=1}^k y_i[l] h_{t+1}(x_i, l) G_i(\boldsymbol{\alpha}_t) \\
&= -\sum_{i=1}^m \sum_{l=1}^k y_i[l] h_{t+1}(x_i, l) D_{t+1}(i) m \Pi_{s=1}^t Z_s \\
&= (2\epsilon_{t+1} - 1) m \Pi_{s=1}^t Z_s
\end{aligned}$$

where $D_{t+1}(i) = \frac{D_t(i) e^{-\frac{1}{k} \sum_{j=1}^k y_i[j] \alpha_t h_t(x_i, j)}}{Z_t}$ and

$$Z_t = \sum_{i=1}^m D_t(i) e^{\alpha_t(2\epsilon_t^i - 1)}$$

where $\epsilon_t^i = Pr_{j \sim U(k)}[y_i[j] \neq h_t(x_i, j)]$.

Also,

$$\epsilon_{t+1} = Pr_{(i,l) \sim D_{t+1} \times U(k)}[y_i[l] \neq h_{t+1}(x_i, l)] = \mathbb{E}_{i \sim D_{t+1}} \epsilon_{t+1}^i$$

Our h_{t+1} minimize ϵ_{t+1} .

For step size note that

$$\begin{aligned}
\frac{d}{d\eta}G(\boldsymbol{\alpha}_t + \eta e_{t+1}) &= -\frac{1}{k} \sum_{i=1}^m \sum_{l=1}^k y_i[l] h_{t+1}(x_i, l) G_i(\boldsymbol{\alpha}_t + \eta e_{t+1}) \\
&= -\frac{1}{k} \sum_{i=1}^m \sum_{l=1}^k y_i[l] h_{t+1}(x_i, l) G_i(\boldsymbol{\alpha}_t) \exp\left(-\frac{1}{k} \sum_{j=1}^k y_i[j] \eta h_{t+1}(x_i, j)\right) \\
&= -\sum_{i=1}^m \sum_{l=1}^k y_i[l] h_{t+1}(x_i, l) \exp\left(-\frac{1}{k} \sum_{j=1}^k y_i[j] \eta h_{t+1}(x_i, j)\right) D_{t+1}(i) m \Pi_{s=1}^t Z_s \\
&= -\sum_{i=1}^m \sum_{l=1}^k y_i[l] h_{t+1}(x_i, l) \exp(\eta(2\epsilon_{t+1}^i - 1)) D_{t+1}(i) m \Pi_{s=1}^t Z_s
\end{aligned}$$

Thus

$$\frac{d}{d\eta}G(\boldsymbol{\alpha}_t + \eta e_{t+1}) = 0 \Leftrightarrow \sum_{i=1}^m (2\epsilon_{t+1}^i - 1) D_{t+1}(i) \exp(\eta(2\epsilon_{t+1}^i - 1)) = 0 \quad (2)$$

Algorithm 1 Alternative ADABOOST.MH($S = ((x_1, y_1), \dots, (x_m, y_m))$)

```

1: for  $i \leftarrow 1$  to  $m$  do
2:    $D_1(i, l) = \frac{1}{m}$ 
3:   for  $h \in H$  do
4:      $\epsilon_h^i \leftarrow \Pr_{j \sim U(k)}[y_i[j] \neq h(x_i, j)]$ 
5:   end for
6: end for
7: for  $t \leftarrow 1$  to  $T$  do
8:    $h_t \leftarrow$  base classifier minimize  $\mathbb{E}_{i \sim D_t} \epsilon_h^i$ 
9:    $\eta_t \leftarrow$  solution of (2)
10:   $Z_t \leftarrow \mathbb{E}_{i \sim D_t} e^{\eta_t(2\epsilon_t^i - 1)}$ 
11:  for  $i \leftarrow 1$  to  $m$  do
12:     $D_{t+1}(i) \leftarrow \frac{D_t(i) e^{-\frac{1}{k} \sum_{j=1}^k y_i[j] \eta_t h_t(x_i, j)}}{Z_t}$ 
13:  end for
14: end for
15:  $g \leftarrow \sum_{t=1}^T \eta_t h_t$ 
16: return  $sgng$ 

```

Note that ϵ_t^i are multiple of $\frac{1}{k}$ so by change of variable $x = e^{\frac{\eta}{k}}$ we can transform it into a polynomial equation.

The above analysis gives us algorithm 1.

Note that in this case our weak learning condition becomes $\mathbb{E}_{i \sim D_t} \epsilon_h^i < \frac{1}{2}$ for any distribution D_t and $h \in H$. Also when k is large this alternative algorithm is more efficient than the original ADABOOST.MH.

For generalization bound, note that we are dealing with multi-label classification. For any hypotheses h we can see it as a vector of binary classifiers $(h_1, \dots, h_k)$, where $h_l(x) = h(x, l)$. We denote $\Pi_l(H) = \{h(\cdot, l) : h \in H\}$

$$\begin{aligned}
R(h) &= \mathbb{E}_{x \sim D} d(h(x), y) = \sum_{l=1}^k \mathbb{E}_{x \sim D} 1_{h_l(x) \neq y[l]} = \sum_{l=1}^k R(h_l) \\
\hat{R}(h) &= \frac{1}{m} \sum_{i=1}^m d(h(x_i), y_i) = \sum_{l=1}^k \frac{1}{m} 1_{h_l(x_i) \neq y_i[l]} = \sum_{l=1}^k \hat{R}(h_l)
\end{aligned}$$

where d is Hamming distance.

We then can use corollary 6.1 on textbook for every $l \in [k]$. Fix ρ and then for any $\delta > 0$, with prob at least $1 - \delta$ the following holds for all $h_l \in \text{conv}(\Pi_l(H))$

$$R(h_l) \leq \hat{R}_\rho(h_l) + \frac{2}{\rho} \mathfrak{R}_m(\Pi_l(H)) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}$$

$$R(h_l) \leq \hat{R}_\rho(h_l) + \frac{2}{\rho} \hat{\mathfrak{R}}_S(\Pi_l(H)) + 3\sqrt{\frac{\log \frac{1}{\delta}}{2m}}$$

Thus fix ρ and then for any $\delta > 0$, with prob at least $1 - k\delta$ the following holds for all $g \in \text{conv}(H)$

$$R(g) \leq \sum_{l=1}^k \hat{R}_\rho(g_l / \|\alpha\|_1) + \frac{2}{\rho} \sum_{l=1}^k \mathfrak{R}_m(\Pi_l(H)) + k\sqrt{\frac{\log \frac{1}{\delta}}{2m}}$$

$$R(g) \leq \sum_{l=1}^k \hat{R}_\rho(g_l / \|\alpha\|_1) + \frac{2}{\rho} \sum_{l=1}^k \hat{\mathfrak{R}}_S(\Pi_l(H)) + 3k\sqrt{\frac{\log \frac{1}{\delta}}{2m}}$$

Mehryar Mohri
 Foundations of Machine Learning 2020
 Courant Institute of Mathematical Sciences
 Homework assignment 3
 Nov 14, 2020
 Due: Nov 24, 2020 [before class starts].

A. Kernel methods

1. Graph kernel. Let $G = (\mathcal{V}, \mathcal{E})$ be an undirected graph with vertex set $\mathcal{V}$ and edge set $\mathcal{E}$. $\mathcal{V}$ could represent a set of documents or biosequences and $\mathcal{E}$ the set of connections between them. Let $w[e] \in \mathbb{R}$ denote the weight assigned to edge $e \in \mathcal{E}$. The weight of a path is the product of the weights of its constituent edges. Show that the kernel K over $\mathcal{V} \times \mathcal{V}$ where $K(p, q)$ is the sum of the weights of all paths of length two between p and q is PDS (*Hint*: you could introduce the matrix $W = (W_{pq})$, where $W_{pq} = 0$ when there is no edge between p and q , W_{pq} equal to the weight of the edge between p and q otherwise).
2. Pixel kernel. This problem consists of showing that a kernel useful in applications is PDS.
 - (a) Show that $S: (z, z') \mapsto \int_0^{+\infty} 1_{t \in [0, z]} 1_{t \in [0, z']} dt$ is a PDS kernel defined over $\mathbb{R} \times \mathbb{R}$.
 - (b) Use the previous question to prove that the kernel $K_\mu: (\mathbf{x}, \mathbf{x}') \mapsto \prod_{k=1}^N e^{\min(|x_k|^\mu, |x'_k|^\mu)}$, where x_k is the k th coordinate of $\mathbf{x}$, is a PDS kernel defined over $\mathbb{R}^N \times \mathbb{R}^N$, for any $\mu \geq 0$.

B. Boosting

1. Logistic loss boosting.
 - (a) Show that $\Phi: u \mapsto \log_2(1 + e^{-u})$ defines a convex and decreasing function upper-bounding $u \mapsto 1_{u \leq 0}$.
 - (b) Let $\mathcal{H} = \{h_1, \dots, h_N\}$ be a finite hypothesis set serving as a family of base predictors taking values in $\{-1, +1\}$. Define the objective $F(\boldsymbol{\alpha})$ for a boosting-type algorithm using Φ , instead of the exponential function used in AdaBoost, for a given sample of size m and show that it is a convex function of $\boldsymbol{\alpha}$.

- (c) Determine the best direction at iteration t if you apply coordinate descent to F . You should adopt a notation similar to the one used in class and define D_t , Z_t , and $\epsilon_{t,k}$ for $t \in [T]$ and $k \in [N]$. What should be the boosting condition on $\epsilon_{t,k}$ for the best direction k ?
- (d) Show that for any $(u, v) \in \mathbb{R}^2$, the following inequality holds:

$$\Phi(u + v) - \Phi(u) \leq -\Phi'(u)(e^{-v} - 1).$$

- (e) Use that to show the following:

$$F(\alpha_{t-1} + \eta \mathbf{e}_k) - F(\alpha_{t-1}) \leq \frac{1}{m} \sum_{i=1}^m D_t(i) Z_t[e^{-\eta y_i h_k(x_i)} - 1].$$

- (f) To determine the best step η for a given direction k , we can minimize the upper bound of the previous question. Show that this is syntactically the same minimization as the one for finding the best step in AdaBoost. Give the expression of the step at iteration t .
- (g) Give the full pseudocode of the algorithm.
- (h) Give a margin-based generalization bound for this algorithm.
- (i) Compare empirically AdaBoost and this logistic loss boosting algorithm on the same dataset as the one used in the previous homework by choosing T via cross-validation as multiples of 100.

Mehryar Mohri
 Foundations of Machine Learning 2020
 Courant Institute of Mathematical Sciences
 Homework assignment 3
 Nov 14, 2020
 Due: Nov 24, 2020 [before class starts].

A. Kernel methodss

1. Graph kernel. Let $G = (\mathcal{V}, \mathcal{E})$ be an undirected graph with vertex set $\mathcal{V}$ and edge set $\mathcal{E}$. $\mathcal{V}$ could represent a set of documents or biosequences and E the set of connections between them. Let $w[e] \in \mathbb{R}$ denote the weight assigned to edge $e \in \mathcal{E}$. The weight of a path is the product of the weights of its constituent edges. Show that the kernel K over $\mathcal{V} \times \mathcal{V}$ where $K(p, q)$ is the sum of the weights of all paths of length two between p and q is PDS (*Hint*: you could introduce the matrix $W = (W_{pq})$, where $W_{pq} = 0$ when there is no edge between p and q , W_{pq} equal to the weight of the edge between p and q otherwise).

Solution:

Graph kernel For any $i, j \in \mathcal{V}$, let $\mathcal{E}[i, j]$ denote the set of edges between i and j which is either reduced to one edge or is empty. For convenience, let $\mathcal{V} = \{1, \dots, n\}$ and define the matrix $\mathbf{W} = (W_{ij}) \in \mathbb{R}^{n \times n}$ by $W_{ij} = 0$ if $\mathcal{E}[i, j] = \emptyset$, $W_{ij} = w[e]$ if $e \in \mathcal{E}[i, j]$. Then, we can write

$$K(p, q) = \sum_{e \in \mathcal{E}[p, r], e' \in \mathcal{E}[r, q]} w[e]w[e'] = \sum_r W_{pr}W_{rq} = W_{pq}^2.$$

Let $\mathbf{K} = (K_{pq})$ denote the kernel matrix. Since $\mathbf{W}$ is symmetric For any vector $\mathbf{X} \in \mathbb{R}^n$, $\mathbf{X}^\top \mathbf{K} \mathbf{X} = \mathbf{X}^\top \mathbf{W}^2 \mathbf{X} = \mathbf{X}^\top \mathbf{W}^\top \mathbf{W} \mathbf{X} = \|\mathbf{W} \mathbf{X}\|^2 \geq 0$. Thus, the eigenvalues of $\mathbf{K}$ are non-negative. The same holds similarly for the kernel matrix restricted to any subset of $\mathcal{V}$, thus K is PDS. $\square$

2. Pixel kernel. This problem consists of showing that a kernel useful in applications is PDS.

- (a) Show that $S: (z, z') \mapsto \int_0^{+\infty} 1_{t \in [0, z]} 1_{t \in [0, z']} dt$ is a PDS kernel defined over $\mathbb{R} \times \mathbb{R}$.

Solution: $(f, g) \mapsto \int_{t=0}^{+\infty} f(t)g(t)dt$ is a positive semi-definite inner product over the set $L_2(\mathbb{R})$. Let $\langle \cdot, \cdot \rangle$ denote that inner

product. Then, for any $z, z' \in \mathbb{R}$, $S(z, z') = \langle t \mapsto 1_{t \in [0, z]}, t \mapsto 1_{t \in [0, z']} \rangle = \langle \Psi(z), \Psi(z') \rangle$, where Ψ is the feature mapping $\Psi: z \mapsto t \mapsto 1_{t \in [0, z]}$. Thus, S can be expressed as an inner product and is therefore PDS. $\square$

- (b) Use the previous question to prove that the kernel $K_\mu: (\mathbf{x}, \mathbf{x}') \mapsto \prod_{k=1}^N e^{\min(|x_k|^\mu, |x'_k|^\mu)}$, where x_k is the k th coordinate of $\mathbf{x}$, is a PDS kernel defined over $\mathbb{R}^N \times \mathbb{R}^N$, for any $\mu \geq 0$.

Solution: Observe that, for any $\mu \geq 0$, $\min(|u|^\mu, |u'|^\mu) = S(|u|^\mu, |u'|^\mu)$. Thus, for any $\mu \geq 0$ and $k \in [N]$, $(\mathbf{x}, \mathbf{x}') \mapsto \min(|x_k|^\mu, |x'_k|^\mu)$ is a PDS kernel. The sum of these PDS kernels (over all $k \in [N]$) is therefore also PDS and the composition with the power series $\exp$ whose coefficients are non-negative and whose radius of convergence is infinite is also PDS. $\square$

B. Boosting

1. Logistic loss boosting.

- (a) Show that $\Phi: u \mapsto \log_2(1 + e^{-u})$ defines a convex and decreasing function upper-bounding $u \mapsto 1_{u \leq 0}$.

Solution: This is straightforward. For any $u \in \mathbb{R}$,

$$\Phi'(u) = -\frac{1}{\log(2)} \frac{e^{-u}}{1 + e^{-u}} = -\frac{1}{\log(2)} \frac{1}{1 + e^u} < 0.$$

Thus, Φ is decreasing. For any $u \in \mathbb{R}$,

$$\Phi''(u) = \frac{1}{\log(2)} \frac{e^u}{(1 + e^u)^2} > 0.$$

Thus, Φ is convex. For $u \leq 0$, we have $e^{-u} \geq 1$, thus $\log_2(1 + e^{-u}) \geq \log_2(1 + 1) = 1$. For $u > 0$, since $\log$ is increasing, $\log_2(1 + e^{-u}) \geq \log_2(1) = 0$.

- (b) Let $\mathcal{H} = \{h_1, \dots, h_N\}$ be a finite hypothesis set serving as a family of base predictors taking values in $\{-1, +1\}$. Define the objective $F(\boldsymbol{\alpha})$ for a boosting-type algorithm using Φ , instead of the exponential function used in AdaBoost, for a given sample of size m and show that it is a convex function of $\boldsymbol{\alpha}$.

Solution: The objective function can be written for any $\alpha \in \mathbb{R}^N$, $\alpha \geq 0$ as follows:

$$F(\alpha) = \sum_{i=1}^m \Phi \left(y_i \sum_{j=1}^N \alpha_j h_j(x_i) \right).$$

This a convex function of α since Φ is convex and composition with an affine function of α preserves convexity. $\square$

- (c) Determine the best direction at iteration t if you apply coordinate descent to F . You should adopt a notation similar to the one used in class and define D_t , Z_t , and $\epsilon_{t,k}$ for $t \in [T]$ and $k \in [N]$. What should be the boosting condition on $\epsilon_{t,k}$ for the best direction k ?

Solution: For any $\eta > 0$ and direction $k \in [N]$, we can write:

$$F(\alpha_{t-1} + \eta \mathbf{e}_k) = \frac{1}{m} \sum_{i=1}^m \Phi \left(y_i \sum_{j=1}^N \alpha_{t-1,j} h_j(x_i) + y_i \eta h_k(x_i) \right).$$

Thus, the directional derivative of F along $\mathbf{e}_k$ is given by:

$$\begin{aligned} F'(\alpha_{t-1}, \mathbf{e}_k) &= \frac{1}{m} \sum_{i=1}^m y_i h_k(x_i) \Phi' \left(y_i \sum_{j=1}^N \alpha_{t-1,j} h_j(x_i) \right) \\ &= -\frac{1}{m} \sum_{i=1}^m \frac{1}{\log(2)} \frac{y_i h_k(x_i)}{1 + e^{y_i \sum_{j=1}^N \alpha_{t-1,j} h_j(x_i)}} \\ &= -\frac{1}{m} \sum_{i=1}^m y_i h_k(x_i) D_t(i) Z_t \\ &= -\frac{Z_t}{m} [(1 - \epsilon_{t,k}) - \epsilon_{t,k}] \\ &= \frac{Z_t}{m} (2\epsilon_{t,k} - 1), \end{aligned}$$

where $D_t(i) Z_t = \frac{1}{\log(2)} \frac{1}{1 + e^{y_i \sum_{j=1}^N \alpha_{t-1,j} h_j(x_i)}}$. In view of that, the direction maximizing $|F'(\alpha_{t-1} + \eta \mathbf{e}_k)|$ is the one with the minimal weighted error $\epsilon_{t,k}$. For coordinate descent to succeed, we need $\epsilon_{t,k} < \frac{1}{2}$ for that best direction.

- (d) Show that for any $(u, v) \in \mathbb{R}^2$, the following inequality holds:

$$\Phi(u + v) - \Phi(u) \leq -\Phi'(u)(e^{-v} - 1).$$

Solution:

$$\begin{aligned}
\Phi(u+v) - \Phi(u) &= \log_2 \left[\frac{1 + e^{-(u+v)}}{1 + e^{-u}} \right] \\
&= \log_2 \left[\frac{1 + e^{-u} + e^{-(u+v)} - e^{-u}}{1 + e^{-u}} \right] \\
&= \log_2 \left[1 + \frac{e^{-v} - 1}{1 + e^u} \right] \\
&\leq \frac{1}{\log(2)} \frac{e^{-v} - 1}{1 + e^u} \quad (\log(1+z) \leq z) \\
&= -\Phi'(u)(e^{-v} - 1).
\end{aligned}$$

(e) Use that to show the following:

$$F(\alpha_{t-1} + \eta \mathbf{e}_k) - F(\alpha_{t-1}) \leq \frac{1}{m} \sum_{i=1}^m D_t(i) Z_t [e^{-\eta y_i h_k(x_i)} - 1].$$

Solution: Observe that $D_t(i) Z_t = -\Phi'(y_i \sum_{j=1}^N \alpha_{t-1,j} h_j(x_i))$. Then, using the result of the previous question,

$$\begin{aligned}
F(\alpha_{t-1} + \eta \mathbf{e}_k) - F(\alpha_{t-1}) &\leq \frac{1}{m} \sum_{i=1}^m -\Phi'(y_i \sum_{j=1}^N \alpha_{t-1,j} h_j(x_i)) [e^{-\eta y_i h_k(x_i)} - 1] \\
&= \frac{1}{m} \sum_{i=1}^m D_t(i) Z_t [e^{-\eta y_i h_k(x_i)} - 1].
\end{aligned}$$

(f) To determine the best step η for a given direction k , we can minimize the upper bound of the previous question. Show that this is syntactically the same minimization as the one for finding the best step in AdaBoost. Give the expression of the step at iteration t .

Solution: Minimizing that upper bound is equivalent to minimizing $\frac{1}{m} \sum_{i=1}^m D_t(i) e^{-\eta y_i h_k(x_i)}$, which is the quantity minimized by AdaBoost to determine the step. The expression of D_t is different here but we can immediately use the result known for AdaBoost to determine the step at iteration t : $\eta = \frac{1}{2} \log \left[\frac{1 - \epsilon_{t,k}}{\epsilon_{t,k}} \right]$.

- (g) Give the full pseudocode of the algorithm.

Solution: This is straightforward based on the previous results.

- (h) Give a margin-based generalization bound for this algorithm.

Solution: Straightforward using the ensemble Rademacher complexity bound presented in class: fix $\rho > 0$; for any $\delta > 0$, with probability at least $1 - \rho$,

$$R\left(\frac{\sum_{t=1}^T \alpha_t h_t}{\|\boldsymbol{\alpha}\|_1}\right) \leq \widehat{R}_{S,\rho}\left(\frac{\sum_{t=1}^T \alpha_t h_t}{\|\boldsymbol{\alpha}\|_1}\right) + \frac{2}{\rho} \mathfrak{R}_m(H) + \sqrt{\frac{\log \frac{1}{\delta}}{2m}}.$$

- (i) Compare empirically AdaBoost and this logistic loss boosting algorithm on the same dataset as the one used in the previous homework by choosing T via cross-validation as multiples of 100.

Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 4
Due: April 11, 2006

Problem 1: Perceptron

- (1) Prove that the mistake bound given for the perceptron algorithm is tight. To do that, find a sequence of m examples $(x_1, y_1), \dots, (x_m, y_m) \in X \times \{-1, +1\}$ linearly separable, such that $\|x_i\| \leq R$ for $i = 1, \dots, m$, and such that the number of updates made by the perceptron algorithm is exactly $\frac{R^2}{\rho^2}$.

Problem 2: Boosting

- (1) Let h_t be the base classifier selected by Adaboost at round t . What is the error rate of h_t with respect to the distribution D_{t+1} ? Use this and the definition of weak learners to conclude that $h_{t+1} \neq h_t$.
- (2) Adaboost may lead to significant overfitting in presence of noise. This could be because of the high penalization of misclassified examples. To reduce that effect, one could use instead the following objective function:

$$F = \sum_{i=1}^m G(-y_i f(x_i)), \quad (1)$$

where G is the function defined on $\mathbb{R}$ by

$$G(x) = \begin{cases} e^x & \text{if } x \leq 0 \\ x + 1 & \text{otherwise.} \end{cases} \quad (2)$$

- (a) Show that the function G is convex and differentiable.
- (b) Use F and greedy coordinate descent to derive an algorithm similar to Adaboost.
- (c) Can you compare the reduction of the empirical error rate of this algorithm with that of Adaboost?

Problem 1: Perceptron

[25 points]

- (1) [25 points] Consider the points $x_i = (\underbrace{0, \dots, 0}_{(i-1)}, 1, 0, \dots, 0)$, $i = 1, \dots, m$, on the sphere of radius $R = 1$ in $\mathbb{R}^m$. Then the set is separated with the weight vector $w = (y_1, \dots, y_m)$, margin $\rho = \frac{|w \cdot x_i|}{\sqrt{m}} = \frac{1}{\sqrt{m}}$, and with exactly $m = \frac{R^2}{\rho^2}$ updates.

Problem 2: Boosting

- (1) [25 points] As seen in Lecture 8, at each boosting round t , Adaboost assigns an equal probability mass to the examples correctly classified and those misclassified by h_t . h_t cannot be selected at the next round since its error rate is exactly $\frac{1}{2}$ with that distribution and it would need to be $\frac{1}{2} - \epsilon$ for some $0 < \epsilon \leq \frac{1}{2}$.
- (2) [50 points]
- (a) [10 points] Let $G_1(x) = e^x$ and $G_2(x) = x + 1$. G_1 and G_2 are continuously differentiable over $\mathbb{R}$ and $G'_1(0) = G'_2(0)$. Thus, G is differentiable over $\mathbb{R}$. Note that $G' \geq 0$. Both G_1 and G_2 are convex, thus

$$G(y) - G(x) \geq G'(x)(y - x) \quad (1)$$

for $x, y \leq 0$ or $x, y \geq 0$. Assume now that $y \leq 0$ and $x \geq 0$, then

$$G(y) - G(x) = e^y - (x + 1) \geq (y + 1) - (x + 1) = G'(x)(y - x), \quad (2)$$

since $G'(x) = 1$. Thus G is convex.

- (b) [35 points] The direction e_u taken by coordinate descent after $T - 1$ rounds is the argmin_u of:

$$\left. \frac{dG(\alpha + \beta e_u)}{d\beta} \right|_{\beta=0} = - \sum_{i=1}^m y_i h_u(x_i) G'(-y_i f(x_i)) \quad (3)$$

$$(\text{since } G' \geq 0) \propto - \sum_{i=1}^m y_i h_u(x_i) \frac{G'(-y_i f(x_i))}{\sum_{i=1}^m G'(-y_i f(x_i))} \quad (4)$$

$$\propto - \sum_{i=1}^m y_i h_u(x_i) D_{T-1}(i) \quad (5)$$

$$= -(1 - 2\epsilon_u), \quad (6)$$

with $D_{T-1}(i) = \frac{1}{m} \frac{G'(-y_i f(x_i))}{\sum_{i=1}^m G'(-y_i f(x_i))}$. Thus, the base classifier h_u selected at each round is the one with the minimal error rate over the training data.

The step size β is the solution of:

$$\frac{dF(\alpha + \beta e_u)}{d\beta} = - \sum_{i=1}^m y_i h_u(x_i) G'(-y_i f(x_i) - \beta y_i h_u(x_i)) = 0, \quad (7)$$

which can be solved numerically. A closed form solution can be given under certain conditions, e.g., if

$$\beta \leq \rho = \min_{i \in [1, m]} |f(x_i)|. \quad (8)$$

- (c) [5 points] By definition of the objective function, this algorithm is less aggressively reducing the empirical error rate than AdaBoost.

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 4
Due: April 27, 2007

This is a shorter assignment to leave you more time to work on your project.

Boosting

Suppose we simplify AdaBoost by setting the parameter α_t to a fix value $\alpha_t = \alpha > 0$, independent of the boosting round t .

1. Let γ be such that $(\frac{1}{2} - \epsilon_t) \geq \gamma > 0$ where ϵ_t is defined as in class. Find the best value of α as a function of γ by analyzing the empirical error.
2. For that value of α , does the algorithm assign the same probability mass to correctly classified and misclassified examples at each round? Which set is assigned a higher probability mass?
3. Using the previous value of α , give a bound on the empirical error of the algorithm that depends only on γ and the number of rounds of boosting T .
4. Using the previous bound, show that for $T > \frac{\log m}{2\gamma^2}$, the resulting hypothesis is consistent with the sample of size m .
5. Let s be the VC dimension of the base learners used. Give a bound on the generalization error of the consistent hypothesis obtained after $T = \left\lfloor \frac{\log m}{2\gamma^2} \right\rfloor + 1$ rounds of boosting (*hint*: you can use the fact that the VC dimension of the family of functions $\left\{ \text{sgn}(\sum_{t=1}^T \alpha_t h_t) : \alpha_t \in \mathbb{R} \right\}$ is bounded by $2(s+1)T \log_2(eT)$). Suppose now that γ varies with m . Based on the bound derived, what can you say if $\gamma(m) = O(\sqrt{\frac{\log m}{m}})$?

Mehryar Mohri
Foundations of Machine Learning
Courant Institute of Mathematical Sciences
Homework assignment 4 – Solution
April 27, 2007

This is a shorter assignment to leave you more time to work on your project.

Boosting

[60 points]

Suppose we simplify AdaBoost by setting the parameter α_t to a fix value $\alpha_t = \alpha > 0$, independent of the boosting round t .

1. [20 points] Let γ be such that $(\frac{1}{2} - \epsilon_t) \geq \gamma > 0$ where ϵ_t is defined as in class. Find the best value of α as a function of γ by analyzing the empirical error.

As in class, we can show that

$$\widehat{error}(H) \leq \prod_{t=1}^T Z_t, \quad (1)$$

and that

$$Z_t = (1 - \epsilon_t)e^{-\alpha} + \epsilon_t e^{\alpha}. \quad (2)$$

By definition of γ and the fact that $e^{\alpha} - e^{-\alpha} > 0$ for all $\alpha > 0$,

$$Z_t = \epsilon_t(e^{\alpha} - e^{-\alpha}) + e^{-\alpha} \quad (3)$$

$$\leq (1 - \gamma)(e^{\alpha} - e^{-\alpha}) + e^{-\alpha} \quad (4)$$

$$= (\frac{1}{2} - \gamma)e^{\alpha} + (\frac{1}{2} + \gamma)e^{-\alpha} = u(\alpha). \quad (5)$$

$u(\alpha)$ is minimized for

$$(\frac{1}{2} - \gamma)e^{\alpha} = (\frac{1}{2} + \gamma)e^{-\alpha}, \quad (6)$$

that is for

$$\alpha = \frac{1}{2} \log \frac{\frac{1}{2} + \gamma}{\frac{1}{2} - \gamma}. \quad (7)$$

Tighter bounds on the product of the Z_t s can lead to better values for α .

2. [20 points] For that value of α , does the algorithm assign the same probability mass to correctly classified and misclassified examples at each round? Which set is assigned a higher probability mass?

As in the proof given in class, at round t , the probability mass assigned to correctly classified points is $p_+ = (1 - \epsilon_t)e^{-\alpha}$ and the probability mass assigned to the misclassified points is $p_- = \epsilon_t e^\alpha$. Thus,

$$\frac{p_-}{p_+} = \frac{\epsilon_t}{1 - \epsilon_t} \frac{\frac{1}{2} + \gamma}{\frac{1}{2} - \gamma} \leq \frac{\frac{1}{2} - \gamma}{\frac{1}{2} + \gamma} \frac{\frac{1}{2} + \gamma}{\frac{1}{2} - \gamma} = 1. \quad (8)$$

This contrasts with AdaBoost's property.

3. [20 points] Using the previous value of α , give a bound on the empirical error of the algorithm that depends only on γ and the number of rounds of boosting T .

$$Z_t \leq \left(\frac{1}{2} - \gamma\right)e^\alpha + \left(\frac{1}{2} + \gamma\right)e^{-\alpha} \quad (9)$$

$$= \left(\frac{1}{2} - \gamma\right)\sqrt{\frac{\frac{1}{2} + \gamma}{\frac{1}{2} - \gamma}} + \left(\frac{1}{2} + \gamma\right)\sqrt{\frac{\frac{1}{2} - \gamma}{\frac{1}{2} + \gamma}} \quad (10)$$

$$= 2\sqrt{\left(\frac{1}{2} + \gamma\right)\left(\frac{1}{2} - \gamma\right)}. \quad (11)$$

Thus, the empirical error can be bounded as follows:

$$\widehat{error}(H) \leq \prod_{t=1}^T Z_t \quad (12)$$

$$\leq [2\sqrt{\left(\frac{1}{2} + \gamma\right)\left(\frac{1}{2} - \gamma\right)}]^T \quad (13)$$

$$= (1 - 4\gamma^2)^{T/2} \quad (14)$$

$$\leq e^{-2\gamma^2 T}. \quad (15)$$

4. [20 points] Using the previous bound, show that for $T > \frac{\log m}{2\gamma^2}$, the resulting hypothesis is consistent with the sample of size m .

If $\widehat{error}(H) = \frac{1}{m} \sum_{i=1}^m 1_{y_i f(x_i) \leq 0} \leq \frac{1}{m}$, then clearly $\widehat{error}(H) = 0$. Using the bound obtained in the previous question, if $e^{-2\gamma^2 T} < \frac{1}{m}$, the empirical error is zero. This can be rewritten as

$$T > \frac{\log m}{2\gamma^2}. \quad (16)$$

5. [20 points] Let s be the VC dimension of the base learners used. Give a bound on the generalization error of the consistent hypothesis obtained after $T = \left\lfloor \frac{\log m}{2\gamma^2} \right\rfloor + 1$ rounds of boosting (*hint*: you can use the fact that the VC dimension of the family of functions $\left\{ \text{sgn}(\sum_{t=1}^T \alpha_t h_t) : \alpha_t \in \mathbb{R} \right\}$ is bounded by $2(s+1)T \log_2(eT)$). Suppose now that γ varies with m . Based on the bound derived, what can you say if $\gamma(m) = O(\sqrt{\frac{\log m}{m}})$?

Using the bound proved in class for the consistent case,

$$\Pr[error_D(H) > \epsilon] \leq 2\Pi_C(2m)2^{-\frac{m\epsilon}{2}} \leq 2\left(\frac{2em}{d}\right)^d 2^{-\frac{m\epsilon}{2}}. \quad (17)$$

Setting the right-hand side to δ , with probability at least $1 - \delta$, the following bound holds for that consistent hypothesis:

$$error_D(H) \leq \frac{2}{m} \left(d \log_2 \frac{2em}{d} + \log_2 \frac{2}{\delta} \right), \quad (18)$$

with $d = 2(s+1)T \log_2(eT)$ and $T = \left\lfloor \frac{\log m}{2\gamma^2} \right\rfloor + 1$.

The bound is vacuous for $\gamma(m) = O(\sqrt{\frac{\log m}{m}})$. This could suggest overfitting.

Mehryar Mohri
 Foundations of Machine Learning
 Courant Institute of Mathematical Sciences
 Homework assignment 4
 Due: April 18th, 2008
 Credit: Ashish Rastogi, Afshin Rostamizadeh
 Ameet Talwalkar, and Eugene Weinstein.

1. **Problem 1:** Consider the following formulation of Adaboost. As in class, we start with a training set of labelled examples: $\{(\mathbf{x}_i, y_i)\}_{i=1, \dots, m}$, where $(\mathbf{x}_i, y_i) \in \chi \times \{-1, 1\}$. Let $\mathcal{H} = \{h_1, \dots, h_n\}$ be the set of weak classifiers where $h_j : \chi \rightarrow \{1, -1\}$ (note: we assume a finite number n of weak classifiers, where $m \ll n$). We define an $m \times n$ matrix $\mathbf{M}$ where $M_{ij} = y_i h_j(\mathbf{x}_i)$, i.e., $M_{ij} = +1$ if training example i is classified correctly by weak classifier h_j , and -1 otherwise. Let $\mathbf{d}_t, \lambda_t \in \mathbb{R}^n$, $\|\mathbf{d}_t\|_1 = 1$ and $d_{t,i}$ (respectively $\lambda_{t,i}$) equal i^{th} component of $\mathbf{d}_t$ (respectively λ_t). Let $\mathbf{d}_t^T$ denote the transpose of the vector $\mathbf{d}_t$. Now we define the following algorithm:

- (a) **Input:** Matrix $\mathbf{M}$, Number of iterations t_{max}
- (b) **Initialize:** $\lambda_{1,j} = 0$ for $j = 1, \dots, n$
- (c) **Loop for** $t = 1, \dots, t_{max}$
 - i. $d_{t,i} = \frac{\exp(-(\mathbf{M}\lambda_t)_i)}{\sum_{k=1}^m \exp(-(\mathbf{M}\lambda_t)_k)}$ for $i = 1, \dots, m$
 - ii. $j_t \in \operatorname{argmax}_j (\mathbf{d}_t^T \mathbf{M})_j$
 - iii. $r_t = (\mathbf{d}_t^T \mathbf{M})_{j_t}$
 - iv. $\alpha_t = \frac{1}{2} \ln \left(\frac{1+r_t}{1-r_t} \right)$
 - v. $\lambda_{t+1} = \lambda_t + \alpha_t \mathbf{e}_{j_t}$, where $\mathbf{e}_{j_t}$ is 1 in position j_t and 0 elsewhere.
- (d) **Output:** $\frac{\lambda_{t_{max}}}{\|\lambda_{t_{max}}\|_1}$
- (a) Is this approach of explicitly using $\mathbf{M}$ practical? Why/Why not?
- (b) What does $d_{1,i}$ equal for $t = 1$ for each value of i ?
- (c) In one sentence, explain what is happening in step (c).i.

- (d) $(\mathbf{d}_t^T \mathbf{M})_j$ is called the "edge" of weak classifier j at time t w.r.t. the training examples. What are the max and min values for the edge of a weak classifier at time t ?
- (e) What do large and small values of r_t tell us about the classifier?
- (f) How would you write the combined classifier $H(x)$ as defined in lecture in terms of λ_{tmax} ?

2. **Problem 2:** The explicit mapping between $\mathbf{d}_t$ and D_{t+1} for the algorithm presented in Problem 1 can be defined as follows:

- (a) $j_t \in \operatorname{argmax}_j (\mathbf{d}_t^T \mathbf{M})_j$
- (b) $r_t = (\mathbf{d}_t^T \mathbf{M})_{j_t}$
- (c) $d_{t+1,i} = \frac{d_{t,i}}{1 + M_{ij_t} r_t}$ for $i = 1, \dots, m$
- (a) Let d_- be the probability of error of weak classifier h_{j_t} at iteration t . Define d_- as a summation over entries in $\mathbf{M}$.
- (b) Write an expression for edge r_t in terms of d_- .
- (c) Assuming $\mathbf{d}_t$ is normalized, show that d_{t+1} remains normalized, i.e., $\sum_i d_{t+1,i} = 1$.
- (d) Show that Adaboost sets the edge of the previous weak classifier to 0, i.e., $(\mathbf{d}_{t+1}^T \mathbf{M})_{j_t} = 0$.

3. **Problem 3:**

- (a) Observe the $\mathbf{M}$ defined below, with 8 training points and 8 weak classifiers. As defined in Problem 1, the i^{th} column of $\mathbf{M}$ represents weak classifier i applied to the training points.

$$\mathbf{M} = \begin{pmatrix} -1 & 1 & 1 & 1 & 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 & 1 & -1 & 1 & 1 \\ 1 & -1 & 1 & 1 & -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 & 1 & 1 & -1 \\ 1 & 1 & -1 & 1 & 1 & 1 & 1 & -1 \\ 1 & 1 & -1 & 1 & 1 & 1 & -1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & 1 & -1 \end{pmatrix}$$

Assume that we start with the following initial distribution over the datapoints:

$$\mathbf{d}_1 = \left(\frac{3 - \sqrt{5}}{8}, \frac{3 - \sqrt{5}}{8}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{\sqrt{5} - 1}{8}, \frac{\sqrt{5} - 1}{8}, 0 \right)^T$$

Perform Adaboost using the algorithm defined in Problem 2 using $\mathbf{M}$, $\mathbf{d}_1$, and $t_{max} = 7$. What weak classifier is picked at each round of boosting? Do you notice any pattern?

- (b) What is the norm-1 margin produced by Adaboost for this example?
- (c) Instead of using Adaboost, imagine we combined our classifiers using the following coefficients: $[2, 3, 4, 1, 2, 2, 1, 1] \times \frac{1}{16}$. What is the margin in this case? Does Adaboost maximize the margin?

Mehryar Mohri
 Foundations of Machine Learning
 Courant Institute of Mathematical Sciences
 Homework assignment 4 - solution
 Due: April 18th, 2008
 Credit: Ashish Rastogi, Afshin Rostamizadeh
 Ameet Talwalkar, and Eugene Weinstein.

1. **Problem 1:** Consider the following formulation of Adaboost. As in class, we start with a training set of labelled examples: $\{(\mathbf{x}_i, y_i)\}_{i=1, \dots, m}$, where $(\mathbf{x}_i, y_i) \in \chi \times \{-1, 1\}$. Let $\mathcal{H} = \{h_1, \dots, h_n\}$ be the set of weak classifiers where $h_j : \chi \rightarrow \{1, -1\}$ (note: we assume a finite number n of weak classifiers, where $m \ll n$). We define an $m \times n$ matrix $\mathbf{M}$ where $M_{ij} = y_i h_j(\mathbf{x}_i)$, i.e., $M_{ij} = +1$ if training example i is classified correctly by weak classifier h_j , and -1 otherwise. Let $d_t, \lambda_t \in \mathbb{R}^n$, $\|d_t\|_1 = 1$ and $d_{t,i}(\lambda_{t,i})$ equal i^{th} component of $d_t(\lambda_t)$. Now we define the following algorithm:

- (a) **Input:** Matrix $\mathbf{M}$, Number of iterations t_{max}
- (b) **Initialize:** $\lambda_{1,j} = 0$ for $j = 1, \dots, n$
- (c) **Loop for** $t = 1, \dots, t_{max}$
 - i. $d_{t,i} = \frac{\exp(-(\mathbf{M}\lambda_t)_i)}{\sum_{k=1}^m \exp(-(\mathbf{M}\lambda_t)_k)}$ for $i = 1, \dots, m$
 - ii. $j_t \in \operatorname{argmax}_j (\mathbf{d}_t^T \mathbf{M})_j$
 - iii. $r_t = (\mathbf{d}_t^T \mathbf{M})_{j_t}$
 - iv. $\alpha_t = \frac{1}{2} \ln \left(\frac{1+r_t}{1-r_t} \right)$
 - v. $\lambda_{t+1} = \lambda_t + \alpha_t \mathbf{e}_{j_t}$, where $\mathbf{e}_{j_t}$ is 1 in position j_t and 0 elsewhere.
- (d) **Output:** $\frac{\lambda_{t_{max}}}{\|\lambda_{t_{max}}\|_1}$

5 points Is this approach of explicitly using $\mathbf{M}$ practical? Why/Why not?

Solution: Since n is large, it is not practical to store $\mathbf{M}$.

5 points What does $d_{1,i}$ equal for $t = 1$ for each value of i ?

Solution: $d_{1,i} = \frac{1}{m}$.

5 points In one sentence, explain what is happening in step 3a.

Solution: The distribution over the training samples is being updated and normalized based on the results from the weak classifier chosen in the previous round of boosting.

5 points $(\mathbf{d}_t^T \mathbf{M})_j$ is called the "edge" of weak classifier j at time t w.r.t. the training examples. What are the max and min values for the edge of a weak classifier at time t ?

Solution: $\min = -1$; $\max = 1$.

5 points What do large and small values of r_t tell us about the classifier?

Solution: r_t is the edge of the "best" weak classifier over distribution d_t . A larger (smaller) edge indicates a lower (higher) probability of error for this "best" weak classifier on the training set over d_t .

5 points How would you write the combined classifier $H(x)$ as defined in lecture in terms of λ_{tmax} ?

Solution: Let $f = \sum_i^n \left(\frac{\lambda_{tmax}}{\|\lambda_{tmax}\|_1} \right)_i h_i$. Then $H(x) = \text{sign}(f(x))$.

Problem 2

The explicit mapping between d_t and D_{t+1} for the algorithm presented in Problem 1 can be defined as follows:

1. $j_t \in \text{argmax}_j (\mathbf{d}_t^T \mathbf{M})_j$
2. $r_t = (\mathbf{d}_t^T \mathbf{M})_{j_t}$
3. $d_{t+1,i} = \frac{d_{t,i}}{1 + M_{ij_t} r_t}$ for $i = 1, \dots, m$

5 points Let d_- be the probability of error of weak classifier h_{j_t} at iteration t . Define d_- as a summation over entries in $\mathbf{M}$.

Solution: $d_- = \sum_{i: M_{ij_t} = -1} d_{t,i}$.

5 points Write an expression for edge r_t in terms of d_- .

Solution:

$$r_t = \sum_{i: M_{ij_t} = 1} d_{t,i} - \sum_{i: M_{ij_t} = -1} d_{t,i} = (1 - d_-) - d_- = 1 - 2d_-.$$

10 points Assuming d_t is normalized, show that d_{t+1} remains normalized, i.e., $\sum_i^m d_{t+1,i} = 1$.

Solution: When $M_{ij_t} = 1$, $d_{t+1,i} = \frac{d_{t,i}}{1+r_t}$ and when

$M_{ij_t} = -1$, $d_{t+1,i} = \frac{d_{t,i}}{1-r_t}$. Defining $d_+ = (1 - d_-)$ we have:

$$\sum_i^m d_{t+1,i} = \frac{1}{1+r_t} d_+ + \frac{1}{1-r_t} d_- \quad (1)$$

Rearranging expression for r_t from previous question, we have:

$$\sum_i^m d_{t+1,i} = \frac{(1+r_t)}{2(1+r_t)} + \frac{(1-r_t)}{2(1-r_t)} = 1 \quad (2)$$

10 points Show that Adaboost sets the edge of the previous weak classifier to 0, i.e., $(\mathbf{d}_{t+1}^T \mathbf{M})_{j_t} = 0$.

Solution: Based on the definition of an edge we have:

$$(\mathbf{d}_{t+1}^T \mathbf{M})_{j_t} = \sum_{i: M_{ij_t}=1} d_{t+1,i} - \sum_{i: M_{ij_t}=-1} d_{t+1,i} \quad (3)$$

Using the mapping defined at the beginning of this question, we get:

$$= \sum_{i: M_{ij_t}=1} d_{t,i} \frac{1}{1+r_t} - \sum_{i: M_{ij_t}=-1} d_{t,i} \frac{1}{1-r_t} \quad (4)$$

Using the definitions of d_+ and d_- and the expression for r_t in terms of d_- we can simplify:

$$= d_+ \frac{1}{1+r_t} - d_- \frac{1}{1-r_t} = \frac{1+r_t}{2} \frac{1}{1+r_t} - \frac{1-r_t}{2} \frac{1}{1-r_t} = 0 \quad (5)$$

Problem 3

5 points Observe the $\mathbf{M}$ defined below, with 8 training points and 8 weak classifiers. As defined in Problem 1, the i^{th} column of $\mathbf{M}$ represents weak classifier i applied to the training points.

$$\mathbf{M} = \begin{pmatrix} -1 & 1 & 1 & 1 & 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 & -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 & 1 & -1 & 1 & 1 \\ 1 & -1 & 1 & 1 & -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 & 1 & 1 & -1 \\ 1 & 1 & -1 & 1 & 1 & 1 & 1 & -1 \\ 1 & 1 & -1 & 1 & 1 & 1 & -1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & 1 & -1 \end{pmatrix}$$

Assume that we start with the following initial distribution over the datapoints:

$$\mathbf{d}_1 = \left(\frac{3-\sqrt{5}}{8}, \frac{3-\sqrt{5}}{8}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{\sqrt{5}-1}{8}, \frac{\sqrt{5}-1}{8}, 0 \right)^T$$

Perform Adaboost using the algorithm defined in Problem 2 using $\mathbf{M}$, $\mathbf{d}_1$, and $t_{max} = 7$. What weak classifier is picked at each round of boosting? Do you notice any pattern?

Solution: at $t = 1$ we have:

$$\mathbf{d}_1^T \mathbf{M} = \left(\frac{\sqrt{5}-1}{2}, 0, \frac{3-\sqrt{5}}{2}, \frac{3\sqrt{5}-1}{12}, \frac{3\sqrt{5}-1}{12}, \frac{3\sqrt{5}-1}{12}, \frac{1}{2}, \frac{11-3\sqrt{5}}{12} \right)$$

so we pick weak classifier 1. Now, the distribution at round two is:

$$\mathbf{d}_2 = \left(\frac{1}{4}, \frac{1}{4}, \frac{\sqrt{5}-1}{12}, \frac{\sqrt{5}-1}{12}, \frac{\sqrt{5}-1}{12}, \frac{3-\sqrt{5}}{8}, \frac{3-\sqrt{5}}{8}, 0 \right)^T$$

and the edges at round 2 are:

$$\mathbf{d}_2^T \mathbf{M} = \left(0, \frac{3-\sqrt{5}}{2}, \frac{\sqrt{5}-1}{2}, \frac{4-\sqrt{5}}{6}, \frac{4-\sqrt{5}}{6}, \frac{4-\sqrt{5}}{6}, \frac{\sqrt{5}-1}{4}, \frac{5+\sqrt{5}}{12} \right)$$

so we pick weak classifier 3. Continuing this process, we then pick weak classifier 2 in round 3. However, now we observe that $\mathbf{d}_4 = \mathbf{d}_1$, hence we have found a cycle, in which we repeatedly select classifiers 1, 3, 2, 1, 3, 2, ...

5 points What is the norm-1 margin produced by Adaboost for this example?

Solution: $r_t = \frac{\sqrt{5}-1}{2}, t \in 1, 2, 3$. Thus, the coefficients used to combine classifiers in our example are: $[\frac{1}{3}, \frac{1}{3}, \frac{1}{3}, 0, 0, 0, 0, 0]$ and the margin equals the minimum value in the following vector:
 $\mathbf{M} \times [\frac{1}{3}, \frac{1}{3}, \frac{1}{3}, 0, 0, 0, 0, 0]^T$, which is $\frac{1}{3}$.

10 points Instead of using Adaboost, imagine we combined our classifiers using the following coefficients: $[2, 3, 4, 1, 2, 2, 1, 1] \times \frac{1}{16}$. What is the margin in this case? Does Adaboost maximize the margin?

Solution: $\mathbf{M} \times [2, 3, 4, 1, 2, 2, 1, 1]^T \times \frac{1}{16} = \frac{3}{8}$ for all training points. This margin is greater than the one generated by Adaboost. Therefore Adaboost does NOT always maximize the norm-1 margin.